Cadence® Analog IC Design Flow

Course Version 1.0

Lecture Manual **Revision 1.0**

Table of Contents

Cadence Analog IC Design Flow

This page does not contain notes.

Module 1

About This Course

This page does not contain notes.

Course Prerequisites

Before taking this course, you need to

- Have knowledge of Linux commands
- Have a good understanding of circuit design concepts
- Understand device physics and the IC fabrication process
- Have good problem-solving skills

This page does not contain notes.

Course Objectives

In this course, you will:

- Describe the GUI of the Virtuoso Design Environment
- Attach the required PDK with the Virtuoso Studio Design Environment
- Design a schematic from the transistor level using the Virtuoso Studio Schematic Editor
- Create testbench setup and simulation environments using the ADE Explorer and Assembler
- Perform pre-layout and post-layout simulations using the Spectre® tool and analyzing them using the Virtuoso VA window
- Tune the design parameters using Real-Time Tuning assistant to meet the specifications
- Generate layout from schematic using Auto Place & Route(Auto P&R) assistant and perform routing
- Debug and clear the DRC and LVS errors using iPegasus™ and Pegasus™ Verification System
- Perform Parasitic Extraction from the layout using the Quantus™ Extraction Solution
- Generate the GDSII file for tapeout

This page does not contain notes.

Course Agenda

- Schematic Design and Symbol Creation
  - Submodule: Introduction to Virtuoso Studio Design Environment
  - Submodule: Schematic Design
  - Submodule: Creating Symbol from a Schematic
  - Lab: Setting up the *cds.lib* File for Virtuoso
  - Lab: Creating a New Library and Cell View
  - Lab: Designing a Schematic Using the Virtuoso Schematic Editor
  - Lab: Creating a Symbol
- Schematic Testbench and Modifying DUT Parameters
  - Lab: Creating Testbench Schematic
  - Lab: Modifying DUT Parameters
- Pre-Layout Simulation
  - Submodule: Simulation Setup in the ADE Explorer
  - Submodule: Simulation Setup in the ADE Assembler
  - Submodule: Real-Time Tuning(RTT) and Backannotating of the Tuned Parameters
  - Lab: Setting Up ADE Explorer and Assembler for Testbench Simulations
  - Lab: Adding Expressions and Analyzing the Results
  - Lab: Tuning Design Parameters Using Real-Time Tuning Assistant
  - Lab: Updating Design with Tuned Parameters (Backannotation)

This page does not contain notes.

Course Agenda (continued)

- Layout Design
  - Submodule: Device Placement Using Auto P&R Assistant
  - Submodule: Device Placement Using Generate All From Source (GFS)
  - Submodule: Exploring Various Methods of Routing
  - Lab: Generating Layout from Schematic
  - Lab: Performing Pin Placement
  - Lab: Routing the Design in Different Methods
- Physical Verification
  - Submodule: Design Rule Checking Using iPegasus DRC
  - Submodule: Design Rule Checking Using Pegasus Verification System
  - Submodule: Layout Versus Schematic Using Pegasus Verification System
  - Lab: Running Design Rule Check (DRC) Tests
  - Lab: Running Layout Versus Schematic (LVS) Tests
- Parasitic Extraction, Post-Layout Simulation, and Generating GDSII File
  - Submodule: Parasitic Extraction
  - Submodule: Creating Design Configurations with the Hierarchy Editor
  - Submodule: Post-Layout Simulation
  - Submodule: Generating GDSII File
  - Lab: Extracting Parasitics Using Quantus Tool
  - Lab: Creating a Config View for Post-Layout Simulation
  - Lab: Performing Post-Layout Simulations Using ADE Assembler
  - Lab: Generating GDSII File

This page does not contain notes.

Software and Licenses

For the software and licenses used in the labs for this course, go to:

https://www.cadence.com/en_US/home/training/all-courses/86388.html

If there is additional information regarding the specific software, it is detailed in the lab document and/or the README file of the database provided with this course.

This page does not contain notes.

Become Cadence Certified by Earning a Digital Badge

Digital badges indicate mastery in a certain technology or skill and give managers and potential employers a way to validate your expertise.

- Cadence Training Services offers digital badges for our popular training courses.
- Your digital badge can be added to your email signature or social media platforms like LinkedIn or Facebook.

Benefits of Cadence Certified Digital Badges

- Validate expertise
  - Expand career opportunities
- Professional credibility
  - Stand apart from your peers
- For more information, go to www.cadence.com/training or email es_digitalbadge@cadence.com.

How do I register to take the exam?

- Log in to our Learning Management System, click on the course in your transcript, and go to the **Content** tab to locate the exam.

How long will it take to complete the exam?

- Most exams take 45 to 90 minutes to complete. You may retake the exam multiple times to pass the exam.

How do I access and use the digital badge?

- After you pass the exam, you get a digital badge and instructions on how to place it on social media sites.

How is the digital badge validated?

- Credly validates the digital badge as issued to you by Cadence and includes the details of the criteria you completed to earn the badge.

cadence

This page does not contain notes.

Icons Used in This Class

- Best Practice
- Language/ Command Syntax
- Concept/ Glossary
- Frequently Asked Questions/ Quiz
- Error Message
- Problem & Solution
- Quick Reference
- GUI and Command
- How To
- Lab List

Throughout this class, we use icons to draw your attention to certain kinds of information. Here are the icons we use, and what they mean.

This page does not contain notes.

Module 2

Schematic Design and Symbol Creation

This page does not contain notes.

Module Objectives

In this module, you will:

- Explain the Integrated Circuit (IC) design
- Outline the Analog IC Design flow
- Invoke the Virtuoso® Studio Design environment
- Identify specific files such as *.cdsinit*, *.cdsenv*, and the *.cadence* directory that let you customize your design environment
- Analyze the Command Interpreter Window (CIW)
- Use the Library Manager to create, copy, open, rename, and delete libraries, cells, and cell views
- Use the Virtuoso Schematic Editor XL to capture design schematics and create associated symbols using library components

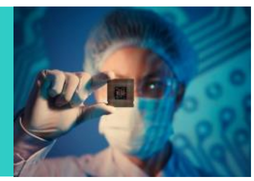
This page does not contain notes.

What Is Integrated Circuit (IC) Design?

Integrated circuit (IC) design is the process of interconnecting circuit elements to perform a specific desired function.

- IC design is at the heart of every intelligent device and supports almost every vertical, from autonomous vehicles to healthcare to space and defense.
- The IC development process starts with defining product requirements, designing, implementing, verification, signing off, and production.
- There are two main domains of IC design—**digital** and **analog**—and most real-world ICs are a mix of the two.

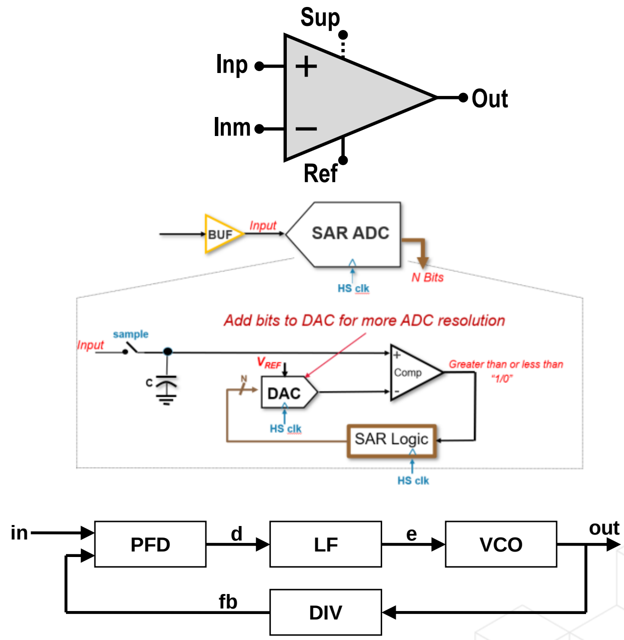
The IC design process is complex and error-prone, leading designers and manufacturers to continuously refine and optimize each stage.



This page does not contain notes.

What Is Analog IC Design and Its Applications?

Analog Integrated circuit (IC) design consists of active (diodes and transistors), and passive (resistors, inductors, capacitors) devices that use analog signals for amplification, filtering, and signal processing.



Applications of Analog ICs

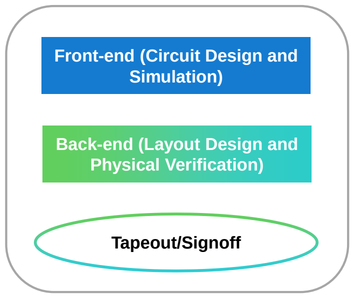
- Operational Amplifiers (Op-Amp) and Audio Amplifiers
- Linear Regulators (Voltage and Current Regulators) and Power Management circuits
- Oscillators
- Active Filters
- Data Converters (ADC, DAC)
- Phase Locked Loops (PLL)

This page does not contain notes.

How Does the Analog IC Design Flow Work?

The analog IC design flow consists of two distinct phases:

- **Circuit design**: Assembles and interconnects building blocks to implement the ICs required functionality. (Front-end)
- **Physical design**: Creates the layout of the interconnected devices using various metal layers. (Back-end)

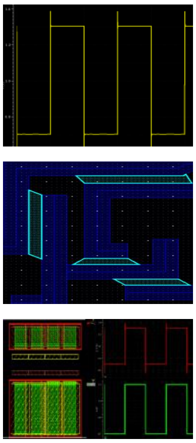


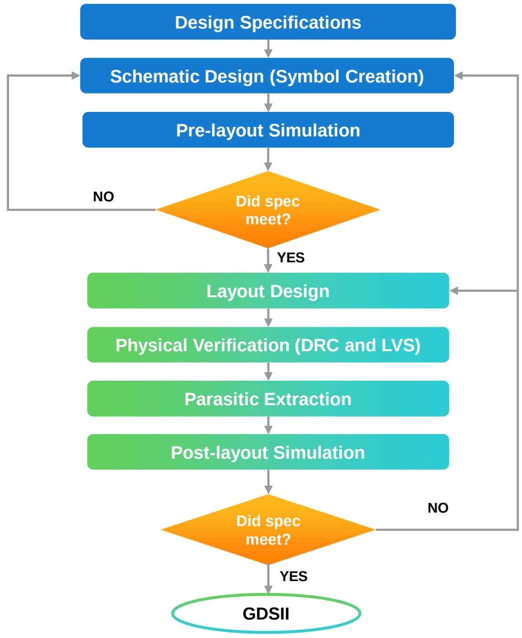
How can we effectively design analog blocks within an integrated circuit (IC)?

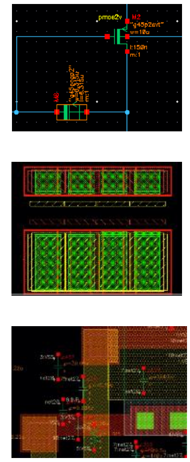
This page does not contain notes.

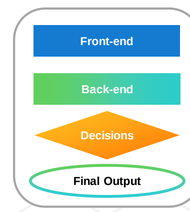
Analog IC Design Flow

PARAMETER	MIN	MAX
Gain		1000
Bandwidth	50KHz	500MHz
Supply Voltage	1.77V	5.5V
Supply Current	1.23mA	10.26mA
Propagation Delay (τ)	16.7ns	43.7ns
Chip area		$2\mu m^2$









cādence

This page does not contain notes.

What Are Design Specifications?

Design specifications are the set of requirements that describe the functionality, interface, and overall architecture of a circuit to be designed.

Design spec is the document that:

- Outlines the expected constraints that define the design's intended behavior, functionality, and performance.
  - Provides a clear understanding of what the design must achieve.
- Serves as a reference point for engineers, developers, and stakeholders throughout the design process to ensure the final product meets required standards.

Universal Amplifier IC ZT480

Principal Function: Amplification and conversion of differential input voltages to current.

Applications:

- Transducer for sensor applications
- Modular signal conditioning
- Impedance converter

Technology:

- GPDK90

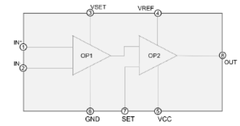
Features:

- Instrumentation amplifier with a wide input voltage range
- Output current limitation.
- Supply voltage: 6 to 35V(V_{CC})
- Reference voltage (V_{REF}) source 5 to 10V (Adjustable)
- Operating temperature range: -50°C to +90°C
- Two package variants: SOP and SSOP

Electrical Specifications:

Parameter	Symbol	Conditions	Min.	Max.	Unit
Quiescent Current	I_{CC}	T = -50 to +90°C	-	2	mA
Gain	G_A	-	4.9	5.1	
Common Mode Rejection Ratio	CMRR	-	80	-	dB
Power Supply Rejection Ratio	PSRR	-	80	-	dB
Offset Voltage	V_{OS1}	-	-	±2	mV
Input Bias current	I_B	-	-	120	nA
Output Voltage Range	V_{OUT1}	V_{CC}<10V	0	V_{CC}-5	V
		V_{CC}≥10V	0	V_{REF}	V
(Stages 1 and 2)	V_{OUT2}	V_{CC}<19V	0	V_{CC}-5	V
		V_{CC}≥19V	0	14	V
Load Capacitance	C_{L1}	-	-	250	pF
	C_{L2}	-	-	500	nF

Block Diagram:



cādence

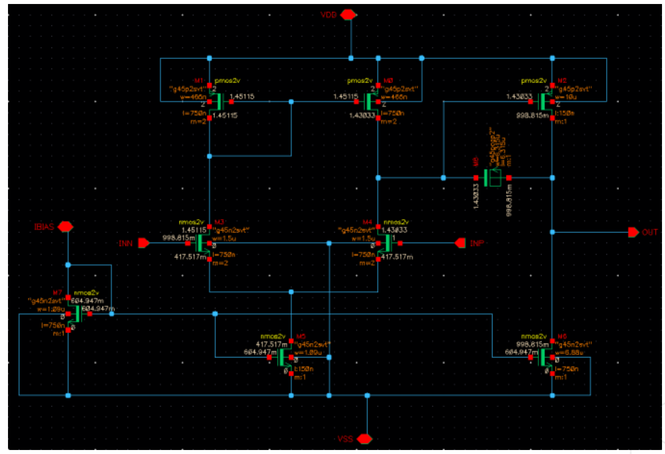
This page does not contain notes.

Example: Design Specifications

In this course, we will learn how to design a **Two-Stage Operational Amplifier** circuit.

For our project, the below are the design specifications/requirements:

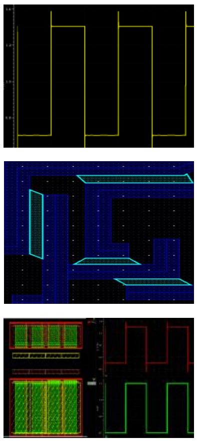
- Slew Rate should be **≥ 50 MV/s**
- DC Open-Loop Gain should be **≥ 60 dB** (1000 V/V)
- Unity-Gain Bandwidth should be **≥ 50 MHz**
- Output Offset should be **≤ ±10 mV**
- Settling Time should be **≤ 50 ns**

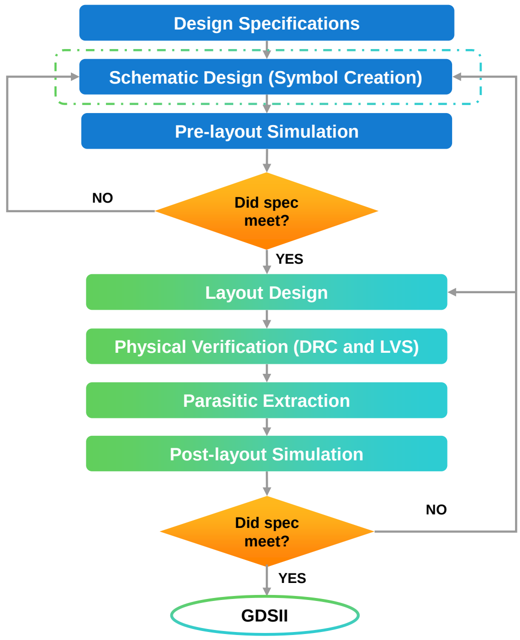


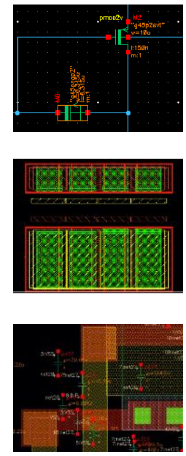
This page does not contain notes.

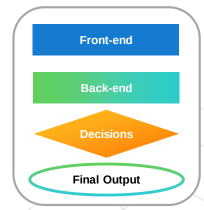
Analog IC Design Flow

PARAMETER	MIN	MAX
Gain		1000
Bandwidth	50KHz	500MHz
Supply Voltage	1.77V	5.5V
Supply Current	1.23mA	10.26mA
Propagation Delay (τ)	16.7ns	43.7ns
Chip area		2µm^2









This page does not contain notes.

Submodule 2.1

Introduction to the Virtuoso Studio Design Environment

cadence

This page does not contain notes.

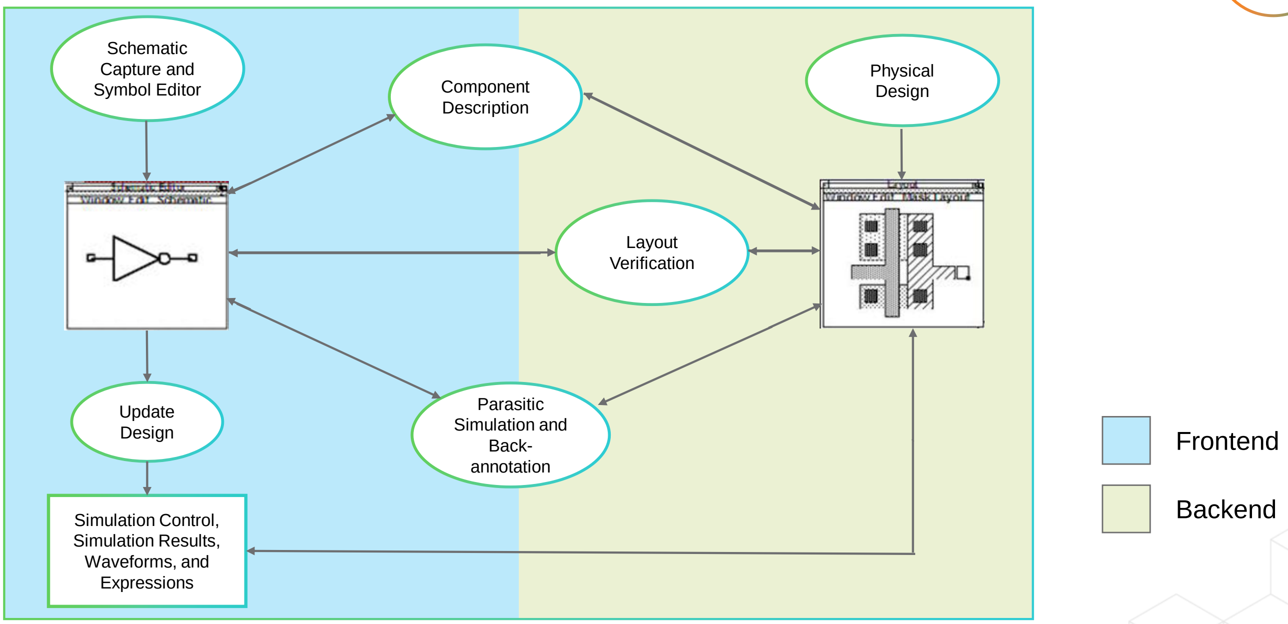
Submodule Objectives

In this submodule, you will:

- Describe the Virtuoso Studio environment and its characteristics
- Launch the Virtuoso Studio Design environment
- Identify and analyze the configuration files for the Virtuoso Studio
- Use the CIW and Library Manager windows to open libraries, cells, and cellviews in the Virtuoso Studio

This page does not contain notes.

Overview of the Virtuoso Design Environment



- On the front end, the design is captured using a schematic graphical interface and simulations are carried out to verify the design.
- On the back end, the design is drawn as it would appear on a mask for IC production. This layout is connected, routed, and verified.
- The parasitics obtained from the layout are back-annotated to the schematic and, conversely, to verify design requirements. Refinements are carried out to the schematic and layout to meet specifications.
- The Virtuoso Design Environment, with its set of tools, can assist you in this complete front-to-back flow of IC design.

Characteristics of the Virtuoso Design Environment

The Virtuoso Design Environment is the underlying structure for the Cadence® design tools for the schematic capture, analog simulation, and layout. It provides a single, integrated environment for accessing all the tools and design data, including the ability to:

- Access the CIW using **`virtuoso`**.
- Customize the CIW with the User Preferences form.
- Use the Library Manager tool to browse the design libraries and open cellviews.
- Create new libraries, cells, and cellviews.
- Start or edit a schematic view or symbol view.
- Start or edit a layout design.
- Run layout verification.
- Start the Virtuoso ADE and run simulations.
- Access the simulation results directly with the Results Browser and the Virtuoso Visualization and Analysis tool.
- Run OCEAN scripts.

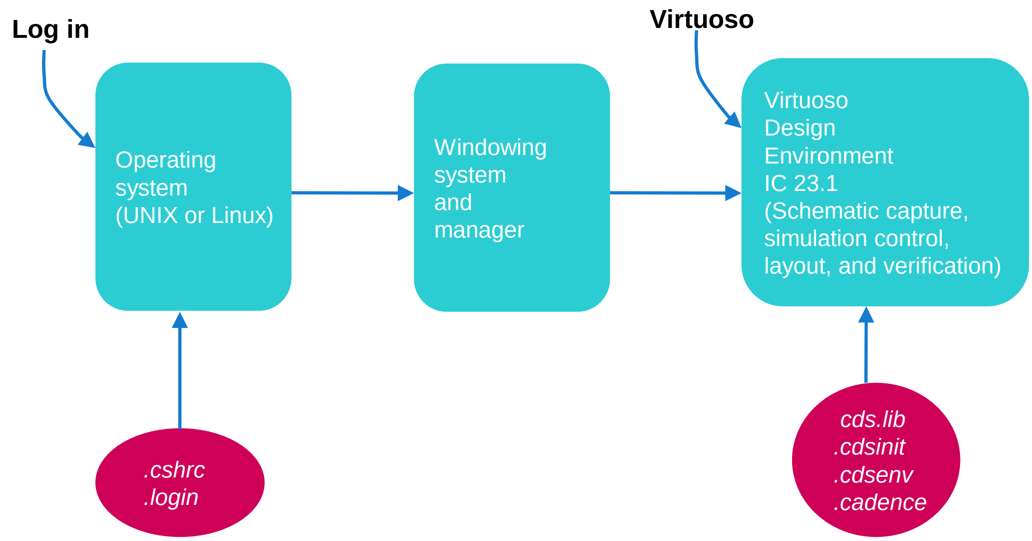
OCEAN stands for Open Command Environment for Analysis. You use a text-based process to run scripts from a UNIX shell, non-graphic terminal, or the CIW to set up, simulate, and analyze circuit data.

Advantages of the Virtuoso Design Environment

- Common software environment for using schematic capture, simulation, layout, and design verification.
- An easy-to-learn and consistent user interface.
- Similar appearance between most forms and windows.
- Communication between software tools.
- Tool windows remain open while running other applications.
- Data can be back-annotated from:
  - Layout to schematic
  - Simulation to schematic
  - Simulation to layout
- Applications can be customized or automated using the SKILL® language or the OCEAN command language.

The SKILL programming language extends and enhances functionality of Cadence tools.

Configuration Files for the Virtuoso Design Environment

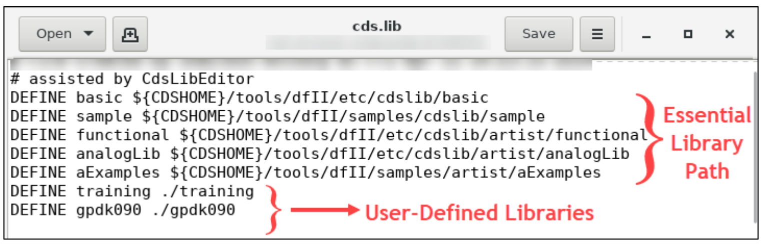


- Several system initialization files configure the operating system environment. For example, the *.cshrc* and *.login* files configure the UNIX or Linux environment when you log in and start an application.
- Other files such as *.cdsinit* customize the Virtuoso Analog Design Environment (ADE) and often load a local *.cdsenv* file, which further customizes each application. The *cds.lib* file sets the paths to the libraries used for a particular project. The user can customize all these files.

What Is *cds.lib* File?

The *cds.lib* file contains design libraries. You normally start the Virtuoso Design Environment from the directory that contains the ***cds.lib*** file.

- The *cds.lib* file contains essential library paths from the software hierarchy.
- You can include the user-defined libraries in the *cds.lib* file.
- The Library Manager lists the libraries present in the *cds.lib* file.
- The *cds.lib* file supports the following keywords: SOFTDEFINE, SOFTINCLUDE, DEFINE, UNDEFINE, ASSIGN, and UNASSIGN.



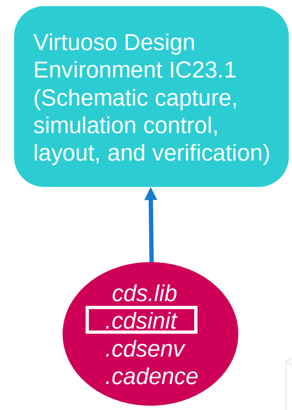
cadence

This page does not contain notes.

What Is *.cdsinit* File?

The Cadence initialization files set environment variables containing user or site SKILL code definitions. When you start the Virtuoso Design Environment, it reads the *.cdsinit* file to set up your configuration.

- The environment variables can be set by calling the schsetEnv or envSetVal in the cdsinit file.
- The *.cdsinit* file:
  - Sets user-defined bindkeys when the design environment is started.
  - Can contain the SKILL commands to customize the environment.
  - Can load a local *.cdsenv* file for customizing various tools.
- The search order prioritized for the *.cdsinit* file is:
  1. */tools/dfII/local*
  2. Current directory
  3. Home directory
- When a *.cdsinit* file is found, the search stops unless a command in the *.cdsinit* file reads other user files.



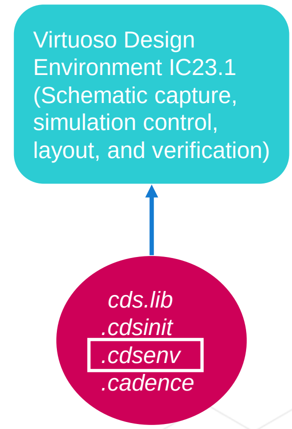
- The Virtuoso Design Environment software reads several files at startup to set up your environment: the *.cdsinit* file, the *.cdsenv* file, and the *.cadence* directory.
- Sample *.cdsinit* files are included in the installed software with examples of statements to copy to your own *.cdsinit* files.
  - A general sample is located at */tools/dfII/cdsuser/.cdsinit.*
  - An additional sample *.cdsinit* file for analog designers can be found at */tools/dfII/samples/artist/cdsinit.*
- For more information about configuring your operating system environment for the Virtuoso Design Environment, see the *Virtuoso Design Environment User Guide*.

What Is *.cdsenv* File?

The Cadence environment file sets default values for the environment variables. The ***.cdsenv*** file contains settings for the various tools in the Virtuoso suite.

The following examples show how various tools' settings can be controlled with statements in the *.cdsenv* file:

```
cdsLibManager.main showCategoriesOn boolean t
spectre.envOpts modelFiles string
     "./pdk/models/spectre/gpdk090.scs"
asimenv.startup projectDir string "./simulation"
layout snapMode string "anyAngle"
```



The search order for the *.cdsenv* file is:

1. *<IC_install_dir>/tools/dfII/etc/tools/binaryName/.cdsenv*
2. *<IC_install_dir>/tools/dfII/local/.cdsenv*
3. *as specified by the CDS_LOAD_ENV environment variable, if set; otherwise,*
   <user_home_dir>/.cdsenv

27

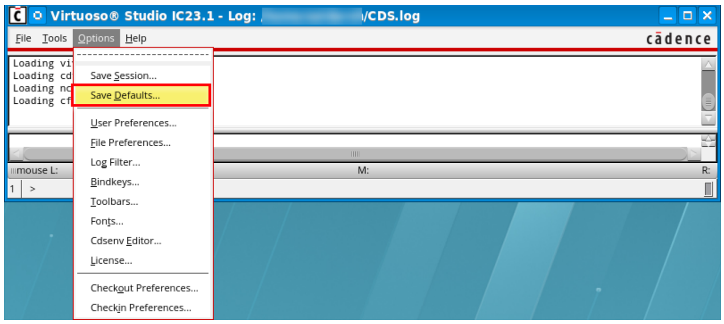
In the *.cdsenv* example file above:

- The Library Manager will show the different categories.
- The model file used for Spectre® simulations is set to **gpdk090.scs.**
- The project directory that stores simulation results is specified to be *./simulation.*
- The snap mode while drawing layouts is set to "anyangle".

Loading a Local *.cdsenv* File

- To load a *.cdsenv* file located in the current working directory (CWD) after you verify that all the files are present before *virtuoso* is started, do any one of the following:
  - Set an environment variable: **`setenv CDS_LOAD_ENV CWD`**
  - Add to the *.cdsinit* file: **`envLoadFile(".cdsenv")`**

  As each *.cdsenv* file is loaded, new values overwrite previously set values.
- A local copy of the *.cdsenv* file can be updated with any changes and saved from the CIW by choosing Options – Save Defaults.



The search order for the *.cdsenv* file is:

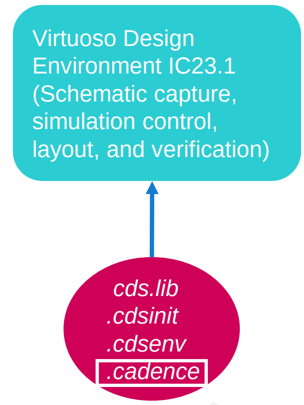
- *<IC_install_dir>/tools/dfII/etc/tools/binaryName/.cdsenv*
- *<IC_install_dir>/tools/dfII/local/.cdsenv*
- *<user_home_dir>/.cdsenv*

.

What Is the *.cadence* Directory?

The *.cadence* directory contains local customization for default setup files. This directory is created when you start the Virtuoso software. It contains the default setup files in the <IC61_InstallDir>/share/cdssetup/dfII directory.

- For each location specified in the *setup.loc* file, the program searches for the *.cadence* directories starting with the CWD, which should always be searched first.
- Your local *.cadence/dfII* directory contains the following subdirectories, which store information from your current working environment:
  - History
  - Virtuoso Visualization and Analysis
  - Workspaces



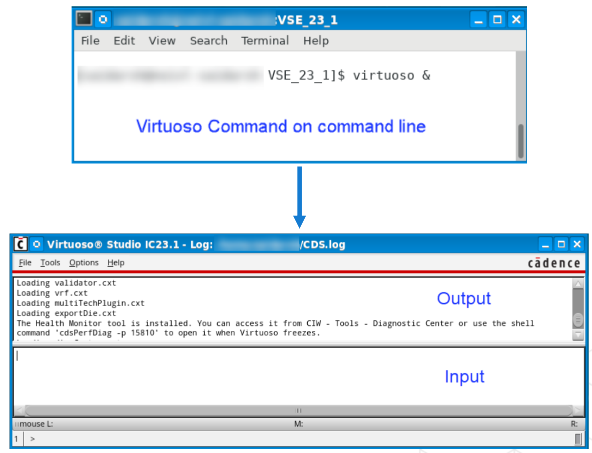
The *setup.loc* file is located in the *<IC61_InstallDir>/share/cdssetup* directory. It defines different locations to search for the *.cadence* directory starting with the CWD and then moving on to the design libraries, and so on. Open your *setup.loc* file to look at the search locations that are set.

Virtuoso Command Interpreter Window (CIW)

On the command line, enter **virtuoso**.

In the IC 23.1 software, the legacy *icfb* and *icms* workbenches are replaced by the *virtuoso* workbench.

- CIW is the starting point for the Virtuoso Design Environment.
- The auto highlighting of the SKILL keywords, strings, and constructs is provided in an expandable, multiline input area.
- Recently opened design windows are listed in the File menu.
- Warnings and error messages are color coded in the output pane.
- Command history can be displayed, and commands can be reused.



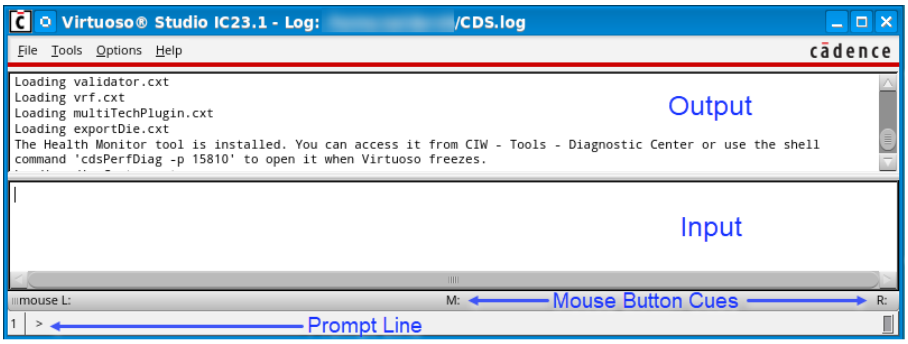
This page does not contain notes.

Virtuoso Command Interpreter Window (CIW) (continued)

- Environment variables in *.cdsenv* are available to control the raising of the CIW to the front of the screen on an error or warning message.

```
ui raiseCIWonError boolean t
ui raiseCIWonWarning boolean t
```

- The default values for both are *nil*, which means that the CIW will not be raised on an error or warning message.

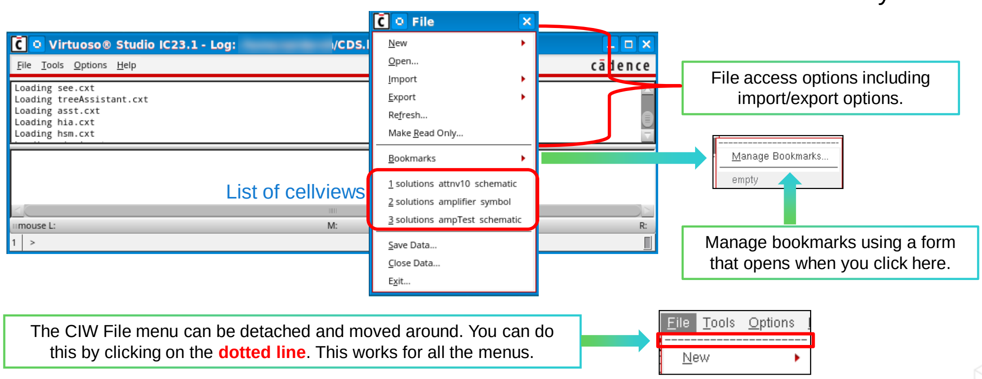


This page does not contain notes.

CIW: File Menu

CIW → File

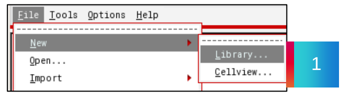
- The list of most recently opened cellviews is located in a history file and appears in the File menu, which can be “torn off” and moved to a convenient location.
- The oldest item falls off the menu when the maximum number is reached in the recently used file list.



- All the menus in the CIW can be “torn off” and viewed separately.
- The recently used file list option is available in the File Preferences form, accessed from the Options menu.

How to Create a New Library from the CIW

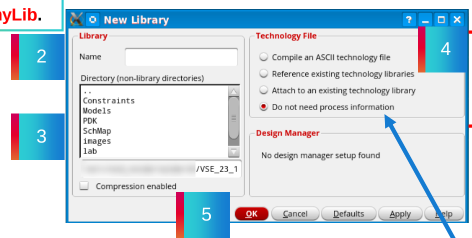
A library is a flat collection of cells. It also contains all the different views associated with each of the cells.



1. Choose **File – New – Library** from the CIW.
2. Give the new library name.
3. Choose the directory you want to create using the Directory structure accessible below the name field.
4. Select the technology file library to be attached to the new library. For physical design and verification, specify the ASCII technology file.
5. Click **OK** to add the new library to the *cds.lib* file and the Library Manager.

Give a name such as **myLib**.

If there is going to be a layout, then choose the corresponding option to attach a tech file or library.



If there are only schematics, there is no need to attach a tech file.

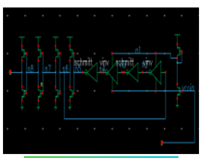
33

You can also create a new library in the Library Manager by choosing **File – New – Library.** The interface form that opens up looks a bit different than when you invoke it from the Library Manager.

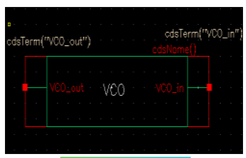
Library Manager

CIW → Tools → Library Manager

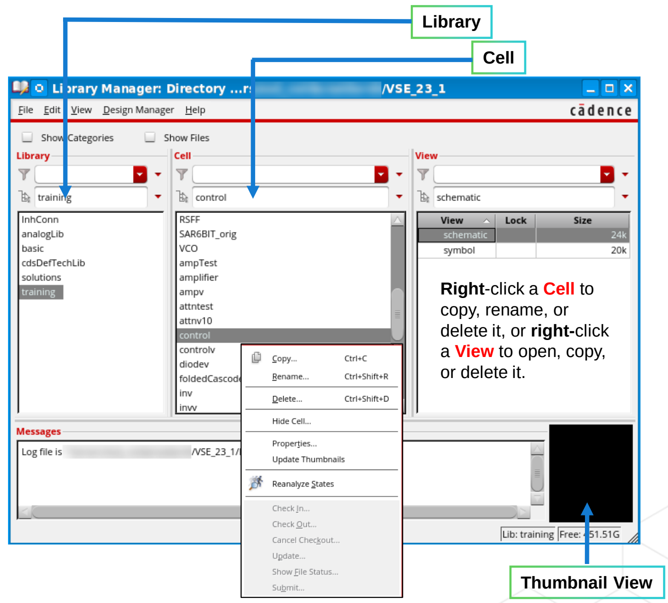
- Invoke the Library Manager to use the graphical library management tool to browse through and select cellviews from different design libraries.
- Design libraries contain cells currently being developed by a particular user or a group or for a particular design project.
- The Library Manager contains Library, Cell and View.



Schematic View



Symbol View



cādence

Library

- A library is a flat collection of cells. It also contains all the different views associated with each of the cells. Reference libraries typically contain well-characterized cells that can be instantiated in many designs. Examples are *analogLib* and *basic* libraries.
- Details of the design hierarchy exist inside the views that contain instances of other cells. The library treats all cells the same.

Cells

- A cell is a logical component in your library. It can be a building block such as a VCO or an amplifier. It can also be the top-level chip name.

Views

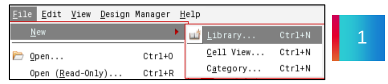
- A view is a particular representation of a cell, such as a layout, symbol, or schematic. An application tool, such as Virtuoso Schematic Editor, creates a schematic view.

You can use the Thumbnail View to look at the architecture of your schematic in a thumbnail format before you open it.

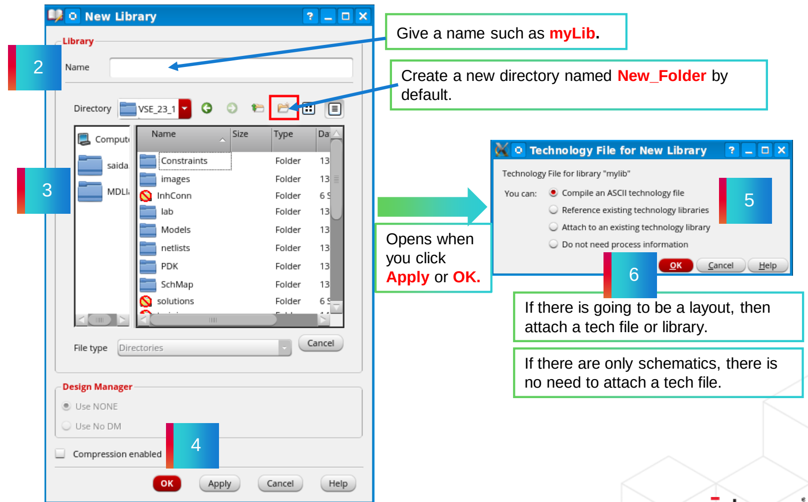
The Cell View section also illustrates the *size* of each view and also indicates whether any of the views is currently opened or locked by a system user.

How to Create a New Library from the Library Manager

Design libraries contain cells currently being developed by a particular user or group or for a particular design project.



1. Choose **File – New – Library**.
2. Give the new library a name.
3. Create a new directory for the library or select an existing one.
4. Click **OK** and select the technology file to attach to the new library.
5. For the Physical Design and Verification, specify the ASCII technology file or technology file library to be attached to the new library.
6. Click **OK** to add the new library to the *cds.lib* file and the Library Manager.



35

cadence

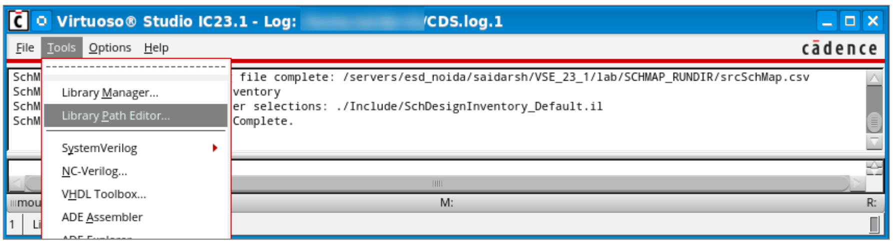
You can also create a new library in the CIW window by choosing **File – New – Library**. The interface form that opens looks a bit different than when you invoke it from the Library Manager.

The Technology File is a large data file that specifies all of the technology-dependent parameters associated with a particular library. Design rules, symbolic device definitions, and parasitic values are some of the technology-specific parameters common to all cells in a library.

How to Create a New Library from the Library Manager (continued)

- The software automatically updates the *cds.lib* file:
  - When you create a library by choosing **File – New – Library** in the CIW.
  - When you copy one library to another name.
  - When you rename a library.

This file contains the paths to all libraries used in the design session. To access it, choose **Tools – Library Path Editor** in the CIW.



This page does not contain notes.

Submodule 2.2

Schematic Design

This page does not contain notes.

Submodule Objectives

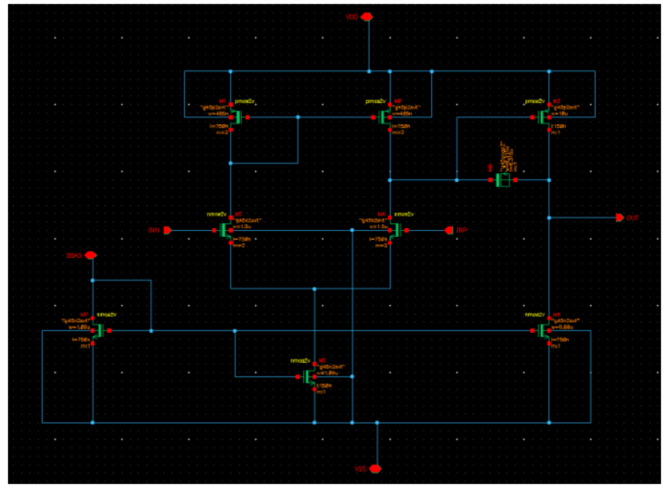
In this submodule, you will:

- Create and open a new cellview in the Virtuoso Studio Schematic Editor
- Add Instances and connect them using the wire command
- Connect and categorize pins for good design connectivity
- Edit the properties and instances
- Check and save the schematic design

This page does not contain notes.

What Is a Schematic Design?

A schematic is a graphical representation of an electrical circuit that shows the components (transistors, resistors, capacitors, etc.) and their connections.



In other words, it is a circuit diagram that uses symbols and wires to represent the functionality and connectivity of an electrical circuit.

- It is the first phase of the architectural design process that lays the groundwork for the rest of the flow.
- It is an essential stage for understanding the circuit's functionality, troubleshooting issues, and modifying the design.

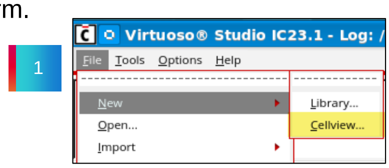
cādence

This page does not contain notes.

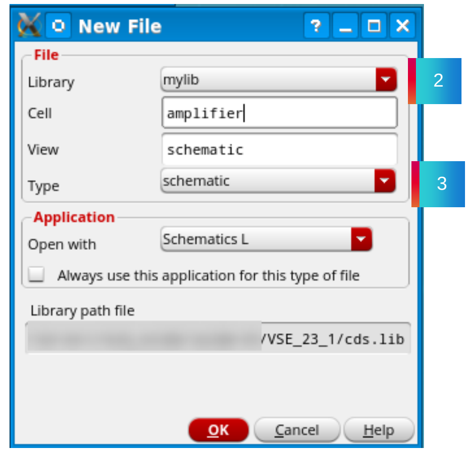
How to Create a New Cellview

The CIW or Library Manager lets you access any cellview.

1. Choose **File – New – Cell View** to display the New File form.



2. Specify the *Library name, Cell name and View name*.
   - The path to the *cds.lib* file is displayed in the form and is not editable.
3. Modify the *Type* field to create a *layout*, *verilog*, *symbol*, *schematic*, *vhdl*, or *ahdl* view.



You can choose to create a new design cellview in the **Schematics L** or **Schematics XL**. This example shows how to open a new cellview in the *Schematics L*.

40

cādence

To create a new cellview, follow these steps:

1. Create a new cellview from the *Library Manager* or *CIW.*
2. Specify the library name, cell name, view name, and tool to use.
   The path to the *cds.lib* file is displayed in the form and is not editable.
3. Modify the *Tool* field to create a *layout*, *verilog*, *symbol*, *schematic*, *vhdl*, or *ahdl* view. For a schematic, use *Schematics L*. This automatically enters *schematic* into the View name text field.

   Although *schematic* is automatically entered into the View name text field by default, you have the option to name the view anything you want. For example, the view name can be altered to *schem1*, and the process repeated for *schem2*. As such, there are two or more schematic views for the same cell name. This allows parallel circuit designs within the same cell name. A common symbol for both schematics is then used within the hierarchy until the final schematic is selected. This feature is useful in exploratory designs where the final circuit topology has not yet been finalized.

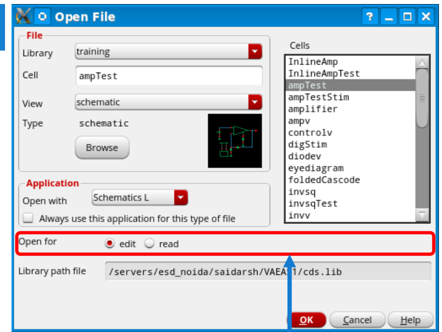
How to Open a Design Schematic

A schematic design is a graphical representation of an electronic circuit design. It can be opened from the Library Manager.

You can open a schematic in one of the four ways:

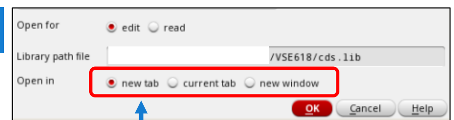
1. Select the schematic view of the cell in the Library Manager and open it either by:
   a. **Double**-clicking the mouse to open the schematic directly, or
   b. **Right**-clicking and selecting **Open**. Using Open With allows the selection of the Schematics L or XL, ADE assembler, and ADE explorer. You can also open the schematic in read-only mode.
2. Use the **File – Open** menu in the CIW and select either Schematics L or XL, ADE Explorer, or ADE Assembler to open a schematic.
3. To open another schematic from an already open Schematic L or XL, choose **File – Open**.

2



Choose between edit and read-only modes.

3



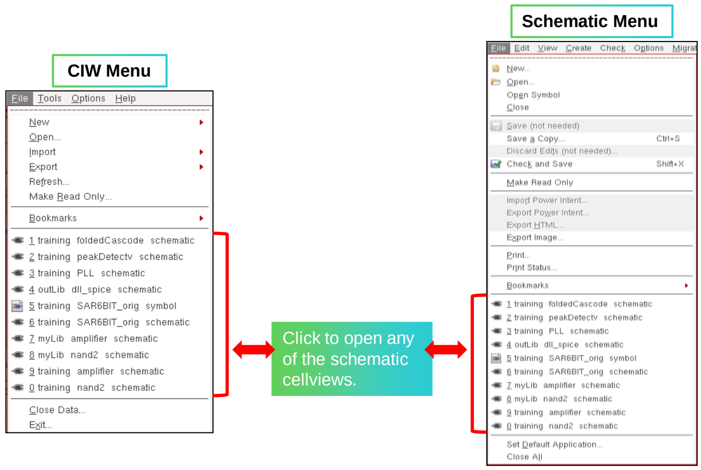
Open a tabbed schematic or open a schematic in a new window.

cadence

This page does not contain notes.

Accessing Opened Cellviews

- You can select from the most recent 10 cellviews using the File menu.
- You can do this from the CIW and an opened Virtuoso Schematic Editor window.

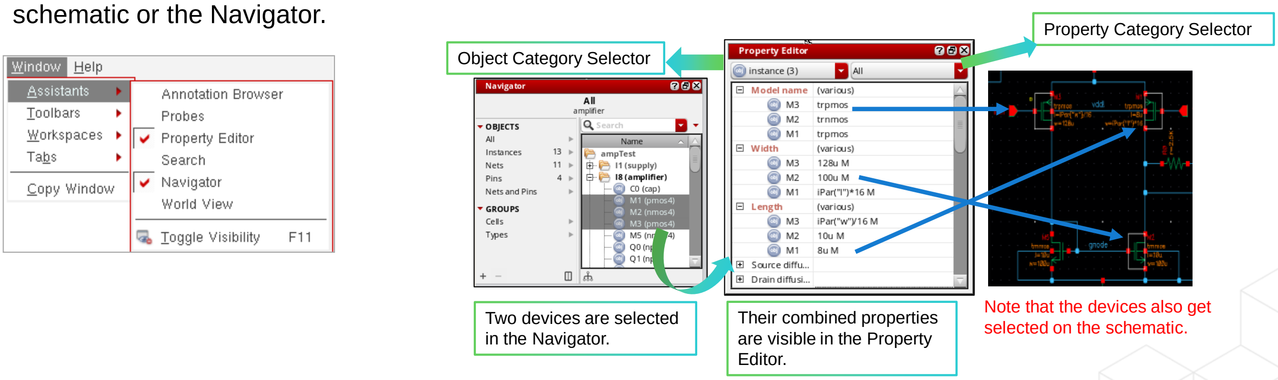


This page does not contain notes.

The Property Editor Assistant

Schematic Editor menu bar → **Window** → **Assistants** → **Property Editor**

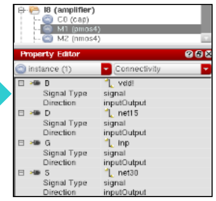
- The Property Editor assistant provides a flexible framework for defining a collection of objects and their associated properties.
- Using this, you can directly edit individual property values on single or multiple objects that you select on the schematic or the Navigator.



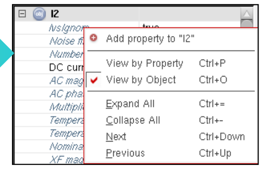
- The *Property Editor* displays information for the currently selected set of objects. If no objects are selected in the design canvas, the *Property Editor* displays the properties of the current cellview.
- The properties can be accessed using the Edit Object Properties form or the Property Editor assistant.
- Using this assistant, you can compare values across multiple selected objects. You can add or delete properties directly in the Property Editor.
- You can filter properties by object type using the Object Category Selector to include the type of object whose properties you want to view.
- You can further filter the properties you want to view using the Property Category Selector. With this, you can filter property display using the following property category options: *Attributes*, *Properties*, *CDF Parameters*, *Connectivity* or *All* properties. Choices are available in the property editor change based on the object you choose, whether a design instance or a wire or a pin.
  - Attributes: A combination of the basic and system property sections found in the Edit Object Properties form and represent the core characteristics of a selected object.
  - Properties*:* Contains details of any user-defined properties for the current objects.
  - CDF Parameters: Displays details of Component Description Format (CDF) parameters. This enables you to view and edit them as you would database attributes and properties.
  - Connectivity: Applicable only to pins and details such as *Pin Name*, *Signal Type*, and *Direction.*

Filtering and Viewing Objects in the Property Editor

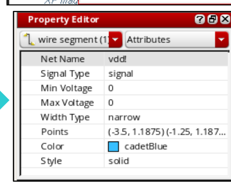
- You can choose the **Connectivity** filter to view the pin connectivity, the signal type and the direction of connectivity for any of the instances.



- To see all properties associated with the selected objects, grouped by object name, you can **right-click** on the property name and select **View by Object**.
  - The default is View by Property.



- You can change the Color and Style of the wires in the Property Editor by clicking on the field and choosing from the provided pull-down list.



- The connectivity filter is disabled if you use select nets or pins in the Navigator.
- Select **View by Object** to display information grouped by object rather than by property. This mode gathers all the properties of an object together under a single entry.

What Are Schematic Window Icons and Bindkeys?

Icons are small graphical representations of a window, file, application, command or state.

Bindkeys are used to tie commonly-used commands to single keys, key combinations, or mouse sequences. You can use the bindkeys to automate repetitive tasks and speed up your work.

- The Virtuoso Schematic Editor GUI tool interface provides both icons and bindkeys to simplify schematic capture.
- The icons appear near the top edge of the schematic editing window.
- Bindkeys speed up the flow of schematic capture.
- Bindkeys are installed with a default set of functions.
- Functions can be customized.
- A full set of programmed bindkeys can be viewed or changed using the Bindkey Editor, accessible from the CIW.

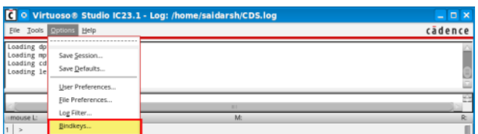
cadence

This page does not contain notes.

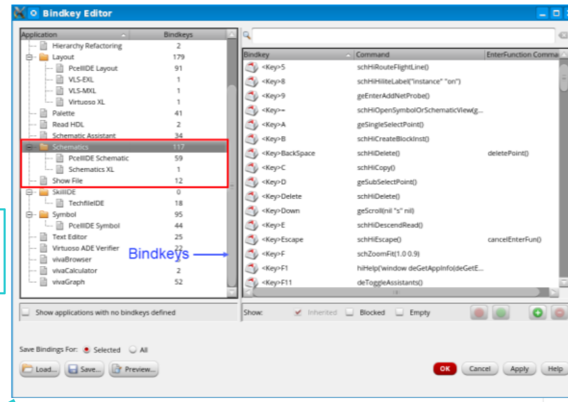
The Bindkey Editor

CIW → Options → Bindkeys

- Bindkeys tie commonly used commands to single keys, key combinations, or mouse sequences and can be used to automate repetitive tasks and speed up your work.
- Choose **Schematics** to see the available bindkeys defined using the SKILL functions.



Use the associated bindkeys using your keyboard.



cādence

For more information on Bindkey usage, please refer to the *Virtuoso Schematic Editor User Guide.*

Add Instance Form

Go to the Schematic Editor menu bar → **Create** → **Instance**, or

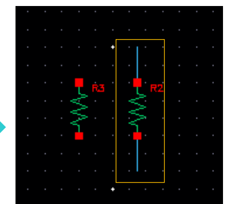
Click the icon in the Schematic Editor toolbar or

Press the **i** bindkey.

In the Add Instance form:

- Enter parameter units, such as *ohms* or *meters* (implicit based on the type of component added).
- Enter multipliers as *1k* (not 1 k) so that *k* is not mistaken as a variable.
- Add *wire stubs* for all or specified terminals.

Choose your placement orientation.



With wire stubs at terminals.

47

- Design components are generally instances of a *symbol* cellview and might be design primitives. Here are some properties associated with the design component instances:

Property	**Example**
Library Name	*analogLib*
Cell Name	*res*
View Name	*symbol*
Instance Name	*R2*

- The instance name is assigned automatically unless explicitly specified.
- Analog design primitives are in the *analogLib* library, which is included wherever the ADE software is installed in the path *<install_dir>/tools/dfII/etc/cdslib/artist*.
- Include this path in your library search path to use *analogLib* components.
- The system prompts for component parameters when instantiating the components. Attach multiplier suffixes, such as *k* for 1000, to numerical quantities.
- Use the **Rotate**, **Upside Down**, and **Sideways** buttons to change the orientation of your components as they are placed in the schematic.

Ignoring Component Instances

Schematic Editor menu bar → **Edit** → **Ignore Instances**, or

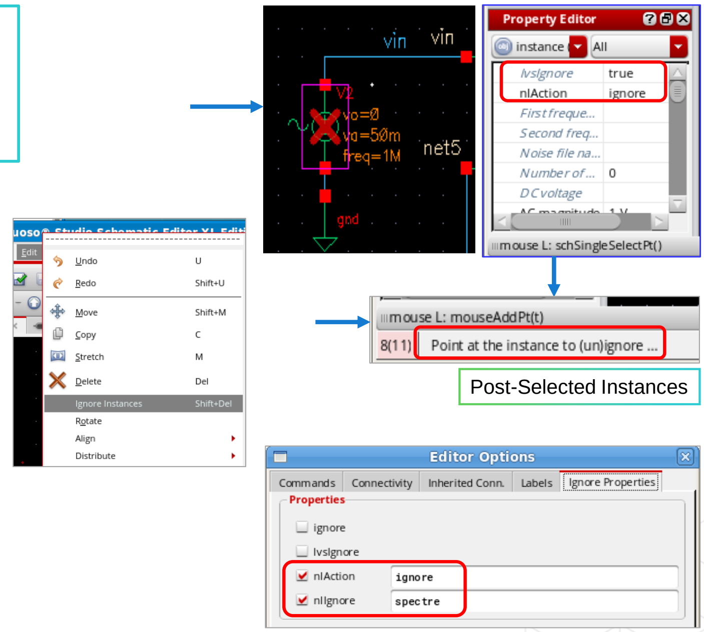
Press the **Shift+Del** bindkey.

The Property Editor indicates your action.

- For pre-selected instances, select the Instance and then choose the **Edit – Ignore Instances**.
- For post-selected instances, select **Edit – Ignore Instances** and then click on the Instance.

The Editor Options Form:

- In the Schematic menu, choose **Options – Editor**.
- The Editor Options form opens.
- You can add and remove the ignore properties from the selected instance.
- You can see the toggle in the ignored status of a given instance.



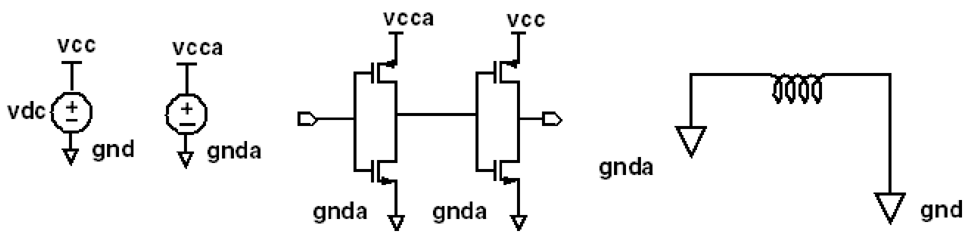
Alternatively, Ignore Properties can be found by **right-**clicking on the selected instance that you want to *ignore*.

Adding Source and Ground Instances

Sources, taps, and grounds are *instances* of cells.

Sample source cells are in the *analogLib* library.

- You can choose from independent, dependent, and piecewise linear (PWL) sources.
- You can choose tap and ground cells, which are used to establish global nets.
- An instance of the cell ground (*gnd)* is required in the design for DC convergence.



- **Ground**

  Always include the symbol *gnd* (found in the *analogLib* library). Analog simulators require that all nodes in the circuit must have a DC path ground. This is represented as *node 0* in the Spectre circuit simulator. Use other ground symbols, such as *gnda*, for a ground that is connected to the reference ground through an analog circuit.

- **Voltage Sources**

  Include all your DC and transient voltage and current sources in the schematic. There are many types of voltage sources in the *analogLib*. For example, some of the independent voltage sources are *vdc*, *vsin*, *vpulse*, *vexp*, *vpwl*, and *vpwlf*. Each source has a current equivalent that begins with the letter *i*. There are also equivalent dependent sources.

  All sources generate input waveforms, except for the *pwlf* sources, which stimulate a circuit using a text file of data tables. It is not necessary to include sources in the schematic, although this is often convenient. Attaching a stimulus file to the final netlist is addressed in the analog simulation section of this course.

- **Voltage Taps**

  Use tap symbols to transfer voltages and currents throughout the design without using wires. Voltage tap symbols, such as *vcc*, *vdd*, *vcca*, and *vccd*, are in the *analogLib* library.

Rotating and Flipping Components

Go to the Schematic Editor menu bar → **Edit** → **Rotate**, or

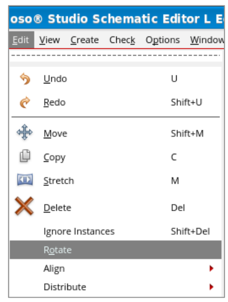
Click the icon in the Schematic Editor toolbar, or

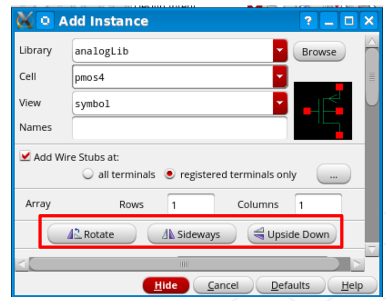
Go to the Schematic Editor menu bar → **Create** → **Instance** → **Rotate/Sideways/Upside Down**

- To flip from the Add Instance form using the Sideways and Upside Down buttons, use the right mouse button to rotate the component counter-clockwise after the *move, copy,* or *rotate* commands are used.
- Additionally, the Rotate, Sideways *flip,* and Upside Down *flip* operations are also executable in conjunction with the following operations:

I	Add Instance
C	Copy
M	Stretch
Shift-M	Move

Note: Shift+R and Control+R must be used in conjunction with the **I**, **C**, **M** or Shift+M commands.

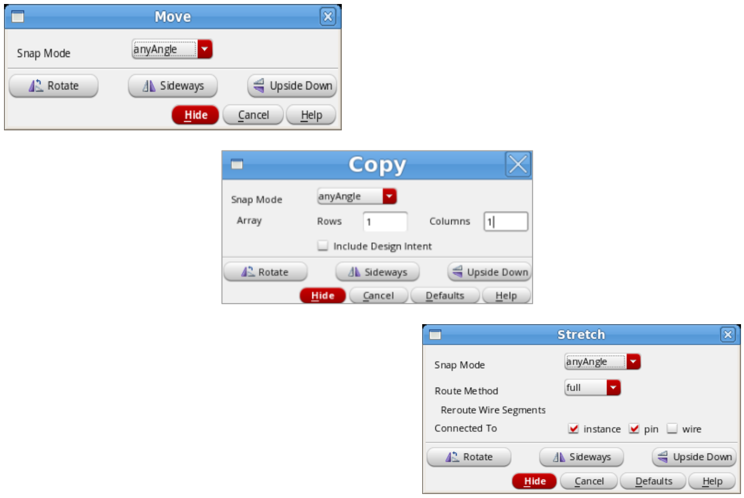




cādence

This page does not contain notes.

Rotating and Flipping Components (continued)



Options for rotating and flipping while editing objects

Examples of rotating and flipping components

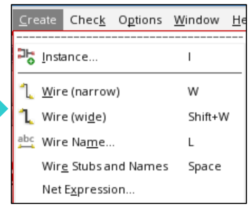
Combination	Action
▪ Shift-M, then R	Moves and rotates counterclockwise
▪ Shift-M, then Shift-R	Moves and flips sideways
▪ Shift-M, then Control-R	Moves and flips upside down
▪ M, then R	Stretches and rotates counterclockwise
▪ M, then Shift-R	Stretches and flips sideways
▪ M, then Control-R	Stretches and flips upside down
▪ C, then R	Copies and rotates counterclockwise
▪ C, then Shift-R	Copies and flips sideways
▪ C, then Control-R	Copies and flips upside down
▪ I, then R	Adds instance and rotates counterclockwise
▪ I, then Shift-R	Adds instance and flips sideways
▪ I, then Control-R	Adds instance and flips upside down

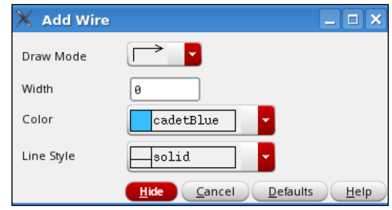
Connecting Components Using Wires

Schematic Editor menu bar → **Create** → **Wire**, or

Click the icon in the Schematic Editor toolbar, or

Press the w bindkey.

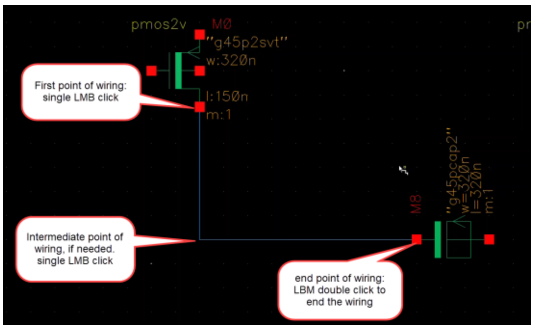




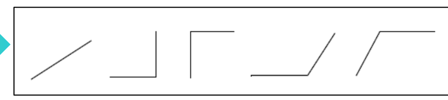
Use the **W** bindkey for narrow and **Shift+W** for wide Wire.

In the wire creation mode, select wire, then press **F3** to bring up this form, which allows you to change the **Draw Mode**, among other options.

You can also use the schematic toolbar buttons to create *Narrow* and *Wide* wires.



Use the ***middle mouse*** button to change the draw mode on the fly.



This page does not contain notes.

Creating Wire Names

Schematic Editor menu bar → **Create→ Wire Name** or

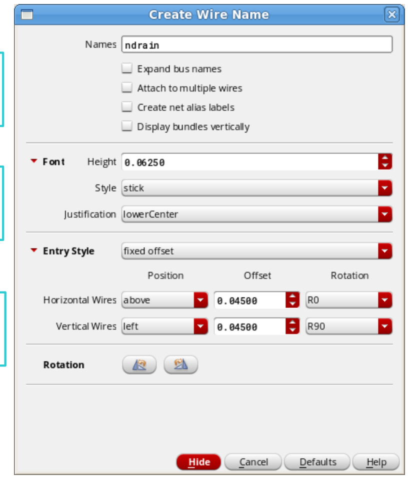
Click the icon in the Schematic Editor toolbar.

- Names specifies one or more names for wires.
- You can specify multiple names by separating each name with a space.
- You can also use a bus expression to name an array of wire names.

For creating wire names, choose **Create – Wire Name.**

Create names and change the **Font** and **Entry Style**.

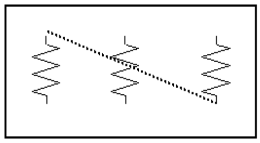
Also, change the way you would like the name to be displayed.



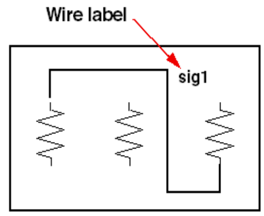
You can create narrow or wide wires in the schematic.

Wiring and Wire Labels

Full routing is the default mode.



Route entered



The system routes

When you label a wire, the entire net takes on the label name. Otherwise, the tool assigns a net name for you. If the router cannot find a path between two points, a dotted *flight line* establishes the connectivity only.

- Click on the intermediate points along the path to guide the router to yield a solid line of connectivity.
- Use the Command Options icon from the toolbar or F3 key to modify the wiring options.

Note: First, you must be in the wire mode: press **W**, then **F3**.

Draw wires between the instance pins and schematic pins to connect them. Use wide wires to indicate multiple signals on a wire; the system does not force or check this. Draw wires at any angle, but most designers frequently restrain wires to orthogonal lines.

- **Using route methodology**

  The *route draw* mode chooses two points in your design, and then it automatically routes a wire around components. If a routed net remains dotted, it is because there is no clear routing channel. This can happen if the instances are too close or overlap the selection boxes. To solve this, move the components farther apart to give a routing channel.

  Route method options exist to wire together two points immediately (the default) or indicate many points to route together later in a single step. More information on route methods is included in the design entry reference documentation.

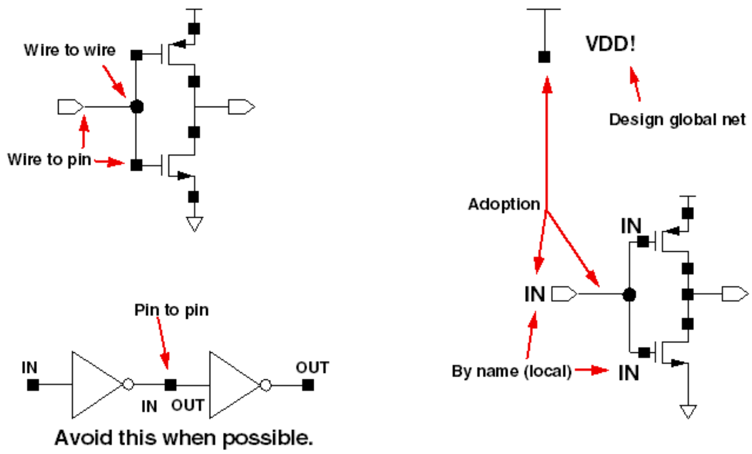
- **Wire labels**

  Labeling wires gives the corresponding net a meaningful name in the simulation results data. Otherwise, nets are system named. There is some control over the automatically generated names, but these might not be as meaningful as custom names.

  Click the **Command Options** icon in the schematic window or press the **F3** key to change the default wiring setup.

Interconnecting Components

Schematic pins and global symbol pins force wires to adopt their names.



- **Physical connectivity**

  All physical connections are made by wire-to-pin, wire-to-wire, or pin-to-pin connections.

- **Connectivity by name**

  If two wires are labeled with the same name, they become part of the same net when connectivity is established.

- **System-assigned names**

  If a net is unnamed, the system generates a name such as *net100* or *net7*. Optionally, change the base name from *net* to something else. If a wire is connected to a schematic pin, then the pin name is used to name the net by adoption when connectivity is established.

- **Global nets**

  Any net or pin name that ends in an exclamation point will be part of a global net when connectivity is established. Global nets are automatically connected through the hierarchy without the use of wires. For example, voltage taps have symbol pin names that end in an exclamation point (!). If a wire is connected to a pin that has a global name, the pin name is used to name the net by adoption. This is how voltage and ground signals are propagated throughout a design.

Note: A net named *net!* is not connected to a net named *net*.

Create Pin Form

Schematic Editor menu bar → **Create** → **Pin**, or

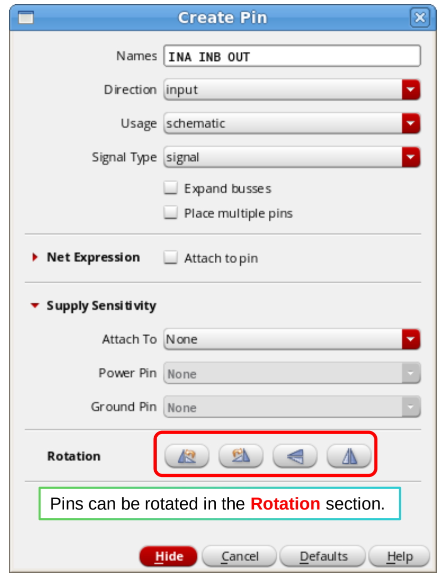
Click the icon in the Schematic Editor toolbar, or

Press the **p** bindkey.

Pins have a user-defined Pin Name and a Direction (*input, output, switch, inputOutput, jumper, and tristate*).

- **Rotate Left**: Turning a pin in 90-degree increments counterclockwise with respect to the reference point.
- **Rotate Right**: Turning a pin in 90-degree increments clockwise with respect to the reference point.
- **Flip Vertical**: Mirrors the pin across an imaginary vertical line.
- **Flip Horizontal**: Mirrors the pin across an imaginary horizontal line.

The Supply sensitivity section determines the power and ground terminal values that are connected to pin terminals.



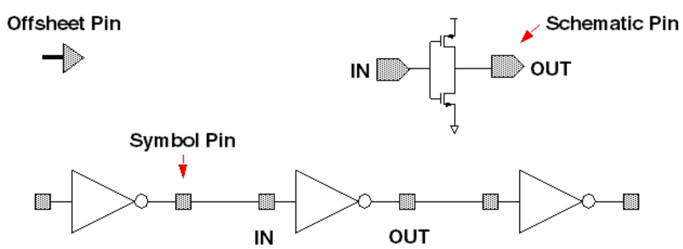
cādence

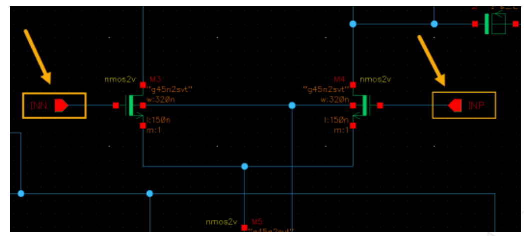
Expand Busses and Multiple Pin Options

- The **Expand busses** option determines how bus names are interpreted.
- By default, it is not checked on, and the tool interprets a bus name as a single name and generates a single pin that includes all the bits in the bus range.
- Enabling the **Place multiple pins** option allows you to place multiple pins automatically in one operation.

Categorizing Pins for Connectivity

- Pins can be categorized as follows:
  - Schematic pins provide ports to a schematic.
  - Symbol pins provide ports to a symbol representing a schematic and are connection points to the symbol in a hierarchical design.
  - Offsheet pins are used in large designs without hierarchy.
- Use the Create Pin form to create pins.
- Pin names and directions must match all cellviews of a cell.





Accessed using a toolbar button or using **Create – Pin** from the menu.

Functions

For analog designers, pins have two primary functions:

- Pins represent connection points between different cellviews, such as schematic, symbol, and layout representations. Using named pins identifies equivalent input, output, and I/O ports throughout the design environment.
- Pins provide connection points for objects that are hierarchically instantiated.

Pin properties

Pins have a pin name, pin type, and pin direction. These should be consistent throughout your design.

Multiple-sheet design with offsheet pins

The *Virtuoso Schematic Editor L User Guide* includes a section on multiple-sheet design methodology and information on the *offsheet* pin type. For more information about the Virtuoso Schematic Editor software, see Cadence® Help.

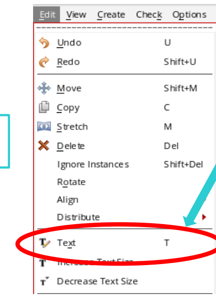
Pins (*ipin*, *opin*, *iopin*, *sympin*) now come from the basic library.

Note: Pin type *switch* is maintained only for backward compatibility. Its use is discouraged.

Using Direct Text Edit

Schematic Editor menu bar → **Edit→ text**

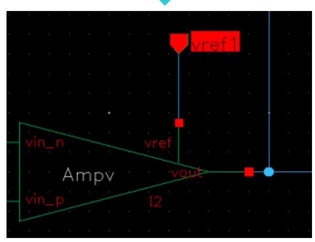
Choose **Edit – Text** and click on the text of Schematic.



You can directly edit the text on:

- Instance Names, Instance Labels and Instance Parameters
- Wire Labels
- Pin Names
- Multiline Notes

You can select and change text directly in the schematic canvas itself.



- You can also use the **right** mouse button to pop-up a form with the command.
- **Double**-click on the text or parameter to start the Direct Text edit. Click on the text you want to change. It will be highlighted. Enter the new text. You can change *cdsParam()* elements without using the Edit Object Properties form.

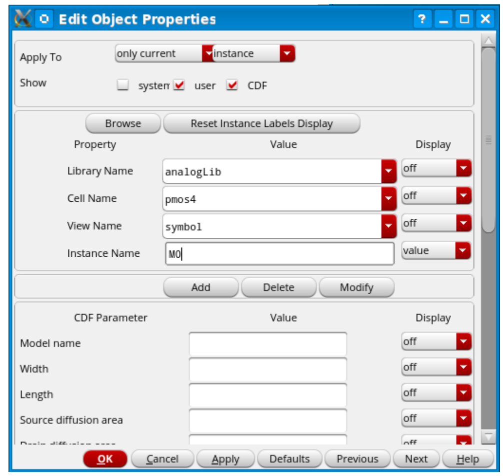
Edit Object Properties

Schematic Editor menu bar → **Edit→ Properties → Objects**

Press the **q** bindkey.

In the Edit Object Properties form:

- You can view the cell properties.
- You can view CDF parameters and set their values.
- You can choose how you want the instance properties to be displayed.
- You can select multiple objects to edit, and then invoke this form. The Next and Previous buttons highlight single objects in a selected set.

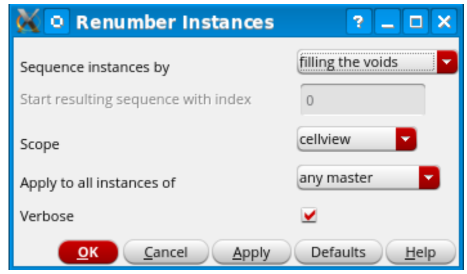


Here, the schematic is chosen that contains a *pmos4* cell, and this cell is selected to the schematic from the *analogLib* library.

- You can update either single or multiple objects in a selected set. Use **Next** and **Previous** on the Edit Object Properties form to scroll through the selected objects and update them. One object at a time is highlighted in the schematic window. The most common changes concern component parameters, pin name, and pin direction.
- Use the **Stretch**, **Copy**, **Move**, **Delete**, and **Rotate** commands to update your design. These commands are located under the **Edit** menu in the schematic window. To display an options form that is associated with any of these commands, use the **Cmd Options** icon or press the **F3** key while these commands are active.
- To select a component, wire, pin, or label:
  1. Edit the component or property with the middle mouse button pop-up menu. Use the **q** bindkey or choose **Edit – Property**.
  2. Delete the component by selecting the target and pressing the **Del** key.
  3. Replace the component with another cell with the property editor or by deleting and replacing.
- To select a component or label to be moved:
  1. **Single**-click with the left mouse button, and then drag.
  2. Choose a menu command, such as **Edit – Select – By Property**.
  3. Use implied selection. This occurs when an object is highlighted as a result of being near the cursor.

Renumbering Instances

Schematic Editor menu bar → **Edit → Renumber Instances**



The renumber sequence depends on the order in which the symbols were added to the schematic.

You can set up the following options:

- Sequence instances: Fill the voids, X+Y+, X+Y, and so forth.
- Start resulting sequence with index: (the number you choose for the starting point).
- Scope: Cellview, hierarchy, library.
- Apply to all instances: same master, any master.
- Verbose: When enabled, reports to the CIW log the old and new numbering for each renumbering sequence.

This page does not contain notes.

How to Select Multiple Instances

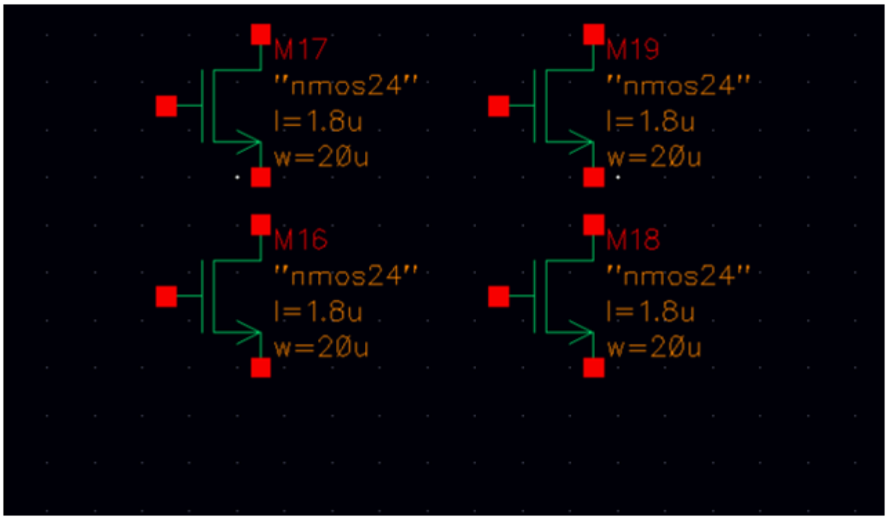
Selection of several consecutive components, pins and wires can be done in the schematic editor.

Multiple components, pins, and wires can be:

- **Selected in a single motion** by using the left mouse button to drag a selection box over the elements.
- **Selected or added to the existing selection** by holding down the **Shift** key and **left**-clicking.
- **Deselected** by holding down **Control** and **left**-clicking or using the **Control** and **D** bindkey.

This page does not contain notes.

Selecting Multiple Instances

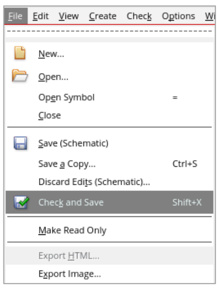


This page does not contain notes.

What Is Schematic Checking?

Schematic checking is a critical step in the design process. You can either check a single cellview or descend through the hierarchy to check all the cellviews in your design.

It is imperative to use **File – Check and Save** to save your schematic and ensure that you have followed design creation rules.



Performing a schematic check accomplishes the following:

- **Updates connectivity**: When connectivity is established, wires and pins in the design entry window become associated with logical connections called *nets*.
  - It is necessary to correct connectivity problems prior to going on to the next design phase.
- **Schematic rules checks**: This process checks the schematic with a set of rules. You can access the rules from the Schematic Rules Checks Setup form. Among others, the checks include:
  - Logical checks, such as *floating input pins* and *shorted output pins*.
  - Physical checks, such as *unconnected wires* and *overlapping instances.*
  - Name checks, such as *instance name syntax.*
- **Cross-view checks**: This option checks the pin consistency between different views of the cell. Pin names and directions must match between cellviews.

In **IC6.1.8**, You must update the connectivity of the design explicitly by running one of these check commands: *File – Check and Save*, *Check – Current Cellview*, or *Check – Hierarchy.* This is the batch connectivity mode.

In **ICADVM20.1**, there are two modes in which the connectivity of a design can be updated: incremental and batch. The default is incremental connectivity mode, which means that the connectivity in your design is updated automatically as you create and edit the design.

The current status of incremental connectivity mode is indicated on the status bar at the bottom of the schematic window.

- *On* means that incremental connectivity is enabled.
- *Off* means that incremental connectivity is disabled.
- *Paused* means that incremental connectivity has been paused. The status of incremental connectivity mode changes to *Paused* when a batch connectivity check is required.

Submodule 2.3

Creating Symbol from a Schematic

This page does not contain notes.

Submodule Objectives

In this submodule, you will:

- Analyze the limitations of transistor level schematic
- Create a symbol from the schematic design

This page does not contain notes.

Terms and Definitions

Primitive	A component or cellview, such as a resistor, capacitor, or transistor, that has no hierarchy below the current level.
Flat Schematic	A schematic containing only primitive components.
Hierarchy	A design using structure or design data at multiple levels.
Symbol	A cellview used in a schematic to replace design data or structure, such as a schematic cellview or behavior.
ASG	Automatic Symbol Generator, a software tool that generates a symbol from the schematic.

This page does not contain notes.

Limitations of a Flat Schematic

- The entire design must be captured with primitives.
- Repeated structures must be captured for every occurrence.
- Large designs become inefficient or impractical.
- Large designs can require enormous resources, such as data storage.
- Edits must be applied to every instance of the repeated structure.
- It becomes difficult to locate or correct design problems.

- A flat schematic becomes inefficient as more and more components are added to the schematic structure. The design of repeated structures becomes inefficient because each occurrence must be captured separately. Likewise, any edit to a repeated structure must be repeated over and over again. If the flat schematic has a large number of devices, then it becomes difficult and time-consuming to locate a specific device to make a correction to the design.
- A complex system, such as a microprocessor, a cell phone, a long shift register, or a frequency synthesizer, becomes difficult to conceptualize at the transistor level. It is easier to design such a system in blocks that represent transistor-level schematics.
- The block-level schematic requires a symbol to represent each block of transistor-level schematics. The symbols are then interconnected at the next level of the hierarchy.

Advantages of Using a Hierarchy

- A symbol can represent a simple or complex schematic.
  - A single schematic design can be used for repeated cells.
  - Edits or design corrections to repeated structures require only a single correction, which propagates through all structures.
- A block-level diagram can represent an entire system, a subsystem, a small circuit, or a repeated structure.
- Team members can be assigned blocks that can be captured independently.

This page does not contain notes.

Design Requirements of Symbols in a Hierarchy

Each symbol must:

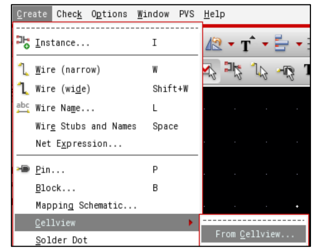
- Properly reference a component, schematic, or behavioral view.
- Correctly pass all connectivity information in and out.
- Have symbol pins that correspond correctly to the pins and their associated names on the schematic view.
- Reference a cellview that has been checked and saved.
- Be checked and saved as a symbol view.
- Have parameters that contain valid expressions or values where the symbol is instantiated.

- Each symbol must properly reference a component, schematic, or behavioral description. The symbol must correctly pass connectivity information between signals applied to the symbol pins in a schematic to the subsequent level the symbol references. To realize this requirement, the symbol must have an exact corresponding relationship between the pins on the symbol and the pins of the schematic of the same cellview. The pins must have the same name and signal direction.
- If the symbol references a schematic view, the schematic must be checked and saved. If the symbol references a behavioral view, the behavioral view must be checked and saved. Likewise, the symbol view must be checked and saved.
- If a symbol references a schematic having instances that require parameter values, the instance of that symbol must contain valid expressions or values at the next level.
- The term cell bindings refer to the special software which recognizes that the symbol has actually replaced the schematic (or other cellview) within the hierarchy. This software operates in the background and generates these “invisible” bindings.

Caution: Often, when an existing symbol is deleted and replaced by a new symbol, the cell bindings are no longer valid. Yet these invisible bindings still exist. Run another check and save to update the bindings.

Symbol Generation

Schematic Editor menu bar → **Create** → **Cellview** → **From Cellview**



In ASG, a template file is used for symbol creation. There are different symbol template files for different tools in the Virtuoso® Design Environment.

To make sure the *analog* symbol generation template is used in your design, put the following command in your *.cdsinit* file:

```
schSetEnv( "tsgTemplateType" ")
```

Note: Failure to set this results in `"analog"` a digital symbol generation.

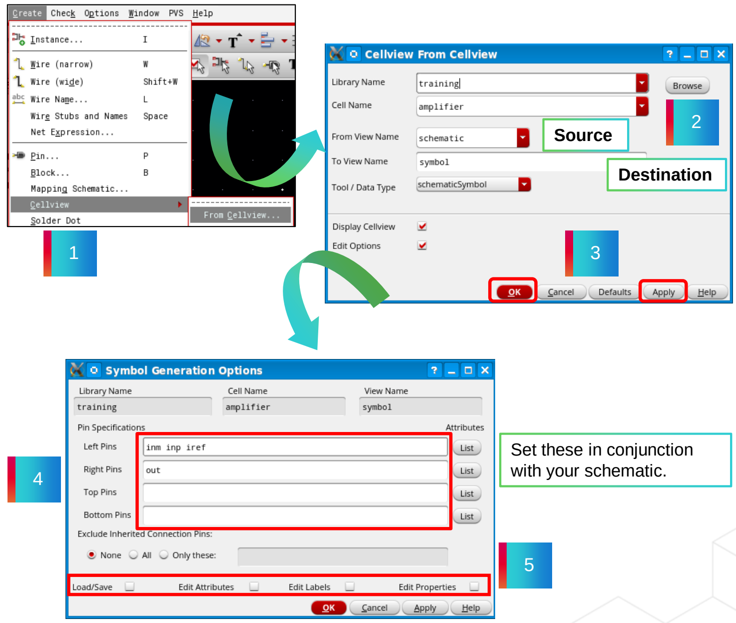
Notice that the From View Name and To View Name fields can be modified in the Cellview from Cellview form. This provides a way to create other views, such as *behavioral* or *ahdl* from your schematics.

This page does not contain notes.

How to Generate a Symbol

A symbol can be created automatically from a schematic. Automatic cellview creation can be useful for both top-down and bottom-up design approaches.

1. Choose **Create – Cellview – From Cellview**.
2. Establish the source and destination views.
3. Click **Apply** or **OK** to open the Symbol Generation Options form.
4. Designate pin locations.
5. Enable additional options.

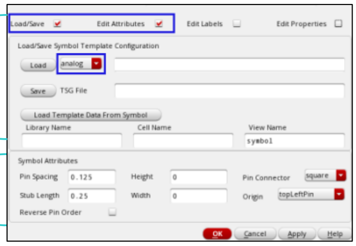


Notice that the From View Name and To View Name fields can be modified in the Cellview from Cellview form. This provides a way to create other views, such as behavioral or ahdl, from your schematics.

Symbol Generation Options: Enabling Options

Load/Save

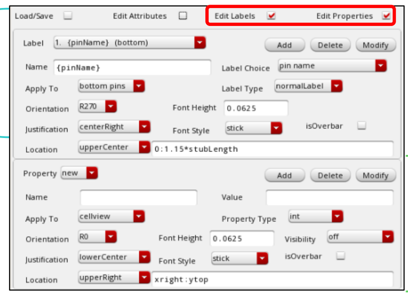
Edit Attributes



Use the checkboxes as indicated here, on the Symbol Generation Options form to view additional options.

Ensure that the analog template is used.

Edit Labels



Edit Properties

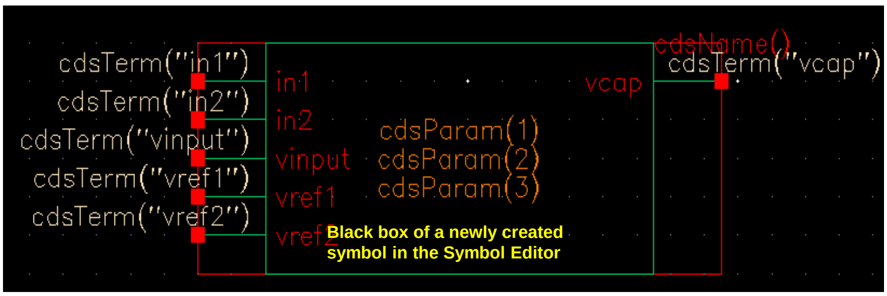
This page does not contain notes.

Characteristics of Symbols

Interpreted labels on the symbol act as placeholders for different types of information to be displayed in the schematic.

- cdsTerm() labels display pin names or the net names.
- cdsParam() labels display parameters of an instance.
- cdsName() labels display the instance or cell name.

Symbols are created and visible in the **Virtuoso Symbol Editor**.



- An automatically generated symbol cellview includes pins, a rectangular graphic, and labels. It can be modified using the symbol editor.

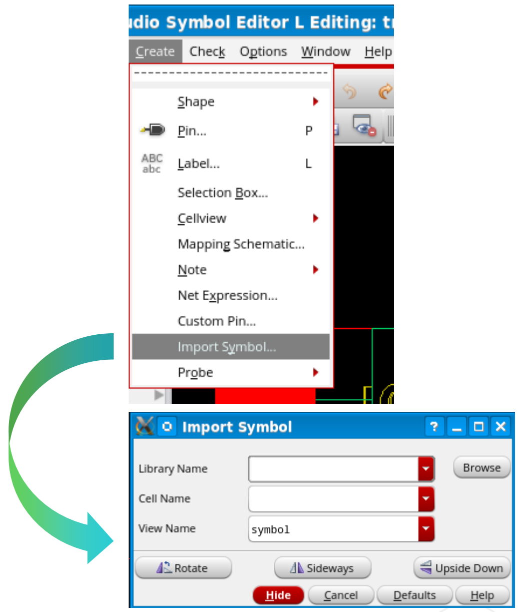
  Note: The generation depends on the symbol template selected either in the *.cdsinit* file or by the Symbol Generation form.
- There is some control over how automatically generated symbols are drawn. By default, pins are placed at the left side of the symbol if their directions are *input*, at the right side of the symbol if their directions are *output*, and on top of the symbol if the directions are *InputOutput*. Options exist to change the pin appearance and order of the pins.
- *Interpreted Labels* allows information to appear near the symbol after it is placed in another cellview. For the label generation template, there are three label types:
  - *cdsTerm()* labels display pin names or the net names pins connect to.
  - Each *cdsParam()* label can display a parameter of the instance. There can be multiples of this label.
  - *cdsName()* labels display the instance or cell name.
- All placeholder labels can be rearranged so that labels on instantiated instances are moved accordingly. These three labels have meaning in the Analog Design Environment (ADE). They display certain attributes, which are covered later.
- A selection box is drawn around the symbol and can be edited. It defines the symbol's selectable region after it is placed in another cellview.

Importing a Symbol from a Different Schematic

Schematic Editor menu bar → **Create** → **Import Symbol**

A design *Symbol* of a schematic capture can be imported from any library in the Symbol Editor.

- In the Import Symbol form, choose pull-down menus or the **Browse** button to set the Lib:Cell:ViewName of the symbol you want to import.



This page does not contain notes.

References

Virtuoso Schematic Editor full course reference

- Virtuoso Schematic Editor Training

Virtuoso Schematic Editor Training Bytes

- Virtuoso Schematic Editor Video Channel

Virtuoso Schematic Editor User Manual

- Virtuoso Schematic Editor User Guide

These links are accessible from the Reference section of the course interface.

cadence

This page does not contain notes.

Module Summary

In this module, you learned how to

- Invoke the Virtuoso Studio Design environment
- Use specific files such as *.cdsinit*, *.cdsenv* and the *.cadence* directory that let you customize your design environment
- Access the Command Interpreter Window (CIW)
- Open the Library Manager to create, copy, open, rename, and delete libraries, cells, and cell views
- Use the Virtuoso Schematic Editor L and XL to capture design schematics and create associated symbols using library components

This page does not contain notes.

Labs

Lab 2-1 Setting Up the *cds.lib* File for Virtuoso

Lab 2-2 Creating a New Library and Cell View

Lab 2-3 Designing a Schematic Using the Virtuoso Studio Schematic Editor

Lab 2-4 Creating a Symbol

This page does not contain notes.

Module 3

Schematic Testbench and Modifying DUT Parameters

This page does not contain notes.

Module Objectives

In this module, you will:

- Identify the importance of creating a testbench circuit
- Create a testbench circuit
- Edit a hierarchical design to modify DUT parameters

cadence

This page does not contain notes.

Terms and Definitions

Hierarchy	A design in which symbols represent schematics or behavioral views. The design has data that exists at different levels.
Descend Editing	Pushing into a symbol for editing and seeing only the contents of the symbol.
Editing in Place	Pushing into a symbol for editing while still seeing the schematic visible at the parent level of hierarchy.

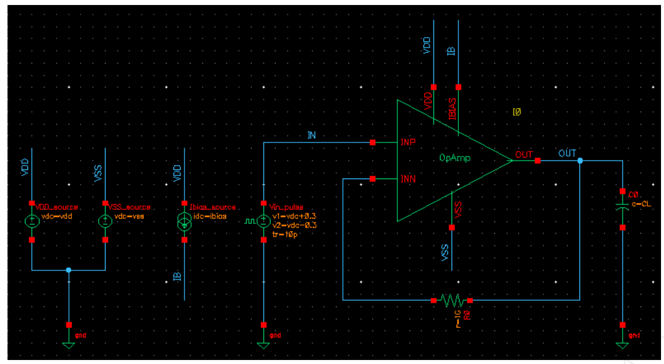
This page does not contain notes.

What Is a Testbench Circuit?

A testbench circuit is a circuit that is verified to determine whether it will perform according to the given specifications.

A testbench is a circuit or a design that validates the functionality of analog/mixed-signal designs.

- Here, we need to inject stimulus into the design and collect the results.
  - To inject stimuli, we need components like sine/square wave input (vpulse), DC sources (VDC, IDC), ground, etc.
- One of the purposes of creating a symbol for a schematic is to use it in a testbench.



This page does not contain notes.

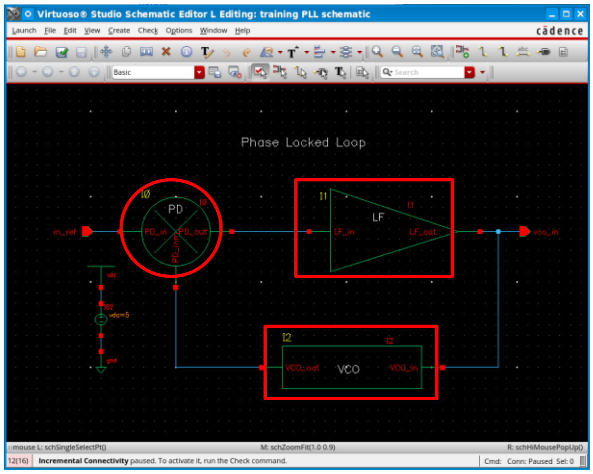
Overview of Designing a Hierarchy

- You can create a symbolic representation of the design.
- A symbol can contain a primitive component, a schematic, or a behavioral view.
- The symbol can be repeated on all levels.
- A single symbol can represent a large number of components.
- Alter an existing hierarchy with the Hierarchical Editor.

- A symbol or a set of symbols can represent the design schematic. The design structure can exist at different levels, which allows the design to be easily conceptualized.
- Rather than having one enormous impractical schematic using primitives for the entire design, the design appears as functional blocks at different levels.

Example of a Hierarchical Design

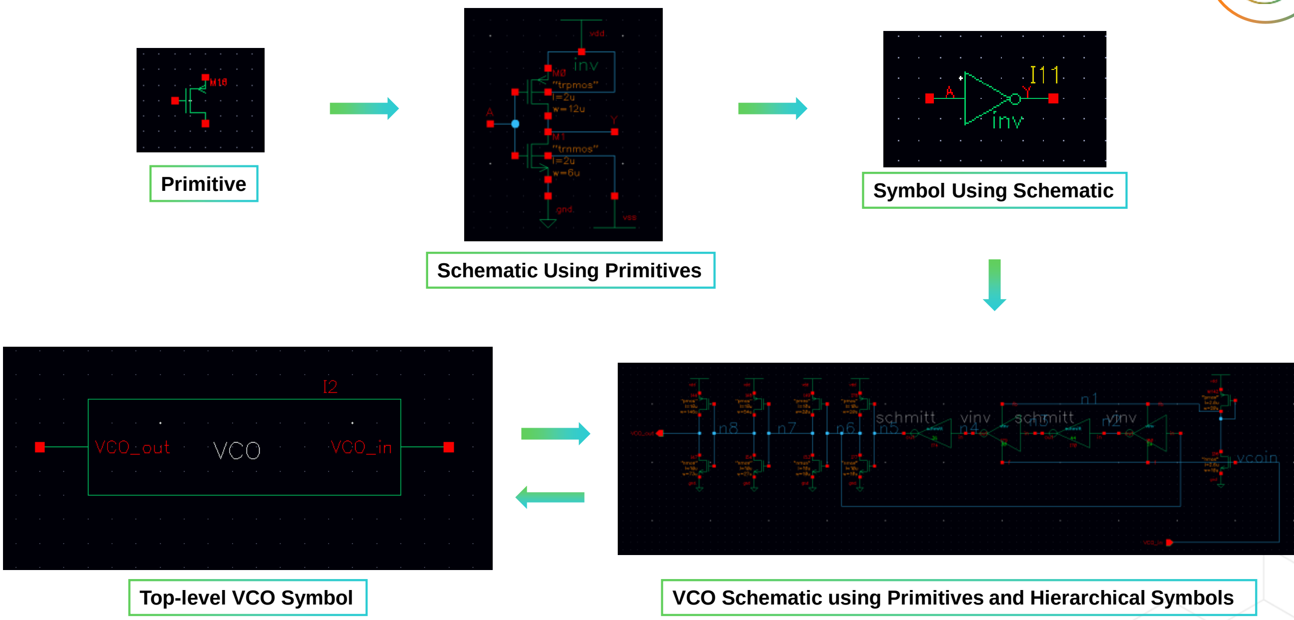
- Here is an example of a top-level view for a design that relies heavily on hierarchical design concepts.
- You have a clear view of the system, and you can descend into specific subcircuits to view or edit the internal circuits.



Schematic PLL with three major blocks, each referencing a different cellview.

The hierarchical cellviews can be schematics or Verilog-A definitions.

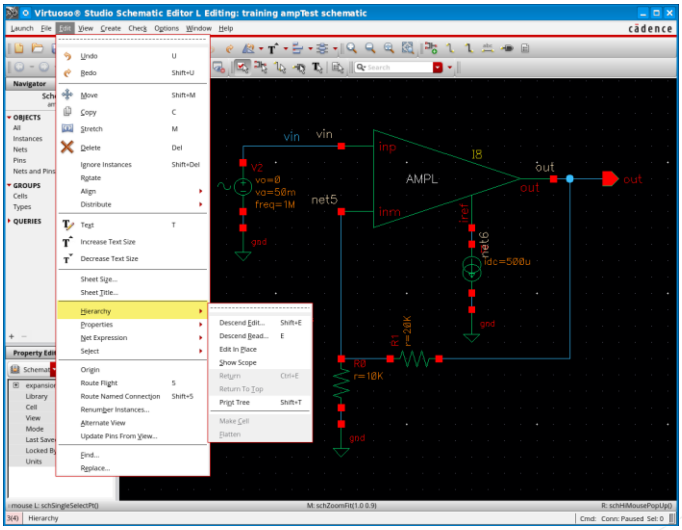
Basics of Schematic Hierarchy



- A hierarchy is the design data of a complex system organized into simple and manageable data at different levels. A hierarchy simplifies the complex structure of a system. It often reduces the storage requirements for the data. It also simplifies and reduces the time to design the system.
- Primitives are the basic design elements and exist at the bottom of the hierarchy. The design does not descend below this level. A schematic can consist entirely of primitives. Such a schematic is also referred to as a flat or *primitive-level schematic*. For large systems, for example, using a 16-bit analog-to-digital converter, it is difficult to capture the design with a flat schematic.
- A design of a complex system can consist entirely of a single schematic. The flat schematic can be simulated by including sources for power and stimulus.
- A design using only a flat schematic (without hierarchy) is inefficient when repeated structures are used. Such a schematic becomes difficult to manage as the circuit complexity increases.
- A schematic can use symbols of primitives and symbols for other schematics. Such a schematic is more efficient in designing repeated structures and complex systems. This use of symbols to represent schematics continues to higher and higher levels of the hierarchy until the *top* level of the design is reached.
- A symbol for a schematic view is required only when that schematic is used within a hierarchy.

Editing the Hierarchy

In the Schematic Editor window, go to **Edit → Hierarchy**.



Design hierarchy comprises many levels of a single design. It enables you to view a smaller division of a larger design in detail.

- Choose **Descend Edit/Read** to traverse to the lower level in the hierarchy.
- Choose **Edit In Place** to edit a symbol in the design schematic.
- Choose **Return to Top** to go back to the top-most level in the hierarchy.

You can use these options to descend into the symbol and make any modifications to the schematic (DUT).

You get multiple options to traverse the hierarchy in both directions.

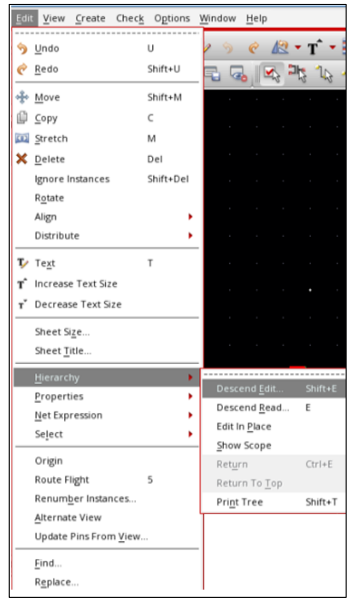
Descending into the Hierarchy

Schematic Editor menu bar → **Edit → Hierarchy → Descend Edit/Read**

Press the **Shift+E** bindkey.

- You can descend, return to the previous level, or return to the top from wherever you are. In all cases, you can choose to descend in edit or read mode.
- You can also descend edit into a symbol by **double**-clicking the left mouse button on the symbol.

You can use Ctrl+E to return one level above in the Schematic window.



An example of the Edit – Hierarchy – Descend Edit command.

cadence

This page does not contain notes.

How to Descend into a Hierarchy to Browse or Edit

The Descend Edit/Read form allows to descend into a lower-level view and open a chosen view in a *new tab*, the *current tab* (default), or a *new window*.

To descend into the hierarchy:

1. Select a component.
2. Choose a command.

   To browse: Edit – Hierarchy – Descend Read

   To edit: Edit – Hierarchy – Descend Edit
3. Select a view.

To ascend to the previous level (higher up in the hierarchy):

- Up one level: Edit – Hierarchy – Return
- Top level: Edit – Hierarchy – Return To Top

Use either **Edit – Hierarchy – Descend Read** or **Edit – Hierarchy – Descend Edit** to descend into the hierarchy one level at a time until the desired level is reached.

cadence

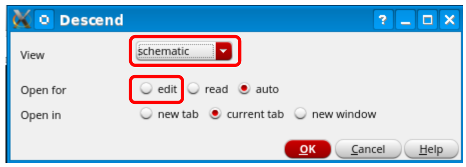
This page does not contain notes.

The Descend Form

The View cyclic field provides choices for the Descend command.

You can choose to descend into the schematic, Verilog-A, or some other definition.

- When a symbol is preselected, the *View* field is provided in the Descend form.



Using Descend – Edit

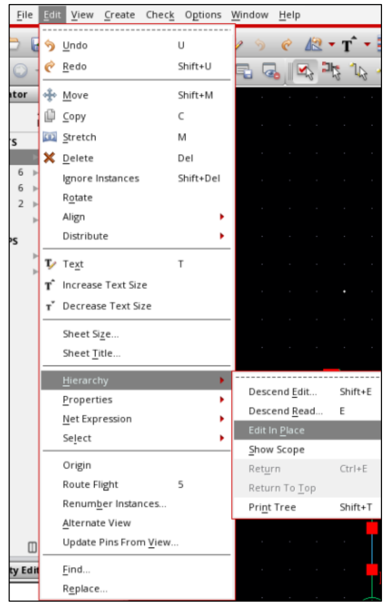
Using Descend – Read

This page does not contain notes.

Edit In Place Command

Schematic Editor menu bar → **Edit → Hierarchy → Edit In Place**

- With Edit In Place, you can edit the target symbol and symbol properties while still viewing it in context with the rest of the schematic.
- This is useful in situations where you have created a symbol that is slightly out of alignment with pins, text, or shapes because you can see the circuit you are connected to as you make adjustments.



Example: Hierarchy – Edit In Place command

This page does not contain notes.

Module Summary

In this module, you learned how to

- Create a testbench schematic
- Edit a hierarchical design to modify DUT parameters

This page does not contain notes.

Labs

Lab 3-1 Creating Testbench Schematic

Lab 3-2 Modifying DUT Parameters

This page does not contain notes.

Module 4

Pre-Layout Simulation

cadence

This page does not contain notes.

Module Objectives

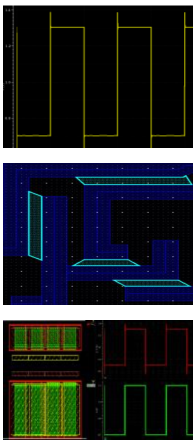
In this module, you will:

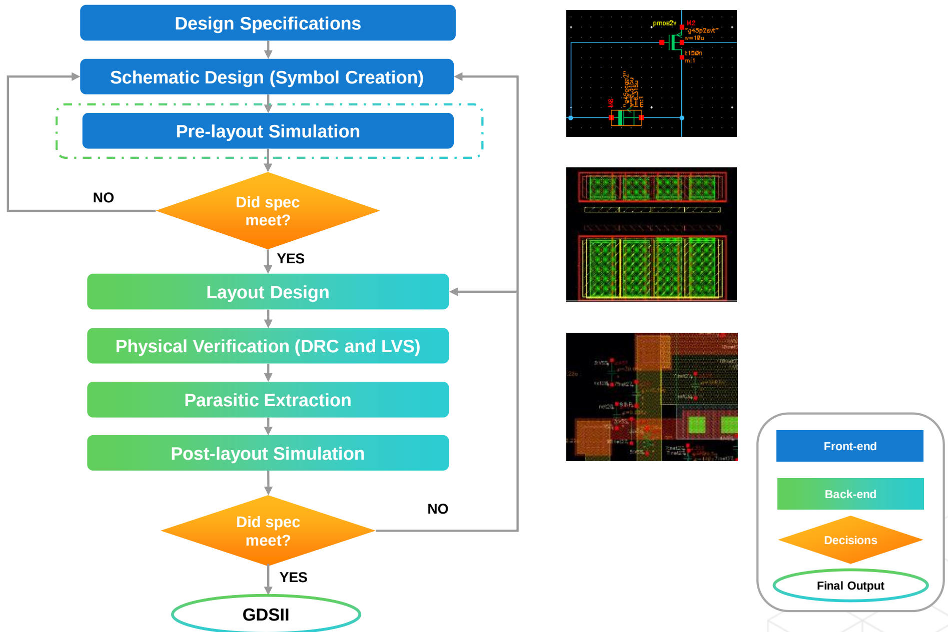
- Identify the capabilities of the Virtuoso® Studio ADE Explorer and Assembler
- Set up ADE Explorer and Assembler simulation environments by:
  - Defining the variables
  - Setting up different types of Analyses: transient, DC, and AC
  - Defining the Model Library
  - Generating the output expressions using the Calculator
  - Defining the Parameters and Outputs
- Run the simulation using the Spectre® simulator
- Analyze the simulation results using the Virtuoso VA
- Dynamically tune the DUT parameters using the Real-Time Tuning assistant

This page does not contain notes.

Analog IC Design Flow

PARAMETER	MIN	MAX
Gain		1000
Bandwidth	50KHz	500MHz
Supply Voltage	1.77V	5.5V
Supply Current	1.23mA	10.26mA
Propagation Delay (τ)	16.7ns	43.7ns
Chip area		2μm²





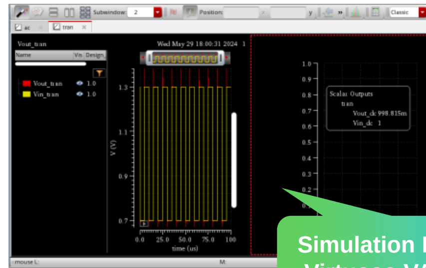
94

This page does not contain notes.

What Is Pre-Layout Simulation?

Pre-layout simulation is a process that verifies the functionality of the design at an early stage.

- It helps the designer to identify and fix issues early in the design process.
- **Spectre** is the simulator that is used for simulating our designs.
- The **Virtuoso ADE Explorer** and **Assembler** are the simulation environments available for performing simulations.
- A common cellview '**maestro**' is used for Explorer and Assembler.
- Assembler allows you to work with multiple testbenches and provides additional analysis capabilities.
- Appropriate licenses are checked out depending on the tools and features used.



Simulation Results in Virtuoso VA window

Test	Output	Nominal	Spec	Weight	Pass/Fail
tran	Vout_tran				
tran	Vin_tran				
tran	Vout_dc	998.8m			
tran	Vin_dc	1			
tran	Slew Rate	50.82M	> 50M		pass
tran	Settling Time	22.14n	< 50n		pass
tran	Offset	-1.185m	range -10m 10m		pass
tran	Pdiss	160.7u	< 200u		pass
ac	Av_ac				
ac	UGB	81.01M	> 50M		pass
ac	Av0	51	> 60		fail

Simulation Results in ADE Results Pane

cadence

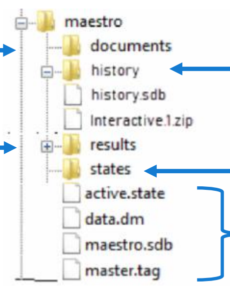
This page does not contain notes.

What Is the *maestro* Cellview Directory Structure?

The simulation setup and non-waveform results are stored in the cellview in a directory structure.

Documents, datasheets, etc., created in the Explorer/Assembler are stored in the document's directory.

The results directory contains the results database (.rdb) and run log (.log) files for history items saved from the maestro view.



The history directory contains history.sdb and .zip files for each history. Each zip file contains the setup information for history. The compressed format of the saving history setup saves space.

The states directory contains the setup states with the Variables and Parameters assistant.

For the *currently* active ADE Explorer setup:

- The XML ***active.state*** file contains details of all the tests in the *currently active setup.*
- The main ***maestro.sdb*** file contains the setup details other than the test details.
- The ***master.tag*** file identifies the cellview as type=maestro.
- The ***data.dm*** file contains other properties of the maestro cellview.

results directory:

- No waveform results are stored in the *results* directory. The *.rdb* file contains the results of all the scalar outputs evaluated using the simulation results returned by the simulator. These details are used to populate the Results tab in the ADE Assembler environment window.
- The *.log* file contains the summary of the simulations run for the saved checkpoint.

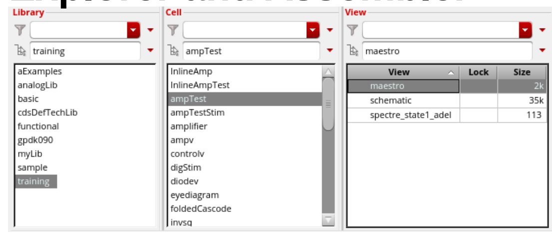
states directory:

- The *setupState* directories created for every state contain an XML state file called *active.state*, which includes details of all the tests in the saved state except the ones that are currently active.
- For every state that is saved in the *states* directory, a setup database (*.sdb*) file and corresponding directory are saved. These files contain details other than test details of the ADE Assembler setup.

Locations for both of these can be set in the **Setup → Save Options** dialog box.

The *maestro* Cellviews in the ADE Explorer and Assembler

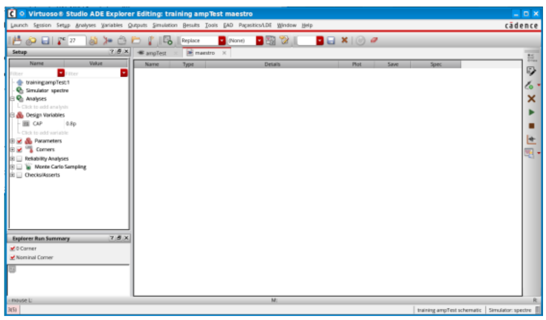
CIW → Tools → Library Manager → choose **Library →** choose **Cell → Maestro view**

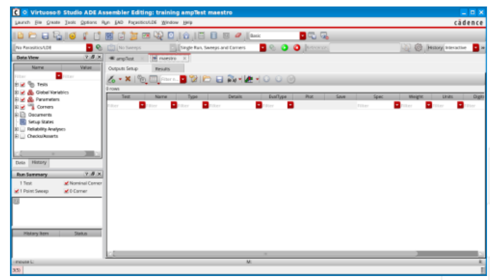


The ADE Explorer and ADE Assembler tools use the new maestro cellviews to establish a common data structure to provide access to the design environment.

Maestro cellview opened in the ADE Assembler test setup.

Maestro cellview opened in the ADE Explorer test setup.





97 © Cadence Design Systems, Inc. All rights reserved.

When you open the maestro view from the Library Manager, it is opened in the same application from which it has been created. If it is created using the ADE Explorer, it will open in the ADE Explorer, and if it is created using the ADE Assembler, it will open in the ADE Assembler.

Capabilities of ADE Explorer and Assembler Environments

Virtuoso ADE Explorer

- The ADE **Explorer** is a highly interactive, single testbench environment.
- Key features are:
  - Displays and analyzes simulation results
  - Parametric sweeps
  - Backannotation to designs
  - Real-Time Tuning(RTT)
  - Corners analysis
  - Monte Carlo analysis
  - Checks and Asserts interface

Virtuoso ADE Assembler

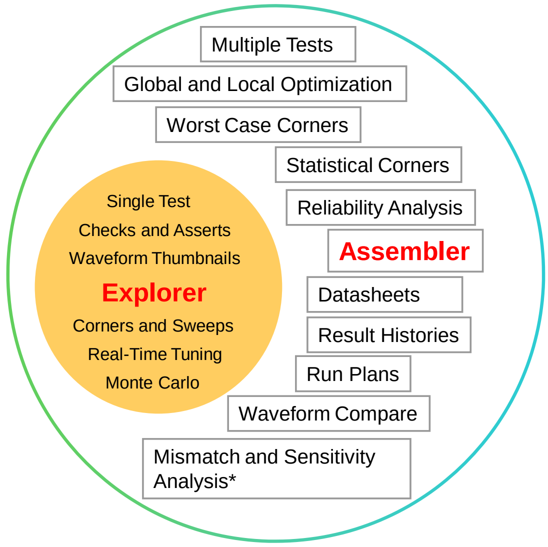
- Other than single runs, the **Assembler** allows you to run:
  - Nested/Parallel Parametric sweeps
  - Corners analysis
  - Monte Carlo (statistical) analysis
  - Optimization runs
- Assembler provides you access to previous simulation results (history):
  - You can load results from previous runs as well as the corresponding testbench setup via the simulation history.
  - You can control the number of saved history points.
- Assembler gives you access to a rich collection of post-processing tools:
  - Use the Virtuoso Visualization and Analysis waveform tool suite.
  - Generate HTML-based datasheets that include spec summaries, results, waveforms, etc.

By default, the Spectre Classic simulator is used to run simulations in both ADE Explorer and Assembler.

The ADE Assembler includes all the ADE Explorer features in addition to its specific capabilities.

- **Virtuoso ADE Explorer:**
  - Single test analysis environment that can be used to run a range of the Spectre simulations, analyze designs over corners, and perform Monte Carlo statistical simulations.
  - You can perform Real-Time Tuning of design variables and parameters.
  - You can also implement a Parasitic-Aware Design flow in the Explorer.
- **Virtuoso ADE Assembler:**
  - Multi-test simulation environment that includes all Explorer capabilities.
  - An interactive cockpit provides access to run sweeps, circuit optimization, create worst-case corners, analyze yield, etc.
  - You can also create mini-run plans.

Capabilities of ADE Explorer and Assembler Environments (continued)



*With the **Virtuoso Variation Option**.
This is an additionally licensed feature.

This page does not contain notes.

Submodule 4.1

Simulation Setup in the ADE Explorer

This page does not contain notes.

Submodule Objectives

In this submodule, you will:

- Launch the ADE Explorer environment
- Set up the environment for performing transient and DC analyses
- Run the simulation using the Spectre simulator
- Plot and save the simulation results

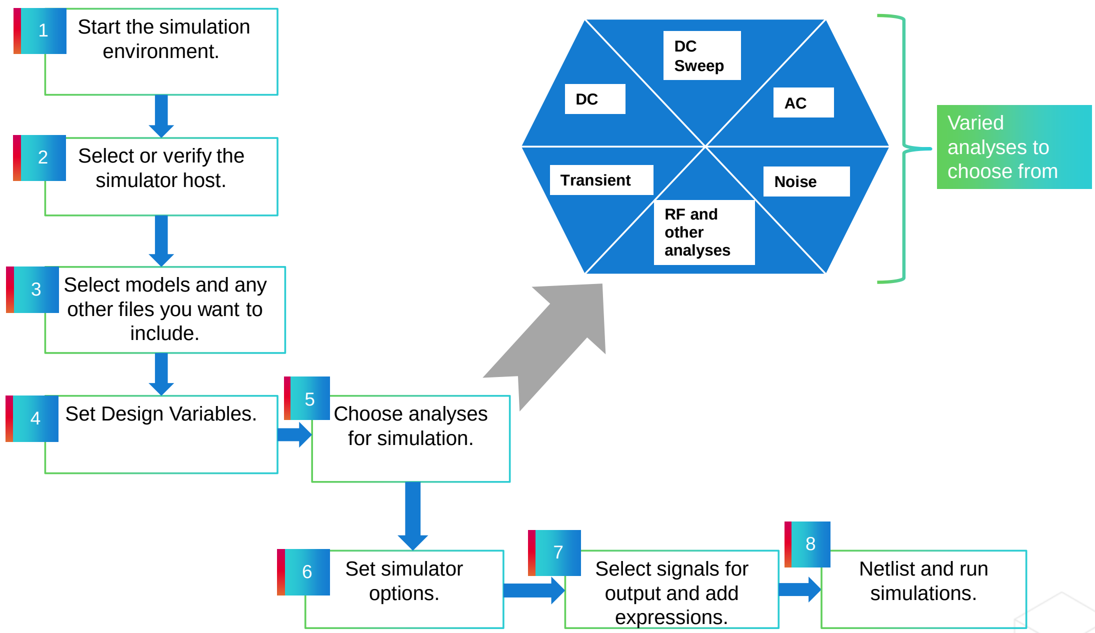
This page does not contain notes.

Terms and Definitions

Analyses	The types of simulation analyses available for a particular simulation engine, such as tran, ac, and dc. Must be selected prior to starting the simulation.
Variables and Parameters	Variables are string values of component parameters that can be set in the ADE environments. Parameters can be used to change fixed values of component parameters without requiring edits to the design.
Spectre	Spectre Classic Simulator: A Cadence® SPICE-type simulation tool for simulating analog circuits.
Accelerated Parallel Simulation (APS)	A Spectre parallelization algorithm for larger analog circuits. The +aps option to Spectre provides results almost identical to Spectre in a much shorter amount of time.
Netlist	A text-based representation of design instances and interconnects used by the simulator host.

This page does not contain notes.

Flowchart: Analog Simulation Flow



cādence

This page does not contain notes.

How to Start the Simulation Environment

To open the Virtuoso ADE Explorer for the first time, firstly, you need to create a new maestro cellview to open the design in the Explorer.

1

Use one of the following methods to create a *maestro* view and open **ADE Explorer**:

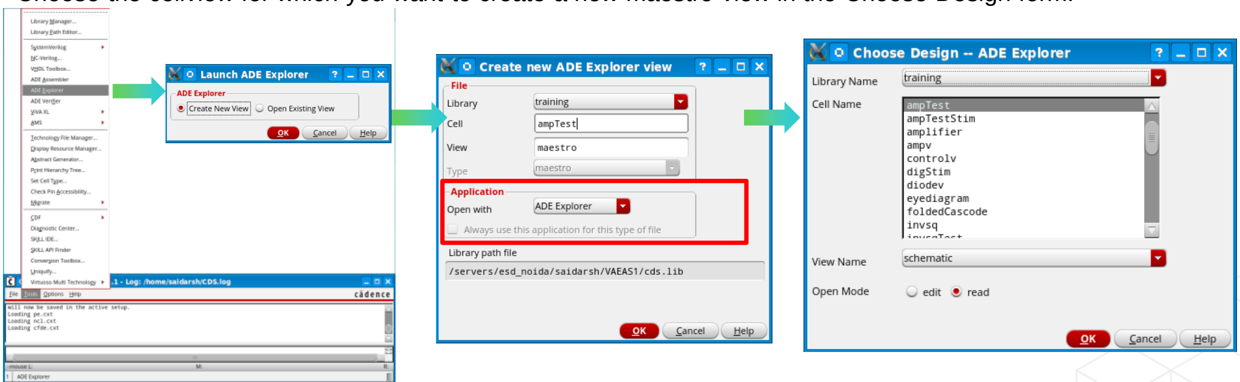
1. Open Explorer from the **CIW**.
2. Open Explorer from the **Virtuoso Schematic Editor**.
3. Open ADE Explorer using the **Schematic Editor workspace**.
4. Load an **existing ADE L setup** in the Explorer.

ADE L states and ADE-XL views can be opened in the Explorer and Assembler. This action migrates the data to a maestro cellview.

How to Open ADE Explorer from the CIW

You can create a new maestro view and open the Virtuoso ADE Explorer from the CIW. This is not a common method.

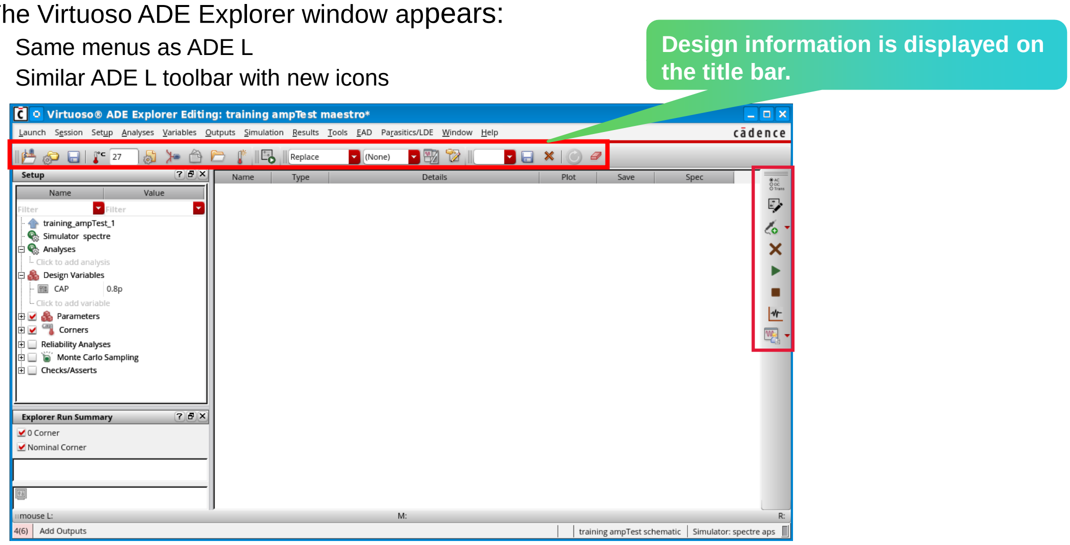
1. Choose **Tools – ADE Explorer** in the CIW. Select the **Create New View** option in the *Launch ADE Explorer* form.
2. Ensure that **Open with** is set to the *ADE Explorer* for your new maestro cellview. Click **OK**.
3. Choose the cellview for which you want to create a new maestro view in the Choose Design form.



This page does not contain notes.

How to Open ADE Explorer from the CIW (continued)

- The Virtuoso ADE Explorer window appears:
  - Same menus as ADE L
  - Similar ADE L toolbar with new icons

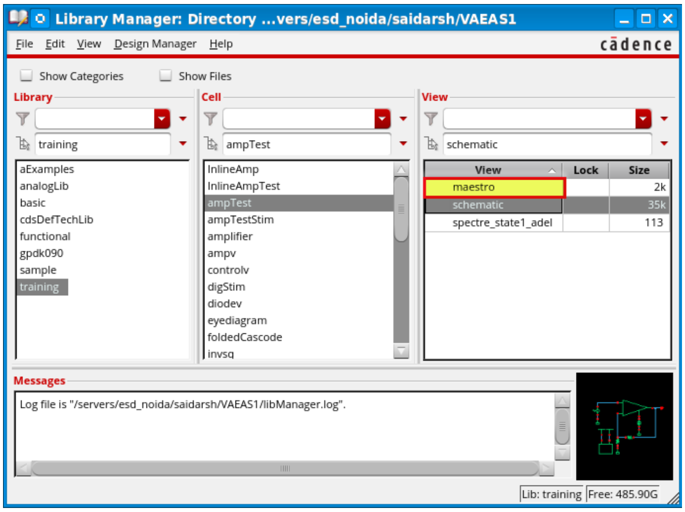


You will be prompted to save any changes you make when exiting the tool.

This page does not contain notes.

How to Open ADE Explorer from the CIW (continued)

- The new **maestro** view is displayed in the Library Manager.
  - Note that you can name this view anything that you would like.
  - Next time, you can **double-**click on maestro for this **cellview** to open it in the ADE Explorer.

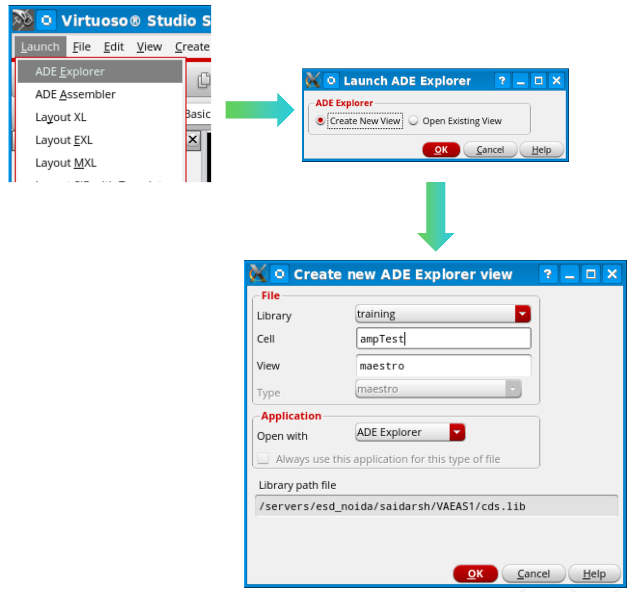


This page does not contain notes.

How to Open ADE Explorer from the Schematic Editor

You can create a new *maestro* view and open Virtuoso ADE Explorer from the schematic window. This is the most common method.

1. Select **VSE** and then choose **Launch – ADE Explorer**.
2. Select **Create New View** in the *Launch ADE Explorer* window.
3. Choose View type as **maestro** and Application as **Open with** (ADE Explorer), which is already specified, choose to open in a *new tab*, *current tab*, or *new window.*

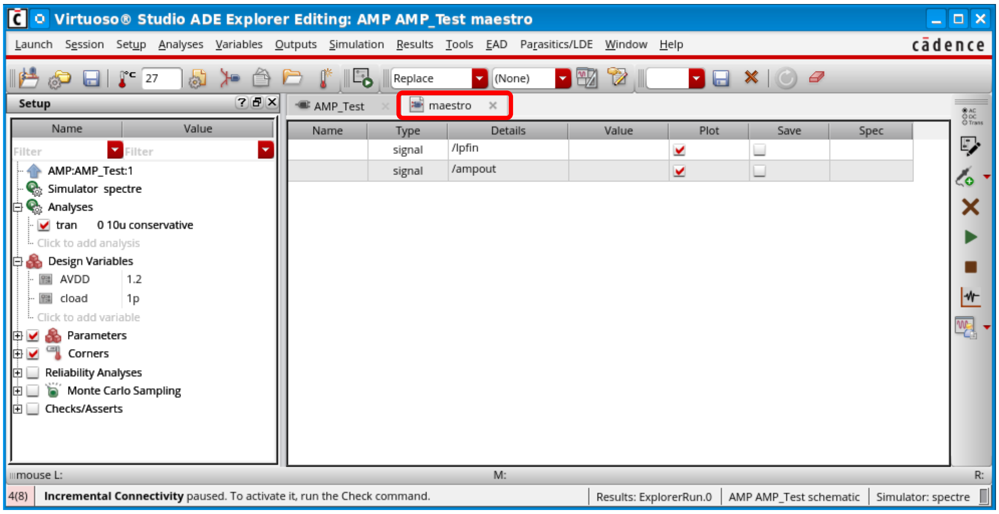


cādence

VSE: Virtuoso Schematic Editor

How to Open ADE Explorer from the Schematic Editor (continued)

Depending on your choice, the ADE Explorer can be tabbed with the design.



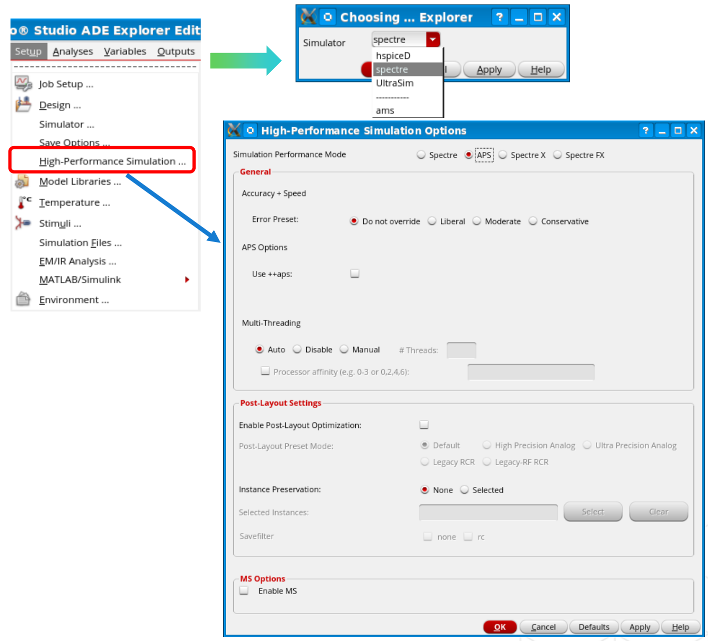
This page does not contain notes.

How to Select Simulator in the ADE Explorer

Spectre is an advanced circuit simulator that simulates analog and digital circuits at the differential equation level.

2 Choosing Simulator

- To select or verify the simulator in the ADE Explorer:
  - Choose **Setup – Simulator** and then select **Spectre** as the Simulator.
- For larger designs and faster simulation, you can use other modes of simulation:
  - Choose **Setup – High-Performance Simulation** and select the Simulation Performance mode as **APS, Spectre X,** or **Spectre FX** instead of Spectre.
    - Spectre FX is not supported in Spectre 20.1 release. It is available in the Spectre 21.1 release.
    - *MS options* can be used for mixed-signal designs.



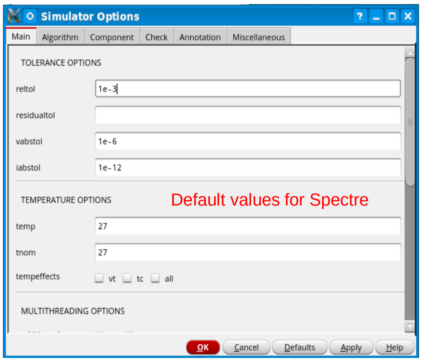
cādence

The High-Performance Simulation menu option is disabled when the simulation runs in the interactive mode.

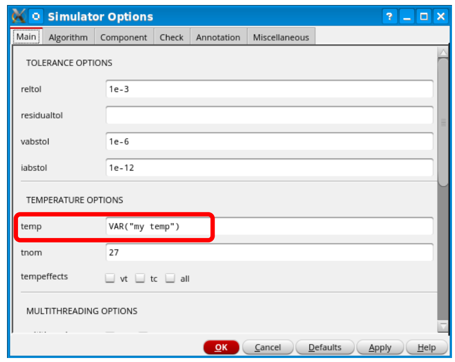
Setting Simulator Options

ADE Explorer menu bar → **Simulation** → **Options** → **Analog**

All defaults



Changing temperature value to a design variable with VAR syntax



- Options related to convergence and data compression are available in the **Algorithm** tab.
  - Simulator Options affects all the chosen analyses and are not analysis specific.

To access global simulator options, such as *tolerance* settings, choose **Simulation – Options – Analog**.

These are simulation options that are common to all analyses.

Tolerances define convergence criteria.

The form on the left is shown with all the simulator default values.

The form on the right is modified in the *temp* field. The *VAR("mytemp")* syntax is used to set a temperature value by referencing a design variable.

The *temp* value is the ambient temperature at which a normal simulation runs.

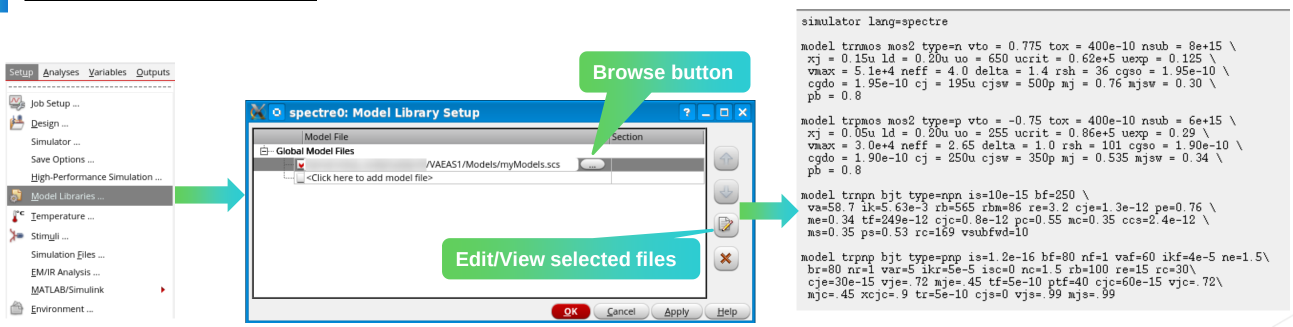
T*nom* is the nominal value at which all circuit parameters are measured. It is defined in the model card for a device. Any value you write in this form is overridden by the value in the model card.

For detailed information on how to set the options on this form, attend the *Virtuoso Spectre Pro* series classes.

Including Model Files in the ADE Explorer

ADE Explorer menu bar → **Setup** → **Model Libraries**

3 **Including Model Files**



112

The model library file is used to specify paths to device model definitions. These are provided by your foundry as model cards.

- Choose **Setup – Model Libraries**.
  - The Model Library Setup form appears. The first time you set up the simulation environment, this form is empty.
- Click the field for the first model. Click the **Browse** button to open the Choose Model File form.
- To view the model file, select the model file. Click the **Edit/View Selected Files** button.
- Select a section to use if the model file you are pointing to contains sections/.lib syntax.

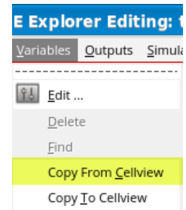
How to Set Design Variables in the ADE Explorer

You can use design variables to set component values. Design variable values are always global to the design.

4 **Specifying Design Variables**

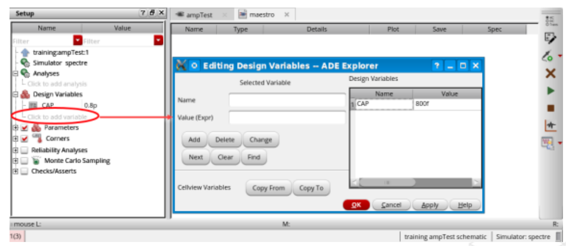
If there are variables saved to the design, retrieve them by:

- Choosing **Variables – Copy from Cellview** on the ADE Explorer menu.



You can also

- Select **Click to add variable** to add a new variable to the **Setup** assistant.



The values of any design variables in the circuit must be set before running a simulation. If values have been set in the past, they appear in the Value column of the Design Variables section of the Setup assistant in the ADE Explorer form.

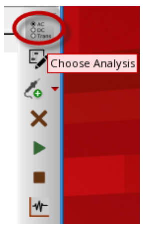
Variables are retrieved from cellview properties and functions such as *pPar*.

How to Choose Analyses in the ADE Explorer

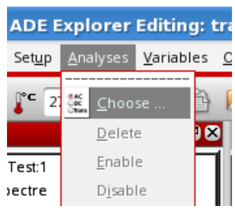
You can perform any one of the following steps to open the *Choosing Analyses* form and set up an analysis in the ADE.

5 To open the **Choosing Analyses** form:

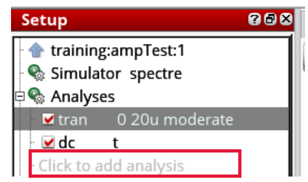
1. Click the **Choose Analysis** icon from the Run toolbar.
2. Choose **Analyses – Choose** from the ADE Explorer menu bar.
3. Click on **Click to add Analysis** in the **Setup** assistant.
4. **Double**-click on an existing analysis in the **Setup** assistant.



OR



OR

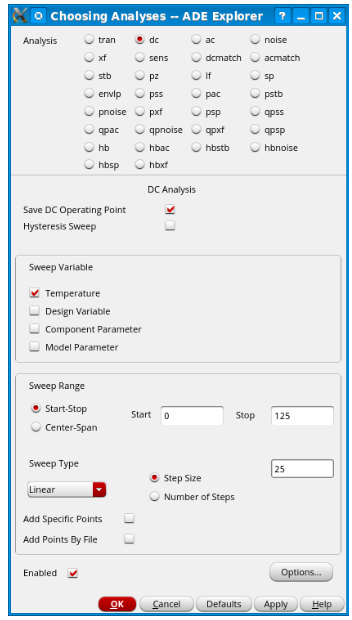


In the ADE Explorer window, **right**-click the test name and choose **Add Analysis**. The Choosing Analyses form appears.

Choosing Analyses: DC

For a DC simulation (*dcOp* or *dc sweep*):

1. Select the **dc** radio button. Select the **Save DC Operating Point** checkbox. Select the **Hysteresis Sweep** checkbox if you want to enable DC hysteresis sweep.
2. If you want a DC sweep, from the *Sweep Variable* field:
   - To sweep circuit temperature, select the **Temperature** checkbox.
   - If it is a **Design Variable**, select it from the list.
   - If it is a **Component Parameter**, select the component from the schematic and select the parameter from the list.
   - If it is a **Model Parameter,** enter the model's name and the parameter in the model.
3. Set *Sweep Range* limits and type.
4. To enter options specific to the Spectre's DC analysis, click **Options**.
5. Click **Apply** or **OK**.



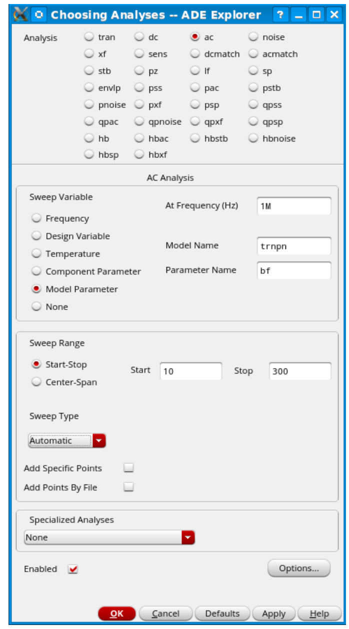
Optionally, select the **Add Specific Points** checkbox to specify a list of values to sweep. Use spaces to separate each value in the field.

Optionally, select the **Add Points By File** checkbox to choose the file containing the values to sweep. You can either enter the path to the file in the textbox manually or click the **Browse** button next to the text box and choose the file.

Choosing Analyses: AC

For an AC sweep analysis:

1. Select the **ac** radio button.
2. Select a sweep variable (usually Frequency).
3. Set a sweep range and type.
4. Set Specialized Analyses to **None**, if it is not already set (for RF simulations).
5. To enter **AC options**, click **Options**.
6. Click **Apply** or **OK**.



cādence

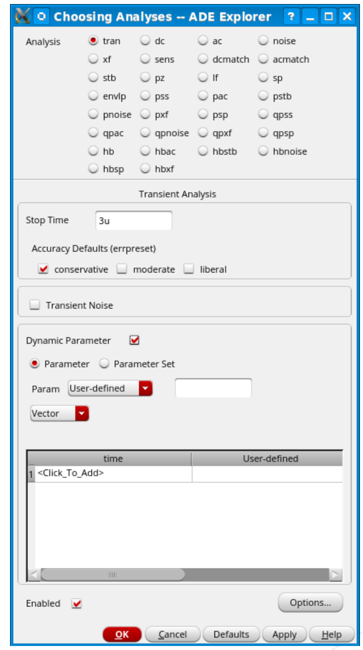
Similar to the DC Analysis Options, clicking the **Options** button here gives you a form that you can use to set *annotation, convergence, state-file,* and *output* parameters relative to your design. You can also add initial condition parameters with the option to skip DC analysis, also available here.

Specialized analyses are analyses used in RF simulations. The Specialized Analyses Cyclic menu gives you access to special analyses carried out on RF circuits such as IP3 and compression distortion calculation. For more information, refer to the *Spectre RF Analysis Using Harmonic Balance* and *Spectre RF Analysis Using Shooting Newton Method* courses.

Choosing Analyses: Transient Analysis

For a transient analysis:

1. Select the **tran** radio button.
2. Select a **Stop Time**.
3. Select **Accuracy**.
4. Optionally, select **Transient Noise**.
5. Optionally, specify the **Dynamic Parameter** to sweep over time.
6. To enter transient options, click **Options**.
7. Click **Apply** or **OK**.



Transient noise provides the ability to examine the effects of noise in the time domain. This analysis is well suited to designs that have disparate frequencies or non-periodic in nature, where SpectreRF analysis is not possible.

The Spectre transient analysis provides the ability to sweep a parameter over time. This can be a design parameter or a simulation tolerance such as reltol. This capability can greatly help with circuits that are difficult to achieve convergence with.

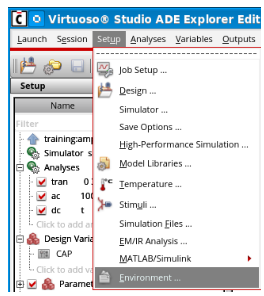
Setting Environment Options

ADE Explorer menu bar → **Setup** → **Environment**

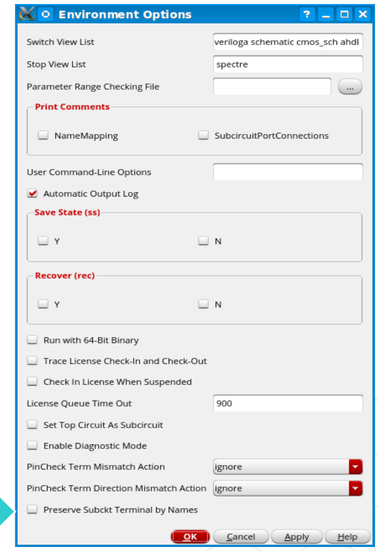
6

Access additional simulation options in the **Environment Options** form. The choices vary based on the simulator being used.

- **Switch View List** and **Stop View List** establish ordering of views for design traversal.
- A ***Parameter Range Checking File*** can be used to limit ranges.
- ***Print Comments*** can be used to print the name mapping of the terminals in the netlist as comments and the connection of each terminal with the net, it is connected to for each subcircuit.
- ***User Command-Line Options*** provide the ability to specify options that are not in the UI.
- ***Save State*** saves circuit information at specific times, or if the simulation aborts.
- ***Recover*** restarts a simulation based on conditions specified in the *savestate* file.



Setup – Environment



Switch View List – List of views used for design traversal. This is used when manually traversing the design and also by the netlisters. Ordering is important.

Stop View List – A list of views that identify a stopping view to be netlisted. This list does not require a particular sequence. The first view on the switch view list here will be the view used as the stop view.

Note: The Switch and Stop View lists are not displayed if your testbench is a config.

The parameter range checking file can be used to set limits on parameters to be checked at the start of the simulation. We provide an example file at:

<install_dir>/tools/spectre/etc/limits/range.lmts.

The *userCmdLineOption* field can be used to specify command-line options for the simulator. This is important if you are working with a version of the simulator that is much newer than the environment.

Savestate and Recover provide the ability to continue a simulation from a specific timepoint. This is beyond the save/restore command, as it saves a solution history, avoiding inaccuracies with the resumed simulation.

The diagnostic mode is valuable when simulation difficulties are encountered. It provides very detailed convergence, timestep and parameter information.

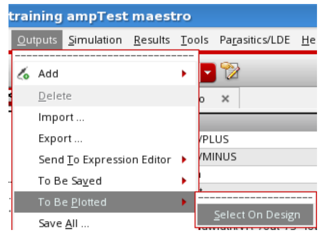
Specifying Output Data

Choose **Outputs → To Be Plotted → Select On Design** in the ADE.

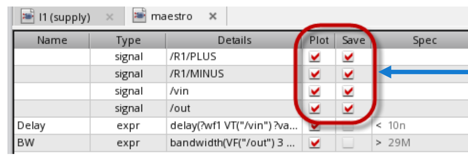
6

To specify outputs for saving and/or plotting:

- Select device pins to save terminal currents or wires to save node voltages. Press **Esc** to end the selection process.
- Be sure the Plot or Save checkboxes in the **ADE Explorer** form are selected for the signals.
- Expressions you create with the calculator can also be plotted or evaluated.
- Outputs can be interactively plotted after the simulation finishes with the **Direct Plot** form.



You can select signals (nodes, terminals, or nets) on the schematic and specify whether you want to save or plot them.



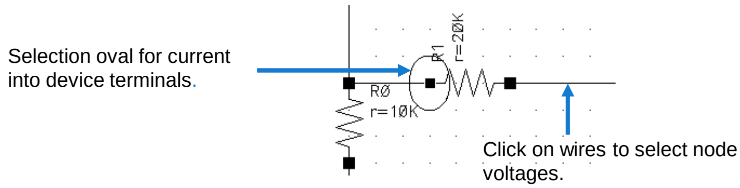
The plot option enables autoplot after the run.

This page does not contain notes.

Probing the Schematic to Save the Output Data

To select outputs from the schematic, choose **Outputs → To Be Plotted → Select On Design**. Select wires to save voltages and terminals for currents.

- When you select a wire, it changes color to indicate it has been selected. When you click on a terminal, a colored oval surrounds it to indicate it is selected.
- You *must* terminate the selection process: Press **Esc**.
- If you accidentally select a signal, you do not want, click it again to deselect it.

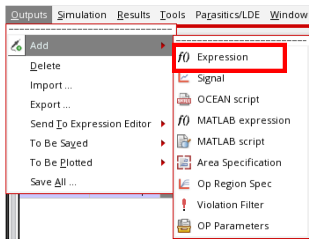


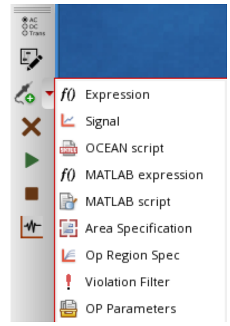
This page does not contain notes.

Adding Output Expressions

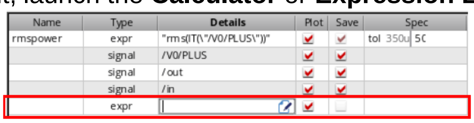
You can add an expression of the type expr to be evaluated in the Output setup.

- To add an expression to the Output, perform one of the following:
  - Choose **Add Outputs – Expression** in the *Run toolbar.*
  - Choose **Outputs – Add – Expression** in the *menu bar.*





- From the newly added output, launch the **Calculator** or **Expression Editor** to build your expression.

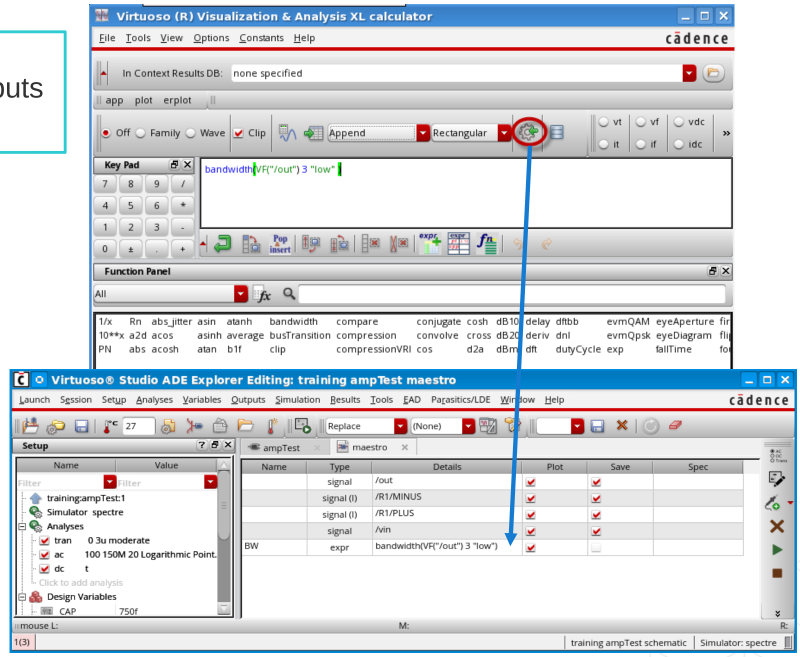


An empty expression row (expr) gets added in the ADE Explorer Outputs with the action mentioned above.

Other types of outputs are also supported.

Sending Expressions to ADE Outputs from the Calculator

Calculator → Send buffer expressions to the ADE Outputs icon.



This page does not contain notes.

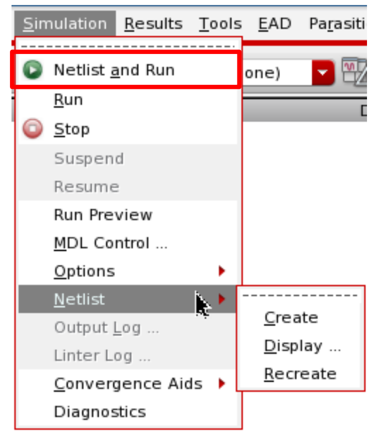
Netlisting in the ADE

ADE Explorer menu bar → **Simulation** → **Netlist**

8

To create a netlist, choose the following options:

Simulation – Netlist – Create	Performs an incremental netlisting that only netlists schematics that have changed since they were last netlisted.
Simulation – Netlist – Recreate	Netlists the entire design starting over from scratch. This mode finds all topological and value changes.
Simulation – Netlist and Run	Performs an incremental netlist and simulation. You can also click the Netlist and Run icon.
Simulation – Netlist – Display	Displays the created netlist in your chosen text editor.



cadence

Manually creating a netlist is not required prior to running a simulation; you can choose to do so if you only want to view the connectivity and not run a simulation.

Running the Simulation

ADE Explorer menu bar → **Simulation** → **Netlist and Run**

Click the **Run Simulation** icon.

- If changes were made to the design and a **Check & Save** was not performed, you would be prompted to do so before the simulation progresses.

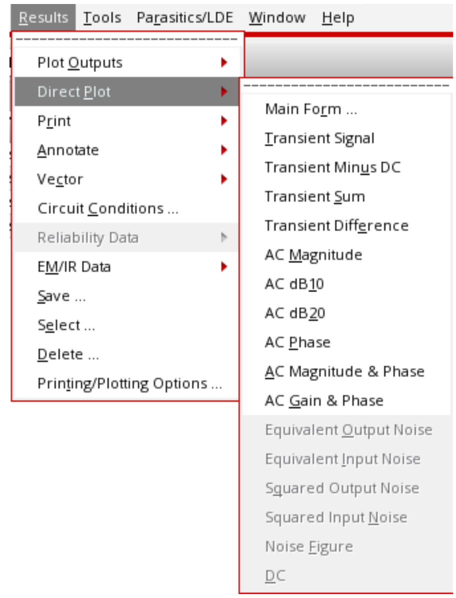
- If outputs were preselected for plotting in the Outputs pane of the ADE Explorer, then the waveform display, Virtuoso Visualization and Analysis automatically appear when the simulation is completed.
- To stop the simulation at any time, click **Stop**.

This page does not contain notes.

Plotting Signals from the ADE

ADE Explorer menu bar → **Results** → **Direct Plot**

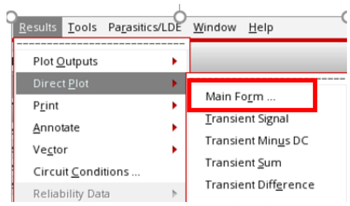
- You can use the **Direct Plot Form** (described on the next slide) or interactive plotting picks.
- You can also access this form from individual results table cells.



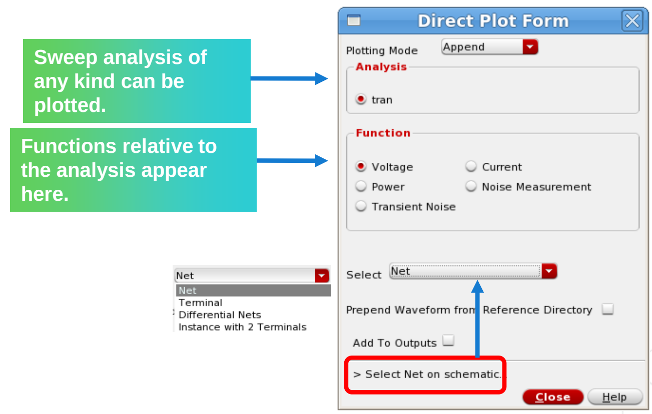
This page does not contain notes.

Direct Plot: Main Form Option

ADE Explorer menu bar → **Results → Direct Plot → Main Form**



- With the **Direct Plot Form**, you can quickly plot common waveforms after simulation.
- The **Direct Plot** form has different commands available in its **Function** section.



126

The Direct Plot Form can only be accessed from the ADE after the simulation has run. If you try to access it before the simulation, an error message appears saying that no analysis data is selected or found.

The Direct Plot Form changes dynamically based on the type of option chosen.

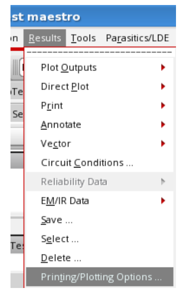
Pressing the **Esc** key in the Schematic window closes the Direct Plot Form.

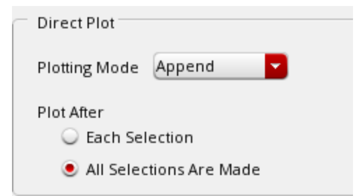
Choosing Printing/Plotting Options

ADE Explorer menu bar → **Results** → **Printing/Plotting Options**

There are two modes for plotting using the interactive plot commands:

- Immediately plot a signal you select.
- Defer plotting until all selections are complete.
  (The **Esc** key is used to indicate completion of selection.)



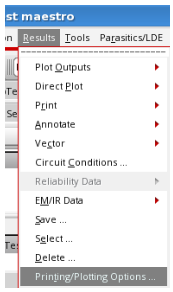


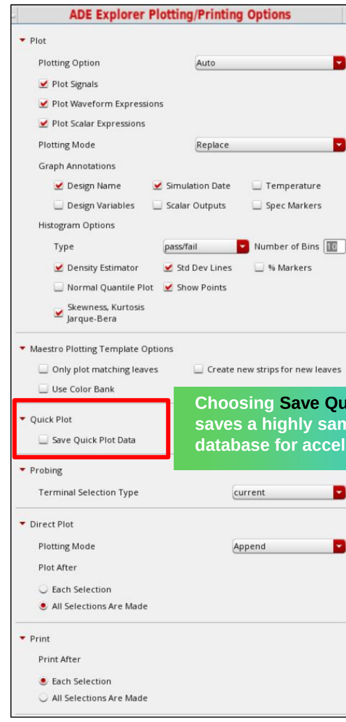
Note: The Direct Plot main form will always plot signals when selecting objects on the schematic.

This page does not contain notes.

Using the Plot After Simulation Option

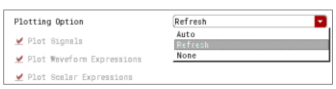
- Choose **Results – Printing/Plotting Options.**

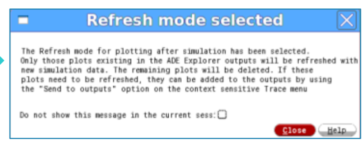




Choosing **Save Quick Plot data** saves a highly sampled transient database for accelerated plotting.

- When you choose **Refresh** as the plotting option, the existing graph from a previous simulation is refreshed in the same window with new simulation results.





When you change the cyclic option to **Refresh**, a pop-up menu appears, providing details on that mode. You can also choose to set this option to **Auto** or **None**.

Using Refresh also saves any customized settings you might have made on the graph (markers, attributes, strips, subwindows, etc.).

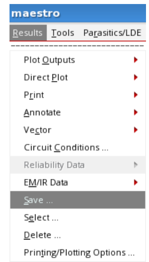
When you choose **Auto**, autoplotting is done according to the plotting mode specified.

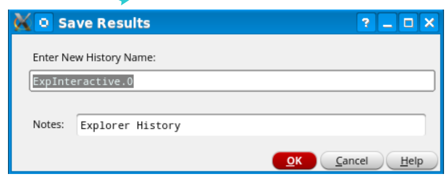
None will suppress all automatic plotting.

Saving Results in the Explorer

ADE Explorer menu bar → **Results** → **Save**

- Explorer saves only the last simulation run results with the **ExplorerRun.0** directory name. For example:
  - **`<projectDir>/libname/cellname/maestro/results/maestro/ExplorerRun.0`**
- This ***ExplorerRun.0*** directory is overwritten with the latest simulation results.
- You can save on-demand results (histories) that can be loaded later.
- In the **Save Results** form:
  - Enter **New History Name** to store current simulation results in that directory.
  - Specify **Notes** description that can be viewed in the **History** tab of Data View in Assembler.





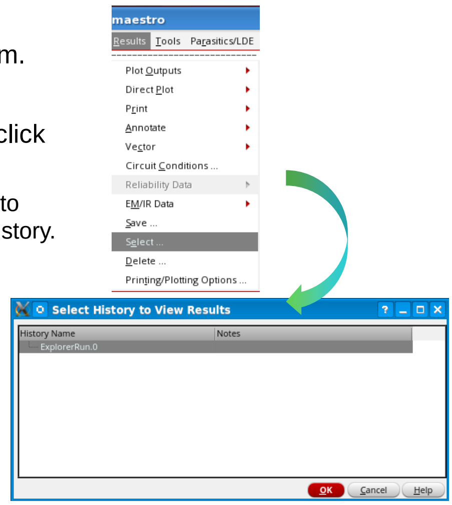
- After running simulations, exit and then reopen the maestro view in Explorer:
  - Results from the single-point simulation are loaded automatically.
  - For the multipoint run, choose **Results – View – ExplorerRun.0** to load results.

- In the *Enter New History Name* field, specify a history name to store the simulation results of the current simulation
- In the *Notes:* field, specify a description of this history.

Viewing Saved Results

ADE Explorer menu bar → **Results** → **Select**

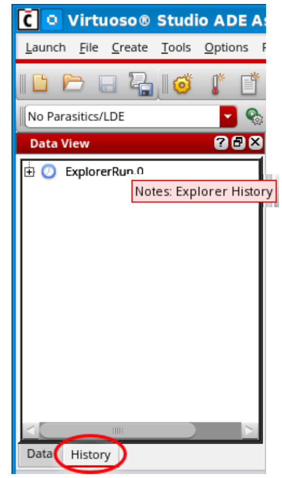
- You can view a previously saved result/history item.
- From the **Select History to View Results** form, choose desired history name to view results and click **OK**.
  - The Explorer Setup and Output section are updated to show the settings from the selected simulation run history.

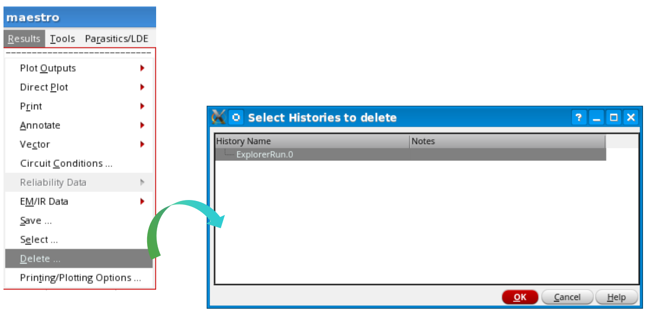


This page does not contain notes.

Saved Results and Deleting Saved Results

- Results saved in Explorer are visible on the **History** tab in ADE Assembler.
  - Hover your cursor over the history name to read notes.
- Choose **Results – Delete** in the ADE Explorer to delete unwanted previously saved history/result.
  - Select the History item to be deleted and click **OK**.





cadence

This page does not contain notes.

Submodule 4.2

Simulation Setup in the ADE Assembler

This page does not contain notes.

Submodule Objectives

In this submodule, you will:

- Launch the ADE Assembler environment
- Set up multiple tests, analyses, and simulations in the ADE Assembler
- Create Output Expressions
- Run the simulation
- Plot and view the simulation results
- Analyze the results using the spec-related features
- Switch between the ADE Explorer and Assembler

cadence

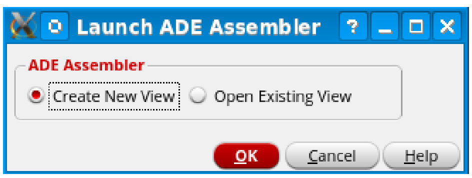
This page does not contain notes.

How to Start the Virtuoso ADE Assembler

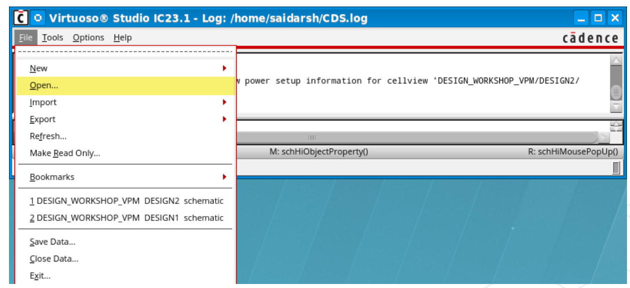
You can launch ADE Assembler from the Virtuoso Design Environment Command Interpreter Window (CIW) or from your schematic editing window.

- The ADE Assembler can be started directly from the CIW (**Tools − ADE Assembler**) or from an opened schematic (**Launch − ADE Assembler**), either of which prompts a user to:
  - Create a New (*maestro*) View.

    or
  - Open an Existing (*maestro*) View.



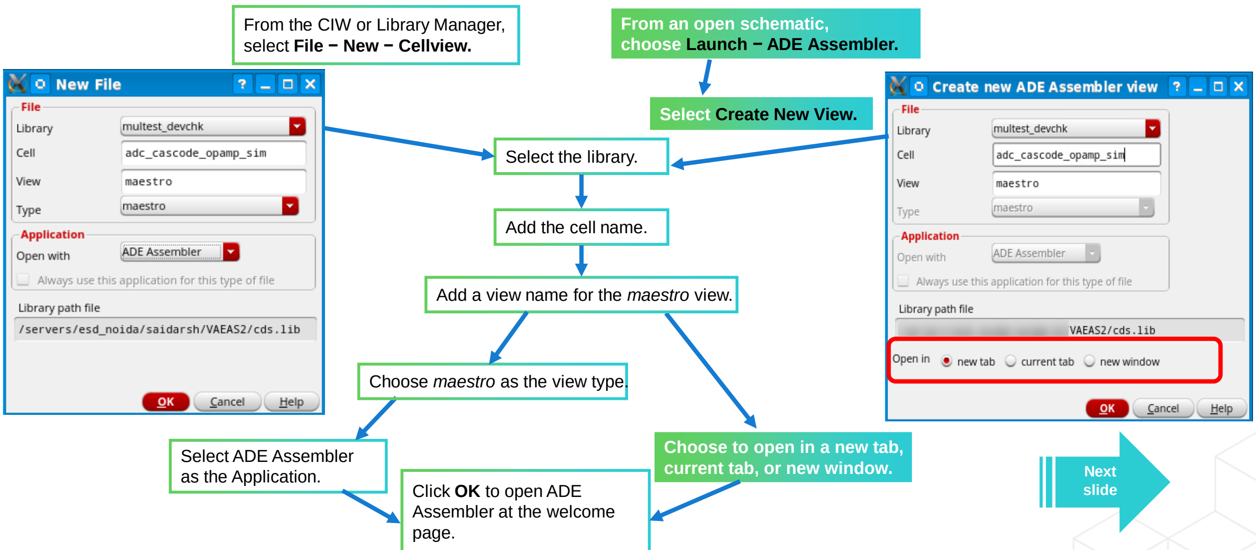
- Other ways to launch ADE Assembler are given below:
  1. Opening an existing *maestro* view from the **CIW (File − Open)** and choosing **ADE Assembler** as the application to open with.
  2. Opening from the **Library Manager,** selecting the view and executing **File − Open** or **double**-clicking.
  3. Translating from an ADE Explorer environment:
     - Any settings will be automatically loaded in the Assembler session.



This page does not contain notes.

Creating a New *maestro* CellView for ADE Assembler

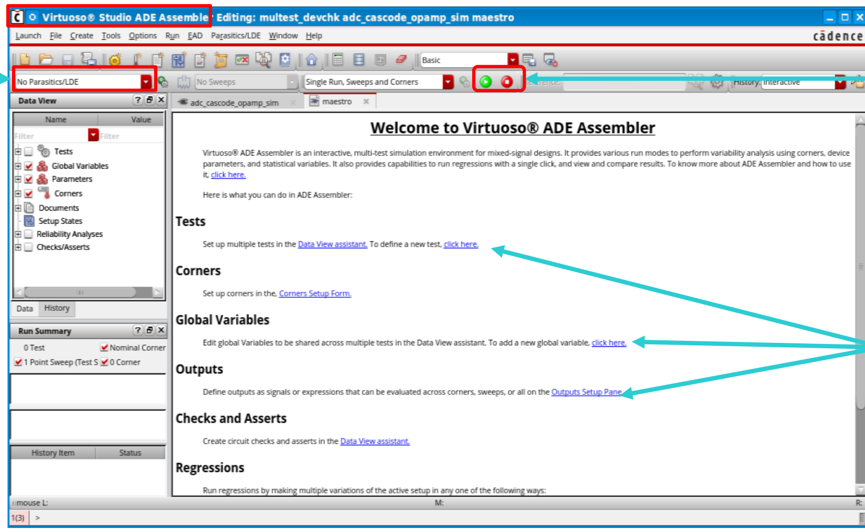
The first task in setting up an ADE Assembler session is creating the *maestro* view, which can be stored in any library and assigned arbitrary cell and view names (of type *maestro*).



- You can create a maestro view in multiple ways. You can use the *Library Manager* or the CIW and use the New File form for a given design schematic.
- You can alternatively create a maestro view from an already open schematic and can choose to open the *maestro* view in a new tab, new window, or the current tab itself.
- Note that the *Open in* options are available only when you create a new *maestro* view from an existing schematic view.

Welcome to Virtuoso ADE Assembler

When creating the *maestro* view, ADE Assembler opens a Welcome to Virtuoso ADE Assembler page with links to testbench configuration interfaces.



The Welcome to Virtuoso ADE Assembler panel is replaced by the Outputs Setup assistant when the first test is defined.

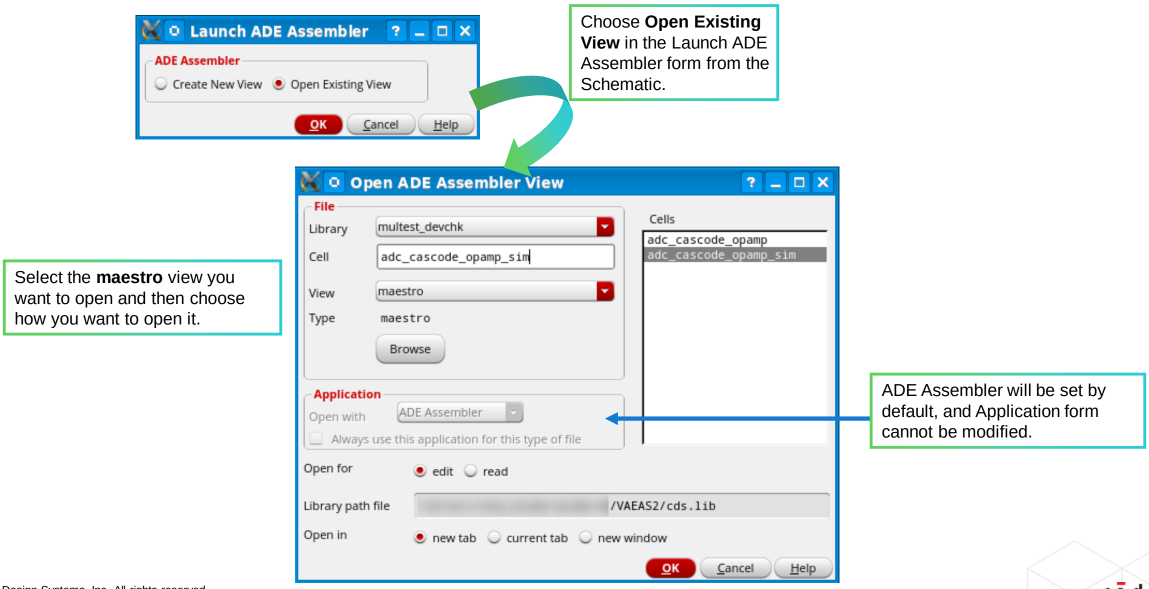
When you click on the *Setup Links* that are shown in the snapshot above, you are redirected to or a form corresponding to the action or setup to be performed opens up.

The links provided are for:

- Tests (access to Data View assistant)
- Corners Setup Form
- Global Variables Setup
- Outputs Setup Pane
- Creating checks/asserts
- Regressions (local sweeps, run plan assistant)

Opening an Existing *maestro* Setup

You can open an existing setup only once the maestro view has been created for that cell.



Virtuoso remembers the application that was last used to save each maestro cellview and, by default, opens the view in the same application next time.

The ADE Assembler title bar displays the text *Editing* if the ADE Assembler view is opened in edit mode. The ADE Assembler title bar displays the text *Reading* if the view is opened in read-only mode.

A *constraint* view is automatically created when you create a maestro view.

Test Definition in the ADE Assembler

A test is identical to a single ADE Explorer session.

It typically contains the following:

- A design that you will simulate:
  - This is usually comprised of a DUT and testbench.
- A target simulator:
  - ADE Assembler supports all Cadence® provided simulators as well as an interface for third-party tools.
- A single analysis of each type that your simulator supports (dc, ac, tran, pss, etc.).
- Design variables, device models, outputs and expressions, and simulation options.

Important: Multiple tests can be specified, and each can have a unique setup. The design, simulator, variables, outputs, save options, etc., of one test can be entirely different from the other tests in the same session.

Note that a single ADE Assembler session can include tests using a different simulator.

This means that you can specify one of your tests to use Spectre (Spectre-APS, other flavors), another to use AMS Designer, and another to use a proprietary simulator.

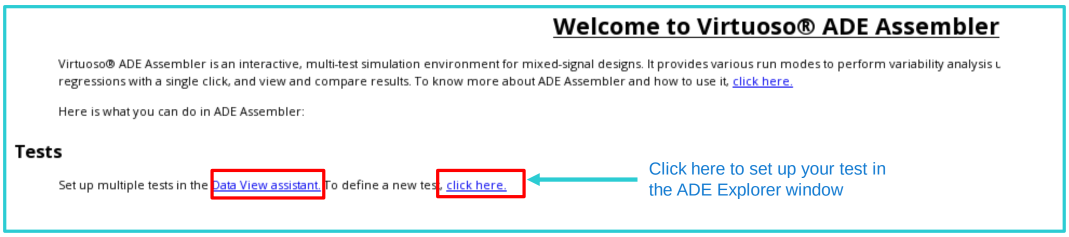
The setup of a test can include one or more of the following:

- The design testbench defined in a design schematic
- Analyses to verify DUT operation
- Device models and other included files
- Selecting a Simulator
- Simulator-specific options
- Design variables and parameters
- Global options for saving data
- Output signals and expressions

How to Set Up Test, Analyses and Simulation Information

The ADE Assembler gives you the flexibility to set up your tests, related analyses, and simulation parameters in two ways.

1. Use the **Data View assistant** in the ADE Assembler.
2. Translate to the **Virtuoso ADE Explorer Editing** environment using the **click here** link to create and add a single test (multiple analyses) in that environment.



You can switch between both environments anytime while retaining all the setup information.

cadence

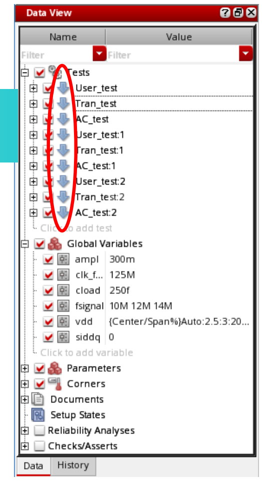
You are encouraged to try this out in the labs. In this module and its corresponding lab, we discuss a little bit on both these use models.

What Is the Data View Assistant?

The **Data View** assistant is where all the test setup information is visible and accessible.

- **Test(s):** Each testbench has a unique design.
- **Analyses:** Each test includes one or more analyses (tran, ac, dc, etc.).
- **Variables:** Global and local variable values can be enabled, disabled or modified.
- **Parameters:** Parameters are used for varying component values without requiring design edits.
- **Corners:** Can be added and enabled/disabled.
- **Reliability Analyses:** Set up and run stress and aged simulations.
- **Checks/Asserts:** Check for circuit violations using circuit checks and device Spectre assert statements.

Mutiple tests are set up and enabled.



Click **down arrow** to change to the ADE Explorer single test environment.

In the Data View assistant, you do not set up outputs; those are defined in the outputs pane in the Outputs Setup section. You can use it as a single user interface to quickly view and set up commonly used setup data.

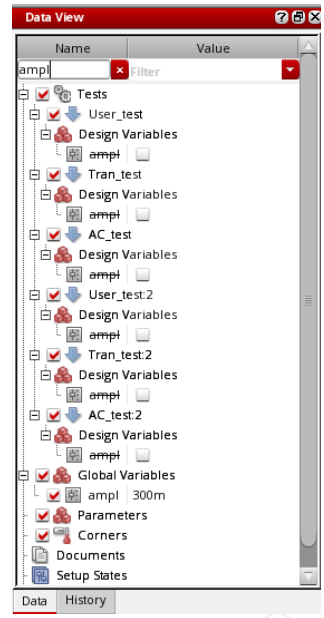
- The name of the test setup is determined by you, and it does not need to match the name of the design. The name of the design (Lib/Cell/View) is visible in a tooltip when hovering over the test name.
- Device models, included files, global options for saving, and other simulation-specific options are specified in the Test Editor, which is described in the next slide.
- Parameters are different from design variables in that they provide the ability to vary component values without editing the design. They can also be correlated to one another.
- Parameterization is used extensively for design optimization runs.

Filtering Data in Data View Assistant

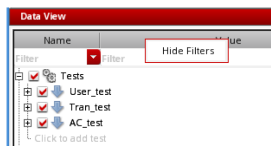
The Data View assistant includes search filters for the **Name** and Value columns that you can use to filter the setup data based on the specified criteria.

These filters are displayed by default and remain applied when you transition from ADE Assembler to ADE Explorer and vice-versa.

- Name: Includes names of the setup elements, such as variable, parameter, corner, etc.
- Value: Includes the values of the elements defined in the setup.



If you don't want to use the filters, **right**-click on the column headers and choose Hide Filters.



Related Environment Variables:

- *maestro.gui nameDisplayWidthInDataView* → The two columns are separated by a divider bar. Use this to control the width of the divider bar.
- *maestro.gui showFilterInDataView* → You can enable/disable the filters.
- *maestro.gui autoExpandSetupStatesSubTree* → The setup states sub tree is collapsed if this variable is not set.

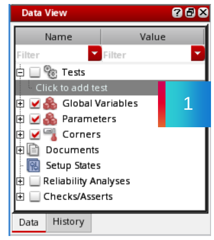
How to Add a Test in the ADE Assembler

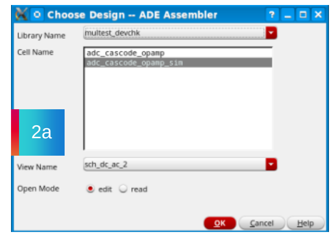
Your testbenches should be available in the Schematic Editor so that you can add tests to verify your DUT.

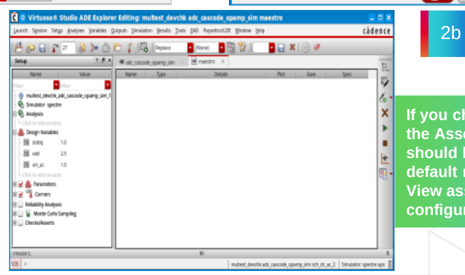
You can add a single (or multiple) test(s) in the Data View assistant in the ADE Assembler.

1. Click on ***Click to add test*** under Tests.
2. In the Choose Design form:
   a. Choose the schematic ***View Name*** that contains the testbench for which you want to configure your test.
   b. Click **OK**.

Your test gets created with a default name, and the cockpit automatically transitions you to the ADE Explorer Editing environment to set up your test.







If you choose to go back to the Assembler, your test should be visible with a default name in the Data View assistant; you can configure it there.

Your first test could also have been set up using the ADE Explorer Editing window if you use the hyperlink on the Welcome page.

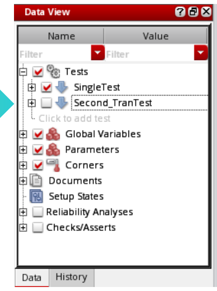
To add a second test, click on the **Click to add test** link again in the Data View assistant.

In the ADE Explorer Editing window, you cannot add multiple tests as it is a single test environment. You can simply configure tests in this environment.

When you delete a test, all associated setup parameters, including any outputs that you have set up, are deleted.

Enabling Specific Tests in the Data View Assistant

- Once you define a test(s), they are displayed in the Data View assistant as an expandable tree.
  - Click on the **+** or **–** icon to expand/collapse details about the test.
- Tests may be individually enabled or disabled by checking or unchecking the box in front of the test in the Data View assistant.

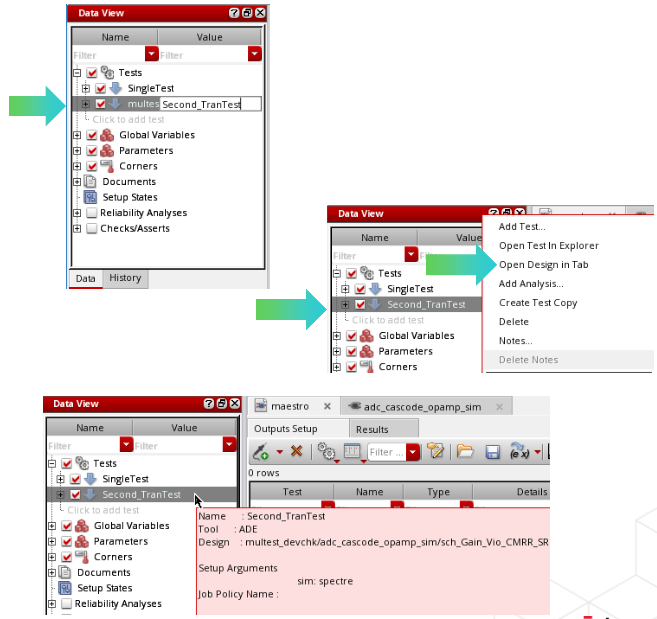


The test *Second_TranTest* will not be run when the Run button is pressed.

This page does not contain notes.

Renaming Tests in the Data View Assistant

Tests can also be renamed in the ADE Explorer Editing window.



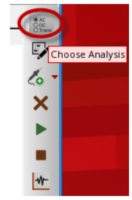
- Tests can be *renamed* in the Data View assistant by clicking twice on the test name. A new name can be entered directly in the inline editor that is invoked.
- The design corresponding to a test can be opened in a tab by
  - **Right**-clicking on the test and
  - Choosing the menu item in the pop-up form, **Open Design in Tab**.
- Placing the mouse over the particular test will invoke a tooltip showing details about the test.

Tests can also be renamed in the ADE Explorer Editing window.

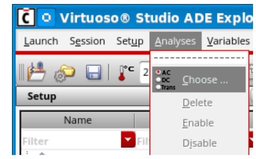
Using the ADE Explorer Editing Window to Configure Analyses

ADE Explorer menu bar→ **Analyses** → **Choose**

Click the **Choose Analyses** icon.



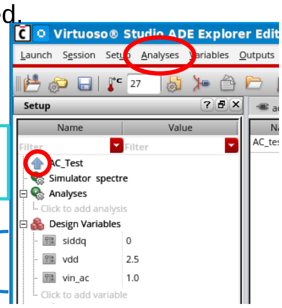
Or

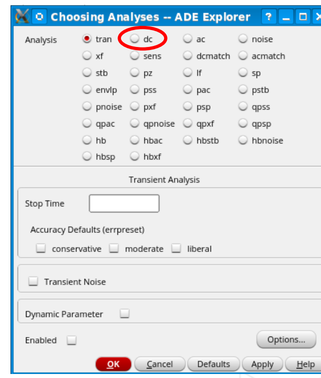


- To add new analyses in the ADE Explorer Editing window, click the **Choose Analyses** icon or select **Analyses – Choose**. All of the supported analyses for your target simulator will be listed.
- You can also set up model files, simulators, analog options, etc., using this environment.

Click the **Up** arrow if you want to complete the setup in the Data View assistant.

Design variables coming in from the design schematic.

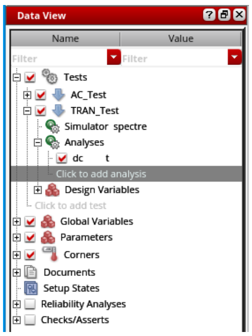


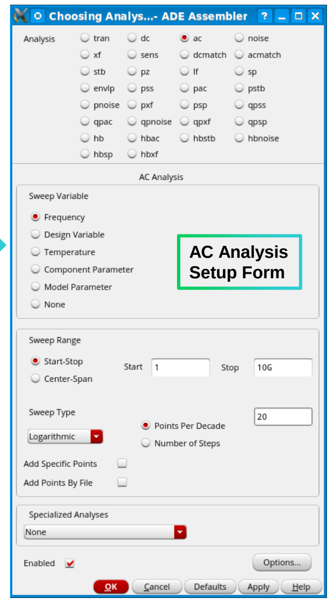


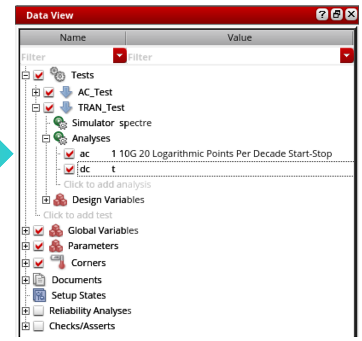
DC Analysis setup is visible in the Explorer.

This page does not contain notes.

Adding Analyses in the Data View Assistant







Expand the test and click on the **Click to add analysis** section of the Data View assistant to open the Choosing Analyses form also.

Feel free to explore other setup methods in the lab, especially the context-sensitive menu accessible from the Data View assistant. For more information on setting up an analysis, refer to the *Virtuoso ADE Explorer and Assembler S1: ADE Explorer and Single Test Corner Analysis* course.

Local and Global Variables

Local variables are variables that are local to a specific test.

- Local variables are also called **Design** variables.
- When a test is opened for editing in the ADE Explorer, the variables shown in the Setup assistant and used for simulations are the local design variables of that test.

Global variables are variables that are global across all the tests.

- Global variables (when enabled) override local design variables of the same name without having to modify the value in the test itself.
- Global variables are not supported in the ADE Explorer.

- Variables can be added or deleted in the Data View assistant for each test.
- Variable values can also be added/edited/overridden in the Variables and Parameters assistant in the Virtuoso ADE Assembler environment.
- You can sweep both local and global variables.

Variables are retained when transitioning between the Explorer and the Assembler.

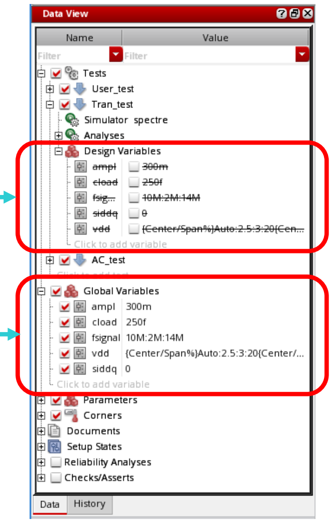
Variables in the Data View Assistant

The Data View assistant in the ADE Assembler lets you add, change and delete design variables.

- Variables are displayed as either:
  - Local Design Variables: Associated with individual tests.
  - Global Variables: Applicable to all tests.
- Variable values can be edited by **double**-clicking on the variable value.
- Values do not always need to be numeric. You can use the calcVal function to create expressions.

Variables local to a specific test

Variables global across all tests

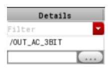


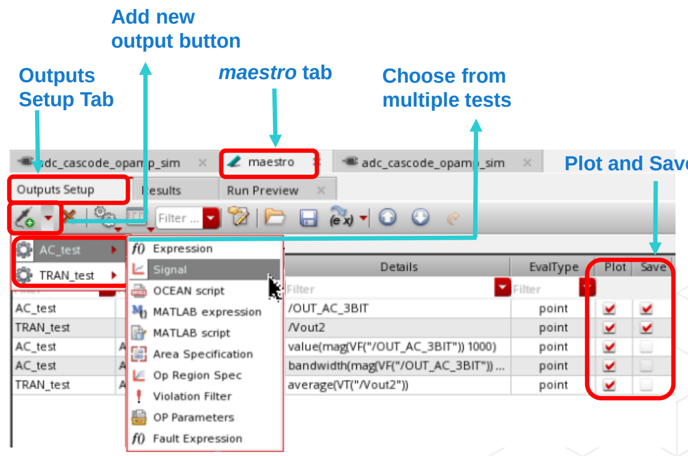
In this example, the design variables are overridden by global variables.

- Local design variables are specific to each test and can be overridden by global variables.
- Design variables also appear in the Variables and Parameters assistant, and their values may be changed there as well. Any changes in the Variables and Parameters assistant are reflected in the Data View assistant and vice versa.

Adding Multiple Signals to Be Plotted

Here, you see how a signal can be added to the Outputs Setup tab to be plotted. The signal needs to be added from the schematic. This schematic that pops up when you select a signal to add is the one that correlates to the test that you chose.

1. On the **Outputs Setup tab** of the **maestro** tab, click the down arrow next to the **Add new output** button.
2. Select the test (in the example shown, AC_test) and select **Signal**. You can now see a signal in the **Type** column.
3. **Double**-click on the new blank field in the Details column and click the **Browse** button to bring up the testbench schematic. Here you can select the signal net you want to be plotted. 
4. Select the checkboxes for **Plot** and **Save** as needed. Now you can simulate.

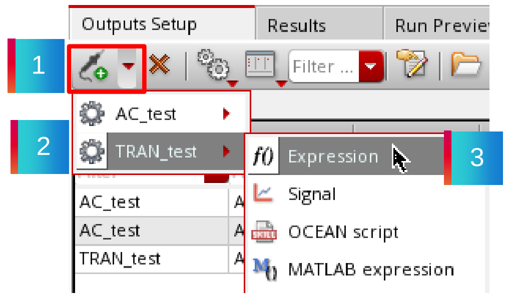


- You can select the **plot** and/or **save** checkboxes for signals/expressions that you want to plot and/or save.
- The Browse button in the Expressions/Signal/File column pops up a relevant schematic or brings up another Choose a File form, depending on the Type of Output that you choose for your setup.

How to Create Expressions in the ADE Assembler

An Output expression can be added or modified using the **Outputs Setup** tab of the Outputs pane. You can type an output expression directly in the *Details* field or use the expression builder to build an output expression.

1. On the **Outputs Setup** tab of the **maestro** tab, click the down arrow next to the **Add new output** button.
2. Select the test for which the measure is to be created. TRAN_test is chosen in the snapshot that you see here.
3. Select **Expression**. This will add a new row to the Outputs table with the *Type* set to expr.

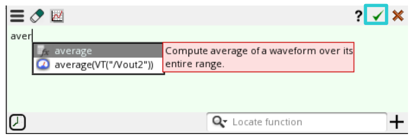


- When you add a new expression for a particular test, you need to create the expression using the Virtuoso Visualization and Analysis Calculator tool.
- You can choose to plot and/or save the value of this expression. (Plotting is applicable only to multi-iteration simulations such as Monte Carlo, parametric sweeps, and corners.)

How to Create Expressions in the ADE Assembler (continued)

4. **Double-**click in the Details field, and click the **Open expression builder** icon.

5. You can use the Expression Builder to create output expressions.



6. Once done, click the **OK** check mark to add the expression to the outputs setup section.

7. Optionally, supply a name for the expression in the Name field.

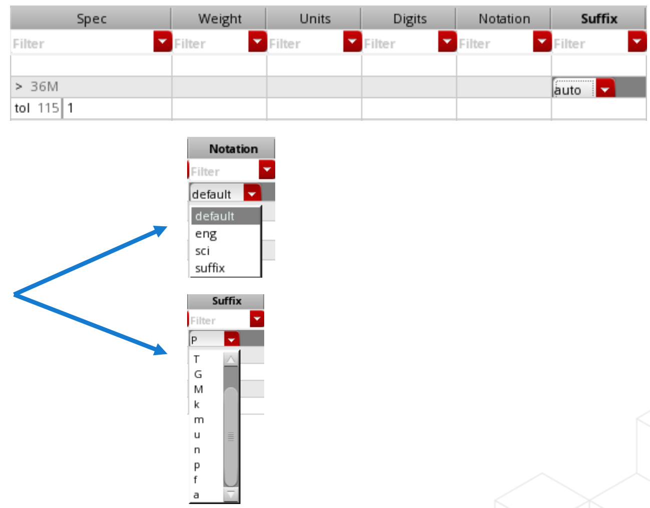
This page does not contain notes.

Spec-Related Features: Outputs Setup Tab

Right-click on the **Pull-down menu** to select the filter

- Weighting factors can be specified.
- The **Units** column lets you specify the units of your measurement.
- The **Digits** column lets you specify the number of significant digits in your measurement.
  Valid values are 2 to 15.
- You can select the notation style in the **Notation** column through a drop-down list.
  Possible values are *suffix*, *engineering,* and *scientific*.
- You can choose to leave the value in the **Suffix** column to auto, or you can choose from a list of values.
- If you choose Notation as engineering and the Suffix as m, then 0.0001 is represented as 0.1m.

Expressions with specs are evaluated to meet pass/fail criteria. You can define a specification limit for each expression under the **Spec** column.

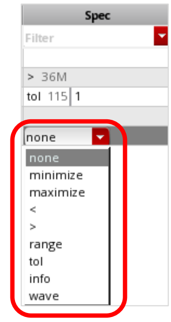


- Weighting provides the ability to place more emphasis on some specs when running optimization. Specs with greater weighting will be met, sometimes at the expense of others.
- Weighting is usually specified for very important specs or specs that are not being met with default weighting. Weights are a positive integer.
- The tool multiplies the cost function for that specification by the integer specified for the weight. This changes the way the tool searches the design space.
- Note that the Assembler can usually find solutions without weighting, and so the guideline is that you run the simulations first with all specifications weights set to 1, which is the default.
- You can set the number of significant digits and formatting for all scalar outputs with the Default Formatting Options form accessible by going to **Options – Outputs Formatting** from the ADE Assembler banner menus.

How to Add Specifications to the Outputs

You can add specifications to any output expression or signal (wave spec only) on the Outputs Setup tab either before or after a simulation.

- Click in the Spec column for the measure, choose a spec type, and set a value or values. Spec types are <, >, *tol (tolerance)*, *range*, *info*, *maximize* and *minimize*.
- If you choose tol or range, then two new input fields pop up in the Spec column.
  - For tolerance, the first field is the midpoint, and the second is the percentage tolerance around that midpoint.
  - For range, the first field is low value, and the second is high value.
- The wave spec option works to compare waveforms in a transient analysis. It provides a numerical pass/fail result.



cadence

- The Spec cell has two parts: a specification type (which you select from a drop-down list) and a target value. The target value is usually numeric.
- Once you set a specification, post-simulation, the Result tab will contain a Pass/Fail column which will indicate whether your simulation has met the specification or not.
- Here, *tol* specifies that the result must be within the specified percentage deviation of the target value.
- The *info* value can be chosen when you want the measured value to appear, but there is no target value or performance objective to meet.
- The *maximize* and *minimize* spec types are used for Optimization routines.
- If a specification is added after simulation, click the re-evaluate button (yellow icon showing a loop of two arrows) in the Results tab to update the display.

Specification Types

Specification	Description
Maximize	Maximizes the value of measurement. The value specified is the minimum value this measurement should have to pass.
Minimize	Minimizes the value of measurement. The value specified is the maximum value this measurement can have to pass.
>	The measurement result needs to be greater than the specified value.
<	The measurement result needs to be less than the specified value.
tolerance	Two values need to be specified – a target value and a % tolerance. Measurement results must be within tol% of the value specified to be considered passing.
range	Requires a minimum and maximum value. Outputs within the min/max are considered to be passing.
info	Prints the measurement value only. No pass/fail information is provided.
wave	Compares waveforms for transient runs and provides a failure count when points are out of a provided tolerance range.

This page does not contain notes.

Creating and Displaying a Netlist

If you want to netlist without simulating, you can do it from the context-sensitive menu accessible from the Data View assistant.

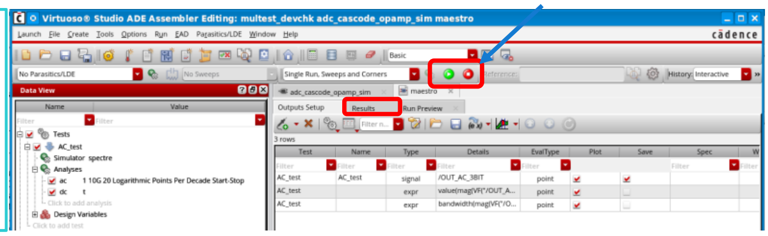
Netlist Task	Action
Incremental Netlisting	**Right**-click on the test and select **Netlist** – **Create** to create a netlist. Only blocks of the design that have changed since the last netlist was created will be re-netlisted.
Forcing Recreation of a Netlist	**Right**-click on the test and select **Netlist** – **Recreate** to force all the blocks in your design to re-netlist.
Viewing a Netlist	**Right**-click on a test and select **Netlist** – **Display**.
Automatic Creation	Click the green **Run Simulation** button on the simulation toolbar. This will netlist and simulate your design in a single step.

- The Assembler creates or updates the simulator input file (final netlist) automatically when you give the command to run a simulation. The input file will include both the structural netlist as well as analysis statements, including model paths, output statements, etc.
- The environment creates hierarchical netlists incrementally: Incremental netlisting is faster than full hierarchical netlisting because netlists are created only for blocks that have changed since the last netlisting job was performed.

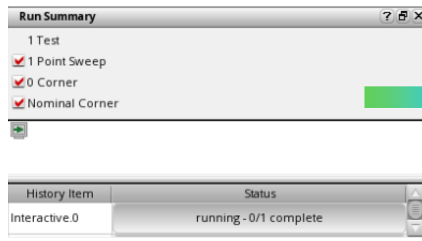
Running a Simulation in the ADE Assembler

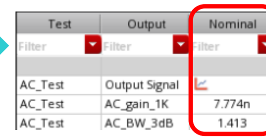
The **Run Simulation** icon can be used to automatically create a netlist of the design and run the simulation for the run mode selected in the **Select a Run Mode** drop-down list.

Run Simulation and **Stop Simulation** icons.



- Click **Run Simulation**.
  - The tool shifts to the **Results** tab.
  - The progress of the simulation is visible in the *Run Summary* window.
- The **Stop Simulation** button can be used to stop the simulation.
  - Any outputs that resolve to scalar values will be shown in the *Results* table.



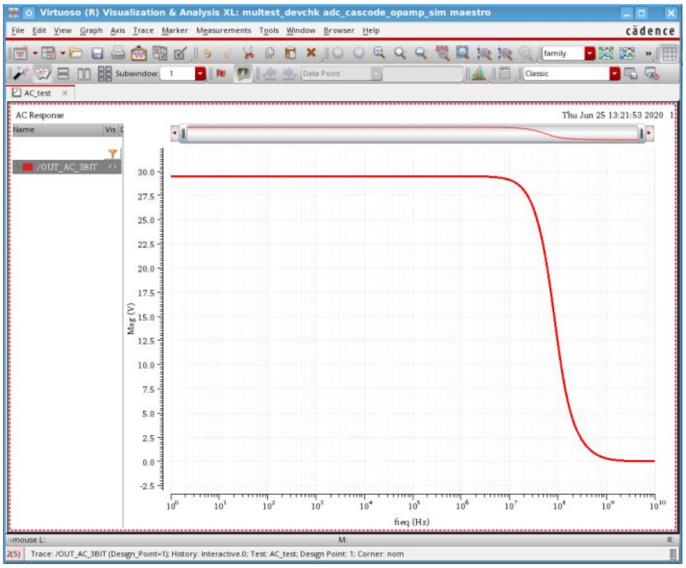


Numerical results appear here. Signals with the *Plot* icon can be plotted in the waveform viewer.

- The Results tab of ADE Assembler will display the signal and scalar values for your output expressions. You can interactively plot a specific waveform by **double**-clicking on the waveform icon.
- The outputs' display can be configured in various ways:
  - You can specify specification limits for your scalar outputs.
  - The number of significant digits for your scalar results, as well as the notation style, can be specified.
  - Signals can be automatically plotted at the end of the run if the Plot Boolean for that signal is selected and if you set the Plotting Option to Auto in the Plotting/Printing Options dialog.
- The usage of these controls is described in more detail in other modules of this course.
- You can run a single simulation from the ADE Explorer Editing window too, which is useful when debugging the setup of a single test.

Automatic Waveform Display for the Plots

If signals are set up to be automatically plotted in the ADE Assembler Plotting/Printing Options form, the Virtuoso VA Graph assistant will be launched at the completion of the run with the signals displayed.

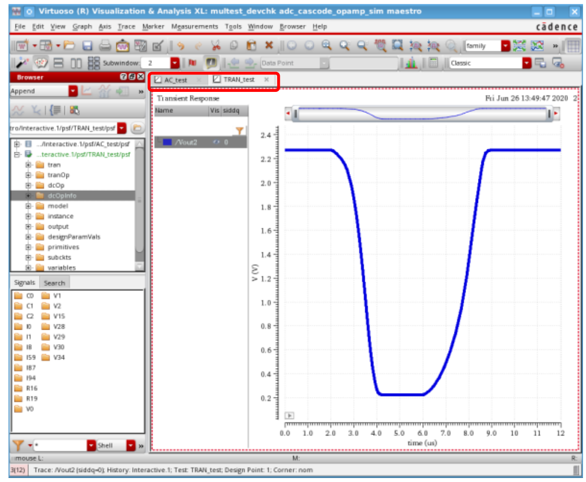


The graph background color and trace width have been modified in this snapshot for better clarity of the image

This page does not contain notes.

Automatic Plotting of Outputs from Multiple Tests

- Signals that are auto-plotted from different tests will be sent to individual tabs of the Virtuoso VA cockpit.
- Auto-plotted signals from each ADE Assembler test go to a separate tab.
- Tabs will be named according to the test.
- In this example, the TRAN_test tab is active.

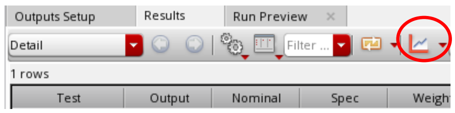


- When plotting multiple corners, the ADE Assembler corner order is maintained.
- ADE Assembler corner parameters and additional sweep parameters are available for sorting and grouping.

How to Manually Display Waveform for the Plots

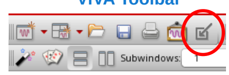
If the plotting option is set to *None* in the *ADE Assembler Plotting/Printing Options* form, then nothing will plot.

- In this case, **double**-click on the **Plot** icon under the result column (Nominal when you don't use corners) for the signals you want to be plotted.
- Alternatively, click the **Plot All** button in the toolbar of the **Results** tab to plot signals in the ViVA window.

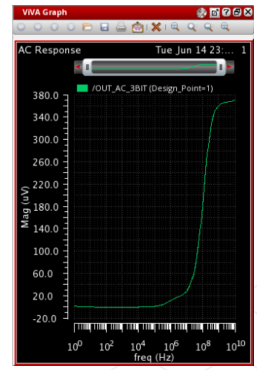


- Click the **Dock Window** button to open the ViVA Graph assistant, docked to the right side of your cockpit, and display your signal in it.

ViVA Toolbar



Docked ViVA Graph assistant



This page does not contain notes.

Interpreting Result Display for Spec Pass/Fail

Numeric values of measurements are displayed with color-shaded backgrounds that indicate whether the measured value meets the specification limits for that expression.

- A result value in **green** means that the specification has passed as it has met the condition you specified.
- A result value in **yellow** means that it is near or within 10% of the value you specified in the Outputs Setup tab in the Spec column.
- A result value in **red** means that the specification has failed as it is more than 10% of the value you specified in the Spec column.
- Some results are generated with a plot icon which indicates that the result data is available for plotting in the Virtuoso® Visualization and Analysis waveform viewer.
  - **Double**-click on the icon to plot the waveform.

cadence

The percentage value corresponding to yellow colorization can be adjusted by the following settings:

In *.cdsenv*:

```
maestro.setupdb percentageForNearSpec int 10
```

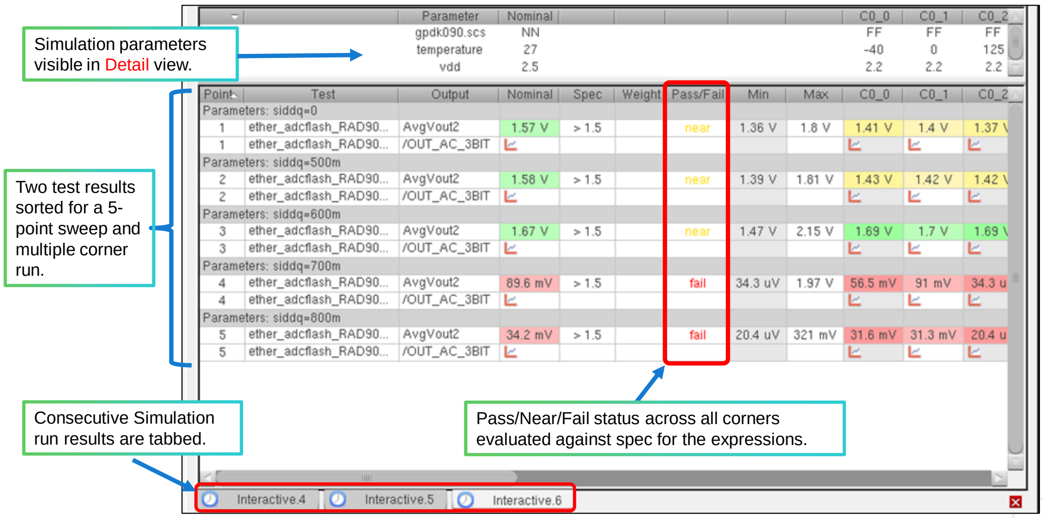
In *.cdsinit*:

```
envSetVal("maestro.setupdb" "percentageForNearSpec" 'int 10)
```

You can set this value to 0 to show only green/red for pass/fail.

Example: Results for a Corner Run with Specs

This is an example of a corner run in the Detail view. You can see the corners listed. Look at the Pass/Fail column, which lists the status.



There are multiple options for displaying results that you can choose from **Select the results view** pull-down menu. This is the Detail option.

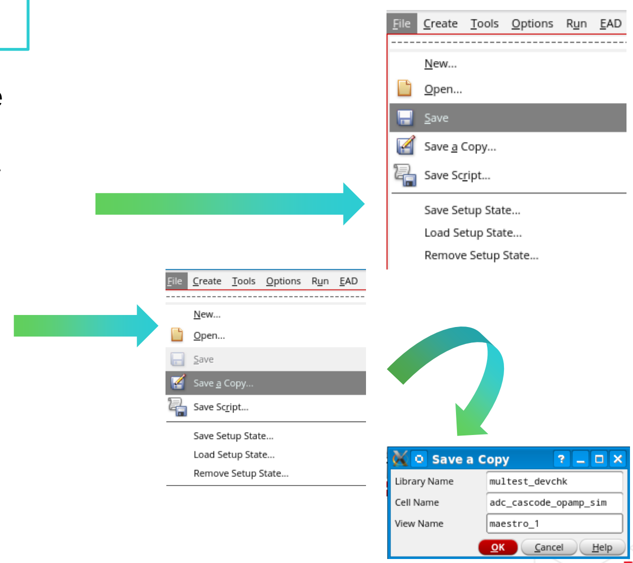
When running iterative simulations such as corners, the *pass*, *near*, or *fail*, the status displayed for output expressions in the Pass/Fail column is based on the status of the output across all corners.

Saving Your Simulation Setup

ADE Assembler menu bar → **File** → **Save** or **Save a copy**

You would most likely not want to repeat all the setup procedures repeatedly.

- You can save your complete simulation setup by simply going to **File – Save** to save in your default maestro view.
- To save your simulation setup to a different maestro view, you can choose **File – Save a Copy**.
- In the *Save a Copy* form, you can specify your new view name. It will be available for you in the Library Manager.
  - This action does not save history data but only the setup database.

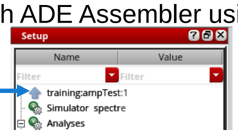


This page does not contain notes.

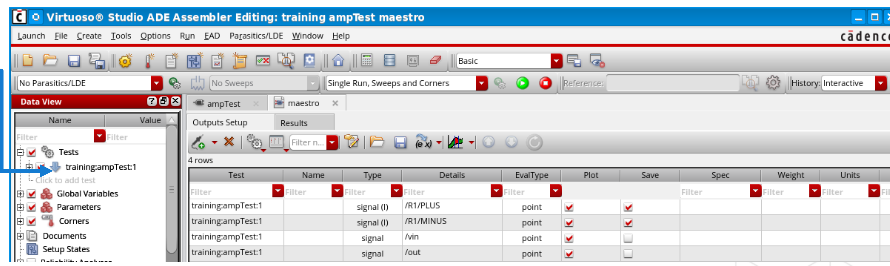
How to Switch Between ADE Explorer and ADE Assembler

In the ADE Explorer, you can work on one test at a time. When you need to work on different tests simultaneously, you can launch the Virtuoso ADE Assembler application that provides a multi-test environment where you can set up and run multiple tests.

- To work with multiple testbenches, launch ADE Assembler using one of these methods:
  - **Launch – ADE Assembler**
  - Click the **blue** upward arrow



- To switch back from Assembler to Explorer, use one of these methods:
  - **Launch – ADE Explorer**
  - Click the **blue** downward arrow

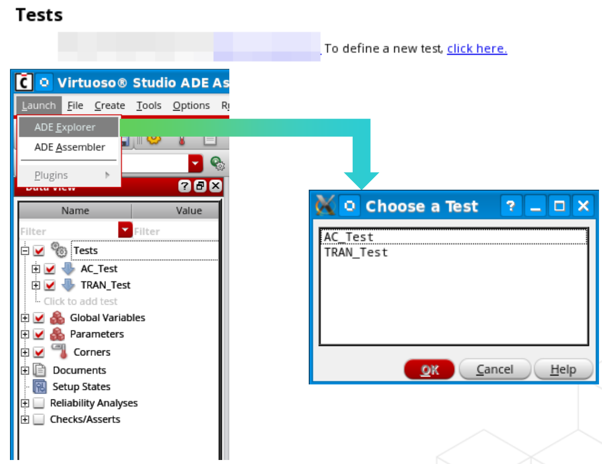


In Explorer, you can work on one test at a time. When you need to work on different tests simultaneously, you can launch the Virtuoso ADE Assembler application that provides a multi-test environment.

How to Transition to the ADE Explorer Editing Window

ADE Explorer is a single testbench, multiple analyses editing window. You can transition to it using the following methods.

1. Use the hyperlink **click here** under Tests, from the Welcome to ADE Assembler page.
2. By using the Launch menu:
   - If you have multiple tests, another form called **Choose a Test** comes up asking you to choose for which test you want to open the ADE Explorer.
3. Choose **Click to add test** from the Data View assistant.
4. **Double-click** on the test name in the Data View assistant.
5. Click on the down arrow preceding the test name in the Data View assistant.

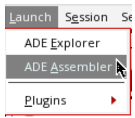


This page does not contain notes.

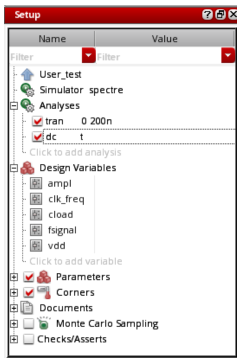
How to Transition from the ADE Explorer Editing Window

To transition from the ADE Explorer Editing window back to the ADE Assembler cockpit, you can:

- Use the **Launch** menu.



- Click the Up arrow in the Setup assistant.



This page does not contain notes.

Submodule 4.3

Real-Time Tuning(RTT) and Backannotating of the Tuned Parameters

This page does not contain notes.

Submodule Objectives

In this submodule, you will:

- Launch and examine the Real-Time Tuning (RTT) assistant
- Tune the parameters
- Backannotate the parameters from RTT

This page does not contain notes.

What Is Real-Time Tuning (RTT)?

Real-Time Tuning helps you to vary the values of variables and parameters and view dynamically updated simulation results.

- You run a nominal simulation once and fine-tune results later by changing variable and parameter values and viewing updated results in real-time.
- You do not need to netlist again.
- Output waveforms are updated in the Virtuoso VA graph window or pinned thumbnails in the schematic when you change variable and/or parameter value, and the simulation completes.
- You can change the plotting mode to compare waveforms after variable and/or parameter change.
- Scalar measurements are updated in the Value column of the *maestro* view.
- It works with a single-point run only.
- On exiting the assistant, simulation results prior to invoking RTT are displayed.

The plotting modes can be Append, Replace, New Window, and New Subwindow.

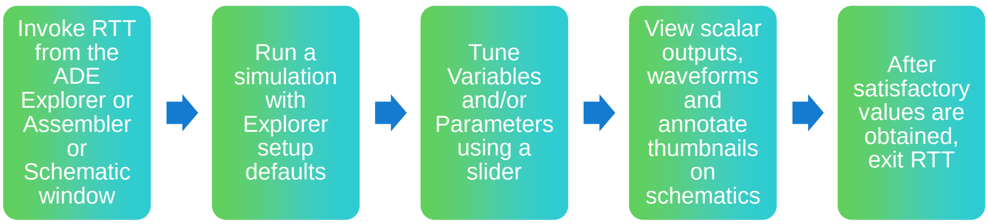
RTT simulation does not evaluate the outputs of type OCEAN.

This feature works with Spectre in SKI interactive run mode.

If reliability is enabled when you use RTT assistant to modify the design, the Spectre batch mode is used to run the RelXpert simulation and the following message is displayed:

*"*Info* Spectre SKI interactive mode is disabled in RTT because RelXpert setting is enabled. Automatically setting the run mode to batch"*

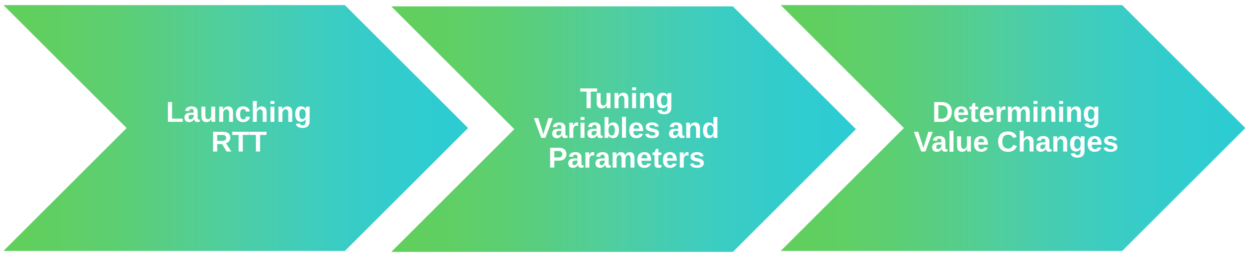
Summary of the RTT Flow



This page does not contain notes.

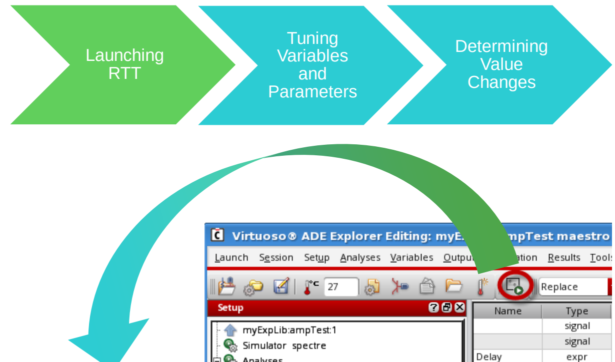
Working with the RTT Assistant

The Real-Time Tuning pane opens as a dockable assistant in the schematic window and ADE Explorer. In the RTT assistant, you can run the simulation once and fine-tune your results later by changing the variable values and simultaneously viewing the updated results.



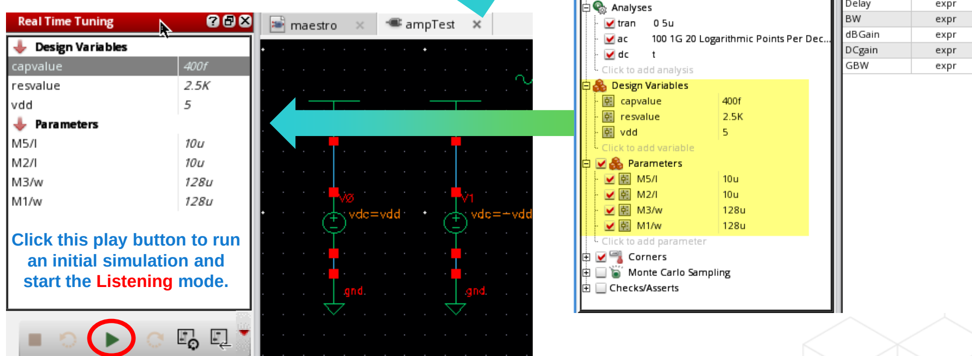
This page does not contain notes.

Launching the RTT Assistant



Consider the Explorer setup shown here in which you define the TRAN, AC and DC analyses, three design variables and four parameters.

- Open the **Real Time Tuning** assistant.
- RTT displays default values for **Design Variables** and **Parameters**.



Now, open the Real-Time Tuning assistant by clicking the RTT icon in the Explorer window. This action opens up the schematic in a separate tab with the Real-Time Tuning assistant on the left.

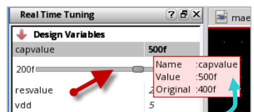
You can toggle between the output centric and schematic centric views.

Tuning Variables and Parameters

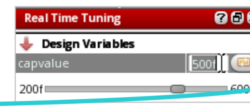
Launching RTT → Tuning Variables and Parameters → Determining Value Changes

1. Click the **Play** button to run a simulation and then enter into *listening mode.*
   a. **Play** button first turns into a button while running simulation and plotting waveforms.
   b. Then it turns into a **Pause** button indicating that simulator is live in background to run simulation based on the changes you make.
2. Change design variable and parameter values using one of these methods:
   a. Hover your cursor over design variable or parameter name/value to get the slider bar. The slider bar gives you min and max range value (+/-50% nom). **Move the slider to choose the desired value**.

As the slider is moved, results are updated with latest values



Changed values are displayed in ***bold****, unchanged in* ***italic****.*

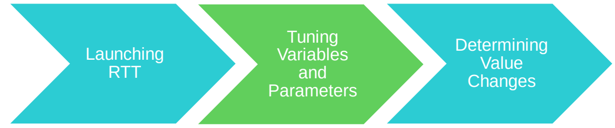


   b. Place a cursor on the changed value to see tooltip.
   c. **Double**-click the value displayed and type in the new value.
   c. Click this button to replace changed value to original value.
   d. The updated value is passed to simulator and an instant simulation run takes place. This simulation is fast because the simulation run does not generate the netlist again.

- You can also **double-**click and type a value greater than the maximum range displayed on the slider bar. The new value is displayed with a red background.
- By default, the simulation will start when the mouse button on the slider is released. This is controlled by the following environment variables:
  - *envSetVal(“maestro.rtt” “simulateOn” ‘cyclic “onMouseUp”)* → Default, variables, and parameters values are passed to the simulator as soon as the mouse button is released.
  - *envSetVal("maestro.rtt" "simulateOn" 'cyclic "adaptiveDelay")* → RTT evaluates the analysis run time from the simulation log in the first run, and then sets the delay time to that.

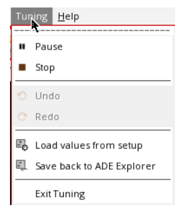
Tuning Variables and Parameters

(continued)



3. When in **Pause** mode, you can only change one value at a time and waveforms are updated accordingly.
   a. To change multiple values together, click the pause button once again.
   b. The button now changes to **yellow Hold** button indicating that the simulation run is on hold and does not happen every time you change a value.
   c. Change multiple values and click this **Hold** button to continue simulation run.
   d. To bring back previous values, use **Undo** and **Redo** buttons.
   e. To bring back the default values from previously saved setup, click the **Load values from setup** button.
   f. Click the **Stop** button to stop simulation and bring back the **Play** button to its default.

Tuning icons in RTT



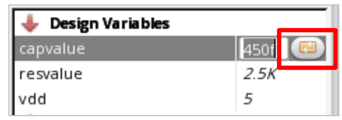
Tuning menu in RTT

This page does not contain notes.

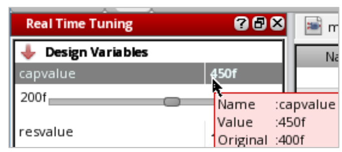
Determining Value Changes

Launching RTT → Tuning Variables and Parameters → Determining Value Changes

- Values in *italics* mean that the value has not changed.
- If the RTT shows values in **bold,** it means that the value has changed from the original value.
- When you **double**-click in the value field after it has changed from the original value, the **Browse** button has a gold icon on it. Click it to return to the original value.



- You can hover your mouse over the item to see its current and original value in a tool tip.

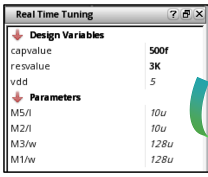


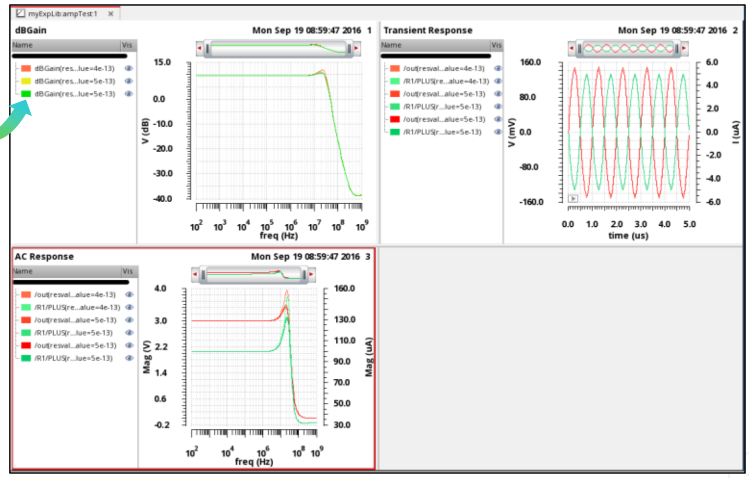
cadence

This page does not contain notes.

Viewing RTT Updated Results in the Virtuoso VA

When the simulation run is complete, the plots are automatically updated in the Virtuoso Visualization and Analysis Graph window. Now, if you change a parameter in the RTT assistant, the graph is updated as per the new values.





As you tune the design, the outputs, waveform thumbnails, and ViVA updates. Previous waveforms on the thumbnails dim.

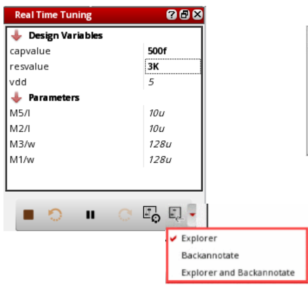
cadence

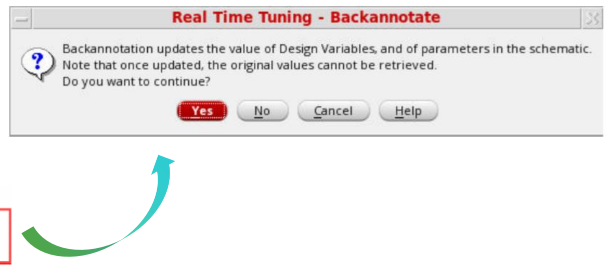
This page does not contain notes.

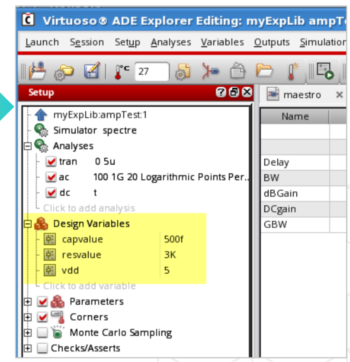
How to Backannotate Variables and Parameters from RTT

After tuning the design using the *Real-Time Tuning* assistant, values of the updated parameter and design variables are displayed in bold. You can send these updated values back to the ADE Explorer and the schematic.

1. Select one of the options from the *Save back* drop-down menu: **Explorer, Backannotate, Explorer and Backannotate.**
2. Click the **Save back** button. A dialog box is displayed. Click **Yes** to confirm the changes.







The CIW also shows a summary.

***Note:** It's important to note that utilizing the RTT assistant and back annotating the updated variables and parameters can also be accomplished in the A D E Assembler. The process remains unchanged.*

Explorer: updates the values of design variables and parameters in the setup assistant.

Backannotate: Updates the parameter values in the schematic. The design variable values are not updated in the schematic.

Explorer and Backannotate: Updates the values of the design variables and parameters in the setup assistant, and that of parameters in the schematic.

When you click the *Save back* button in RTT assistant, only the updated design and parameter values are transferred back into the original setup. RTT does not send the updated results to the ADE Explorer setup. In order to view the updated results in ADE Explorer window, you need to rerun the simulation.

References

Virtuoso ADE Explorer and Assembler full course reference

- S1 ADE Explorer and Single Test Corner Analysis and S1 Spectre Basics
- S2 ADE Assembler and Multi-Test Corner Analysis
- S3 Sweeping Variables and Simulating Corners
- S4 Monte Carlo, Real-Time Tunning and Run Plans

Virtuoso ADE Explorer and Assembler Training Bytes

- Virtuoso ADE Explorer(Channel Video)
- Virtuoso ADE Assembler(Channel Video)

Virtuoso ADE Explorer and Assembler User Manual

- Virtuoso ADE Explorer User Guide
- Virtuoso ADE Assembler User Guide
- Spectre Circuit Simulator Reference

These links are accessible from the Reference section of the course interface.

This page does not contain notes.

Module Summary

In this module, you learned how to

- Set up ADE Explorer and Assembler simulation environments by:
  - Defining the variables
  - Setting up different types of Analyses: transient, DC, and AC
  - Defining the Model Library
  - Generating the expressions using the Calculator
  - Defining the Parameters and Outputs
- Run the simulation using the Spectre simulator
- Analyze the simulation results using the Virtuoso VA
- Dynamically fine-tune the simulation results using the Real-Time Tuning assistant

This page does not contain notes.

Labs

Lab 4-1 Setting Up ADE Explorer and Assembler for Testbench Simulation

Lab 4-2 Adding Expressions and Analyzing the Results

Lab 4-3 Tuning Design Parameters Using Real-Time Tunning Assistant

Lab 4-4 Updating Design with Tuned Parameters (Backannotation)

cadence

This page does not contain notes.

Module 5

Layout Design

This page does not contain notes.

Module Objectives

In this module, you will:

- Use Virtuoso® Studio Layout Suite EXL to build a new layout cell from scratch. It involves:
  - Placing device layouts with the Auto Place & Route(Auto P&R) Assistant
  - Optimizing the pin placement for ease of routing
  - Performing device routing in three different ways:
    - Pin-to-Trunk(P2T)
    - Auto Route using Wire Assistant
    - Manual Routing

This page does not contain notes.

Terms and Definitions

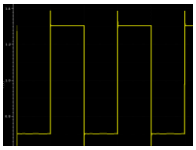
APR	The Auto Place and Route(APR) flow comprises a series of tasks to generate automatically placed and routed layouts. Auto P&R and Routing are the two assistants through which you can quickly design your layout.
PDK	Collection of files used by semiconductor designers to define the manufacturing process parameters and design rules for a specific semiconductor process node. It is crucial for ensuring the designed circuits are compatible with he manufacturing process.
GFS	Generate From Source(GFS) allows users to create layout representations of the schematic design components. It will delete existing components in the layout view and generates everything from scratch.
GSFS	Generate Selected From Source(GSFS) generates selected components from the source without affecting the entire layout.
P2T	Pin-to-Trunk(P2T) routing in Virtuoso Layout Suite is a routing topology style used for connecting pins to trunks efficiently. This style of routing is used in designs to ensure proper signal integrity and to improve overall layout productivity and design efficiency.

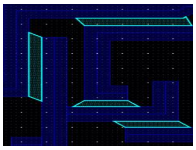
182

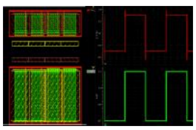
This page does not contain notes.

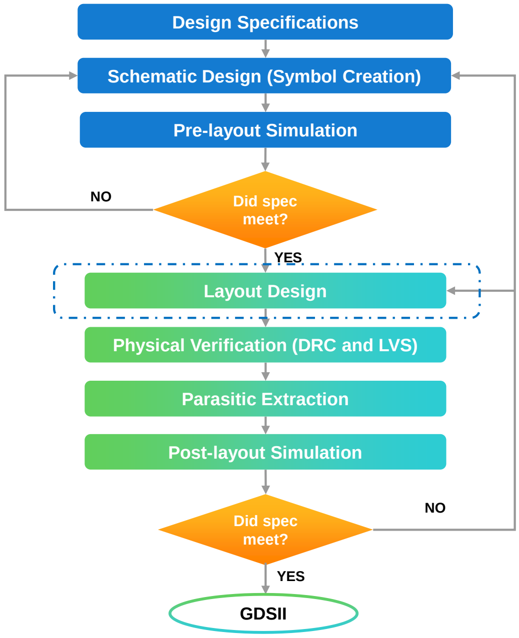
Analog IC Design Flow

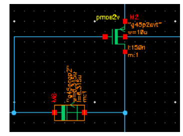
PARAMETER	MIN	MAX
Gain		1000
Bandwidth	50KHz	500MHz
Supply Voltage	1.77V	5.5V
Supply Current	1.23mA	10.26mA
Propagation Delay (τ)	16.7ns	43.7ns
Chip area		2μm^2

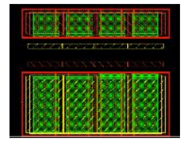


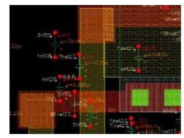


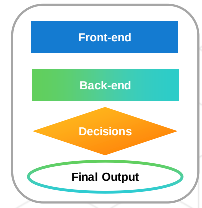










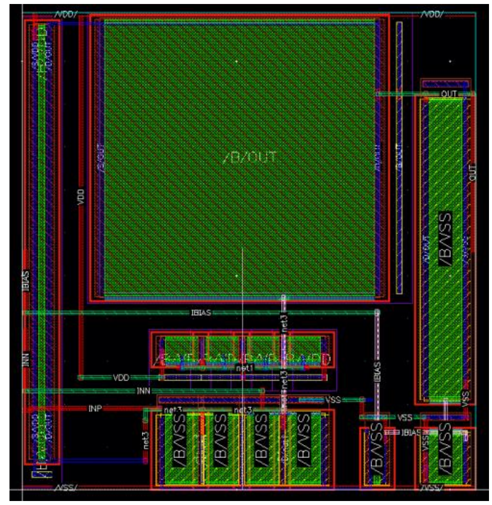


This page does not contain notes.

What Is Layout Design?

Layout design is a crucial stage in IC design flow that focuses on the physical arrangement of schematic blocks.

- Layout design is also termed physical design.
- You draw the geometry of symbols for a circuit, such as transistors, resistors, capacitors, and inductors, and connect them.
  - It involves arranging and routing the devices considering to reduce the parasitic effects.
- The layout also includes additional structures like wells, taps, guard rings, etc.
- The goal of layout design is to design an optimized layout by following design rules.

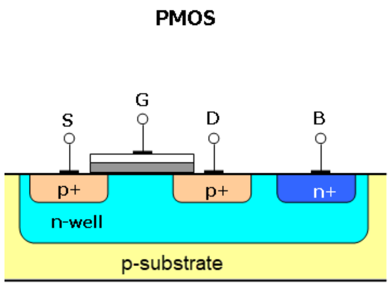


The physical configuration of the layout directly impacts the ICs performance and functionality.

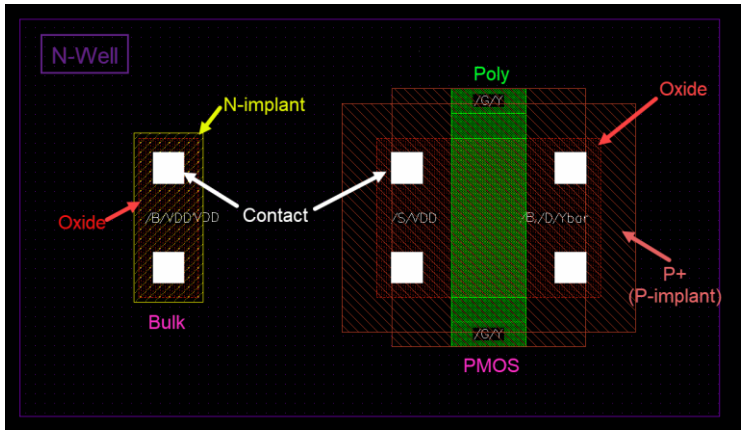
This page does not contain notes.

Example: Layout Design

Let us see what are the different layers that are needed to create a PMOS layout.



PMOS Structure

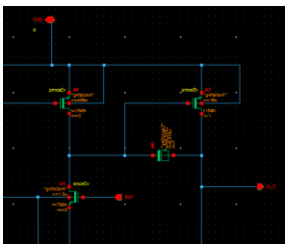


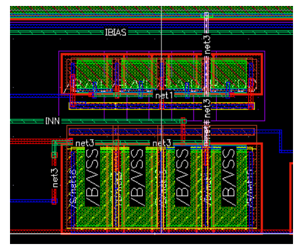
PMOS Layout

This page does not contain notes.

Difference Between Schematic and Layout Designs

Schematic Design	Layout Design
Shows the topological representation of the circuit.	Shows the geometrical representation of the circuit.
Represents the devices used in a design, defining the circuit topology and element interaction.	Involves arranging devices, placing components, and routing interconnections while considering capacitance, inductance, and resistance to reduce parasitic effects.
Aims to achieve the desired electrical characteristics.	Aims to meet the design rules for the fabrication process.
Consists of various primitive transistors and symbols.	Consists of various metal layer polygons.
Requires good knowledge in analog circuits design concepts.	Requires good knowledge of device physics, IC fabrication flow and artistic abilities for optimizing the layout.





This page does not contain notes.

How to Create a Layout EXL in the Virtuoso Studio Environment

To create the Virtuoso Layout EXL for the first time, you need to create a new layout cellview.

Use one of the following methods to create a *layout* view:

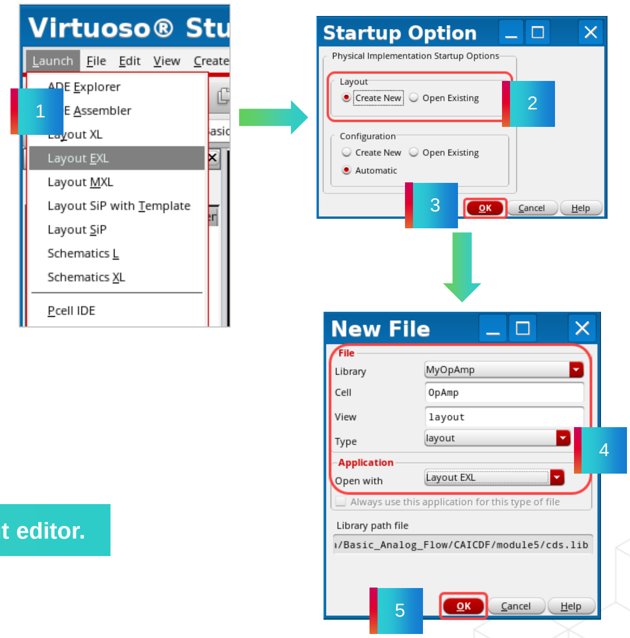
1. From the **Schematic Editor**
2. From the **CIW**
3. From the **Library Manager**

This page does not contain notes.

Method 1: Using the Schematic Editor

You can create a new *layout* view and open it from the schematic window.

1. In the **Virtuoso Schematic Editor** window, select **Launch – Layout EXL**.
2. Select **Create New** in the *Startup Option* window.
3. Click **OK** to open the *New File* form.
4. Choose View type as **layout** and Application as **Open with** (Layout EXL), which is already specified.
5. Click **OK** to open the new layout window.



This is the most common method to invoke the Layout editor.

VSE: Virtuoso Schematic Editor

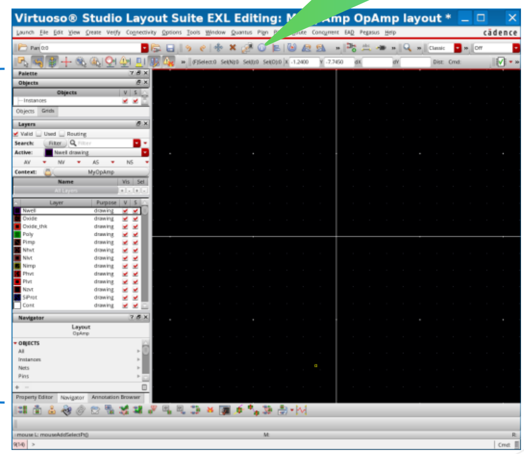
Method 1: Using the Schematic Editor (continued)

- The Virtuoso Layout EXL window appears:
  - Same menus as Layout XL
  - Enhanced toolbars with new features

These are Layout editor toolbars

You will be prompted to save any changes you make changes and when closing the layout window.

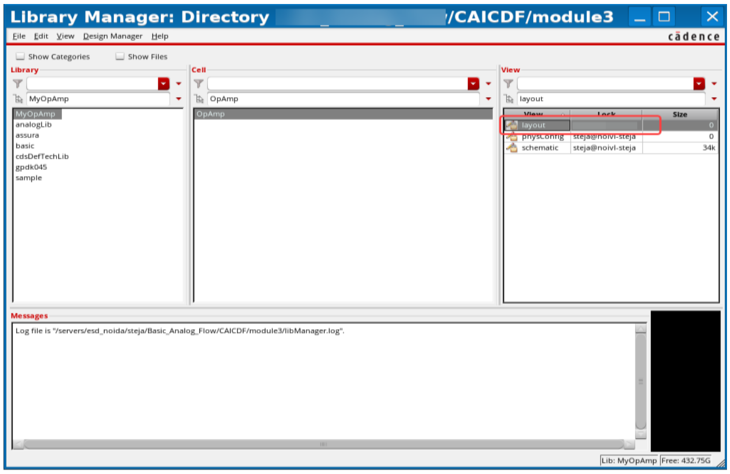
These are Layout Editor Assistants



This page does not contain notes.

Method 1: Using the Schematic Editor (continued)

- The new **layout** view is displayed in the Library Manager.
  - You can name this view anything that you would like.
  - You can **double**-click on the layout for this **cellview** to open.

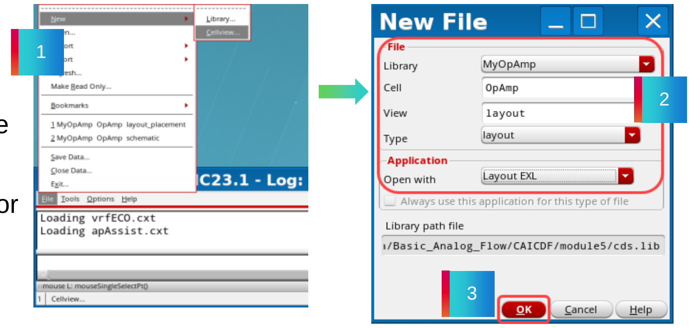


This page does not contain notes.

Method 2: Using the CIW

Open layout from the **CIW**.

1. Choose **File – New – Cellview** in the CIW. The *New File* form opens.
2. Set your preferred **Library** and **Cell** names. In the **View** and **Type** fields, give Layout as your view.
3. Ensure that **Open with** is set to the *Layout EXL* for your new layout cellview. Click **OK**.



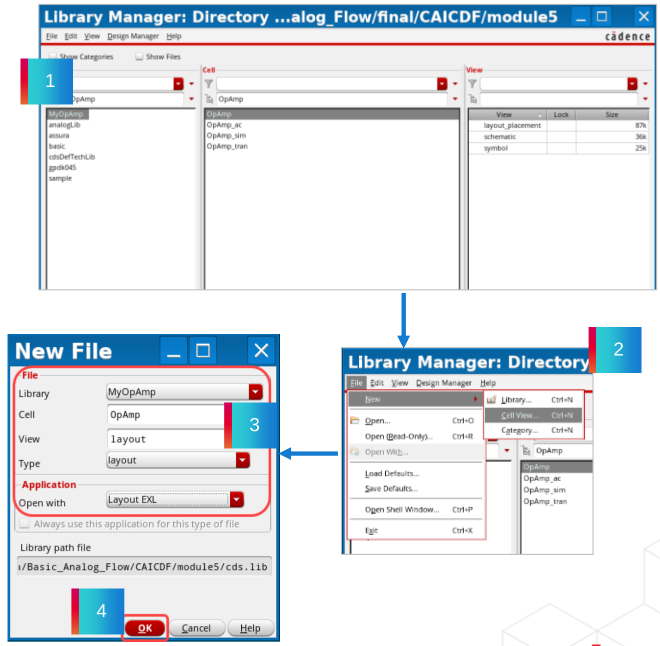
This is not a general practice to invoke the Layout Editor.

This page does not contain notes.

Method 3: Using the Library Manager

You can create a new *layout* view and open it from the Library Manager also.

1. In the **Library Manager** window, select **MyOpAmp** library and **OpAmp** cell.
   - Note that the **schematic** should **not** be selected.
2. Go to **File → New → Cellview** in the Library Manager window.
3. In the **New File** form, specify the **View** and **Type** fields as **layout**, **Open with** field as **Layout EXL**.
4. Click **OK** to launch the layout window.



VSE: Virtuoso Schematic Editor

Submodule 5.1

Device Placement Using Auto P&R Assistant

This page does not contain notes.

Submodule Objectives

In this submodule, you:

- Initialize and place the device layouts using Auto Place & Route (Auto P&R) Assistant
- Optimize the Pin placement using the Pin placement form

cadence

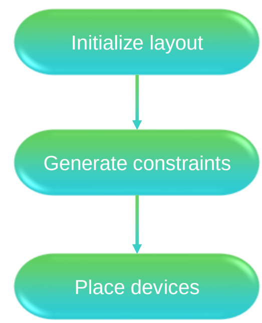
This page does not contain notes.

What Is the Automated Device Placement?

The APR flow (Device APR) in Virtuoso allows users to quickly generate placed layouts that are constraint-compliant, LVS-correct, and follow DRCs as captured in the Virtuoso technology file.

The Automated Device Placer solution in Virtuoso addresses the growing need for layout automation.

- A GUI assistant steps users automatically through the flow to generate a placed layout.
- The Virtuoso Automated Device Placement flow uses the device-level router.
- They are easy to modify to produce the final layout for signoff.
- Placement is driven by Groups (auto-identified or user-defined).
- Optimizes Placement of Groups by changing the Group Aspect Ratio.
- Optimizes for Area (and/or) Wire Length.
- Supports Symmetry Constraints (on/between) Groups or Devices.
- Provides multiple placement results with different Aspect Ratios.



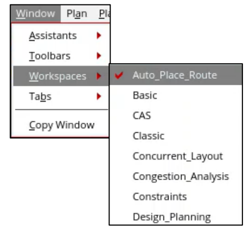
Note: This course talks about the APR features applicable to mature nodes only.

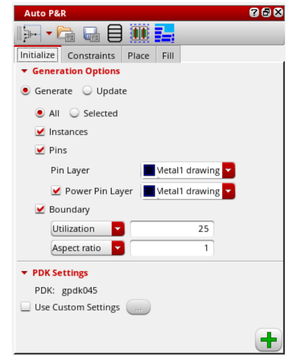
To know about the **APR flow for advanced node layout design,** visit the VLANT3: **Auto Place and Route (APR) for Virtuoso Studio: Device Level**.

The link for the course catalog is provided in the References section.

Auto Device Placement Workspace

Choose **Window -> Workspaces -> Auto_Place_Route** or choose **Auto_Place_Route** from the workspace from the menu bar.



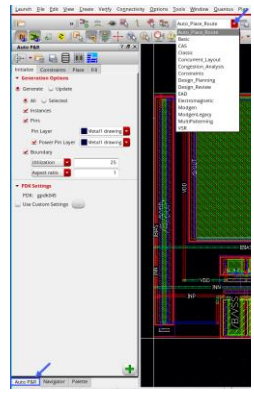


The Auto P&R assistant is the integrated, automatic placement solution in Virtuoso Studio.

The following are the two automatic placement modes:

- **Device:** Runs the automated device placer. This is the default mode.
- **Standard Cell:** Runs the automated standard cell placer.

The Auto P&R assistant allows you to initialize, generate constraints, place, and fill layout designs automatically.

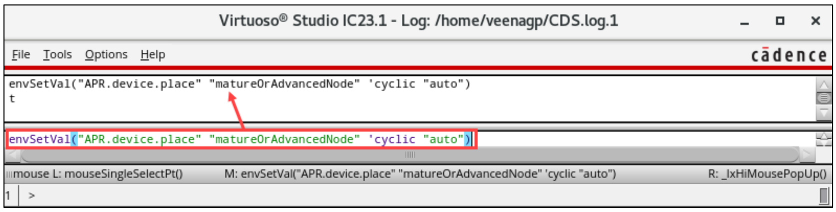


cadence

The tabs and options in the Auto P&R assistant differ based on the selected mode. Each tab represents a task in the automated device placement and routing flow. Follow the steps to generate an automated layout design.

Auto Device Placement Workspace (continued)

- Specify the design process node for the placer by entering the following command in the CIW.
  - When set to `auto`, the tool automatically identifies an appropriate node based on the constraint definitions in the design.
  - The default value is set to `advanced` node.



Note: This can be done using the Cdsenv editor and shown in the lab.

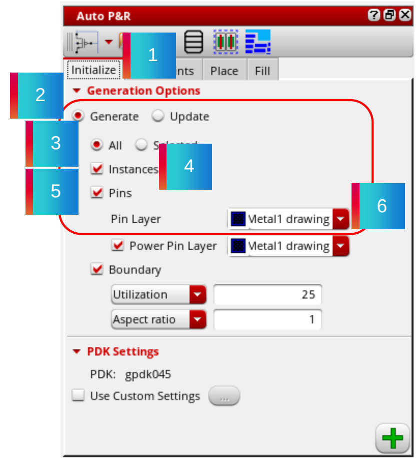
This page does not contain notes.

How to Initialize a Layout in the Automated Device Placement

Initializing and planning a layout involves generating the devices to be placed, generating the constraints and constraint groups to be honored during placement, and defining the placement region.

To initialize a layout view in the automated device placement flow:

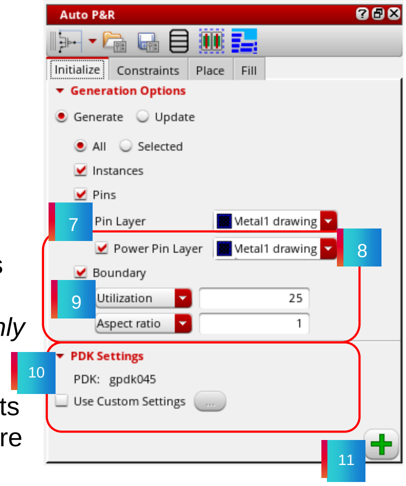
1. Open the **Initialize** tab of the Auto P&R assistant.
2. Expand *Generation Options* and select **Generate** to generate new objects in the layout canvas or *Update* to update the existing objects.
3. Set the scope of layout generation to either **All** or **Selected**.
4. Select **Instances** to generate all instances from the source schematic.
5. Select **Pins** to generate pins during initialization.
6. Select an LPP from the **Pin Layer** list that contains the device pins to be generated. The default value is the first metal layer in the layer stack.

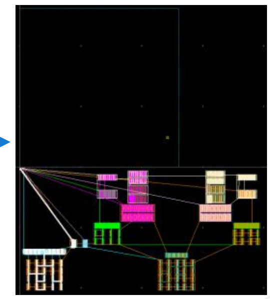


This page does not contain notes.

How to Initialize a Layout in the Automated Device Placement (continued)

7. Select the **Power Pin Layer** to generate power pins.
8. Select an LPP from the **Power Pin Layer** list containing the power and ground pins generated during initialization.
9. Select **Boundary** to generate a PR boundary as per the specified combination of **Utilization** and **Aspect ratio** or **Width** and **Height**.
10. Select Use Custom settings to load PDK settings from a file. (*This option is useful for loading additional CDF parameters that are applicable only at lower nodes*.)
11. Click **Apply** () to generate the selected objects in the layout canvas. All the instances and pins are generated below the PR boundary.





This page does not contain notes.

Generating Constraints and Constraints Groups in the Automated Device Placement and Routing Flow

To plan a layout effectively, generate devices, create device groups and constraints, and use automated device placement and routing with circuit finders. Also, organize structures into device groups with Modgens and symmetry information.

The Auto P&R assistant has preloaded circuit finders that group devices based on criteria. The APR prefix identifies customized finders for the Auto P&R flow. You can also set constraints, constraint groups, and the placement region.

- Use the **Constraints** tab of the Auto P&R assistant to find matching structures and generate constraints.
- Use Modgens to manipulate device arrays for easy matching.
- View Modgens and symmetric structures by default in the Constraint Manager; these constraints are applied during design placement and routing by the automatic device placer and router.

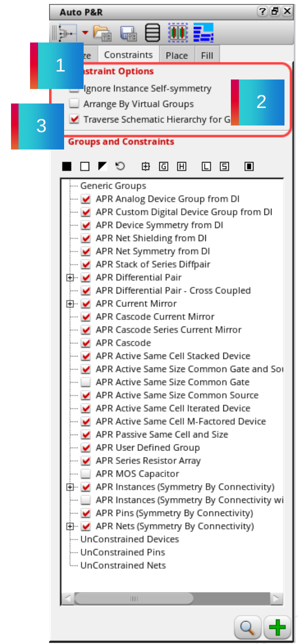
200

For a list of supported circuit finders, refer to APR Circuit Finders.

Generating Constraints and Constraints Groups in the Automated Device Placement and Routing Flow (continued)

To generate constraints:

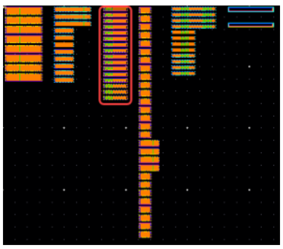
1. Go to the **Constraints** tab of the Auto P&R assistant.
2. Select **Ignore Instance Self-symmetry** to specify that instance self-symmetry constraints are to be ignored.
3. Select **Arrange by Virtual Groups** to preview how the devices will be arranged vertically based on their virtual group before running the placer.

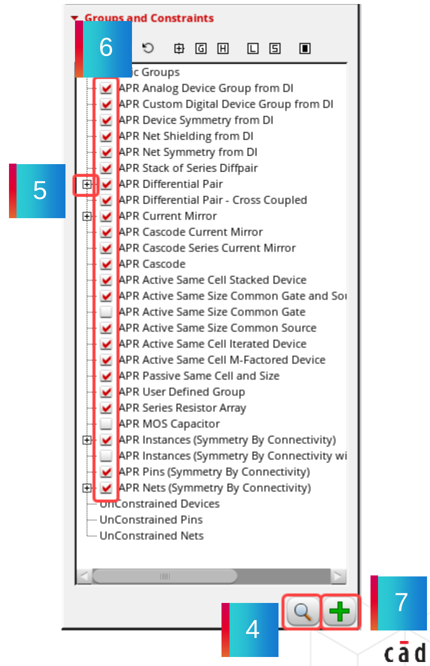


This page does not contain notes.

Generating Constraints and Constraints Groups in the Automated Device Placement and Routing Flow (continued)

4. Click the **find** icon ( ) to search for all matching structures in the source cellview following the order in which they are listed.
5. Click **[+]** to expand each group to view matching device groups.
6. Select the device groups for which constraints are to be created.
7. Click Create ( ) Constraints.





202

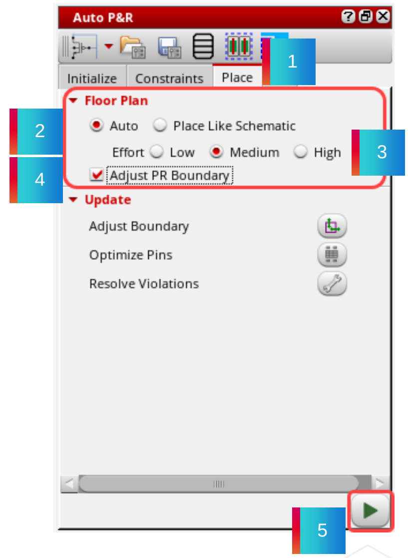
This page does not contain notes.

How to Place the Device Layouts Using Auto P&R Assistant

The auto-placement is done by separating the devices based on the constraints defined in the constraints manager.

To place a layout view in the automated device placement flow:

1. Go to the **Place** tab of the Auto P&R assistant.
2. In the *Floor Plan* section, select the required placement style:
   a) In the ***Auto*** style, the placer will optimize the placement with minimum area.
   b) In the ***Place Like Schematic*** style, the placer places the devices as in the schematic. Here, the area consumed will be more than the Auto style.
3. Set the ***Effort*** levels based on the complex constraints and size of your design.
4. Enable the checkbox **Adjust the PR boundary** to self-adjust the PB boundary based on the placement.
5. Click on the **Play**(▶) button to run the placement.

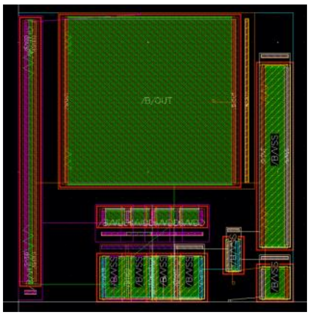


This page does not contain notes.

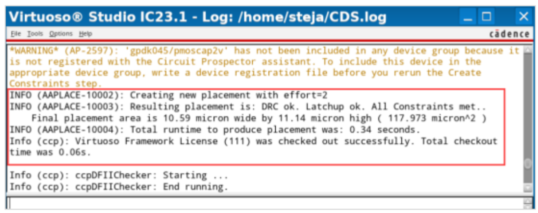
How to Place the Device Layouts Using Auto P&R Assistant (continued)

6. Post-placement updates can be done by using the Adjust Boundary, Optimize Pins, and Resolve Violations buttons under the *Update* section.

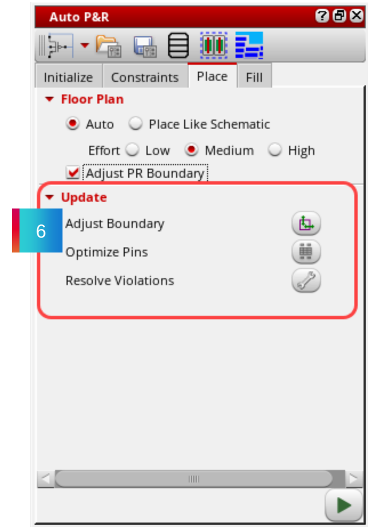
The result of the auto-placement will look as shown below:



Layout Result



CIW Results



To enable the plugin, go to **Launch -> Plugins -> Migrate**. You can use the options in this section to run the assisted flow of the Virtuoso Custom Design Migration solution.

Submodule 5.2

Device Placement Using Generate All From Source (GFS)

This page does not contain notes.

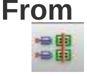
Submodule Objectives

In this submodule, you

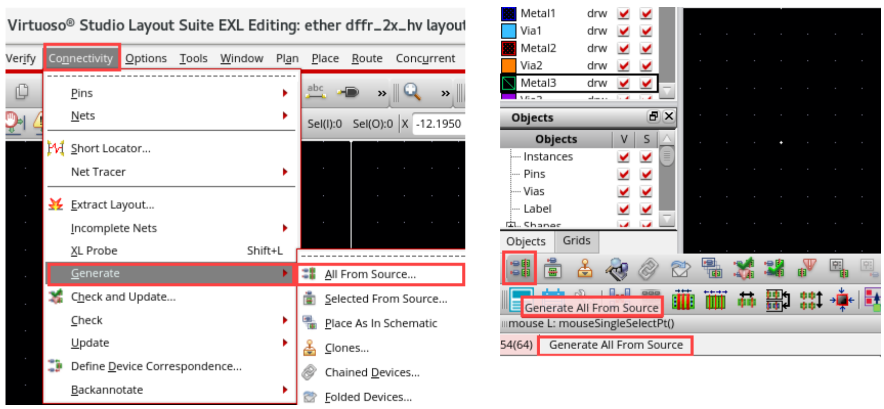
- Generate initial layout devices using the Generate All From Source (GFS)
- Place the layout cells using the align, abut, copy, and move commands

This page does not contain notes.

Generating a Layout from GFS

Generate all from the source by choosing either the **Connectivity – Generate – All From Source** menu command or by clicking the **Generate All From Source** (**GFS**) icon. 

- Use the GFS command to populate the design elements from the schematic to an empty or partially populated layout canvas.

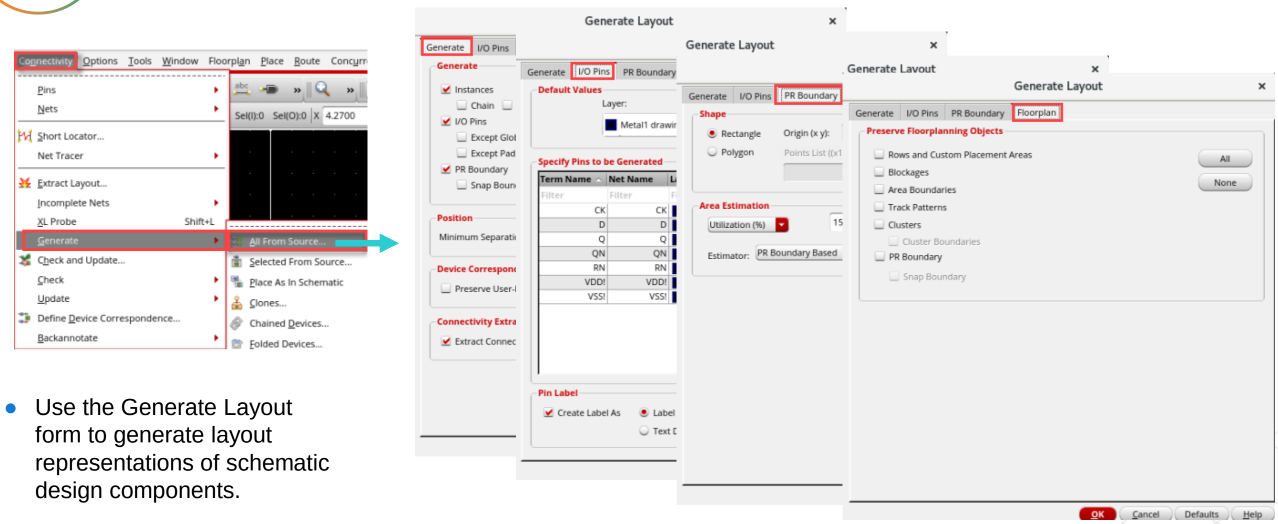


Use the *Generate All From Source* command to generate layout representations of the schematic design components. *Generate All From Source* deletes any existing components in the layout view and generates everything from scratch.

Notice that you can use an ever-increasing number of commands through the use of icons instead of the pull-down menus. You can hover over each icon to see the command mapped to it before clicking on it.

Generate All From Source: Interface

When the blank layout canvas opens, choose the **Connectivity – Generate – All From Source** command. The tabbed Generate Layout form appears.



- Use the Generate Layout form to generate layout representations of schematic design components.

cādence

You can display the Generate Layout form by choosing the **Connectivity – Generate – All from Source** pull-down menu entry or by clicking the **Generate All From Source** Layout XL Toolbar icon at the lower-left corner of the layout canvas.

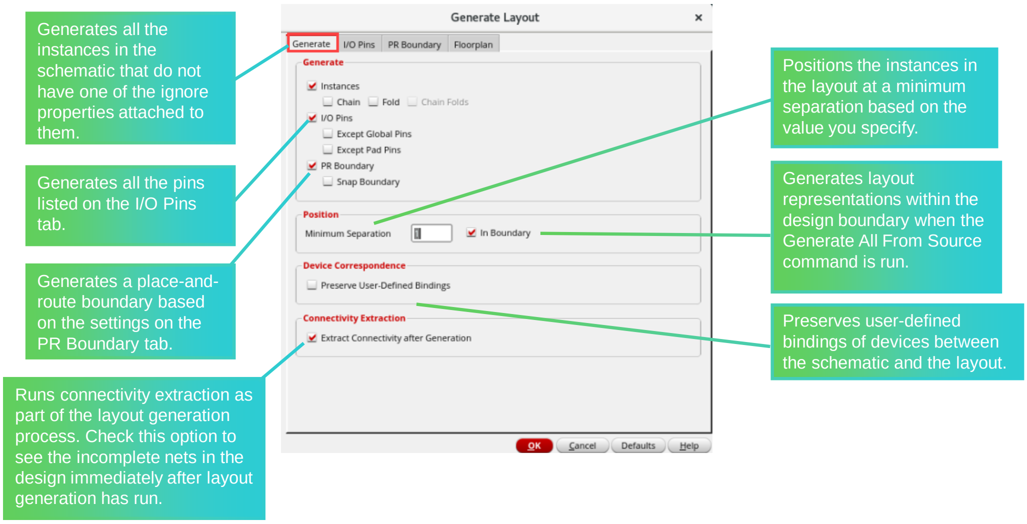
Generate Layout

Use the Generate Layout form to generate layout representations of schematic design components.

The form is split into four tabs:

- Generate
- I/O Pins
- PR Boundary
- Floorplan

Generate All From Source: Generate Tab



You use the **Generate** tab to determine whether or not to generate instances, do automatic chaining and folding, and include I/O pins and boundaries when you run **Generate All From Source**.

You also determine whether or not to automatically extract the physical design when you launch Gen From Source.

- *Minimum Separation* – Positions the instances in the layout at a minimum separation based on the value you specify. You can also control this option using the *lxPositionMinSep* environment variable.
- *In Boundary* – Generates layout representations within the design boundary when the Generate All From Source command is run. You can also control this option using the *lxGenerateInBoundary* environment variable.
- *Preserve User-Defined Bindings* – Preserves user-defined bindings of devices between the schematic and the layout. This option preserves only user-defined one-to-one, many-to-many, many-to-one, and one-to-many device correspondence defined in the Define Device Correspondence form. It does not report missing devices or shapes within a bound group.
- *Extract Connectivity after Generation* – Runs connectivity extraction as part of the layout generation process. Check this option to see the incomplete nets in the design immediately after layout generation has run.

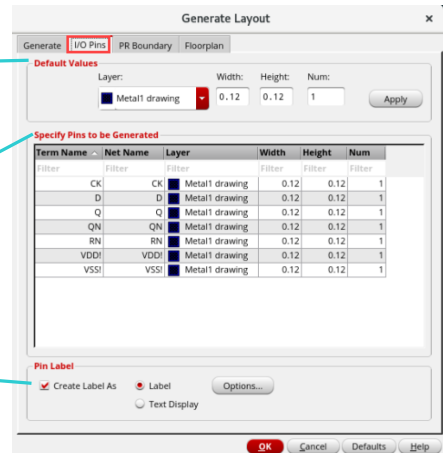
See complete details on the use of this form by clicking **Help** in the lower-right corner of the form.

Generate All From Source: I/O Pins Tab

Specifies attribute values and applies them to all the pins shown in the list box.

Selects pins from the list box and updates the attribute values used when those pins are generated in the layout.

Specifies the type of label generated when you create a pin. Honored by the Generate All From Source and Generate Selected From Source commands.



You use the I/O Pins tab to determine which layers your pins acquire when you run **Generate All From Source**.

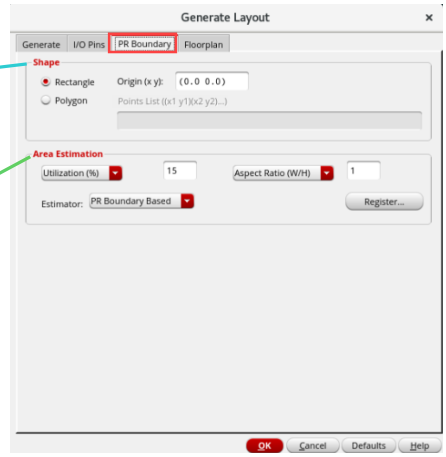
You also determine pin size, number per net, and whether or not to generate labels for pins.

See complete details on the use of this form by clicking **Help** in the lower-right corner of the form.

Generate All From Source: PR Boundary Tab

Specifies whether the place and route boundary is a rectangle or a polygon.

Specifies how the system calculates the shape and size of a rectangular boundary.



211

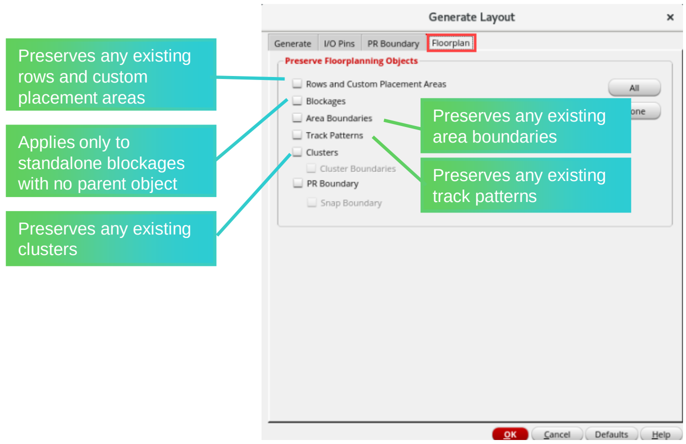
You use the PR Boundary tab to determine the shape and size of the boundary when you run **Generate All From Source**.

You can also let the software determine the approximate size of the boundary by choosing **Utilization (%)** under the Area Estimation pull-down in conjunction with the **Aspect Ratio (W/H)** option if you are not familiar with the boundary size.

If you already know the boundary size, you can use the Width and Length options.

See complete details on the use of this form by clicking **Help** in the lower-right corner of the form.

Generate All From Source: Floorplan Tab



You use the Floorplan tab to preserve floorplanning objects when you run **Gen From Source**.

- *Rows and Custom Placement Areas* – Preserves any existing rows and custom placement areas.
- *Blockages* – Apply only to standalone blockages which have no parent object. A blockage with a parent object is preserved automatically along with its parent.
- *Area Boundaries* – Preserves any existing area boundaries.
- *Track Patterns* – Preserves any existing track patterns.
- *Clusters* – Preserves any existing clusters.
  - *Cluster Boundaries* – Only if clusters are also preserved.

See complete details on the use of this form by clicking **Help** in the lower-right corner of the form.

Abutting and Chaining the Devices

You can use the **Generate Chained Devices** form to select and chain specific devices in your design.

Select the devices you want to chain directly in the layout canvas or in the Navigator assistant and click **Apply** in the form to chain them.

1. Select the devices to be chained, say, for example, select all 4 transistors in the layout canvas or in the Navigator assistant.
2. To chain the devices, in the design canvas, choose the **Connectivity – Generate – Chained Devices** menu command or click the **Generate Chained Devices** icon in the Layout XL toolbar.
3. In the Generate Chained Devices form, click the **Apply** button and place the devices to chain them.

Result: In this example, all the selected transistors are chained and abutted.

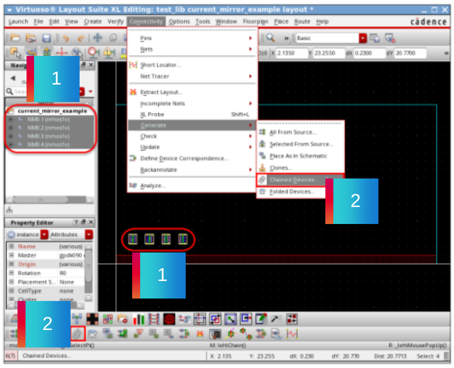
Result: All the selected 4 transistors are chained and abutted, we need to connect these devices, that is done by using the wire command.

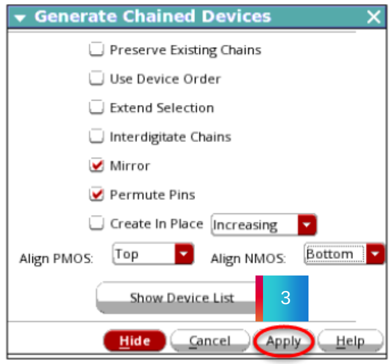
You can use the *Generate Chained Devices* form to select and chain specific devices in your design.

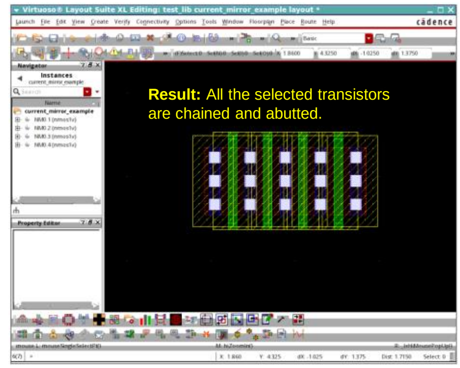
Select the devices you want to chain directly in the layout canvas or in the Navigator assistant and click the *Apply* button in the form to chain them.

Abutting and Chaining the Devices (continued)

- If the majority of the selected devices are oriented vertically, the generated chain is also oriented vertically.
- The devices to be chained need not be bound to a schematic instance.







Result: All the selected 4 transistors are chained and abutted.

Select the devices to be chained, say, for example, select all the 4 transistors in the layout canvas, or in the Navigator assistant.

To chain the devices, in the design canvas, choose the *Connectivity – Generate – Chained Devices* menu command or click the *Generate Chained Devices* icon in the Layout XL toolbar.

In the Generate Chained Devices form, click the *Apply* button and place the devices to chain them.

You can observe that in this example, all the selected transistors are chained and abutted.

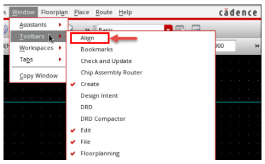
If the majority of the selected devices are oriented vertically, the generated chain is also oriented vertically.

The devices to be chained need not be bound to a schematic instance.

Aligning Objects by Using the Align Toolbar

For performing quick alignment operations, you can use the **Align** command.

In the layout window, choose the **Window – Toolbars – Align** menu command to display it in case it is hidden.



Align toolbar

In the layout window, set these options in the *Alignment* toolbar.

1. Set the *Align Spacing* to **0**.
2. Select *Group0* & *Group0_2*.
3. Click the **Align Right** button.

Result: *Group0* & *Group0_2* are aligned to the right now.

1. Set the *Align Spacing* to **0**.
2. Select *Group0_0* & *Group0_1*.
3. Click the **Align Left** button.

Result: *Group0_0* & *Group0_1* are aligned to the left now.

Use the Align toolbar to align the components.

Let us see how to align the objects by using the Align toolbar.

You can use the *Align* command for performing quick alignment operations.

In the design window, choose the *Window – Toolbars – Align* menu command to display it in case it is hidden.

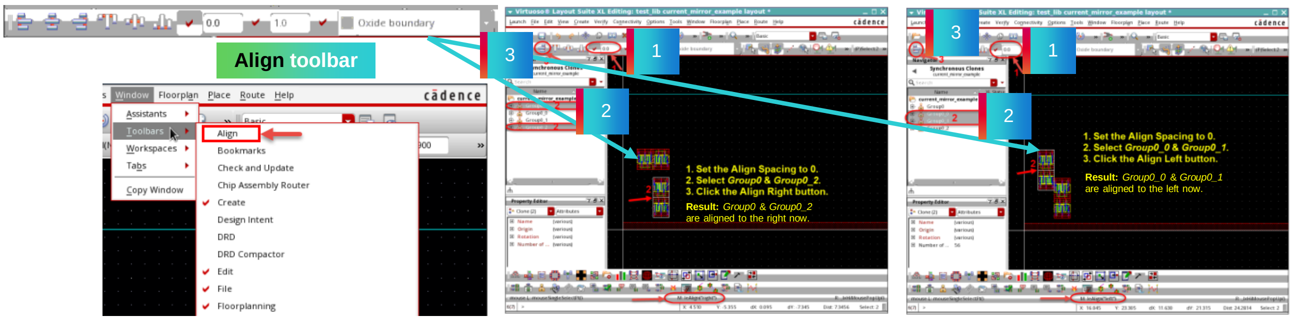
You can use the *Align* toolbar to align the components.

In the design window, set the required options in the *Alignment* toolbar for aligning the objects.

Aligning Objects by Using the Align Toolbar (continued)

Using the **Align** toolbar, you can perform left, right, top, and bottom alignment of the selected objects.

You can also align them vertically or horizontally. The toolbar also provides the option of specifying alignment and orthogonal spacing values.



Using the *Align* toolbar, you can perform left, right, top, and bottom alignment of the selected objects.

You can also align them vertically or horizontally.

The toolbar also provides the option of specifying alignment and orthogonal spacing values.

Firstly, set the *Align Spacing* to *0* in the Align toolbar.

Then select *Group0* and *Group0_2*.

And click the *Align Right* button.

You can see that *Group0* and *Group0_2* are aligned to the right now.

Next, set the *Align Spacing* to *0*.

Then select *Group0_0* and *Group0_1*.

And click the *Align Left* button.

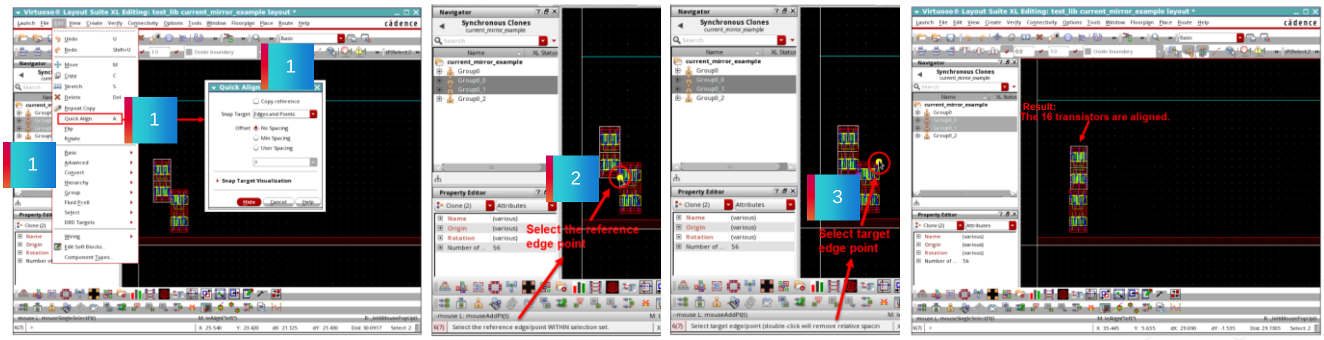
You can observe that *Group0_0* and *Group0_1* are aligned to the left now.

Aligning Objects by Using the Quick Align Command

The **Quick Align** command enables you to use objects, instances, and groups as the reference set and also use them as the target to which the reference set is to be aligned.

1. With the **2 upper clone families** selected, choose the **Edit – Quick Align** menu command or press the bindkey **A** to invoke the Quick Align form.
2. Select the reference edge point WITHIN selection set.
3. Select the target edge point.

Result: This action creates the aligned structure and the **16** transistors are aligned now.



Let us see how to align the objects by using the Quick Align command.

The *Quick Align* command enables you to use objects, instances, and groups as the reference set and also use them as the target to which the reference set is to be aligned.

With the *2 upper clone families* selected choose the *Edit – Quick Align* menu command or press the bindkey *A* to invoke the Quick Align form.

In the Quick Align form, select the reference edge point WITHIN selection set.

Then select the target edge point.

You can observe that this action creates the aligned structure and the *16* transistors are aligned now.

Submodule 5.3

Exploring Various Methods of Routing

This page does not contain notes.

Submodule Objective

In this submodule, you

- Use Pin-to-Trunk Routing
  - Start Pin-to-Trunk Router with Route With Default Lookup / Route With WA Overrides
  - Start Pin-to-Trunk Router with Wire Assistant
- Launch Auto Router on the Selected Nets
  - Route with Default Lookup with default Constraint Group (CG)
  - Route with WA Overrides with seeded Constraint Group (CG)
- Perform Manual Routing
  - Route the layout using the path and wire commands
  - Transition from one routing layer to another using Vias

This page does not contain notes.

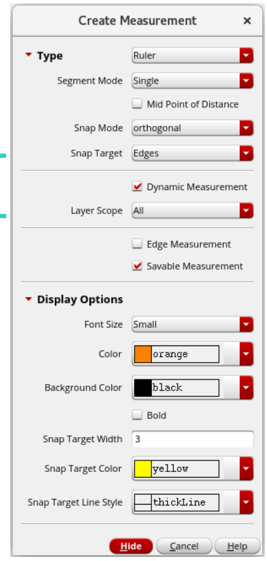
Create Measurement Form

Choose **Tools – Create Measurement** menu command.

The **Create Measurement** form with all the fields expanded:

- Enables Dynamic Measurement
- Specifies Layer Scope:

All	Measurement considers all layers, for example, between M1 and M2
Active	Only active layer in the Palette is considered
Edge	Only the layer underneath the cursor position is considered
Palette	List of LPPs in Palette is considered
Selectable	Only selectable LPPs are considered



- Whether to create a Ruler or Distance or both
- Single/Multiple/Auto Segment Mode
- Find the Mid point of Distance
- Snap Mode
- Snap Target Edges or Points

- When Edge Measurement is On, the length of the edge is displayed rather than the distance between the edges
- Savable Measurement creates rulers that can be saved in the database
- Display Options are controlled here

The Create Measurement Form

Type lets you select the display type of the ruler (environment variable: rulerDisplayType). **Ruler** displays the ruler in the design window. **Distance** displays the distance between two objects on the design window. **Both** displays both the ruler and the distance between two objects on the design window.

Dynamic Measurement, if selected, enables you to dynamically display measurement between the pointer and the closest edges. If you move the pointer between two shapes, the distance between the two edges is displayed. If you move the pointer on an edge of a shape, the edge is highlighted in yellow. Also, the distances between the highlighted edge and the two closest edges are displayed.

Layer Scope lets you select the layers to be considered for dynamic measurement. You can select *All*, *Active*, *Edge*, *Palette*, or *Selectable* layers.

Edge Measurement, if selected, lets you enable the edge measurement mode for rulers. Edge measurement mode allows you to click the edge of an object and a measurement is drawn with the two vertices as the two ends. The first click lets you select the edge and the second click lets you place the measurement on the canvas (environment variable: rulerEdgeMode). **Note:** The Edge Management check box is disabled when the *Snap Target* is None or when the *Segment Mode* is *Multiple* or *Auto*.

Savable Measurement saves the persistent rulers in a cellview when you save the cellview. It is on by default. If this option is off, all the rulers existing in the cellview are deleted when you save the cellview (environment variable: saveRulers).

Display Options let you define the ruler and snap target visualization settings.

Font Size lets you select the font size of the ruler labels. You can choose from *Small*, *Medium*, or *Large* (environment variable: rulerFontSize).

Color lets you select the font color of the ruler labels.

Background Color lets you select the background color of the ruler labels (environment variable: rulerLabelBackgroundColor).

Bold, if selected, displays the ruler labels in bold face (environment variable: rulerFontBold).

Snap Target Width enables you to specify the width of the snap target highlight. The default is 0 (environment variable: targetWidth).

Snap Target Color enables you to select the color of the snap target highlight. The default Environment variable is targetColor.

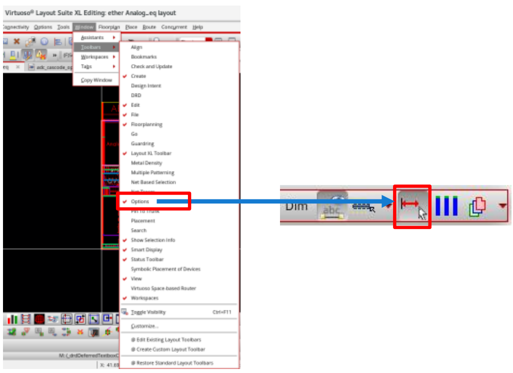
Snap Target Line Style enables you to select the type of snap target highlight. The default is thickline (environment variable: targetLineStyle).

Refer to the *Virtuoso Layout Suite XL: Basic Editing User Guide, Product Version IC23.1*, for more details on these features.

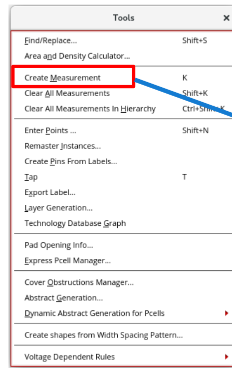
Enabling the Dynamic Measurement

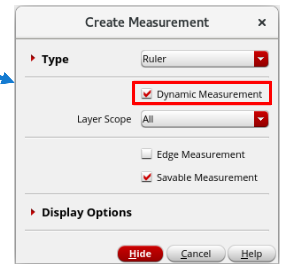
The **Dynamic Measurement** option, if selected, enables you to dynamically display measurement between the pointer and the closest edges in the Create Measurement form.

1. From **Window – Toolbars**, activate the **Options** toolbar and use the **Dynamic Measurement** icon.



2. Invoke the **Tools – Create Measurement** (bindkey "**<Key>K**") menu command and enable the **Dynamic Measurement** button.

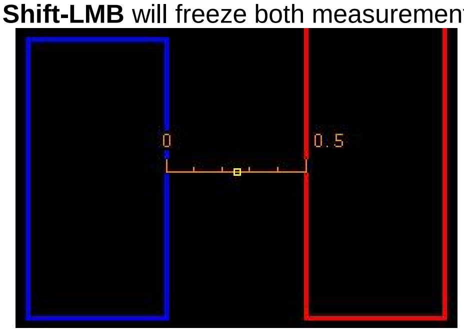


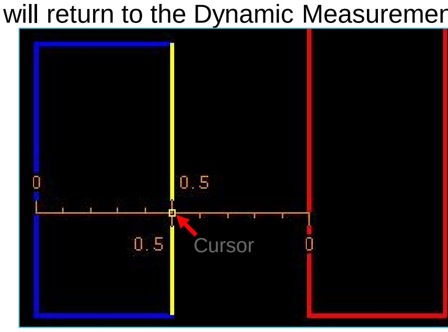


The **Dynamic Measurement** option, if selected, enables you to dynamically display measurement between the pointer and the closest edges in the Create Measurement form.

Using the Dynamic Measurement

1. Invoke the *Create Measurement* form by choosing the **Tools – Create Measurement** menu command or through bindkey “**<Key>K”** and enable the *Dynamic Measurement* option.
2. Move the cursor between 2 edges.
3. The distance between these 2 edges is displayed. To freeze the measurement, press **Shift-LMB**. Then, it will return to the Dynamic Measurement mode.
4. Move the cursor to an edge. The edge will be highlighted in yellow. Distances between this edge and the two closest edges are displayed.
5. Pressing **Shift-LMB** will freeze both measurements. It will return to the Dynamic Measurement mode again.





LMB will create a traditional ruler. To delete all measurements, use **Shift-K**.

In the Create Measurement form, **Dynamic Measurement**, if selected, enables you to dynamically display measurement between the pointer and the closest edges.

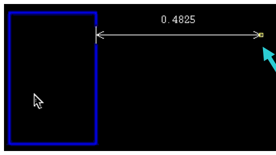
If you move the pointer between two shapes, the distance between the two edges is displayed.

If you move the pointer on an edge of a shape, the edge is highlighted in yellow. Also, the distances between the highlighted edge and the two closest edges are displayed.

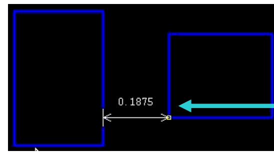
Dynamic Measurement in Create/Edit Commands

Once enabled, **Dynamic Measurement** is displayed in the **Create/Edit** commands. For examples:

1 Create Rectangle



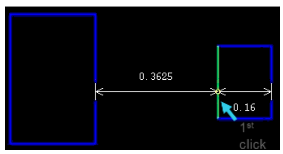
Start Rectangle

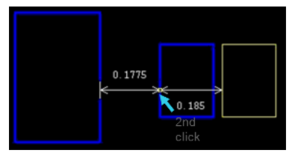


End Rectangle

Measurement between the closest shape and cursor position is displayed.

2 Move Command





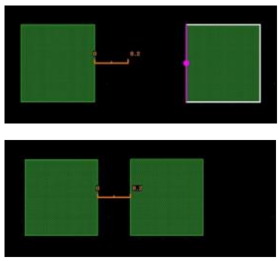
As you move the cursor, Dynamic Measurement shows the distance between the initial and new positions, and the spacing between the rectangle and the closest shape.

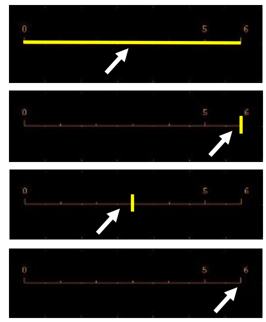
Create Rectangle command: Measurement between the closest shape and cursor position is displayed.

Move command: As you move the cursor, **Dynamic Measurement** shows the distance between the initial and new positions, and the spacing between the rectangle and the closest shape.

Smart Snapping of Ruler in the Quick Align Command

1. Smart snapping in the Quick Align command now supports snapping of the rulers.
2. User can select the ruler, the endpoints, the end edges, or the center line of a ruler as the reference or the target.
3. This feature is useful when the user tries to move or stretch an object and align it to a ruler.





You can align objects by using the **Edit – Quick Align** command or the **Edit – Advanced – Align** command.

Aligning Objects

You can align objects by using the **Edit** – **Quick Align** command or the **Edit** – **Advanced** – **Align** command. The two commands enable you to perform similar tasks but differ in their usage. You can perform the following tasks by using these commands: Align objects to a point, Align objects to an edge, Align objects to a layer, and Align objects to a target axis.

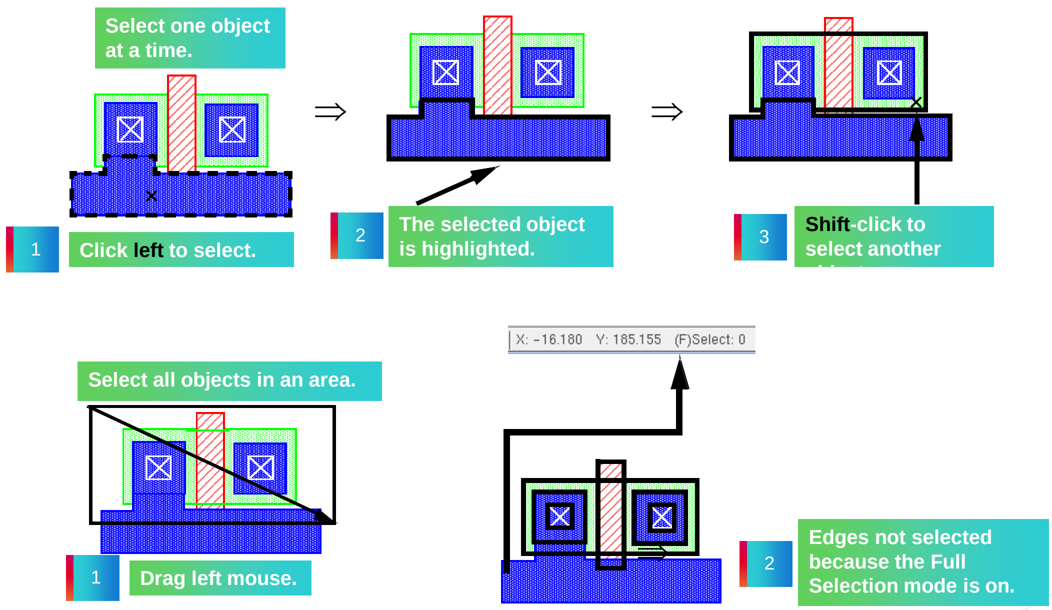
Both the commands work in both pre- and post-selection modes.

The Quick Align command enables you to quickly align objects, in a more interactive manner than the Align command. In comparison with the Align command, the Quick Align command provides the following additional features:

- Enables smart snapping of edges, centerlines, and points within the specified aperture
- Enables cyclic highlighting of the reference and target objects within the aperture distance from the pointer by using the *Spacebar* key
- Enables the alignment with respect to reference or target objects across the hierarchy
- Supports the spacing rules defined in the technology file
- Enables alignment by stretching partially selected objects

The Quick Align command enables you to align an edge, a centerline, or a point to another edge, centerline, or point, whereas the Align command aligns an object bounding box to the bounding box of another object.

Selecting Objects



225

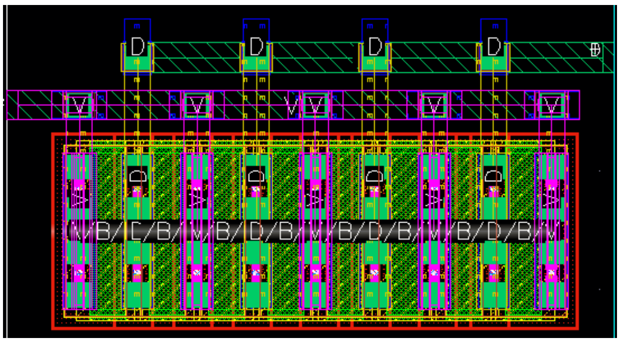
cadence

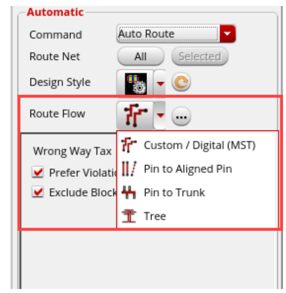
- To select one object, **left**-click inside the shape. This also deselects all other previously selected shapes.
- To select additional objects, press the **Shift** key and **left**-click inside each additional shape.
- To deselect shapes, press the **Control** key and **left**-click inside a shape.
- To select an edge or corner, select **F4** (partial select) and **left**-click on an edge or corner.
- To select multiple shapes, **left**-click and drag to enclose the shapes.
- **F4** toggles full and partial select. If using full select, only shapes which are completely enclosed will be selected. If using partial select, each edge or corner which is enclosed will be selected.
- To deselect individual shapes, press **Control** and **left**-click inside the shape. This also works for deselecting multiple shapes when you use **Control** and left mouse drag to enclose the shapes to deselect.

What Is the Pin-to-Trunk Routing?

The **Pin-to-Trunk** routing topology style is used to connect an individual pin to a trunk.

- Only the connections that are within the scope of a trunk are routed.
- The remaining opens are completed by the **Custom / Digital (MST)** routing style or manually.
- The **Pin-to-Trunk** routing style is usually used when a spine structure is required.





What Is Pin-to-Trunk Routing?

The *Pin-to-Trunk* routing topology style is used to connect an individual pin to a trunk.

Only the connections that are within the scope of a trunk are routed.

The remaining opens are completed by the *Custom / Digital (MST)* routing style or manually.

The *Pin-to-Trunk* routing style is usually used when a spine structure is required.

Starting Pin-to-Trunk Router with Wire Assistant
Composing Trunks

Use the **Compose Trunks** command to convert the selected set of shapes into a trunk object.

The **Pin-to-Trunk** routing step is performed after composing trunks.

To connect the NMOS transistors to the VSS ground rail (net name: **myVss**) in the layout design:

1. Define a trunk for the ground net: select the pin shape **myVss**.
2. Choose **Edit – Wiring – Compose Trunks** or **right**-click and choose **Compose Trunk**.

Result: The pin shape **myVss** is declared as the trunk.

Starting Pin-to-Trunk Router with Wire Assistant – Composing Trunks

You can use the *Compose Trunks* command to convert the selected set of shapes into a trunk object.

The *Pin-to-Trunk* routing step is performed after composing the trunks.

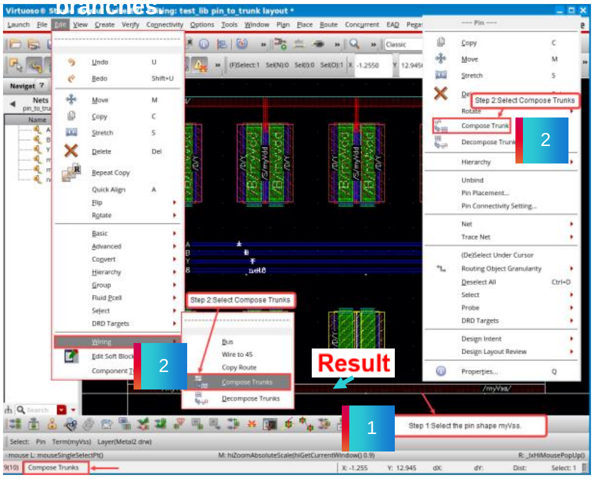
In the layout design, to connect the NMOS transistors to the VSS ground rail, for example, to connect to the net name *myVss*, first, define a trunk for the ground net: For that, select the pin shape which you want to convert as trunk.

Then, choose the *Edit – Wiring – Compose Trunks* menu command or *right*-click and choose the *Compose Trunk* command.

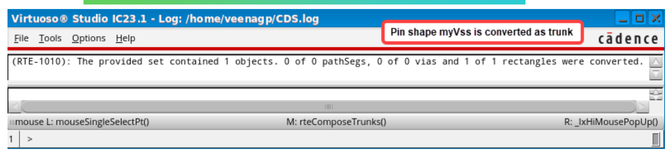
Now, the pin shape *myVss* is declared as trunk.

Result: Starting Pin-to-Trunk Router with Wire Assistant
Composing Trunks

Result: The pin shape **myVss** declared as trunk will be used as the source for all the branches.



Observe the CDS.log file or CIW window for the trunk conversion



Result: Starting Pin-to-Trunk Router with Wire Assistant – Composing Trunks

The pin shape *myVss* which is declared as trunk will be used as the source for all the branches.

You can observe the CDS.log file or the CIW window for the trunk conversion information.

Starting Pin-to-Trunk Router with Wire Assistant

Use the Wire Assistant's **Pin-to-Trunk** router to route the trunk object.

To route the trunk net **myVss** in the layout design:

1. Select the net **myVss** in the Navigator assistant.
2. Choose the **Pin to Trunk** *Route Flow* in the Wire Assistant.
3. Start the **Pin to Trunk** router on the defined trunk **myVss**: Click on the **Selected** button in the Wire Assistant.

Result: The trunk **myVss** is routed.

cadence

Starting Pin-to-Trunk Router with Wire Assistant

You can use the Wire Assistant's *Pin-to-Trunk* router to route the trunk object.

To route the trunk net *myVss*, firstly, select the net in the Navigator assistant.

Then, choose the *Pin to Trunk Route Flow* in the Wire Assistant.

Now, start the *Pin to Trunk* router on the defined trunk *myVss*:

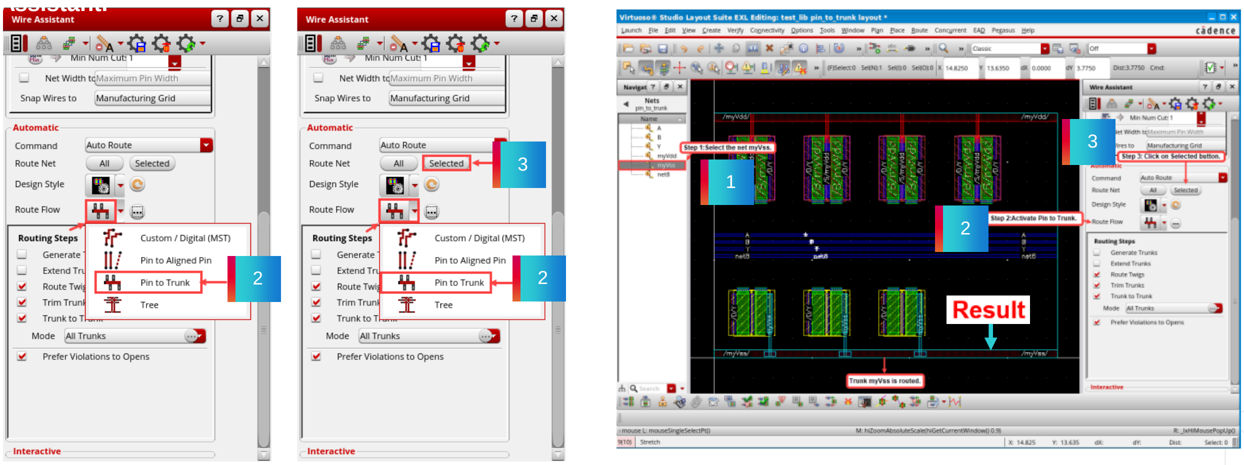
For that, click on the *Selected* button in the Wire Assistant.

You can see that the trunk *myVss* is routed.

Result: Starting Pin-to-Trunk Router with Wire Assistant

3. Start the **Pin to Trunk** router on the defined trunk **myVss**: Click on the **Selected** button in the Wire

Result: Routing for the net **myVss** has been completed using the trunk approach.



Result: Starting Pin-to-Trunk Router with Wire Assistant

When you run the *Pin-to-Trunk* router on the defined trunk net, you see here that the routing for the net *myVss* has been completed using the trunk approach.

Using Auto Router on the Selected Nets
Route With Default Lookup

Use the **Route With Default Lookup** command to route the selected net with default lookup values.

To route the selected net with default lookup values in the layout canvas:

1. From the **Seed Attributes From a Constraint Group** drop-down list in the Wire Assistant toolbar, choose the **virtuosoDefaultSetup** constraint group.
2. From the *Route Flow* groupbox, choose the **Custom / Digital (MST)** option from the Wire Assistant toolbar.
3. Select the net **top** in the Navigator assistant.
4. Now, **right**-click and choose **Route With Default Lookup**.

Result: The router has routed with default lookup values.

Using Auto Router on the Selected Nets – Route With Default Lookup

You can use the *Route With Default Lookup* command to route the selected net with the default lookup values.

To route the selected net with the default lookup values, in the layout canvas:

From the *Seed Attributes From a Constraint Group* drop-down list in the Wire Assistant toolbar, choose the *virtuosoDefaultSetup* constraint group.

Then, from the *Route Flow* groupbox, choose the *Custom / Digital (MST)* option from the Wire Assistant toolbar.

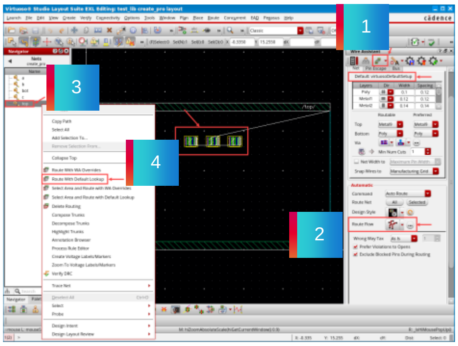
Next, select the net to route, say for example, net *top* in the Navigator assistant.

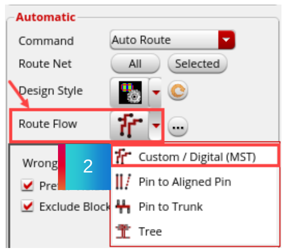
Now, *right*-click and choose the *Route With Default Lookup* command.

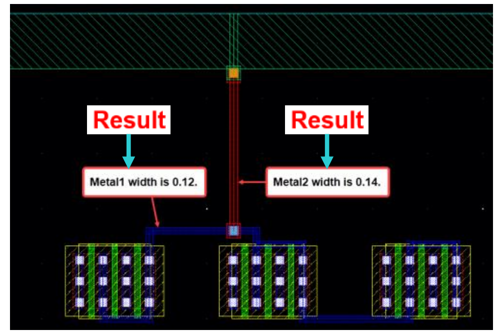
You can notice that the router has routed with the default lookup values.

Result: Using Auto Router on the Selected Nets
Route With Default Lookup

Result: The router has routed with default constraint lookup values for







Result: Using Auto Router on the Selected Nets – Route With Default Lookup

In this example, with the *virtuosoDefaultSetup* constraint group and *Custom / Digital (MST)* routing topology set in the Wire Assistant, when you route the net *top* with the *Route With Default Lookup* command, you can see that, the router has routed with the default constraint lookup values for routing.

Here *Metal1 Width* is *0.12* and *Metal2 Width* is *0.14* which is taken from the default constraint group.

Using Auto Router on the Selected Nets
Route With WA Overrides

Use the **Route With WA Overrides** command to route the selected nets with the Wire Assistant Overrides.

To route the selected net with the Wire Assistant Overrides in the layout canvas:

1. From the **Seed Attributes From a Constraint Group** drop-down list in the Wire Assistant toolbar, choose the **dsn:M1_M3_200n (Net)** constraint group.
2. From the *Route Flow* groupbox, choose the **Custom / Digital (MST)** option from the Wire Assistant toolbar.
3. Select the net **top** in the Navigator assistant.
4. Now, **right**-click and choose **Route With WA Overrides**.

Result: The router has routed with WA Overridden (PRO) values.

Using Auto Router on the Selected Nets – Route With WA Overrides

You can use the *Route With WA Overrides* command to route the selected nets with the Wire Assistant Overrides.

To route the selected net with the Wire Assistant Overrides, in the layout canvas:

From the *Seed Attributes From a Constraint Group* drop-down list in the Wire Assistant toolbar, choose the seeded constraint group, for example, *dsn:M1_M3_200n (Net)* constraint group.

Then, from the *Route Flow* groupbox, choose the *Custom / Digital (MST)* option from the Wire Assistant toolbar.

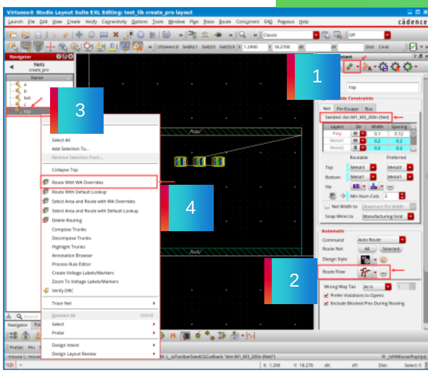
Next, select the net to route, for example, net *top* in the Navigator assistant.

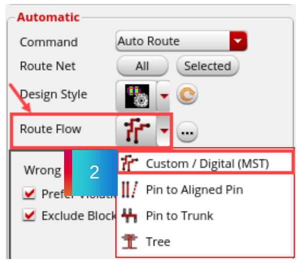
Now, *right*-click and choose the *Route With WA Overrides* command.

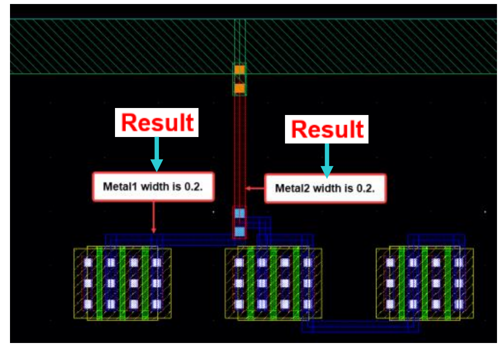
You will notice that the router has routed with the WA Overridden values.

Result: Using Auto Router on the Selected Nets
Route With WA Overrides

Result: The router has routed with the Wire Assistant Overrides.







Result: Using Auto Router on the Selected Nets – Route With WA Overrides

In this example, with the seeded constraint group and *Custom / Digital (MST)* routing topology set in the Wire Assistant, when you route the net *top* with the *Route With WA Overrides* command, you can see that the router has routed with the seeded constraint group values for routing.

Here, both *Metal1* and *Metal2 Widths* are *0.2,* which are taken from the seeded constraint group.

Connecting to Target Pins with the Create Wire Command

Use the **Create Wire** command to connect to the target pins.

1. In the design canvas, select the net **D** in the Navigator assistant.
2. In the design canvas, choose the **Create – Wiring – Wire** menu command or use the **Create Wire** icon in the Create toolbar or use the bindkey **P** to invoke the Create Wire form.
3. Start routing from either the **NM0** or the **PM0** device toward pin **D**.

Result: As you near pin **D**, you see the possible connecting targets appear. As you move your mouse pointer around the pin, the targets will cycle.

You can use the Ctrl+Space key combination to cycle the targets.

Connecting to Target Pins with the Create Wire Command

You can use the Create Wire command to connect to the target pins.

In the design canvas, select the net *D* in the Navigator assistant.

Then, choose the *Create – Wiring – Wire* menu command or use the *Create Wire* icon in the Create toolbar or use the bindkey *P* to invoke the Create Wire form.

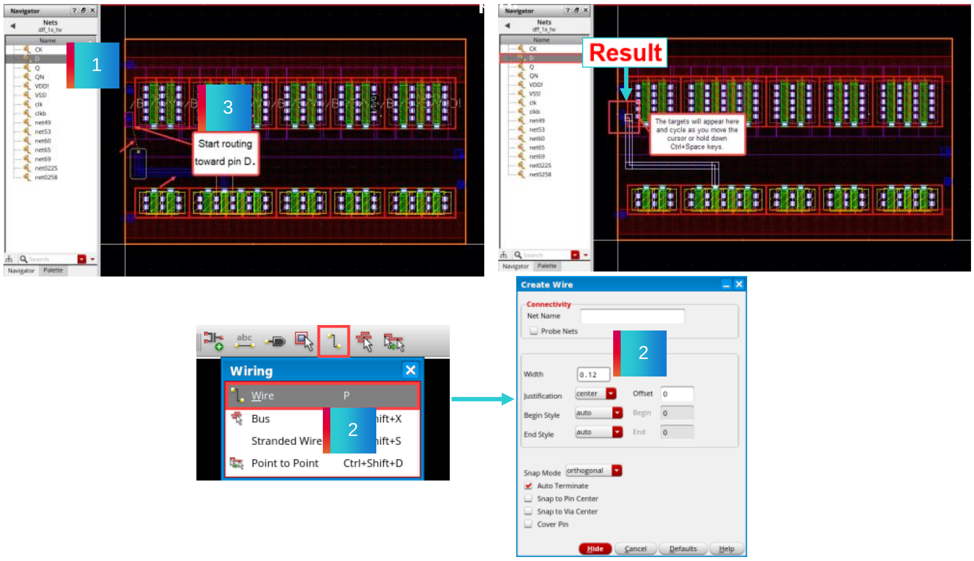
When you start routing from either the *NMOS* or the *PMOS* transistor toward pin *D*, you can see that, as you near pin *D*, you see the possible connecting targets appear.

As you move your mouse pointer around the pin, the targets will cycle.

You can use the *Ctrl+Space* key combination to cycle the targets.

Result: Connecting to Target Pins with the Create Wire Command

You can use the connecting targets to connect to the



Result: Connecting to Target Pins with the Create Wire Command

As you see in this example, after selecting the net *D* in the Navigator assistant, when you start routing from one of the transistor device, when you near pin *D*, you see the possible connecting targets appear which are cycling when you move your mouse pointer around the pin or use the *Ctrl+Space* key.

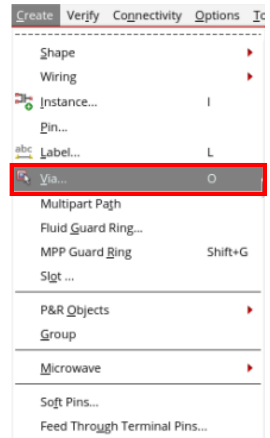
You can use the connecting targets to connect to the pin.

Creating Vias

You can access **Create > Via** in the GUI or use the **bindkey O**.

Vias are the elements that connect one or more routing layers together. You can create either a single via from metal 1 to metal 2 or you can create a via stack from metal 1 to metal n, where n is the total number of routing layers.

Vias can be created manually and interactively or automatically.



cadence

This page does not contain notes.

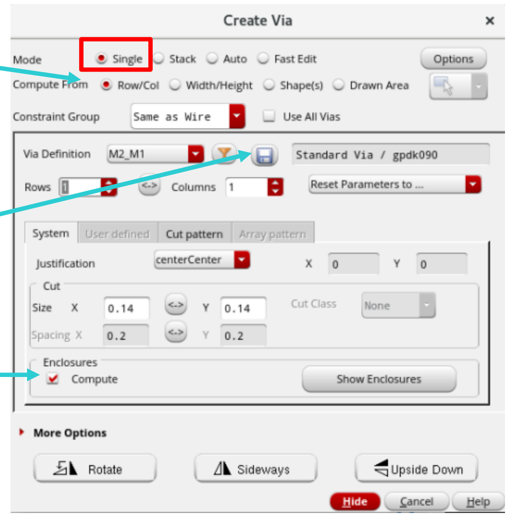
Creating Vias: Single Mode

Choose Create – Via

You can create auto vias in Single, Stack, or Auto Mode.

Stores newly created vias (variants) as viaDef.

When off, allows enclosures to be less than minimum.



See the Notes section on this page.

Contacts (Vias) are defined in the technology file. You may place any contact or via individually or use Auto Via. Auto Via will place the correct number of via cuts and layer enclosure to fit the area of the intersecting wires.

- **Single Mode** – Pulls up the via Definition options. The Via Definition field displays the valid viadefs and valid via variants in a tree-like structure from the effective technology file. The listed viaDefs are ordered by mask number. Only the via Defs or via variants satisfying any filtering options specified through the Filter widget are displayed. Each node in the tree represents a standard or custom viaDef. A node with a plus sign (+) represents a parent standard or customer viaDef with child viaVariants. If a parent viaDef is not a valid via in the current active constraint group, the viaDef is grayed out and cannot be selected. Choosing a viaDef or via variant from the cyclic field selects the via from the effective technology file.
- **Compute -** Enables you to create auto vias at the intersection of shapes in the Single and Stack modes.

If this checkbox is selected:

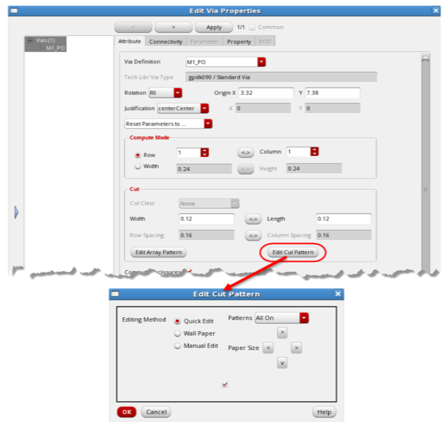
- In **Single mode**: You can select a Via Definition and specify the Width, Length, Row Spacing, and Column Spacing values for the via cuts.
- In **Stack mode**: You can specify the Start Layer and the End Layer of the via stack. So you can predefine the viaDefs that will be used for each stacked via.

Create Via: Cut Pattern

The Cut Pattern can be edited for already placed vias as well.

Select a **via**, open the **Edit Via Properties** form, and click **Edit Cut Pattern**.

Placed vias can be edited



This is how you control via density

The Cut Pattern can be edited for already placed vias as well.

Select a via, open the Edit Via Properties form and click **Edit Cut Pattern**.

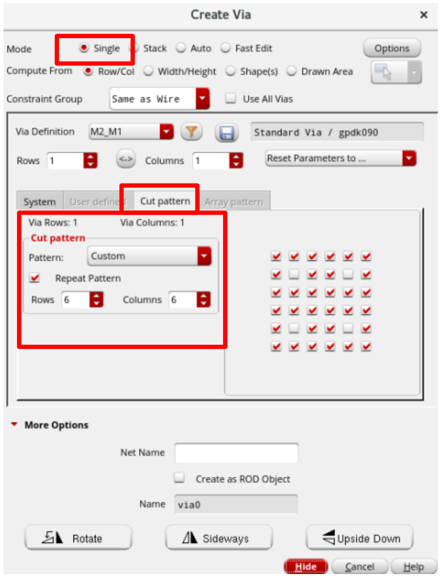
Edit Cut Pattern Form

Note: Edit Cut Pattern is applicable only for standard vias.

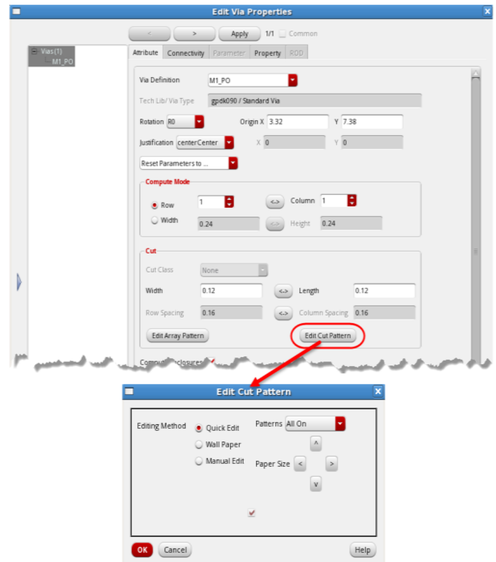
- **Quick Edit** allows you to edit the cut pattern in conjunction with the *Patterns* cyclic field to select a predetermined pattern.
- **Wall Paper** allows you to edit the cut spaces cut by cut or use the *Patterns* cyclic field to select a predetermined pattern.
- **Paper Size** allows you add or delete rows or columns to the cut pattern when *Wall Paper* is selected.
- **Manual Edit** displays all of the rows and columns. You can edit the cut spaces cut by cut or use the *Patterns* cyclic field to select a predetermined pattern.
- **Patterns** lets you select a predetermined pattern, such as All On, All Off, Top, Bottom, Left, Right, H Stripes, V Stripes, Chess Board, U-L Corner, U-R Corner, L-L Corner, L-R Corner, Cross, Farm, and As Is.

Refer to the *Virtuoso Layout Suite L User Guide*, Product Version IC 23.1 for details on these features.

Create Via: Cut Pattern Forms



Placed vias can be edited



This is how you control via density

The Cut Pattern can be edited for already placed vias as well.

Select a via, open the Edit Via Properties form and click **Edit Cut Pattern**.

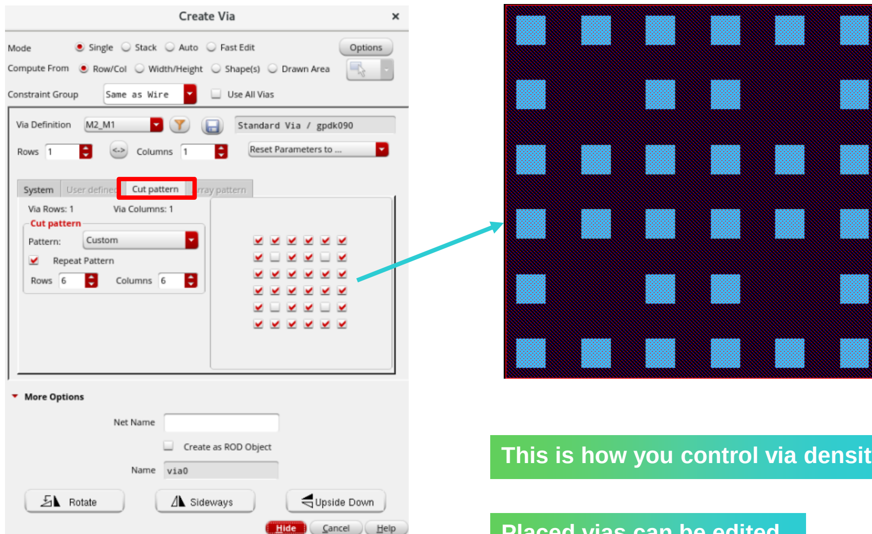
Edit Cut Pattern Form

Note: Edit Cut Pattern is applicable only for standard vias.

- **Quick Edit** allows you to edit the cut pattern in conjunction with the *Patterns* cyclic field to select a predetermined pattern.
- **Wall Paper** allows you to edit the cut spaces cut by cut or use the *Patterns* cyclic field to select a predetermined pattern.
- **Paper Size** allows you add or delete rows or columns to the cut pattern when *Wall Paper* is selected.
- **Manual Edit** displays all of the rows and columns. You can edit the cut spaces cut by cut or use the *Patterns* cyclic field to select a predetermined pattern.
- **Patterns** lets you select a predetermined pattern, such as All On, All Off, Top, Bottom, Left, Right, H Stripes, V Stripes, Chess Board, U-L Corner, U-R Corner, L-L Corner, L-R Corner, Cross, Farm, and As Is.

Refer to the *Virtuoso Layout Suite L User Guide*, Product Version IC 6.1.8 for details on these features.

Create Via: Cut Pattern Example



This is how you control via density

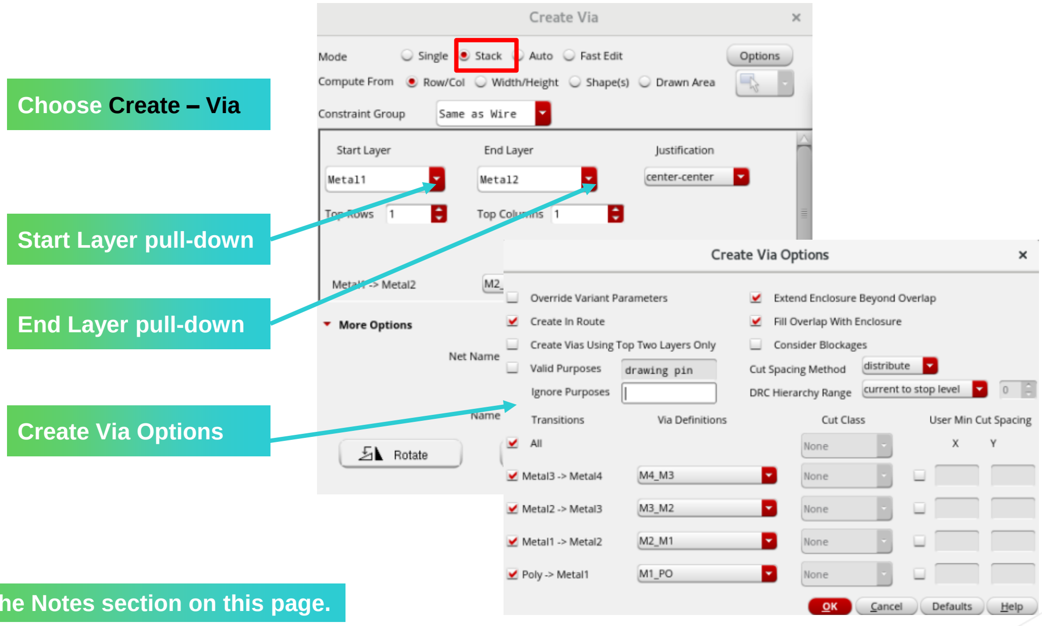
Placed vias can be edited

See the Notes section on this page.

The Cut Pattern can be edited for already placed vias as well.

Select a via, open the Edit Via Properties form and click **Edit Cut Pattern**.

Creating Vias in Stack Mode



See the Notes section on this page.

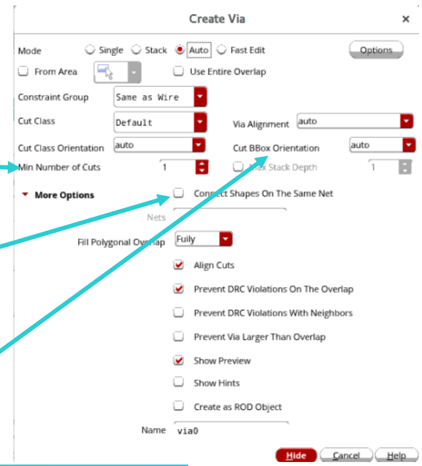
- **Stack Mode** – Selects one of the two layers between which you want to place the via stack. The layer selected in the End Layer list does not appear in the Start Layer list. The End Layer lets you select one of the two layers between which you want to place the via stack. The layer selected in the Start Layer list does not appear in the End Layer list.
- Choose **More Options** to open up the Create Via Options form.

Creating Vias in Auto Mode

Choose **Create – Via**

The size and number of cuts will fill the area where the shapes overlap.

Connects the shapes that are on the same nets.



Specifies the preferred orientation for the Cut BBox as either auto, horizontal, or vertical.

See the Notes section on this page.

243

Auto Mode Options: When you select this option, the form displays the following additional fields.

Use Entire Overlap uses the entire overlap to create the via. If this option is not selected, an intersection between the user area and the overlap box is used to create the via. **Environment variable:** *viaUseEntireOverlap*

Cut Class is a cyclic field that displays cut class choices only if a *cutClasses* constraint has been defined for the cut layer of the selected viaDef in the *layerRules* section of the technology file.

Via Alignment specifies the alignment of the via using the alignment choices: *auto*, *top-left*, *top-center*, *top-right*, *center-left*, *center-center*, *center-right*, *bottom-left*, *bottom-center*, and *bottom-right*. The default is *auto*. **Environment variable:** *viaAutoViaAlignment*

Cut Class Orientation specifies the preferred orientation for the cut class as one of the following: *auto*, *horizontal*, *vertical*, or *uniform in stack*. **Environment variable:** *viaCutClassOrientation*

Cut BBox Direction specifies the preferred direction for the cut box as one of the following: *auto*, *horizontal*, or *vertical*. **Environment variable**: *viaAutoViaCutArrayDirection*

Minimum Number of Cuts specifies the minimum number of cuts to be automatically created for any overlapping area between two shapes. **Environment variable**: *viaAutoViaMinNumCuts*

Align Cuts specifies the alignment of cuts between different tiles of a polygonal overlap. When this option is enabled, cuts between different tiles of a polygonal overlap are aligned. When this option is disabled, the polygonal overlap is decomposed into a set of largest tiles and vias are created independently in each tile, which may result in cuts not being aligned between different tiles. **Environment variable**: viaAlignCuts

Connect Shapes on The Same Net connects the shapes that are on the same nets.

More Options: The **Fill Polygonal Overlap** option is used when the overlap is polygonal. You can choose to either fill in all the tiles of the polygon overlap, *Fully*, or only the tile under the pointer, *Partially*.

Prevent DRC Violations On The Overlap prevents the creation of a via if it introduces DRC violations on the overlap. If this option is not selected, the via is created even if it introduces DRC violations on the overlap. **Environment variable:** *viaPreventDRCOnOverlap*

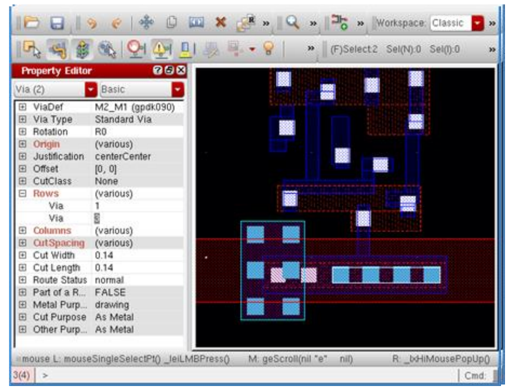
The following buttons are available in the *Single* and *Stack* modes:

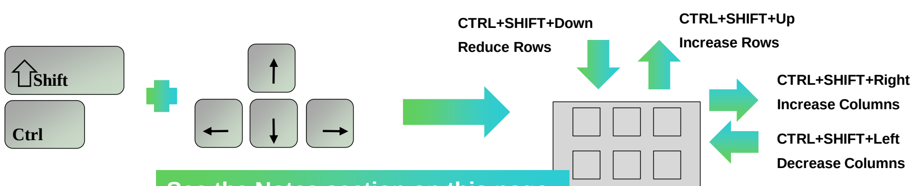
- **Rotate** turns the via 90 degrees counterclockwise. You can also click the middle mouse button to rotate the via.
- **Sideways** mirrors the via or via stack along the X axis. You can also press **Ctrl** and click the middle mouse button to mirror the via or via stack.
- **Upside Down** mirrors the via or via stack along the Y axis. You can also press **Shift** and click the middle mouse button to mirror the via or via stack.

Refer to the *Virtuoso Layout Suite L User Guide*, Product Version IC6.1.7 for more details.

Adjusting Placed Vias

- The number of Rows/Columns of an already placed via can be easily increased using the Property Editor assistant.
- To provide a more user-friendly method, a set of bindkeys has been defined that lets you interactively modify the dimensions of the placed via.





See the Notes section on this page.

Property Editor

If multiple objects are selected, the cyclic field in the top-left corner of the assistant can be used to view only a specific category (Via).

To change the size of a given via, the correspondent parameter should be expanded by clicking the *plus* symbol (in the example above the *Rows* parameter has been expanded) and then the *line* corresponding to the targeted via should be edited. The via which you are editing is highlighted. In the example on this page, a blue frame around the edited via is visible.

When the edit is finished, a click out of the edited field is required to make the changes.

Customized Bindkeys

The bindkeys described here rely on the SKILL code which is included in the Cadence Learning and Support Solution ID 11777642.

Refer to solution #11777642 on support.cadence.com from this link:

http://support.cadence.com/wps/mypoc/cos?uri=deeplinkmin:ViewSolution;solutionNumber=11777642

This SKILL code should be adapted to reflect the specialties of the technology you are using.

Note: In case a bindkey is already in use by the window manager you are using, such as GNOME or KDE, this bindkey will not be available inside the Virtuoso platform.

References

Virtuoso Layout full course reference

- Virtuoso Layout Design Basics
- Virtuoso Layout Pro: T1 Environment and Basic Commands
- Virtuoso Layout Pro: T2 Create and Edit Commands
- Virtuoso Layout Pro: T3 Basic Commands
- Virtuoso Layout Pro: T5 Interactive Routing
- Virtuoso Layout for Advanced Nodes
- Auto Place and Route (APR) for Virtuoso Studio – Device Level

Virtuoso Layout Training Bytes

- How To Use the Dynamic Measurement Feature to Measure Distance
- Quick Align and Align Toolbar
- Virtuoso Pin–To–Trunk Routing

Virtuoso Layout User Manual

- Virtuoso Layout Suite XL: Basic Editing User Guide
- Virtuoso Layout Suite EXL Reference

These links are accessible from the Reference section of the course interface.

This page does not contain notes.

Module Summary

In this module, you learned how to

- Set up and run Auto Place & Route (Auto P&R) assistant for effective device layout placement
- Generate the layout devices from the source (schematic)
- Perform device routing in three different ways
  - Pin-to-Trunk(P2T)
  - Auto Route using Wire Assistant
  - Manual Routing

246

cādence

This page does not contain notes.

Labs

Lab 5-1 Generating Layout from Schematic

Lab 5-2 Performing Pin Placement

Lab 5-3 Routing the Design in Different Methods

This page does not contain notes.

Module 6

Physical Verification

This page does not contain notes.

Module Objectives

In this module, you will

- Describe the importance of the Physical Verification step in the IC design flow
- Verify DRC interactively with the Virtuoso® Studio iPegasus™
- Set up and run the Pegasus™ DRC and LVS in GUI mode
- Debug DRC and LVS violations in the Pegasus Results Viewer

cādence

This page does not contain notes.

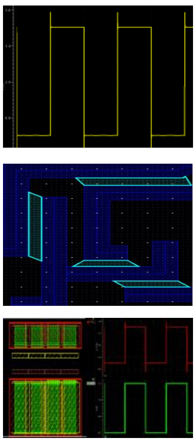
Terms and Definitions

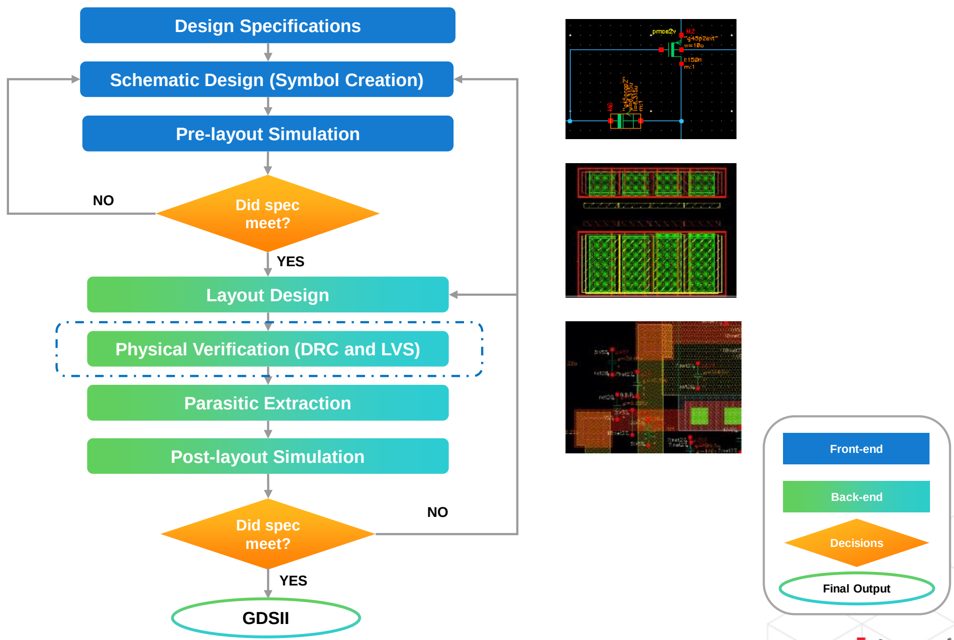
DRC	Design Rule Check (DRC) verifies if the layout design meets the specified design rules and constraints related to spacing, width, overlap, etc.
LVS	Layout Versus Schematic (LVS) is a process used to compare the layout of a design with its schematic representation to ensure they match accurately. It involves comparing the connectivity and geometric properties of layout with the corresponding schematic.
PVS	The Physical Verification System (PVS) is a Cadence® software tool used to ensure the integrity of a layout design in the field of integrated circuits. It performs DRC and LVS checks.
DFII	Design Framework II (DFII) is a software used for designing and manipulating layouts. The DFII files are used for storing and exchanging design data and can be exported/imported.

This page does not contain notes.

Analog IC Design Flow

PARAMETER	MIN	MAX
Gain		1000
Bandwidth	50KHz	500MHz
Supply Voltage	1.77V	5.5V
Supply Current	1.23mA	10.26mA
Propagation Delay (τ)	16.7ns	43.7ns
Chip area		2μm²





This page does not contain notes.

What Is the Physical Verification?

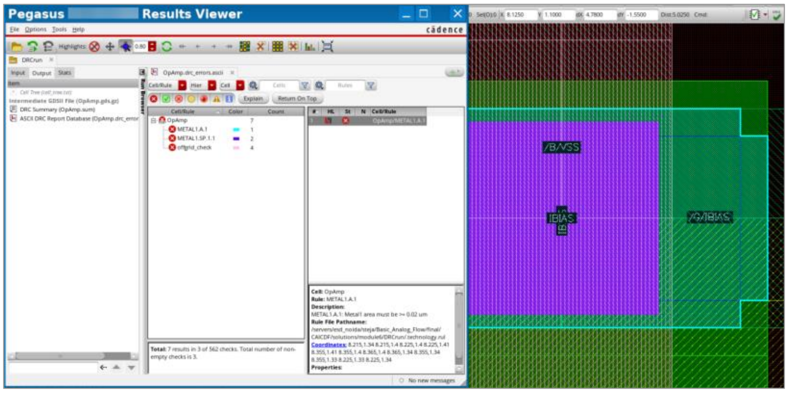
The physical Verification step in the IC design process plays a crucial role in ensuring the manufacturability, correctness, and reliability of the designed layout.

In this step, various errors may be obtained, such as DRC and LVS.

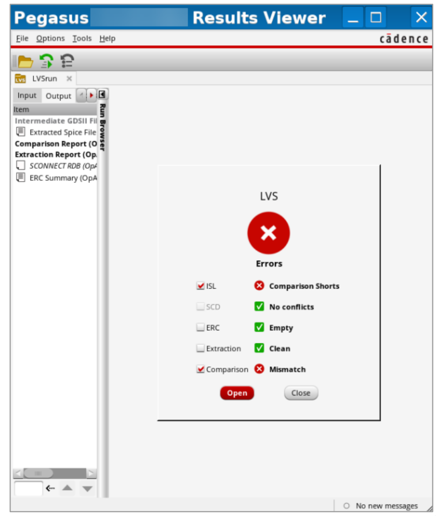
- The errors that are identified during physical verification can lead to manufacturing defects, circuit performance issues, and even device failures.
- The physical verification process involves different types of checks for compliance with the design rules, such as:
  - **Design Rule Check (DRC)**: The DRC checks to ensure the design meets the manufacturing requirements or not.
    - Example: Minimum width, min spacing, min area, end of line spacing, metal jog, etc.
  - **Layout Versus Schematic (LVS)**: The LVS checks whether the layout matches the intended schematic or not.
    - Example: Short circuit, open/flat pins or routing layers, etc.

This page does not contain notes.

Example: Physical Verification



Sample DRC Error



Sample LVS Error

This page does not contain notes.

Submodule 6.1

Design Rule Checking Using iPegasus DRC

Module 3 − Design Rule Checking

Submodule Objectives

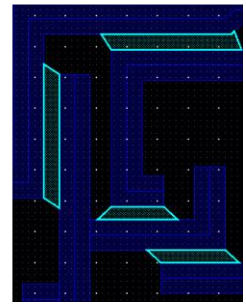
In this submodule, you

- Set up iPegasus DRC run Options form
- Create snapshots and run the iPegasus DRC check
- Debug and fix DRC errors and sign off your design using the iPegasus DRC

In this module, you outline various modes of running Pegasus DRC in the Virtuoso platform, set up and run Pegasus DRC in GUI and batch modes, save and load the Presets files for Pegasus DRC, list all the Pegasus Configurator features, examine the Pegasus DRC Waivers Solution, apply the Pegasus Layer Viewer for debugging rules codes, debug DRC results in Pegasus DRC Results Viewer, define Pegasus DFM, HPM and Metal Fill concepts, use the Pegasus Design Review platform, and set up a batch and GUI triggers in Pegasus DRC.

Design Rule Check (DRC)

- DRC – Verifies if the layout design meets the dimensional and manufacturing margins defined in the DRC rule decks.
  - Examines each layout shape against the DRC rules.
  - Flags any rule violation. Eg: minimum spacing, width, overlap, etc.
- PVS/Pegasus Rule decks:
  - Coded in PVL (Physical Verification Language).
  - Available for all major foundries.
  - Vary with manufacturing process nodes/versions.
- A PVS/Pegasus job can be started from Virtuoso, Innovus™, or Design Review
  - GUI mode or batch mode.
- Minimum data required for hierarchical DRC run.
  - Layout DB → DFII, GDSII, or OASIS
  - DRC Rule Deck → Foundry or Custom



Recommendation: Start DRC with the child cells and progress up the hierarchy.

256

What is Design rule checking or DRC?

DRC determines if a layout design satisfies the rules defined by the foundry. This typically implies that a DRC run examines the shape of each layer in the layout against the DRC rules, and flags any violations of rules such as minimum spacing, minimum width, minimum overlap, and so on. Each semiconductor manufacturing process has its own set of rule sets, also known as rule decks, and ensures sufficient margins, such that normal variability in the manufacturing process does not result in chip failure.

Regarding PVS or Pegasus Rule decks – generally, they are coded in PVL language. PVL stands for Physical Verification language. These decks are available for all major foundries in the market. For accurate runs, users must ensure that the rule decks are available in the right versions.

Most of the PVS or Pegasus jobs can be invoked from one of the Cadence platforms, like Virtuoso, Innovus, or Design Review, and can run in both GUI and batch modes. To run a hierarchical DRC, you need a minimum of two inputs: a layout database and a rule deck. The layout DB could be in the DFII, GDSII, or OASIS format.

In the DRC debugging flow, it is recommended to begin with child cells first, and progress up the hierarchy.

What Is the iPegasus DRC for the Virtuoso Studio?
Features

The iPegasus DRC is a Pegasus-based Virtuoso integrated, interactive signoff DRC check performed on the layout during the design phase using a foundry-qualified rule deck.

- Captures and validates design intent and ensures design convergence.
- **Performance Gain:**
  - No OA-to-GDS streamout.
  - 2X faster than Pegasus Interactive.
- **Unified Virtuoso GUI and iPegasus DRC Toolbar:**
  - Quicker DRC closure with the double engine – DRD and Pegasus signoff checker.
  - Efficient and Configurable – snapshots are customized pre-compiled rulesets.
  - Flexible – area, rules, layers, and symmetry-based optimization.
  - Optional density and connectivity checks.
- **Advanced Node Challenges: Coloring (MPT) Rules Supported**
- **Annotation Browser for Debugging:**
  - Improved error visualization.
  - Signoff DRC tab with search and scope features.
- iPegasus DRC license NOT required for designs under certain limits (process-node dependent)!

Virtuoso iPegasus DRC, offers a comprehensive Pegasus-based, in-design, integrated, and interactive sign-off quality DRC check on your designs, using the foundry-qualified rule decks early in the design cycle, to reduce overall turnaround time. Thus, Virtuoso iPegasus DRC ensures faster and more high-quality layouts. iPegasus captures and validates the design intent and ensures design convergence by performing in-design interactive DRC checks on the layout, immediately after editing the layout.

Coming to the performance, Virtuoso iPegasus DRC is proven to be 2x faster compared to Pegasus Interactive, with much shorter edit–check–fix cycles. Compared to the traditional signoff mode, the iPegasus model is faster, since there is no stream-out process here. That is, the DRC can be run directly on the layout database, without OA-to-GDS data translation. “Virtuoso iPegasus DRC” lets you edit the layout while conducting a DRC check, and run DRC on either the editable or the read-only cell view.

Virtuoso iPegasus DRC comes with a unified Virtuoso G U I and DRC toolbar, which is more efficient and configurable.

“Virtuoso iPegasus DRC” is an area-specific check. Use the G U I options to run “Virtuoso iPegasus DRC” on complete design, or custom specified area.

The G U I offers further options to optimize your DRC run – based on the list of rules and layers.

The G U I provides access to two DRC engines - DRD and signoff checker.

“Virtuoso iPegasus DRC signoff checker” uses a pre-compiled design rule deck called “Snapshots.” Hence, there is no run-time overhead for rule deck compilation, thus fastening the DRC run.

Virtuoso iPegasus DRC expanded verification package includes optional density and connectivity checks.

Coloring rules are supported in the “Virtuoso iPegasus DRC” utility.

An enhanced Annotation Browser is the default debugging environment for Virtuoso iPegasus DRC; that offers an improved error visualization feature and a streamlined debugging experience. The Annotation Browser User interface has a signoff DRC tab, with standard toolbar features like search, scope, etc.

Lastly, if you have Virtuoso XL, EXL, or MXL licenses, you don’t need a separate Virtuoso iPegasus DRC license to run DRC, for most of your runs. With some limitations, the user can run iPegasus DRC, if the checked design area is within certain limits, based on the process node. However, larger designs require the complete Virtuoso iPegasus DRC license.

iPegasus DRC: Faster Path to Signoff

Goal: To capture and validate design intent and ensure design convergence by performing in-design DRC checks on the layout while and after editing the layout.

iPegasus DRC is a productivity enhancer

- Uses sign-off quality foundry rule deck
- Reduces design + verification ToT

iPegasus DRC instant response time

- Finds DRC errors instantly

iPegasus DRC improves the layout quality

- Improves the QoR of your layout by pushing the DRM to its limits

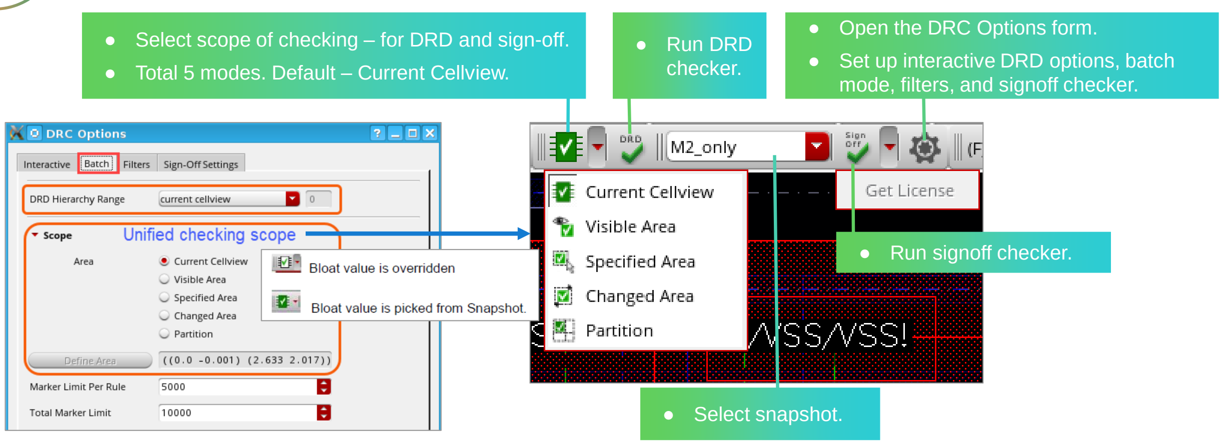
iPegasus DRC is equivalent to Pegasus signoff verification.

cadence

With various features, “Virtuoso iPegasus DRC” offers a faster sign-off path. The goal is to capture and validate design intent, and ensure design convergence, by performing in-design signoff quality DRC checks on the layout immediately after editing the layout. With fewer DRC signoff runs and pre-tape-out layout reworks, “Virtuoso iPegasus DRC” is a great productivity enhancer. By pushing the DRM to its limits and instantly reporting the DRC errors, “Virtuoso iPegasus DRC” improves the quality of your layout. However, note that final sign-off physical verification DRC is recommended.

iPegasus DRC Toolbar

Virtuoso Environment → Toolbars → iPegasus DRC toolbar



You can use the unified iPegasus DRC toolbar to set up the run in either DRD mode or signoff checker mode.

While the DRC signoff checker uses the foundry-certified rule deck, DRD uses the technology file as the input rule deck.

Let us look at various fields in the iPegasus DRC toolbar. Click the right-most settings icon to invoke the DRC options form. As the name indicates, you must set up your run using various fields in this form.

Use the leftmost icon in the toolbar to set the scope of the run. The five options give the flexibility to run DRC on a Specified Area, that ranges from the *Visible Area* to a *Current Cellview*. In addition to the area-specific checks, it lets you check the edited portion of the layout through the *Changed Area* option, and use the *Partition* mode when the layout is split between designers.

Current Cell view mode is the default selection. A green background in the current cell view icon shows that the bloat value is picked from Snapshot, whereas the white background shows that the bloat value is overridden. You may select the area mode from the batch tab of the DRC options GUI, or the toolbar drop-down, and both will be in sync.

To start the DRD run, click the Run DRD checker icon.

The DRD tool provides real-time dynamic DRC feedback to facilitate correct-by-construction layout design.

Use the Run signoff checker icon to start your signoff DRC run. You may run the signoff DRC on the complete foundry rule deck or a customized subset called snapshot. You may create your snapshots using the GUI options, or they are delivered as part of PDK. Available snapshots are listed in the snapshots field on the toolbar. By default, the Snapshots are sorted alphabetically. You may select a snapshot from the list and start the signoff DRC run.

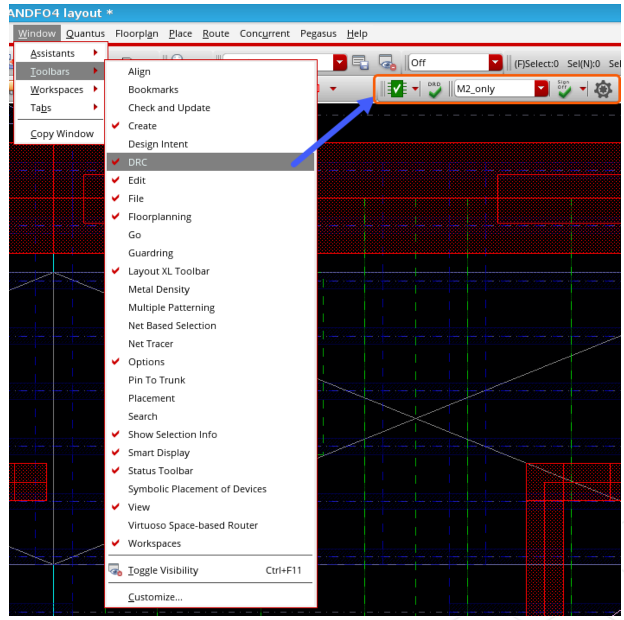
At the end of the run, iPegasus DRC lets you view and debug violations through the Annotation Browser.

COS Video (20481899): Introduction to the DRD User Interface

Invoking iPegasus DRC Toolbar

Virtuoso Environment → Window → Toolbars → DRC

When the Pegasus and Virtuoso setups are available, the iPegasus toolbar comes decked into the layout window. No other tool setup is required.

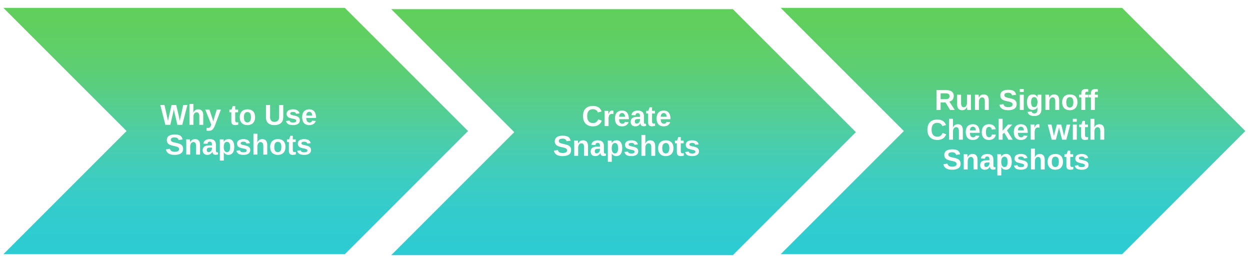


When the Pegasus and Virtuoso setups are available, the iPegasus DRC toolbar comes decked into the layout window. No other tool setup is required. If the toolbar is absent, go to the Windows – Toolbars drop-down in the Virtuoso Environment and select DRC. If the DRC selection is unavailable under Toolbars, this indicates that you did not point to the proper Pegasus installation.

How to Create Snapshots for the iPegasus SignOff Runs

A snapshot is a pre-compiled DRC/fill deck that is loadable only by Pegasus.

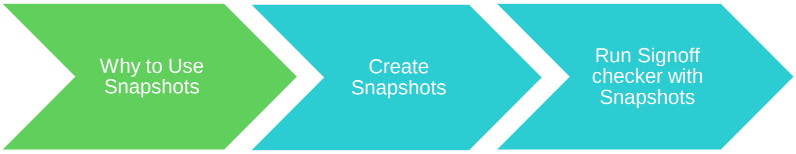
Snapshots allow the iPegasus direct access to design rules during signoff checks without recompiling. Let us see how to create and use snapshots to run DRC/Fill.



Let us see how to create and manage Snapshots for the iPegasus signoff runs.

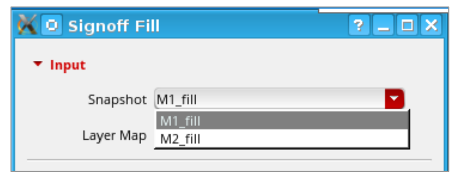
A Snapshot captures DRC or fill rules in a pre-compiled binary format, which only Pegasus understands. This format allows iPegasus, direct access to design rules, without re-compiling them from the foundry rule deck. This helps save compilation time, offers increased flexibility, and ensures optimal Virtuoso usage. In this section, let us see how to identify, create, manage, and use snapshots.

Step 1: Significance of Snapshots



- **Snapshots** – created by users or the CAD team.
  1. iPegasus DRC Run Options form → Create Snapshot.
  2. iPegasus Signoff Fill form → Create Snapshot.
  3. CAD admins can remove snapshot fields with shell variables.
- Advantages of using snapshot:
  - Reflects application intent
  - Eliminates run-time overhead by rule deck compilation
  - Faster switching between snapshots
  - Re-usability

By default, items in the snapshot combo field are sorted alphabetically.



262

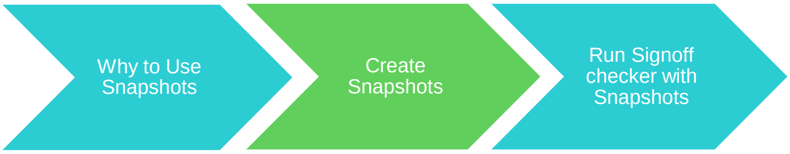
The first step is to identify Snapshots.

In a practical case, the Snapshots could be created by users or the CAD team. The users can develop Snapshots from the options in the "iPegasus Run Options form," or the CAD team can create them for the users, and control the Snapshot accessibility to target users, using certain variables.

For signoff DRC runs, all the approved Snapshots will be listed under the "Snapshot combo" field, in the "iPegasus DRC Toolbar." For the signoff fill runs, the Snapshots will be listed under the input Snapshot field, in the SignOff fill form. By default, items in the Snapshot combo field are sorted alphabetically. During the design phase, there are several advantages of using Snapshots to run the signoff DRC and fill checks, instead of the complete rule deck.

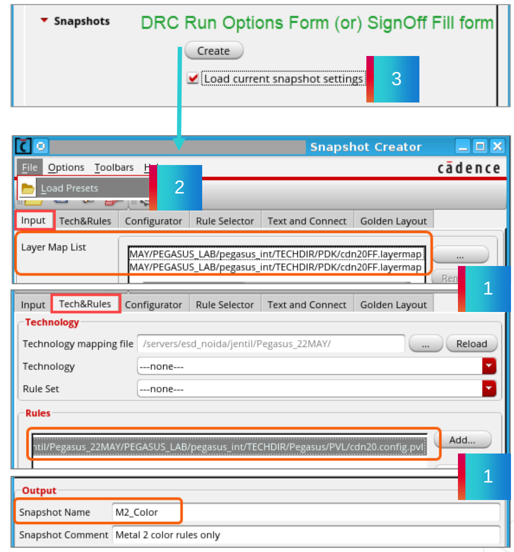
Snapshots enable targeted DRC and Fill runs that reflect the application intent. Since snapshots are pre-compiled rule decks, it saves compilation time. Snapshots enable faster switching, and enable re-usability of the custom rule sets. Thus, snapshots save valuable run time and lead to optimized signoff runs.

Step 2: Create Snapshots



There are four ways to create a snapshot:

1. Interactive creation via Snapshot Creator.
   - Minimum inputs: Layermap file, Rule Deck, Snapshot name.
   - Built-in intelligence to validate the inputs.
2. Preset file with config setup
   - Snapshot Creator: Load Presets icon / File → Load Presets.
3. Load current snapshot settings
4. Command Line



The next step is to create a Snapshot. There are multiple ways to do it.

"Snapshot Creator" allows you to create the Snapshot interactively. It can be invoked by clicking the "Create" button, under the Snapshot section in the "DRC Run Options form, or the Signoff fill form. The "Snapshot Creator" comes with multiple tabs. Three minimum inputs are required to create a snapshot - the Layermap file, Rule Deck, and a name for the Snapshot. The Snapshot Creator form has built-in intelligence, that checks input validity before generating Snapshots. You need to feed the "layer map" files under the input tab, Rule Deck through Tech and Rules tab, and the snapshot name through the output field in the bottom half of the form. Then click "Apply" or "OK." Snapshot gets created, and populated under the appropriate "Snapshot field", on the Signoff DRC toolbar or the Signoff fill form.

You may also configure the "Snapshot Creator" form with a preset file. To load the preset, you can interactively click the "Load Presets" icon on the "File" toolbar, or select "Load Presets" under the "File" dropdown. The preset file loads the pre-saved configuration into the "Snapshot Creator form."

Like creating Snapshot using a preset file, "iPegasus" allows you to load the current active Snapshot setting, while invoking the Snapshot Creator form. This feature allows you to create a Snapshot from a reference Snapshot setting. This option is enabled by default in the DRC Run Options form or the Signoff fill form, when iPegasus identifies an existing active Snapshot. An alternative mode of creating the Snapshot is from the command line.

You can directly open the Pegasus Interactive Snapshot form without invoking Virtuoso (standalone mode) using the following command:

UNIX> pegasusgui -pegasusintSnapshot

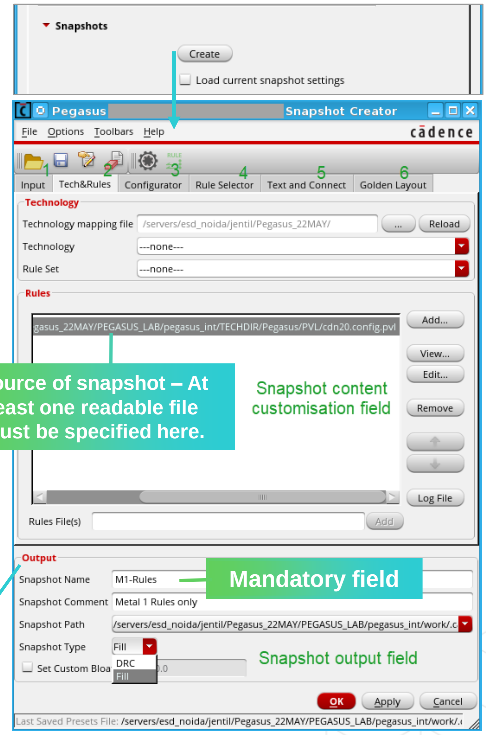
Step 2: Create Snapshots – from GUI

Why to Use Snapshots → Create Snapshots → Run Signoff checker with Snapshots

DRC Options form – Sign-Off Settings tab Or **SignOff Fill form → Snapshots → Create → Snapshot Creator form**

The **Snapshot Content Customization** field has six tabs:

1. **Input:** Polygon handling info
2. **Tech&Rules:** Technology design rules deck
3. **Configurator:** To override the default rule-deck setting
4. **Rule Selector:** Interactive rule selection
5. **Text and Connect:** Text-based connectivity
6. **Golden Layout:** For graphical DRC



- **Snapshot name, comment, location, and type**
- **To override the default bloat value**

264 © Cadence Design Systems, Inc. All rights reserved.

cadence

Let’s look at the GUI-based Snapshot Creation in detail. This Snapshot Creator form is another derivation of the “Pegasus run submission form.” However, the fields are specific to Snapshot creation. “Snapshot Creator form” can be divided into two halves – the “Snapshot Content Customization field” at the upper half, and the “Snapshot output setup field” at the lower half. The “Content Customization Field” has six tabs - Input, Tech and Rules, Configurator, Rule Selector, Text and Connect, and Golden Layout.

The Snapshot output fields remain the same for all six tabs under the Snapshot Content Customization section.

The “Input” tab defines polygon handling info between Virtuoso and Pegasus, including the mandatory layer map list, and the optional object name table. By default, Pegasus will pick up the Layermap file from the technology.

The “Tech and Rules” tab describes the primary rule deck for the Snapshot creation. At least one readable rule deck file must be specified, under the technology or rules section. If no valid rule deck is set up, “iPegasus” flags missing required inputs, by boxing the fields with the red highlight.

The “Configurator” tab can override the default rule-deck setting with a “configuration file.”

The “Rule Selector” tab, lets users interactively select the rules for Snapshot creation. There are wildcard features to make blanket selections as well.

The “Text and Connect” tab defines text-based connectivity criteria for iPegasus, and the “Golden Layout” tab describes golden layout pattern files for graphical DRC.

The “Output” field contains the final Snapshot name declaration, Snapshot comment, the Snapshot directory path, Snapshot type selection, and a checkbox for defining custom bloat value.

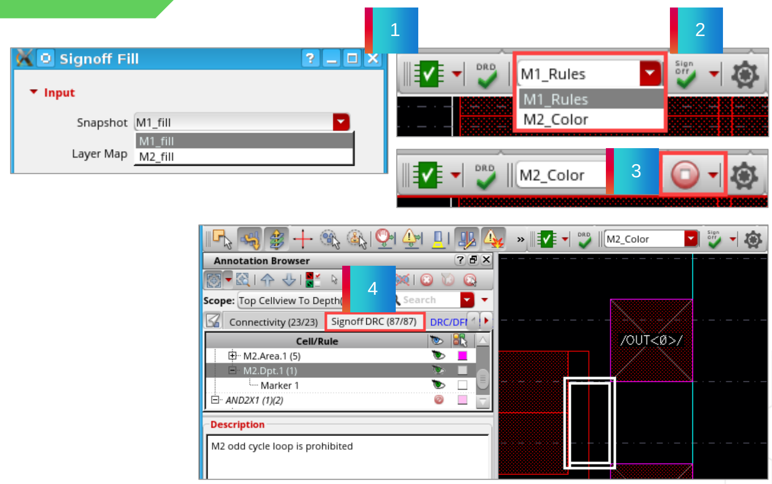
Snapshot name is a mandatory requirement. Two selections are available under the Snapshot type drop-down – DRC and Fill. Make your selection as per your application.

Once the form is set up, click “Apply” or “OK.” The Snapshot gets created and populated under the appropriate “Snapshot fields” for signoff DRC or fill runs. Once the snapshots are generated, close the form.

Step 3: Run iPegasus DRC with Snapshots

Why to Use Snapshots → Create Snapshots → **Run Signoff checker with Snapshots**

1	Select the snapshot (optional)
2	Run Signoff checker / Signoff Fill
3	Run-in-progress
4	Debug DRC / Simulate with Fill
5	Repeat till the design is clean



cādence

You are set to run the iPegasus signoff runs with snapshots. Available snapshots are listed under the snapshots field on the iPegasus toolbar or the Signoff fill form.

By default, the Snapshots are sorted alphabetically. You may select a snapshot from the list and start the signoff DRC or Fill run.

To start the DRC run, click the signoff checker icon. While the run is in progress, the signoff checker icon turns red. The icon returns to its original form once the run is complete, and the results get loaded under the Signoff DRC tab of the Annotation browser.

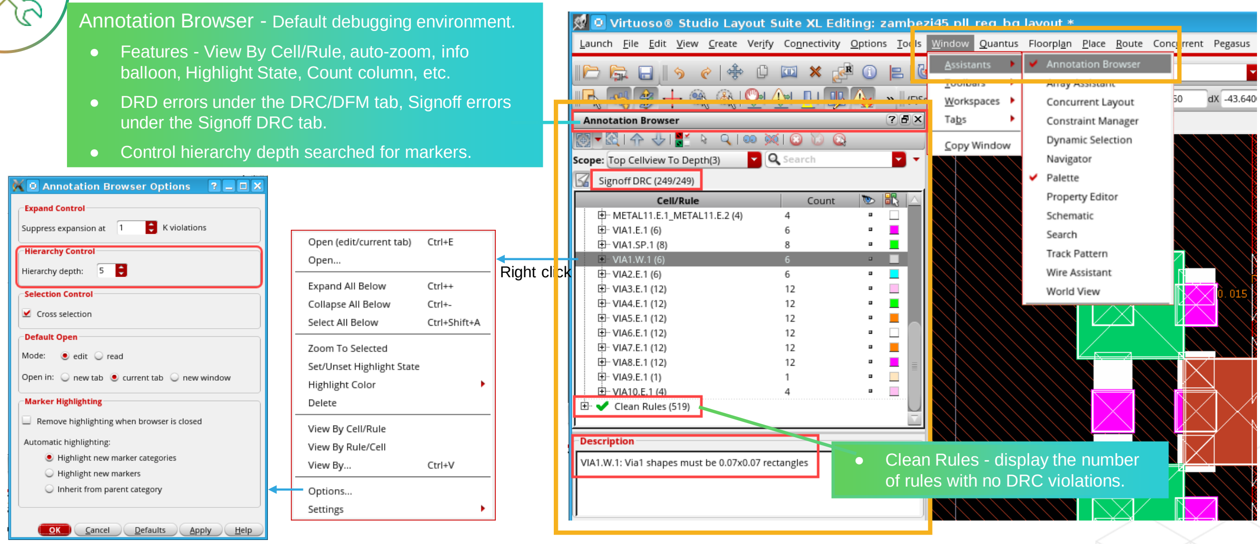
Click the generate icon on the signoff Fill form to start the signoff fill. The fill results get loaded in the signoff fill tab in the Annotation browser. Debug the results and repeat the process till the design is clean.

Annotation Browser: For Debugging the iPegasus Violations

Virtuoso Layout → Window → Assistants → Annotation Browser

Annotation Browser - Default debugging environment.

- Features - View By Cell/Rule, auto-zoom, info balloon, Highlight State, Count column, etc.
- DRD errors under the DRC/DFM tab, Signoff errors under the Signoff DRC tab.
- Control hierarchy depth searched for markers.



For error viewing, iPegasus uses Virtuoso's “Annotation Browser.” Annotation Browser is the native browser for Virtuoso tools. It is not invoked automatically at the end of iPegasus runs unless activated. Do it in the Virtuoso layout suite from the Window drop-down, select Assistants, and then Annotation Browser.

Once activated, at the end of the iPegasus runs, be it DRD, signoff checker, or signoff fill, the Annotation Browser triggers with violations populated. For each type of run, the results get loaded in distinct tabs. DRD results are loaded in the DRC DFM tab, whereas signoff checker results are in the Signoff DRC tab. The fill results are loaded in the Signoff fill tab.

The “Annotation Browser” reports the markers and lets you view and analyze DRC, Antenna, Density, ERC, and fill violations. The selected markers are auto-zoomed and highlighted in the layout. You may start debugging.

The annotation browser has several Additional features. You may list the violations in a view-by-cell and view-by-rule fashion.

Regarding the scope of the browser, the hierarchical markers can be viewed in the Annotation Browser in the hierarchy view, from the top cell view to the selected depth.

The count column displays the number of violations for each rule. The count column can be sorted based on the number of violations. Clean Rules to display the number of rules with no DRC violations.

The info balloon feature can be activated from the Dynamic Display form, invoked from the options drop-down, to help list the details on the layout suite, such as the tool name, a short message, and the violation description.

The hierarchical markers can be viewed in the hierarchy view in the Annotation Browser.

In the browser pane, you can control the Highlight State and the color used to draw the markers in the layout canvas.

Annotation Browser: Advanced Features

- Improved data storage during AB hierarchical traverser.
- CDS Environment Variables:
  1. To fix column sizes in AB.

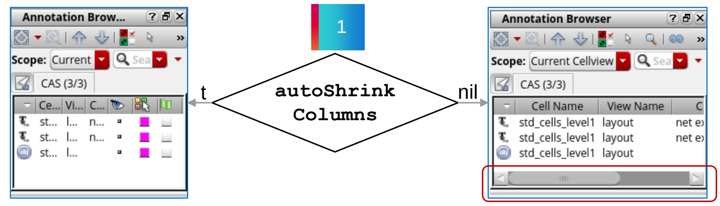
     `autoShrinkColumns t | nil`

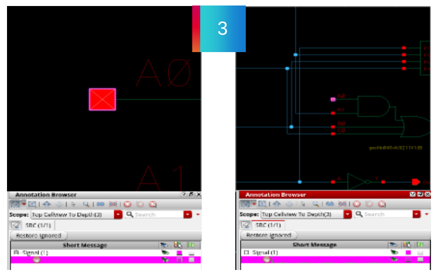
  2. To allow descending/ascending cell view by **double**-clicking the dimmed marker.

     `descendOnDoubleClick t | nil`

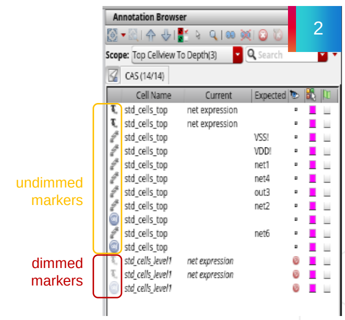
  3. To define the max zoom factor.

     `autoZoomMaxScaleRatio 7 | <number>`





autoZoomMaxScaleRatio 7 autoZoomMaxScaleRatio 50



Let’s look at some of the advanced features in the Annotation Browser.

The new algorithm for the Annotation Browser addresses the way data are stored during hierarchy traversal in large designs. It helps traverse the hierarchical path faster.

There are C D S environment variables meant to address specific tasks.

Variable auto Shrink Columns can be used to keep the size of individual columns unchanged even as the Annotation Browser's size is shrunk or expanded. The default value of this variable is Boolean t, which stands for true. That means when the Annotation Browser is wide and then shrunk, the columns are shrunken as well. Use the Boolean nil, to avoid columns being automatically shrunk.

There are two types of markers: the “undimmed" or dark markers, present in the current edited cell view, and the "dimmed" or light markers, present in the top or bottom hierarchies but not in the current edit cell view. With the C D S environment variable descend On Double Click, set to true, you can allow the descending or ascending cell view by double-clicking the dimmed markers. The Annotation Browser performs a descent into the cell view containing the marker and is auto-zoomed. Nil is the default value for this variable.

By default, the auto zoom feature in Annotation Browser is limited to 7 times the size of the marker box. However, with the environment variable auto Zoom Max Scale Ratio, you can increase this max zoom to get a better overview of the marker. The default value of the variable is 7; however, it can be set to a higher zoom factor of user choice.

Submodule 6.2

Design Rule Checking Using the Pegasus Verification System

This page does not contain notes.

Submodule Objectives

In this submodule, you

- Set up and run Pegasus DRC in GUI mode
- Debug DRC errors in the Pegasus DRC Results Viewer

This page does not contain notes.

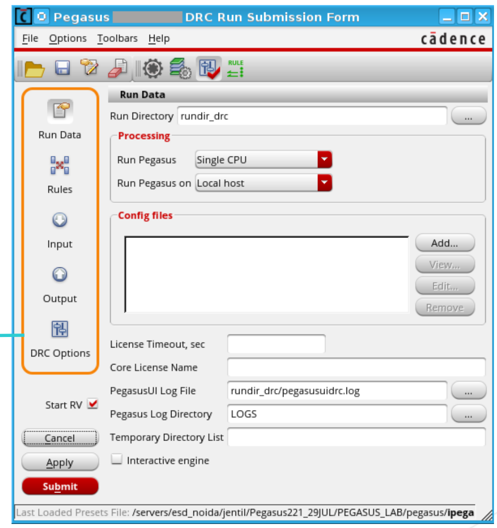
Invoking the DRC Run Submission Form

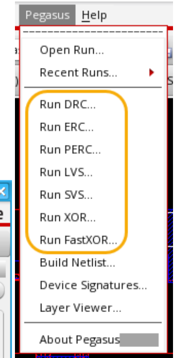
Virtuoso/Innovus/ Pegasus Design Review → Pegasus → Run DRC → Pegasus DRC Run Submission Form

Standalone Pegasus GUI → DRC

- This is the starting point for setting up and executing the Pegasus DRC run.
- This is the basic look of the Pegasus form for all the verification runs – whether it is DRC/LVS/ERC, etc.
- You need to set up this form to submit the DRC run.

The submission form has five sections:
1. Run Data
2. Rules
3. Input
4. Output
5. DRC Options





cadence

How do you invoke the Pegasus DRC Run Submission form?

You may invoke the form from the Virtuoso, Innovus, or Pegasus Design Review platforms, where you click on the Pegasus drop-down on the toolbar, then select Run DRC, and the GUI comes up.

This is the basic look of the Pegasus form for all the verification runs – whether DRC, LVS, ERC, etc.

It has five sections: Run Data, Rules, Input, Output, and DRC options.

Under the Run data section, you set up run details such as run directory, CPU and processing details, config and log files, interactive engine, etc.

Use the processing section to run Pegasus with a Single CPU or to distribute across multiple CPUs.

Use the Config files field to specify the Pegasus configuration file for the run.

Use the Core License Name field to specify the license to be used for CPU scaling.

The Pegasus UI Log File field can be used to specify the name of the Pegasus log file. By default, it is named Pegasus UI, followed by the type of form dot log, and is saved to the Run Directory.

Use the Pegasus Log Directory field to specify the name of the log directory, or use the file browser button to select it. The default value of the Pegasus Log Directory is LOGS.

Select the Interactive engine checkbox to run the Pegasus Interactive engine. It is recommended to enable this checkbox when Pegasus DRC is run on block-level for better performance.

There is a provision to choose the Results Viewer as the error viewer.

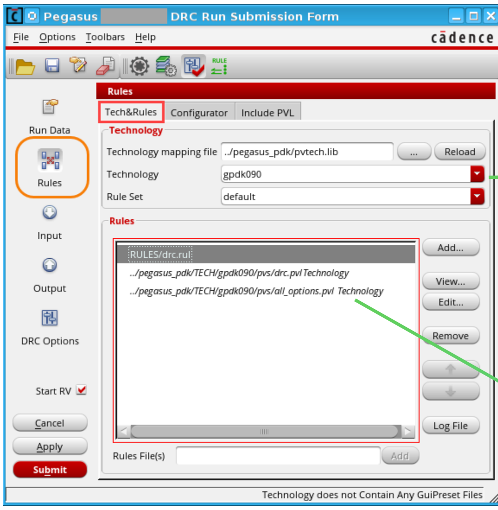
Before submitting the run, you must set up all the Pegasus run form sections.

DRC Run Submission Form

Rules – *Tech&Rules*

Select the appropriate technology and rule decks for the Pegasus run.

Pegasus DRC Run Submission Form → Rules → Technology & Rules



Sample Tech Map File - Eg: pvtech.lib

```
DEFINE tech065  /home/alpha/…/…/tech065
DEFINE tech090  /home/alpha/…/…/tech090
```

Technology

- Select the Technology mapping file if not pointed by default.
- The Technology field will list directories from the mapping file. Select one.
- Select a Rule Set from the selected Technology directory (ref. `techRuleSets file`).

Sample techRuleSets file

```
pegasusRuleSet( "default"
  (DrcRules "drcMain.pvl")
  (LvsRules "lvs.pvl"))
pegasusRuleSet( "antenna"
  (DrcRules "offgridcheck.pvl" "antenna.pvl"))
```

Rules

- Add/remove/view/edit custom/foundry rule file(s) to Pegasus run. (Optional is technology/rule set is specified above.)
- Standalone or append to the Technology Rule Set.
- Multiple rule decks are supported for a DRC run.

cādence

The Rules section in the Pegasus DRC Run Submission form is used to specify the rule deck file for your DRC run. There are three tabs under the Rules section: Tech And Rules, Configurator, and Include PVL. Let us look at Tech & Rules.

Under the Tech And Rules tab, there are two halves: Technology and Rules. You may specify a rule file through either of these sections. If you select a rule file with technology/ruleset combination, then the Rule file selection is not necessary under the Rules field. Any rule file you specify with the rules field will be in addition to the DRC rule pointed to by the technology/ruleset. You can specify the main rule file using the technology/ruleset and options or switches in the additional rule file.

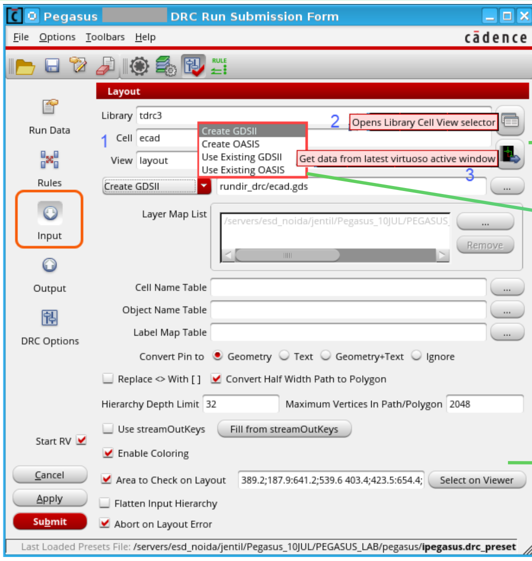
Under the technology field, specify the technology mapping file – for example, pvtech.lib – that contains the technology library names and locations. Click the Reload button to reload the technology file, and refresh technology, and rule set details accordingly. Then select the technology you want to use in the run. Then select the rule set from the list of rule sets defined for the technology specified in the technology field. The rules are defined in the file techRuleSets in a technology directory.

Moving on to the next section, type the name of the rule deck in the Rules Files text field, and click Add. The rule deck will be displayed in the text box below. Alternatively, you may browse the rule deck file location as well. If the rule file you have selected is empty or does not contain DRC rule checks, the Rules block shows a red border outline, and if you take the mouse to that rule file, a tooltip is shown that the rule deck file does not contain any rules, or the rule deck file does not contain DRC rule checks respectively.

You can add multiple rule deck files to your DRC run. There are options to remove or view the Rules Files as well.

DRC Run Submission Form
Input

DRC Run Submission Form → Input → Layout



Input layout information – 3 options
1. Enter Library, Cell, and View info.
2. Open *Library Cell View* selector.
3. Get data from the latest Virtuoso active window.

Input layout format – 4 options:
- Create GDSII file
- Create OASIS
- Use Existing GDSII
- Use Existing OASIS

- Enable Coloring – Only for Adv nodes. Virtuoso passes the predefined mask data to Pegasus.
- Select area(s) to check DRC on layout – manually or enter the coordinates.

When you click the Input icon, the Layout section is displayed, where you can set multiple input layout-specific fields.

In the first half, you enter the input database details like a library, cell, and view names. By default, the data is grabbed from the currently open layout window. You may also open and make a selection at the Library Cell View selector or click the appropriate field to get data from the latest Virtuoso active window.

Available Layout Input Format selections are Create GDSII, Create OASIS, or Use Existing GDSII or OASIS. Depending on the format selection, the form reorganizes to reflect minor changes.

“Cell Name Table,” “Object Name Table,” and “Label Map Tables” can be manually entered or browsed to from the custom locations.

Four options exist to convert the Pin, to Geometry, Text, Geometry + Text, or Ignore.

There is an option to convert angular brackets to square brackets in text records during the translation from the DFII library to the OASIS file.

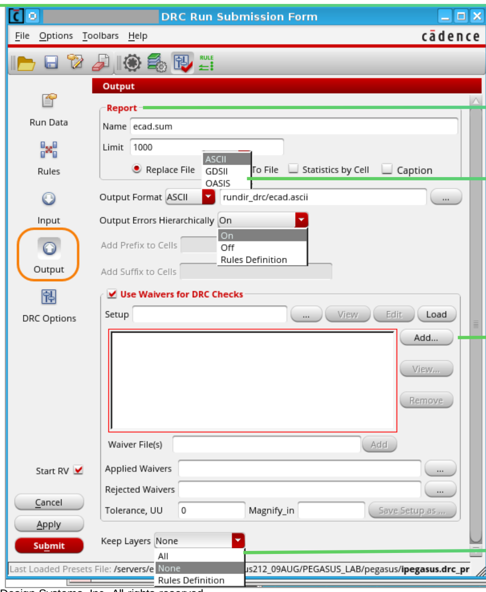
The “Enable Coloring” selection is available only for advanced node releases of Virtuoso. This is to enable the multiple mask feature in advanced nodes. When this feature is selected, Virtuoso passes the predefined mask data to Pegasus, and when unselected, all data on a layer is seen as if it is on the same mask.

Use the Area to check on the Layout option to select part of the layout either with the mouse or through coordinates and run DRC only in that part of the layout. By default, this field is deselected. You can select multiple regions.

By default, “Abort on Layout Error” is selected, which aborts the Pegasus run when it encounters fatal errors. The fatal errors can be missing cell masters, duplicate cell references, absolute angle for an instance placement, absolute magnification for an instance placement, absolute width parameters for a path, or rule precision values inconsistent with the input database precision.

DRC Run Submission Form
Output

DRC Run Submission Form → Output



Report
- Name of the report file
- Limit of lines in the report

Output Format Control
- ASCII file report / GDSII data file of errors / OASIS file

DRC Waivers
- Waivers block is disabled by default; and is activated only when:
  - *Output → Output Errors Hierarchically* is On
  - *Input → Flatten Input Hierarchy* is disabled
  - *Run Data → Interactive engine* is selected

```
pegasus -drc -interactive -bwf <waiver_setup_file> pegasus.rul
```

Keep Layers
- To control visibility of layers (including intermediate) in Pegasus Layer Viewer
- 3 options: All, None, Rules Definition
- Enabled only if *Run Data → Interactive engine* is selected.

273

cadence

Cadence Learning and Support link to the video on DRC Waivers: How to Use PVS DRC Waivers?

Cadence Learning and Support link to the video on PVS Layer Viewer: How PVS Layer Viewer Works as a Great PVL Debugger?

When you click the Output icon, the Output section is displayed in the Pegasus DRC Run Submission form. You can enter the name by which you want the summary report to be generated or set a limit on the number of lines in the report. You may also choose to replace or append to old reports for fresh runs.

In the next section, you can select the format in which you want to see the output. The options are ASCII, GDSII, OASIS, and Annotation Browser. The path and filename for these files are automatically created, but you may change them with the Browse button and by specifying the new path. An ASCII file is needed as input to the DRC error browser for debugging of any errors.

Pegasus supports block-level Waiver flow through the Pegasus interactive engine, which can be run from the command line or GUI. The command-line option is provided here.

In the GUI mode, select the **Use Waivers for DRC Checks** checkbox to activate the waiver section. To enable it, set the Output Errors Hierarchically option to **On** in the Output section, and select the Interactive engine checkbox in the Run Data section. You may then choose to enter or browse to the name of the waiver setup file in the optional "setup field" text box and click **Load**. The required fields like the Waiver Files, Applied Waivers, Rejected Waivers, and Tolerance fields are populated with the values of the Waiver Setup file. Otherwise, you may enter those details one by one. After providing the required details, submit the Pegasus run.

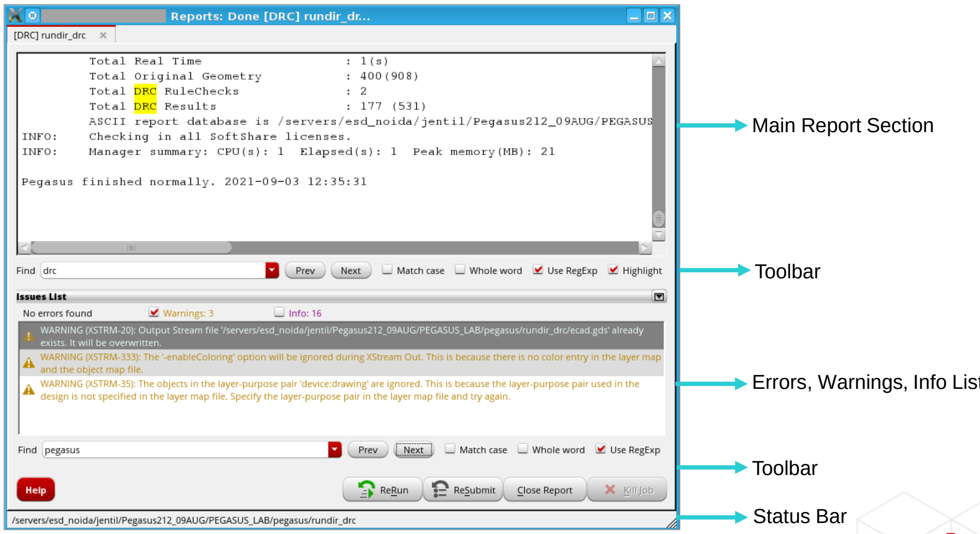
The Keep Layers feature helps control the visibility of layers, including intermediate layers, in Pegasus Layer Viewer. The three options in the drop-down menu are All, None, and Rules definition. All allow you to keep all layers, including intermediate layers. None does save any layers. This is the default. Rules Definition allows you to keep layers on the basis of the *keep_layers* command in the rules file.

DRC Debugging

Reports Window

DRC Run Submission Form → Submit → Reports

The Reports window comes up when you apply/submit the DRC run. The window displays the summary of inputs to help debug startup issues.



274

After the DRC run is completed, the DRC report of the run is generated for debugging DRC errors. The DRC report window is divided into three parts: the Main Report section, the Toolbar, and the Status Bar.

The Main Report section shows the log of the current job. Errors and warnings are automatically highlighted in this field.

The toolbar consists of multiple options to find a keyword or sentence in the report to highlight the search result or the Next and Previous buttons to navigate the report.

Click the arrow button next to Issues List, to expand and view the list of errors, warnings, and info. By default, this field shows errors and warnings.

ReRun would restart the job in the same run directory, with the same run options.

ReSubmit provides quick access to the DRC run submission form.

Hovering the mouse over the ReRun and ReSubmit buttons will display tooltips.

Close Report closes the Log window, and Kill would exit the current job. Click the Help button to access Cadence Doc Assistant.

The Status bar shows absolute paths to the run directory.

DRC Debugging

Modern DRC rule decks consist of more than 6000 checks distributed across sections like main DRC rule decks, Antenna rule deck, Pad rule deck, etc. Let's look at some of the best practices to effectively debug and correct DRC mismatches.

- Usually, there are at least three DRC rule decks for one technology:
  - Main DRC rule decks
  - Antenna rule deck
  - Pad rule deck, which is needed only for the top-level checks
- Modern DRC rule decks consist of more than 6000 checks, which roughly can be divided into the following parts:
  - Backend checks: Geometrical checks of Poly, OD and Implants
  - Frontend checks: Geometrical checks of metals
  - Latch-up checks: Connection to substrate and wells
  - Offgrid-checks: Grid-related checks, checking for notches and jogs
  - ESD checks: Checks for I/O-related nets, guardrings and separation from the logic
  - Density-checks: Checks for metal, poly and od density

cadence

Usually, there exist at least three types of DRC rule decks per technology, and they are the main DRC rule decks, the Antenna rule deck, and the pad rule deck, which is needed only for the top-level checks.

Modern DRC rule decks consist of more than 6000 plus checks, which roughly can be categorized under backend checks, frontend checks, Latch-up checks, off-grid checks, ESD checks, and Density checks.

Solution: DRC Debugging Flow

Use this recommended flow in the Virtuoso® platform:

1. Make your DRC run.
2. Start Virtuoso and bring up the design in the layout viewer.
3. Start the DRC Debug Environment from the Virtuoso platform.
4. In the error browser list, click on the cell or rule that you want to start viewing errors for.
5. Select the next arrow (right arrow) to have Virtuoso display the first instance of the error.
6. The error is highlighted in the Virtuoso viewer window.
7. Select the desired status to apply to that error and make your edits.
8. Continue steps 4, 5, 6, and 7 until you are ready to move to the next cell or rule violation.
9. Continue until you have reviewed all errors for all cells and rule violations.
10. Make any remaining necessary layout edits.
11. Save your work, rerun DRC, and repeat these steps as necessary.

Recommendation: Start DRC debug with the child cells and progress up the hierarchy of the design.

Here is a recommended DRC debug flow in a standard Virtuoso platform. Please read through the lines.

It is always a better practice to first array the DRC list on the complexity scale, and then address the most complex violations at the beginning. Even if you need a significant change – say a cell movement – they are not overlooked till the last moment.

Submodule 6.3

Layout Versus Schematic Using the Pegasus Verification System

cadence

This page does not contain notes.

Submodule Objectives

In this submodule, you

- Set up and run the Pegasus LVS in GUI mode
- Debug LVS errors in the Pegasus LVS Results Viewer

This page does not contain notes.

What Is Layout Versus Schematic (LVS)?

Layout Versus Schematic (LVS) compares the device and connectivity between the circuit layout and the schematic and guarantees that the layout represents the circuit you desire to fabricate. Before comparison, LVS converts layout and schematic info to specific netlist formats.

1. Layout netlist: DFII/GDSII/OASIS/Extracted Layout Netlist
2. Schematic netlist: CDL/SPICE, Verilog, DFII

→ **LVS (Extraction & Comparison)** →

1. LVS Report – Extraction & Comparison
2. ERC Summary
3. Extracted SPICE/SCH netlist files

- LVS uses Virtuoso® StreamOut and CDLout utilities to convert the DFII layout and DFII schematic into GDSII and CDL, respectively. If the schematic input is Verilog, use v2cdl to convert it to CDL.
- LVS extracts the devices and connectivity information from GDSII and creates a SPICE netlist.
- Compare the layout netlist and the schematic netlist and create a mismatch report.
- PVS/Pegasus supports mixed netlists.
- LVS job can be run from the command line or GUI, invoked from the PVS/Pegasus menu in Virtuoso, Innovus™, or PDR.

Layout Versus Schematic, shortly known as LVS, compares the layout netlist with the schematic netlist. LVS check provides device and connectivity comparisons between the physical representation, the circuit layout, and the logical representation, the schematic. The schematic is considered Golden. For the physical circuit to behave as intended, the two design representations must be identical. For the purpose of comparison, LVS converts the layout and the schematic info to specific netlist formats.

What are the required input data to start an LVS job?

You need to have both layout and schematic information, plus the input/output data filenames of the design in the LVS rule file. The layout information can be in DFII, GDSII, OASIS format, or one of the accepted extracted Layout Netlist formats. For schematic data, LVS supports CDL, SPICE, or Verilog netlists. Cadence PVS or Pegasus also allows mixed netlists in SPICE, CDL, and Verilog formats and are order-independent.

The LVS application is a two-step process. Extraction and comparison.

LVS uses Virtuoso StreamOut and CDLout utilities to convert DFII layout and DFII schematic into GDSII and CDL, respectively. If the Verilog netlist is used, then the v2cdl utility is used to convert Verilog to CDL netlist. Then LVS extracts the devices and connectivity information from GDSII layout using the extract rules deck, and creates a SPICE netlist of the layout. This is followed by comparing the layout netlist and the schematic netlist. If differences are found, a mismatch report is created. The differences could be device mismatch, parameter mismatch, connectivity mismatch, shorts, opens, etc. You can see and debug the mismatches using the LVS Debug Environment.

In addition to the Extraction and Comparison report files, there is the option to create other output files, such as the ERC summary file, extracted Layout SPICE file, schematic netlist files, and the log file.

The LVS job can be run from the command line or GUI, invoked from the PVS or Pegasus menu in Virtuoso, Innovus, or PDR.

LVS Flow: Extraction, Reduction, and Comparison

- **Extraction:**
  - Recognizes and extracts drawn device shapes/layers/nets and connectivity info from the input layout geometry.
  - Runs layout DB with polygon areas through **area-based logical operations** to define device recognition layers, device terminals, wiring conductors, vias, and pin locations to determine the components represented in the layout.
  - The layers (metals or vias) that form devices can have various measurements performed, and these measurements can be attached to these devices.
  - Extraction report (*reportName*) summarizes the extraction process, including reports on issues about stamping conflicts, diff labels on the same net (a common issue), malformed devices, etc.
- **Reduction:**
  - An intermediate step between extraction and comparison.
  - The tool combines the extracted components into possible series and parallel combinations and generates a netlist representation of the layout (extracted netlist, for example, SPICE).
  - A similar reduction is performed on the "source" Schematic netlist.
- **Comparison:**
  - Compares the extracted layout netlist to the schematic netlist and report differences.
  - If the two netlists match, the circuit passes the LVS check.
  - Mathematically, the netlists are compared by performing a **Graph isomorphism** check to see if they are equivalent.
  - LVS Comparison report (*reportName.cls*) summarizes the differences and other info about compared cells.

280 cādence

Let's take a detailed look at LVS flow – the flow consists of three steps – layout extraction, layout, and schematic reduction, and then finally, netlist comparison.

In the extraction step, the drawn layout layers, devices, nets, and connectivity info are extracted from the layout geometry data input to LVS. Logical operations are used in the extraction algorithm to define device recognition layers, device terminals, wiring conductors, and vias and pin locations from the input layout database. They are used to determine the components represented in the layout. The metals or vias layers that form devices can have various measurements performed, and these measurements can be attached to these devices. The extraction report summarizes the extraction process and includes reports on issues like stamping conflicts, different labels on the same net, malformed devices, etc.

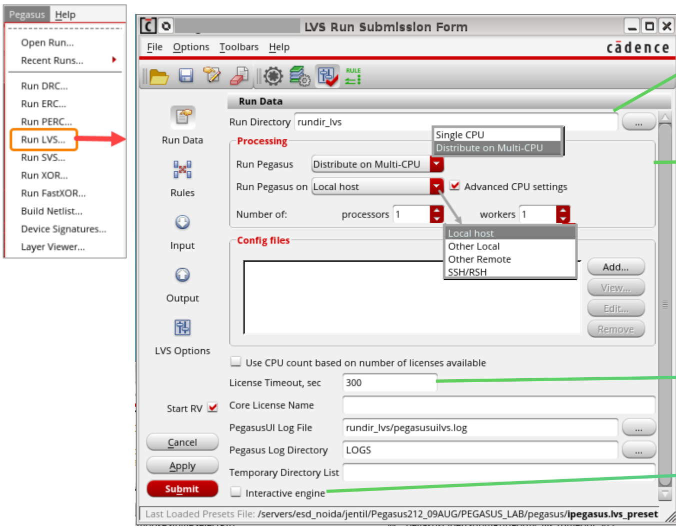
The reduction is an intermediate step between Extraction and Comparison. In this step, the tool combines the extracted components in the layout netlist into possible series and parallel combinations and generates a netlist representation of the layout, like SPICE. A similar reduction is performed on the "source" Schematic netlist as well.

Finally, the comparison step, as the name indicates, compares the extracted layout netlist to the schematic netlist and reports differences. If the two netlists match, then the circuit passes the LVS check. Internally, a Graph isomorphism check is performed to compare the netlists to see if they are equivalent. LVS Comparison report summarizes the differences and other info about compared cells.

Setting Up CPU Processing

LVS Run Submission Form → Run Data → Processing

The tool combines single processor performance with distributed processing/multithreading to accelerate DRC/LVS turnaround of advanced designs.



Specify Run Directory.

Set up Processing: Select the number of CPUs and machine.

- Run Pegasus: Select run mode.
  - Single CPU (default) and Distribute on Multi-CPU (DP) modes.
- Run Pegasus on options:
  - Local host: Run on the local machine (default).
  - Other Local: Run as a part of a batch script on the local machine.
  - Other Remote: Run from a Remote machine.
  - SSH/RSH (in DP mode only): Secure/Remote shell modes.

- License Timeout, sec – has license queuing waiting time. Default is 300 seconds.
- Specify log filename and directory.
- Temporary Directory list.

Enable Pegasus Interactive engine.

- Recommended for block-level LVS.

281

In the Pegasus LVS Run Submission form, click the **Run Data** icon to display the Run Data section. Under this section, the user defines the run directory, processing details, and log file. Specify the Run Directory where the files from the run will be placed. Or, use the file browser button to select it.

Moving on to the Processing section, you may set to run the Pegasus job from a single CPU or distribute it across multiple CPUs, using options in this section. To run Pegasus with a single CPU on a Local host is the default selection. Under the Run Pegasus drop down, you can select from Single CPU or Distribute on Multi-CPU options. Under either selection, you have three options to run Pegasus, on a local host or machine: “other local,” where you run the job as a part of a batch script on the local machine, or “other remote,” to start the run from a remote machine. To view more details on setting up the CPU processing, refer to a similar section under the DRC module.

Note that all five Pegasus products run in single-processor (SP) mode from the various GUIs and command line. However, only DRC and XOR can be set to run on distributed processing as well, from the command line.

The Pegasus U I Log File field can be used to specify the name of the Pegasus log file. By default, it is named Pegasus U I, followed by the type of form dot log, and is saved to the run directory.

Use the Pegasus Log Directory field to specify the name of the log directory or use the file browser button to select it.

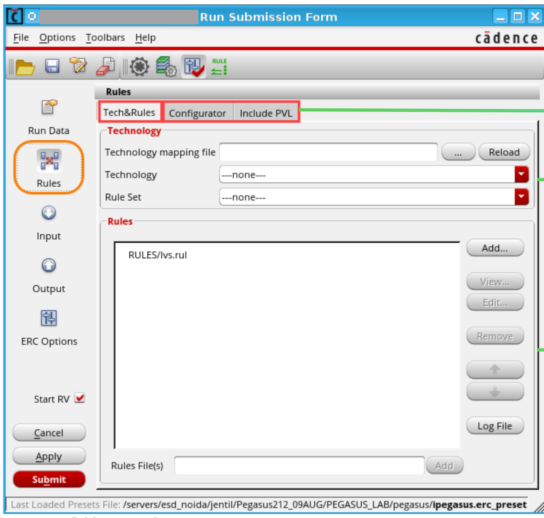
There is a provision to specify the temporary directory list for the run.

Select the ‘**Interactive engine**’ checkbox to run Pegasus Interactive engine. It is recommended to enable this checkbox when Pegasus LVS is run on block-level for better performance.

LVS Run Submission Form
Rules

Select the appropriate technology and rules for the Pegasus LVS run.

LVS Run Submission Form → Rules → Tech&Rules / Configurator / Include PVL



Configurator
- To customize the LVS Run form

Include PVL Rules
- Supplement or Override existing rules
- Enable the Include PVL Rules entry field

Technology
- To set the relevant Technology mapping file/Technology and rule set
- Applicable for foundry technologies

Rules
- To set custom rule deck files

cādence

The “Rules” section is used to specify the rule deck file for your LVS run. There are three tabs under the Rules section: “Tech&Rules,” “Configurator,” and “Include PVL.”

Under the “Tech&Rules” tab, there are two halves. “Technology” and “Rules.” You may specify a rule file through either or both of these sections. If you select a rule file with a mapping file plus technology and ruleset combination, then the “Rule file” selection is not necessary under the “Rules” field. Any rule file you specify with the “Rules” field will be in addition to the rule deck pointed to by the technology/ruleset combination earlier. You can add multiple rule deck files to your LVS run. There are options to remove or view the rules files.

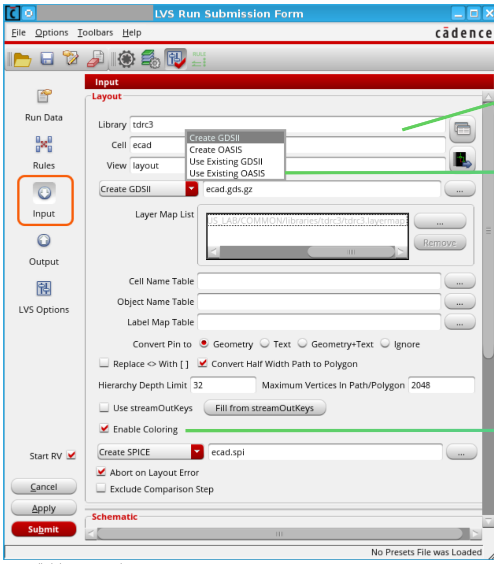
Moving on to the next tab under the “Rules” section – “Pegasus Configurator” is a mechanism to customize the Pegasus GUI to allow the selection of items that you can customize in a set of rules. The customization requires a configuration file containing instructions in a specific syntax to enable Pegasus to create a custom configuration GUI.

The last tab in the “Rules” section of the “Pegasus LVS Run Submission form” is titled “Include PVL.” When you select the “Include PVL Rules” checkbox, the options in this tab get activated, which enables users to add additional rules to be passed to the Pegasus run. When the run is submitted, the rules defined here are added to the “control file.”

For more details on each section, refer to the corresponding slides under the DRC module.

LVS Run Submission Form
Input Layout

LVS Run Submission form → Input → Layout



Layout: Cellview information
- Library, Cell, and View
- Select DFII Lib Mgr to browse to layout cellview

Layout Input Format options:
- Create GDSII (Virtuoso StreamOut tool)
- Create OASIS
- Use Existing GDSII
- Use Existing OASIS

- Enable Coloring – Only for Adv nodes.
- Virtuoso passes the predefined mask data to PVS/Pegasus

In the "Pegasus LVS Run Submission form," when you click the "Input" icon, the input section with two halves – one each for layout and schematic inputs – is displayed with multiple fields. Let's first take a look at the layout input fields.

In the first half, you select the input database details, such as a library, cell, and view names. You can also choose the input from the "Library Manager."

Available layout input format selections are "Create GDSII," "Create OASIS," "Use Existing GDSII or OASIS," and "Direct Read From Memory." Depending on the format selection, there will be minor changes to the form.

"Cell Name Table," "Object Name Table," and "Label Map Tables" can be manually entered or browsed from the custom locations.

There are four options to convert the Pin to – "Geometry, Text, Geometry + Text or Ignore."

There is an option to convert angular brackets to square brackets in text records during the translation from the DFII library to an OASIS file.

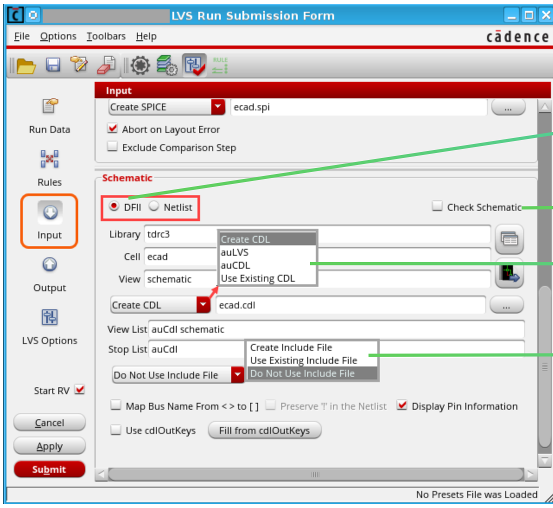
The "Enable Coloring" field enables the multiple mask feature in advanced nodes. When this feature is selected, Virtuoso passes the predefined mask data to Pegasus, and when unselected, all data on a layer is seen as if it is on the same mask.

By default, "Abort on Layout Error" is selected, which aborts the Pegasus run when it encounters fatal errors. Fatal errors can be missing cell masters, duplicate cell references, or absolute angles for an instance placement. The list includes absolute magnification for an instance placement, absolute width parameters for a path, or rule precision values that are not consistent with the input database precision.

LVS Run Submission Form

Input Schematic – DFII

LVS Run Submission Form → Input → Schematic (DFII)



Note: Pegasus LVS/SVS/PERC GUI supports mixed Virtuoso dfII/Verilog/CDL schematic input.

- UI supports Virtuoso dfII (CDBA and OA), converts it to CDL.
- No manual translation to CDL needed.

Check Schematic: Read schematic netlist to check its integrity.

CDL netlist creation options:
- Create CDL (standard CDL netlister – Virtuoso CdlOut) – default.
- auLVS/auCDL (Pegasus internal netlister)- Emulate Assura behavior.
- Use Existing CDL.

How to use mixed dfII/Verilog/CDL schematic input for LVS:
- Specify the dfII schematic as the primary schematic.
- Include a file, that specifies paths to external CDL/Verilog.
- Pin info between dfII & external Verilog/CDL cells must be consistent.

Example – include_file.txt:

```
.include_verilog "<path>/control_in.v"
.include "<path>/std.cdl"
```


Moving on to the Schematic half under the Input section of the Pegasus LVS Run Submission form, mark the Check Schematic checkbox if you want the schematic netlist to be read early in the flow to check its integrity. In this case, if no errors are detected when reading the schematic, the flow continues. If there are errors in the netlist, Pegasus terminates.

There are a couple of schematic format options to choose from DF2 and Netlist. Let us start with DF2. When you specify DFII as the schematic format, you need to set the additional fields, Library, Cell, and View, to make the selection. The view can either be a schematic view or a config view. Alternatively, you may also invoke the DFII Library Manager to make the selection.

There are four options available under CDL netlist creation from a schematic view: a standard CDL netlister is Virtuoso CDLOut, and it is called when the “Create CDL” option is selected in the Input section. An internal CDL netlister is called in when auCDL or auLVS options are specified in the Input section. There are fields to enter view lists and stop lists.

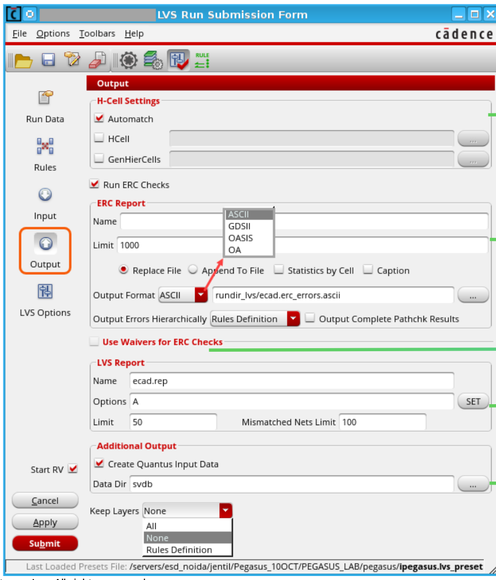
If you want to specify an additional Verilog, CDL, or SPICE file to be used along with the DFII database, there are a few options. You may select from Do Not Use Include File, or Use Existing Include File, and specify the path, or select **Create Include File**.

Click on **Fill from cdlOutKeys** to fill the CDL fields with corresponding values from cdlOutKeys.

LVS Run Submission Form

Output

LVS Run Submission Form → Output



- H-Cell Automatch: Pegasus tries for automatic cell name bindings (PVL command: automatch).
- If Automatch is not used, cells outside the hcell file are flattened.
- HCell: Pegasus uses the hcell file for cell name binding.
- GenHierCells: To have Pegasus generate (with a user-defined or default script) its own hcell list for cell name binding.

Options ERC run: Report Name, Format, limit, etc.

Waivers for ERC checks, if any.

LVS Report:
- Name of LVS Extraction report.
- SET the Output options.

For Pegasus to create Quantus input data.

When you click the **Output** icon in the “Pegasus LVS Run Submission form,” the output section is displayed.

Starting with the “H-Cell Settings,” click the **Automatch** checkbox if you want Pegasus to do automatic cell name correspondence. The H-Cell feature lets Pegasus use a “hierarchy cell” file for the cell name correspondences. The third option, “GenHierCells,” allows Pegasus to generate the H-Cell list automatically for cell name correspondence. You can specify a script file that Pegasus will use to auto-generate this H-Cell list.

There is another command-line-only option, namely genMatchOnlyHierCells, to generate H-Cell file with matching cells only. This option does not accept script files as input, and it must be used with the automatch option.

The “Run ERC Checks” option controls whether ERC rules are run or ignored during the extraction stage. By default, this checkbox is selected. You can enter the name of the summary report to be generated, set the limit on the number of lines for the report file, and replace or append to the existing file, etc. You may also select the Output Format; available options are ASCII, GDSII, OASIS, and OA.

The “Use Waivers for ERC” checkbox lets you invoke the waiver flow for the ERC run. Refer to the waiver section in the DRC module for more details.

Under the “LVS report” section, you can specify the name of the summary report and set the options. Click the **SET** button to select report options from the “LVS Report Options Selector” form.

Under the “Additional Output” section, click on the “Create Quantus Input Data” checkbox to create data for use with Quantus Parasitic Extraction. At the “Data Directory” field, specify the location of the Quantus data directory. The default name of the Quantus data directory is “svdb.”

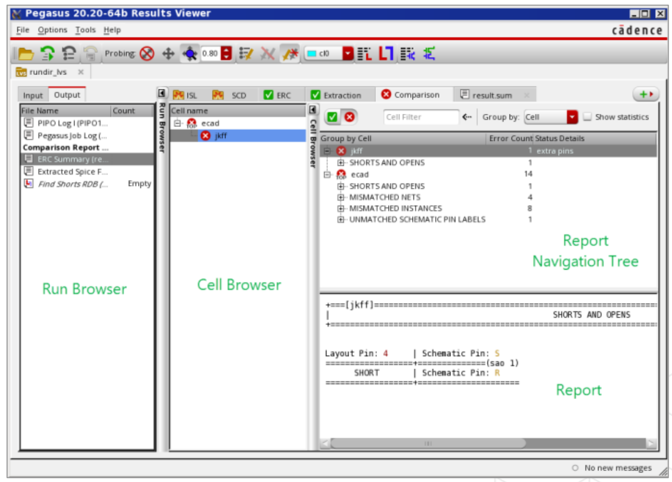
Click here for the Cadence Learning and Support link to the video on Keep Layers:

[How PVS Layer Viewer Works as a Great PVL Debugger?]

Pegasus Results Viewer for LVS, ERC, and SVS Runs

Results Viewer is the single-entry point to all results viewing and management tools in Pegasus. Pegasus Results Viewer is invoked from the Pegasus menu in Virtuoso, Innovus, or PDR.

- Pegasus RV GUI for LVS, ERC, and SVS
- ISL (Interactive Short Locator) tab
  - Only if Shorts in the design
- SCD (Stamping Conflicts Debug) tab
  - Only if Stamping Conflicts in the design
- ERC, Extraction, and Comparison tabs
- Probe Errors in Layout and Schematic
  - Reports section
  - Pegasus Layout Netlist Viewer
  - Pegasus Schematic Netlist Viewer
  - Probing form



The Results Viewer is the single-entry point to all results viewing and management tools in Pegasus. You can view and analyze DRC, Antenna, Density, XOR, FastXOR, LVS, ERC, and PERC, violations, Extraction, and Comparison Results, Stamping Conflicts, and Shorts using Pegasus RV. The Pegasus Results Viewer GUI varies for different runs.

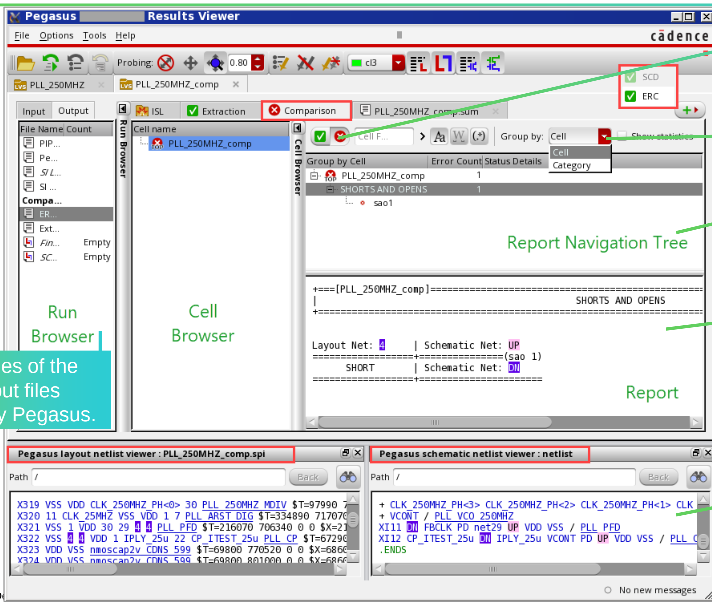
Here is a snapshot of the Pegasus Results Viewer GUI for LVS, ERC, and SVS Runs.

Pegasus Results Viewer for LVS runs consists of different tabs that include the Interactive Short Locator tab, Stamping Conflicts Debug tab, ERC tab, Extraction tab, and Comparison tab. Layout and Schematic LVS errors can be probed using various features available in the Reports Section of Extraction and Comparison tabs, or using Pegasus Layout and Schematic Netlist Viewers, or the Probing form.

Pegasus LVS/ERC Results Viewer

Comparison Tab

Pegasus Results Viewer → Comparison tab



Comparison Mismatched tab list Mismatched nets, pins and devices plus shorts and opens.

Multiple options to list the error items.

Navigation tree represents report file generated by comparison (<reportName.cls>).

Report section:

- Displays part of comparison report relates to selected item in the Navigation tree.

Lists Input files of the run and output files generated by Pegasus.

Pegasus Layout/Schematic netlist Viewer:

- Used for probing errors.
- Activate respective icons on toolbar.
- Then click on net/label link in report section.

287 © Cadence D

If there were no extraction errors and no shorts or stamping conflicts in the design, then the results Viewer form comes up in the Comparison results mode.

Pegasus RV Run Browser lists Input files of the run and output files generated by Pegasus.

Both extraction and comparison browsers are divided into three sections: Cell Browser, Report Navigation Tree, and Report section. These sections work together to let you view the results of your LVS runs. Cell Browser shows the hierarchy of cells being processed during device extraction in a tree view.

When the Comparison run status is Match, the form reports the comparison status Match, showing a green icon. When the Comparison run status is Mismatch, the Run Status reports comparison status Mismatch, showing a red icon. There could be several lines listing mismatched or unmatched nets, pins, or instances.

The Report Navigation tree represents the dot CLS report file generated by the comparison run. The navigation tree comprises three columns, listing cells compared by LVS, the number of errors found, and the status details.

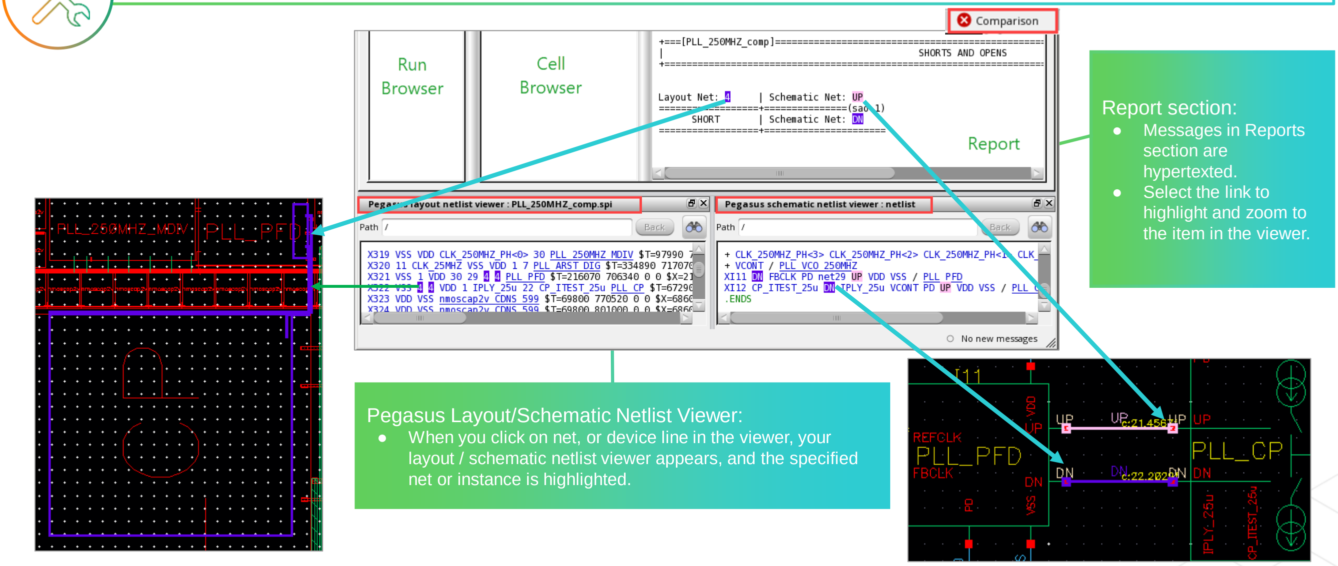
Errors can be shown by cell or category modes in the Navigation Tree.

By default, statistical information of the comparison report is not shown in the Report Navigation Tree. Click on the Show statistics checkbox to change this behavior.

Click on the Pegasus layout or schematic netlist viewer icon in the toolbar, and then click on any net or label link in the Report section. The Pegasus layout netlist viewer appears at the bottom of the Pegasus Results Viewer form. However, the window is detachable, and you can place it anywhere you want.

How to Analyze Comparison Mismatches in LVS RV

Pegasus Results Viewer → Comparison tab **→ Report** section



Report Section shows the device extraction report for the extraction tab; shows the comparison report for the comparison tab. The information which is shown in this section depends on the cell, category, and item selected in the Navigation Tree.

Messages in the Report Section are hypertexted. You can select any link to highlight the net, label, or instance in the designated color, in the corresponding viewer. When a net is highlighted in the layout viewer, the link is highlighted by the same color in the reports section.

You can probe Errors with the Pegasus Schematic or layout Netlist Viewers as well.

When you click on a net or device line in the viewer, your layout netlist viewer appears, and the specified net or instance is highlighted in your layout window.

To search a particular object, click the **Find Text** icon at the top of the window. A 'search' field appears at the bottom of the window. Use the 'Search' field to search an object and use the navigation icons to move up and down to view the search results.

Probing Errors with the Pegasus Schematic Netlist Viewer follows a similar process. Click on the Pegasus schematic netlist viewer icon in the toolbar, and then click on any net or label link in the Report section. The Pegasus schematic netlist viewer appears.

To remove active probes from your layout or schematic netlist viewer, use the 'Clear All Probes' button on the toolbar or the 'Clear Selected' or 'Clear All' buttons on the Probing form.

References

Explore DRC and LVS related full course reference

- Pegasus Verification System
- Physical Verification System
- Physical Verification Language Rules Writer

Explore DRC and LVS related Training Bytes

- How to Set Up and Run iPegasus SignOff DRC?
- Virtuoso DRC Verification Package
- How to Debug DRC Using PVS DRC Debug Environment
- What Is Layout vs. Schematic (LVS)?
- Pegasus Results Viewer 002 – LVS RV Overview

These links are accessible from the Reference section of the course interface.

Explore DRC and LVS related User Manual

- Cadence Pegasus User Guide
- iPegasus User Guide

cādence

This page does not contain notes.

Module Summary

In this module, you learned how to

- Create snapshots for the Virtuoso Studio iPegasus
- Verify DRC interactively with the Virtuoso Studio iPegasus
- Set up and run the Pegasus DRC and LVS in GUI mode
- Debug DRC and LVS violations in the Pegasus Results Viewer

This page does not contain notes.

Labs

Lab 6-1 Running Design Rule Check (DRC) Tests

Lab 6-2 Running Layout Versus Schematic (LVS) Tests

This page does not contain notes.

Module 7

Parasitic Extraction, Post-Layout Simulation, and Generating GDSII File

cādence

This page does not contain notes.

Module Objectives

In this module, you will:

- Set up Quantus™ Extraction Solution for extracting parasitics from the layout
- Extract RC parasitics using Pegasus™-Quantus flow
- Create design configuration using Hierarchy Editor
- Perform post-layout simulations using ADE Assembler
- Generate the GDSII file for tapeout

This page does not contain notes.

Terms and Definitions

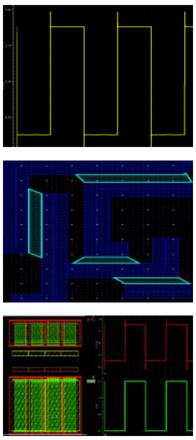
RC	Parasitic Resistance and Capacitance (RC) extraction involves extracting RC values from the layout and then used in calculating delays that affect the signal speed in an IC.
DEF	Design Exchange Format (DEF) file contains design-specific information about a circuit. It serves as a representation of design at any point during the layout process. These files are in ASCII format.
DSPF	Detailed Standard Parasitic Format (DSPF) file contains post-layout data including designed and parasitic devices in the design. These files are used for analyzing and reporting issues that may cause simulation problems.

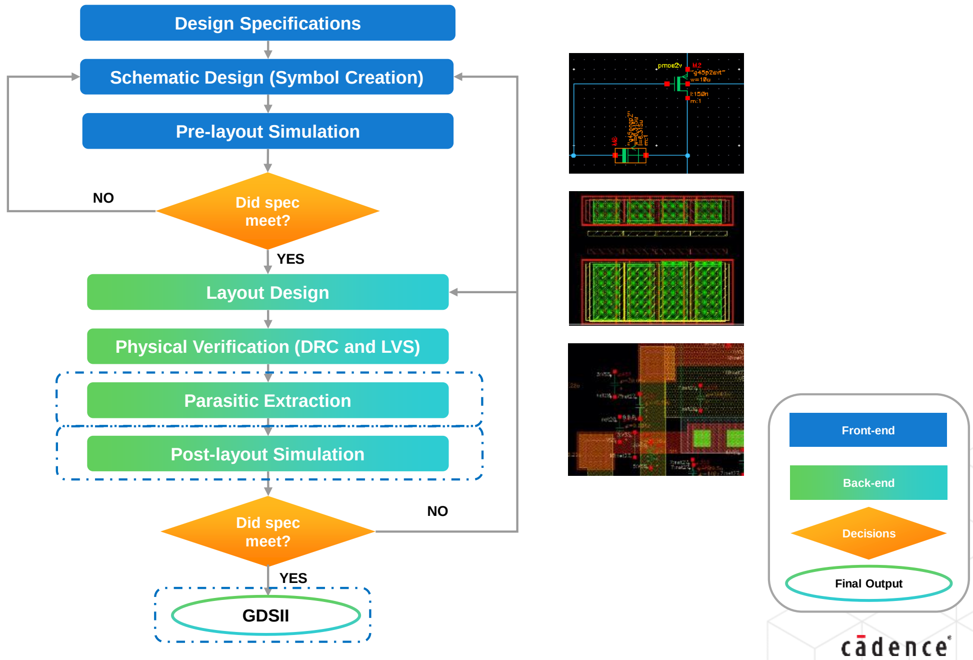
cādence

This page does not contain notes.

Analog IC Design Flow

PARAMETER	MIN	MAX
Gain		1000
Bandwidth	50KHz	500MHz
Supply Voltage	1.77V	5.5V
Supply Current	1.23mA	10.26mA
Propagation Delay (τ)	16.7ns	43.7ns
Chip area		$2\mu m^2$





This page does not contain notes.

What Is the Post-Layout Simulation?

The post-layout simulation is a process of simulating a layout design with extracted physical information to verify the functionality of the final design.

The post-layout simulation is done after the Physical Verification (DRC and LVS) step.

- To perform the post-layout simulation, you must extract parasitics from the final layout design, which leads to an individual stage called *Parasitic Extraction*.
- To perform post-layout simulation, we need to either:
  - Replace the design with the design with parasitic extracted design.
  - Add post-layout simulation files(DSPF) to the simulation environment.
- Run the simulations and analyze the results to ensure they meet the design specifications.

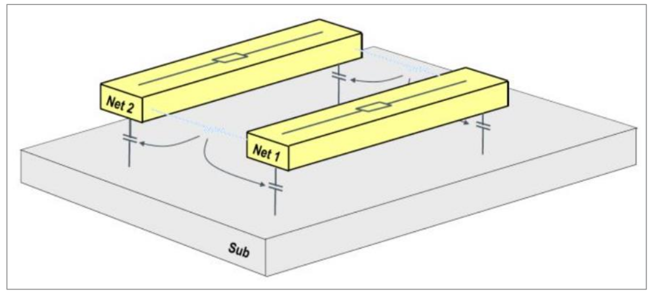


Figure shows the parasitic resistances and capacitances between the layers.

Next, we will learn how to extract parasitics from the layout design.

This page does not contain notes.

Submodule 7.1

Parasitic Extraction

This page does not contain notes.

Submodule Objectives

In this submodule, you

- Set up the Quantus Extraction Run form
- Extract RC parasitics using Pegasus -Quantus flow

This page does not contain notes.

What Is Parasitic Extraction and Its Solution?

Parasitic Extraction is the post-layout calculation of parasitic resistive, capacitive and inductive effects in the designed devices and wiring interconnects of an electronic circuit.

- Goal of extraction + simulation → to derive the total effective delay of nets/cells/devices.
  - Proportional to the extracted RLCK.
  - Total delay – The sum of delays of devices, nets, cells, and second-order device parameters.

- Quantus is the Parasitic Extraction solution from Cadence.
  - Hierarchical / flat extraction for the whole design / selected nets, for block-level / top-down designs.
  - Extraction run in batch (command line) or GUI modes.
  - Supports digital, analog, and RF designs.
  - Accepts multiple input formats – DEF, OA, Assura®, PVS, Pegasus, Calibre, etc.
    - This course is limited to Pegasus-Quantus flow.
  - Allows multiple output formats – DSPF, xDSPF, SPEF, xSPEF, SPICE, Extracted/Smart View, etc.

Let's check out the Quantus Extraction process.

Calculating the parasitic resistive, capacitive, and inductive effects in electronic circuit design devices and wiring interconnects is generally known as parasitic extraction.

What is the goal of parasitic extraction or simulation?

The goal is to derive the total effective delay of the design, which is directly proportional to the extracted parasitic resistive, capacitive, and inductive effects. Total delay will be the sum of the delays of devices, nets, cells, and second-order device parameters. Second-order device parameters include substrate capacitance noise, cross-talk delay, etc.

Quantus is the parasitic extraction solution from Cadence and can perform hierarchical as well as flat extraction for the whole design, or the selected nets in the design, for the block-level and top-down designs, and the cell-level, as well as transistor-level designs.

The extraction can be done in batch mode, that is, from the command line, using the CCL commands, or from the graphical user interface, in the G U I mode.

Quantus extraction solution supports digital, analog, and R F designs; it accepts multiple input formats, like DEF and O A, and the input formats from the Cadence LVS tools, like Assura, PVS, or Pegasus, and the third-party LVS tools, such as Calibre. However, the scope of this course is limited to the Pegasus input to the Quantus.

The combined extracted parasitic RLCK networks are then added to the design netlist to be output in one of the selected formats, which could be DSPF, Interactive DSPF, Transistor-Level DSPF, SPEF, or Transistor-Level SPEF, SPICE, Hierarchical SPICE, Extracted View, LVS Extracted View, Smart View, etc.

Quantus RLCK Parasitic Extraction

Quantus performs:

- RC extraction for cell-level and transistor-level designs.
- Inductance extraction for the transistor-level designs only.
- Capacitance (C) extraction:
  - Quantus geometrically analyzes each conductor in all three dimensions.
  - Generates parameters based on the specific three-dimensional regions.
  - Then passes parameters to the *qrcTechFile* models for capacitance calculation.
  - The models use a halo region to account for all the lateral/multi-level interconnect capacitive effects.
  - The 3D extraction maintains both lumped and fully distributed net-to-net coupled capacitance.
- Resistance (R) and Inductance (L/K) extraction are performed on a design interconnect and vias based on:
  - Layer resistance properties are defined in the *qrcTechFile*.
  - Fracturing and frequency rules are established in the Quantus Command File.
- Combined extracted parasitic RLCK networks are added to the design netlist for output to the selected format.

300

Here's a summary of the RLCK Parasitic Extraction performed by Quantus. Quantus performs RC extraction for cell-level and transistor-level designs, whereas inductance extraction is performed only for transistor-level designs.

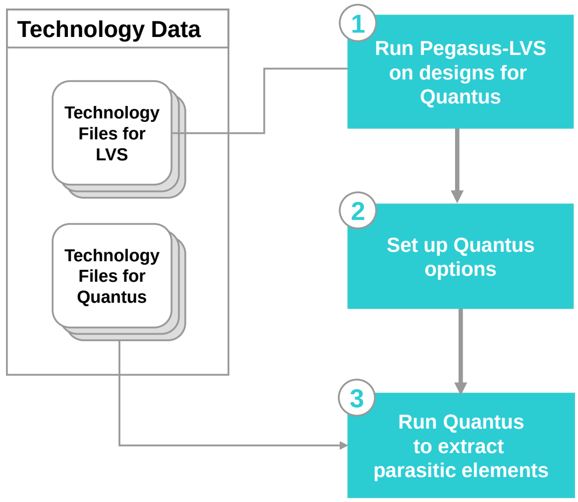
What happens in the capacitance extraction?

Quantus starts by geometrically analyzing each conductor in all three dimensions and then generating the parameters based on the specific 3D regions. Then Quantus would pass the parameters to the models derived from QRC TechFile, for capacitance calculation. These models would then use a halo region to account for all the lateral and multi-level interconnect capacitive effects. Finally, the 3D extraction would maintain both lumped and fully distributed, net-to-net coupled capacitances.

Resistance and inductance extraction is performed on the design interconnect and vias based on two parameters: the layer-resistance properties defined in the QRC TechFile and the fracturing and frequency rules established in the Quantus Command File.

Once the capacitance, inductance, and resistance extraction are done, the combined extracted parasitic networks are added to the design netlist to be output in one of the selected formats, like DSPF, SPICE, Extracted View, etc.

Flowchart: Pegasus-Quantus User Flow
Transistor-Level



`pegasus -lvs -rc_data lvsrules`

This flowchart outlines the main steps of setting up, and running the transistor-level Quantus. To perform parasitic extraction, you must have the technology files, which your design is based upon, and the layout, and connectivity information of the design.

This is a three-step process, starting with running the LVS, on designs for Quantus. For transistor-level extraction, the layout, and the connectivity information is furnished to Quantus, after the LVS verification.

Quantus supports various LVS tools, like PVS, and Pegasus, basically from Cadence, and a third-party tool like Calibre. For a Quantus overview, in this course, we will use Pegasus to run the LVS and proceed further to the Quantus flow.

Moving on to step 2, that is, to set up Quantus options. For Quantus, the technology file, for each process corner, which generally goes by the name, qrcTechFile contains a suite of 2D and 3D capacitance models, and the resistivity info associated with interconnects for that specific process.

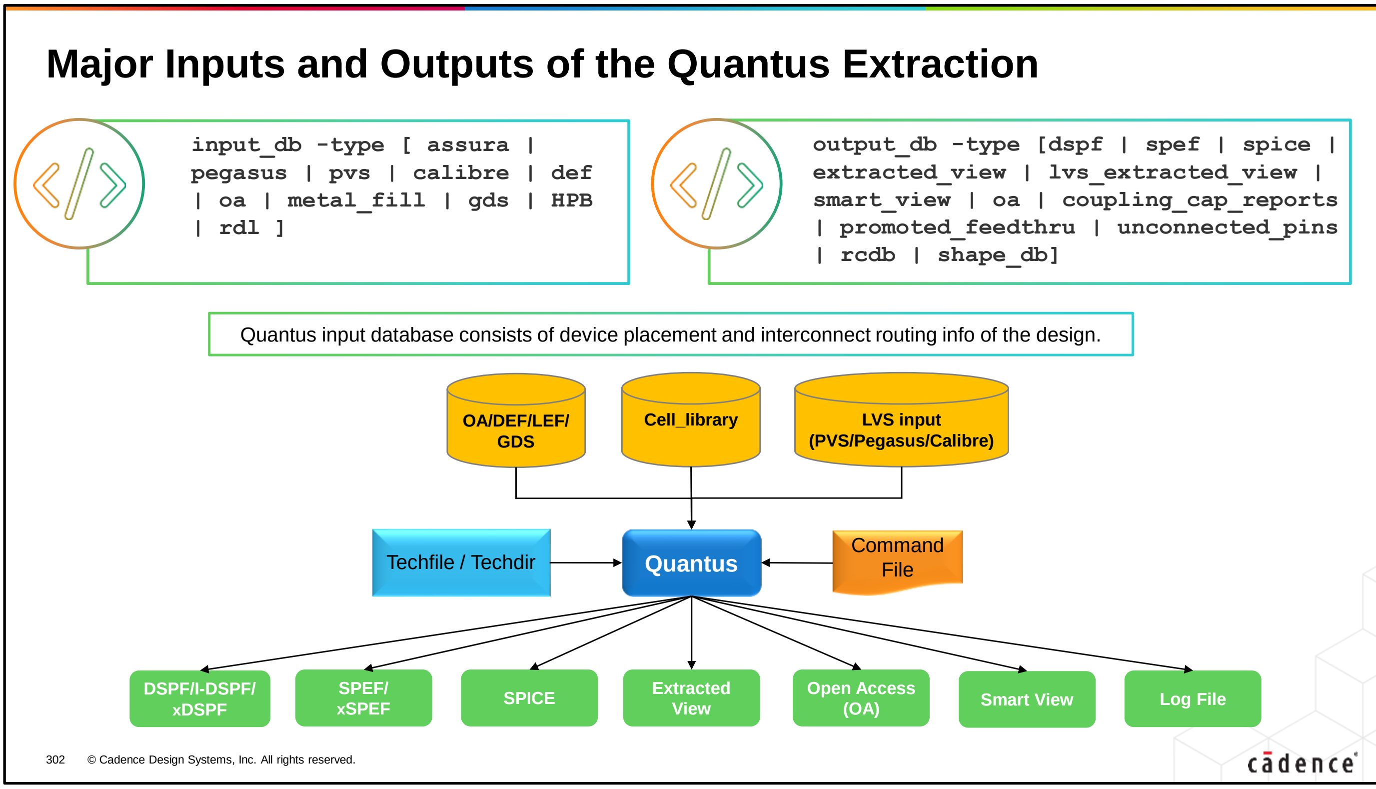
During the extraction, Quantus selects the appropriate model, from the qrcTechFile, and applies it to the specific interconnect configuration, encountered at each location of the design.

qrcTechFile is generated using a process called Techgen, which is Quantus's model-generation software, and is generally provided by the foundry, who provides the technology as well.

If you are interested in knowing more about, how to set up the Quantus technology directory, and how to create the tech files using the Techgen process, kindly refer to Tier 1 of this series, which goes by the name "Overview and Technology Setup for Quantus."

For this user training, in the Tier 2 course, we assume that all the needed technology files, have been created, and are ready for use.

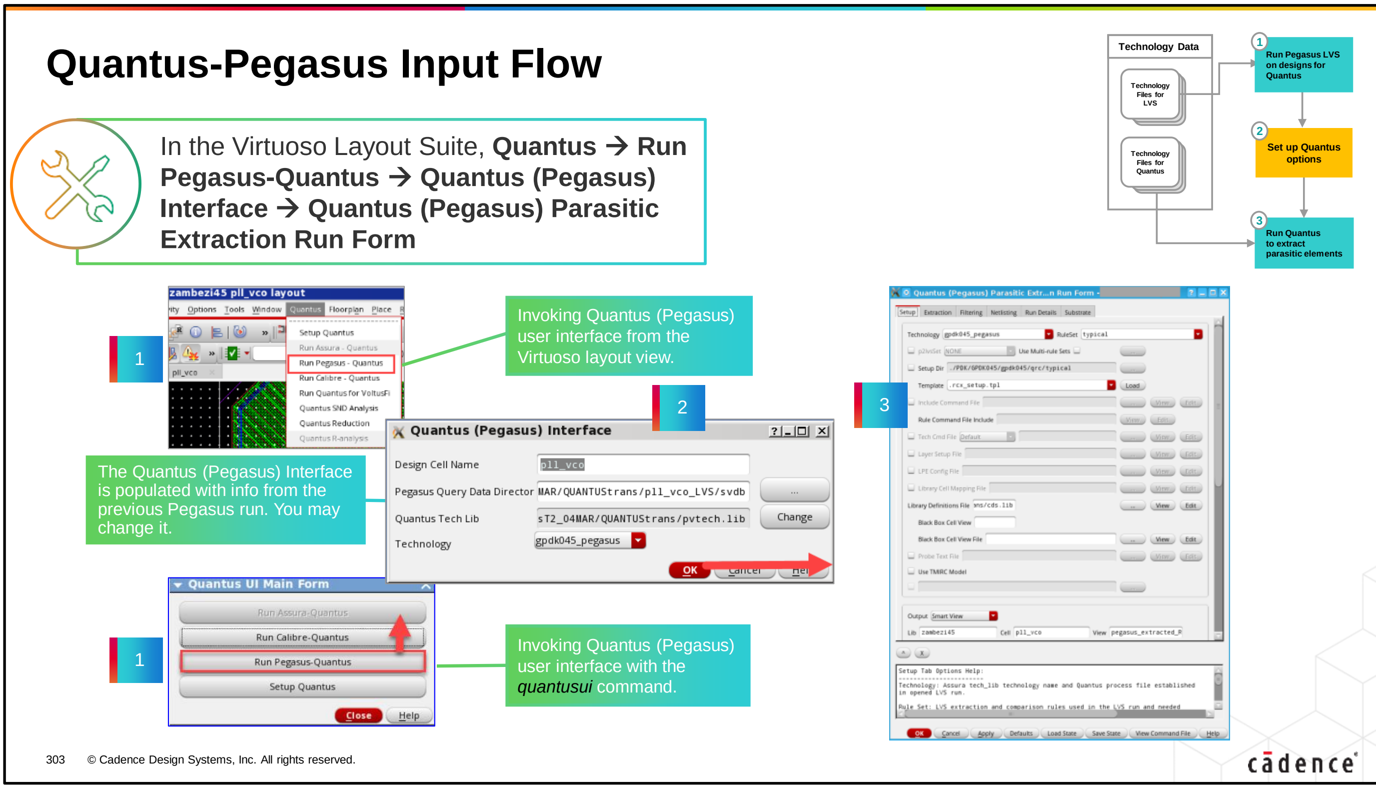
Now we move onto step 3, which is to run Quantus to extract the parasitic elements, and we accept the input from the Pegasus LVS stage, as well as the technology files for Quantus. Although not required, the technology files needed to run the Pegasus LVS, are conventionally put in the same location, where the qrcTechFiles are located. Steps 1, 2, and 3, are the focus of this module.



DEF/LEF/GDS/Metal Fill Input: **DEF Input with LEF Library files and GDS Metal Fill**

output_db -type [dspf | spef | spice | extracted_view | lvs_extracted_view | smart_view | oa | coupling_cap_reports | promoted_feedthru | unconnected_pins | rcdb | shape_db]

input_db -type metal_fill -retain_block_stream_metal_fill ***true*** *| false*: Quantus to consider block GDS metal fills (in addition to the top MFs) for extraction when block DEF GDS are provided.



How to invoke the Quantus-Pegasus Run Form? There are a couple of ways to do that.

The pre-load to launching the Quantus Run Form is to open the Quantus (Pegasus) Interface, and that can be done from the layout view in the Virtuoso environment or through the batch command *quantusui*.

From the Virtuoso layout view, click on **Run Pegasus-Quantus** at the Quantus drop-down menu: that brings up the Quantus (Pegasus) Interface. You may also run *quantusui* from the UNIX prompt, and that brings up the Quantus UI main form.

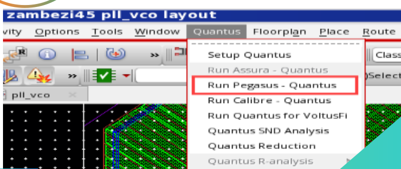
This interface is automatically populated with appropriate info from the previous Pegasus run. You may change it.

Once the fields like design cell name, Pegasus query data directory, Quantus tech lib, and technology details are selected, click **OK** to invoke the Quantus (Pegasus) Parasitic Extraction Run form.

Set up the Quantus extraction particulars through six tabs in the form and start the extraction.

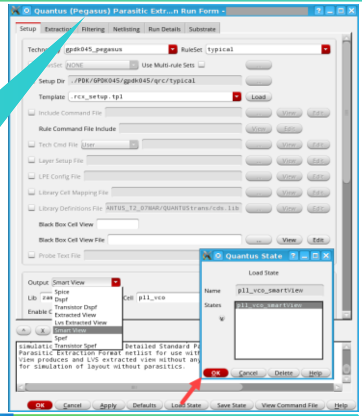
How to Perform Pegasus-Quantus Extraction in GUI

This is a step-by-step representation of invoking the Quantus Run form, setting it up, running the extraction, and analyzing the Quantus Extraction output.



1 Start the Quantus GUI.

1. Quantus supports PVS and Pegasus LVS. The Quantus UI displays “PVS or Pegasus” depending on the tool installation in the PATH.
2. The Quantus GUI run generates a Command File (qrc.<runName>.ccl), with the GUI options.



2 Set up the Quantus UI:

- Use Load State (optional).
- Click OK to start the Quantus run.

3 Extraction in progress.

Click Watch Log File to check the run status.

4 Analyze the Quantus output, for example, xDSPF.

Pegasus Quantus GUI Mode Extraction flow.

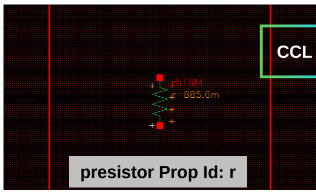
In the Virtuoso environment, click on the **Quantus** dropdown, and select **Run Pegasus Quantus**. Click **OK** in the resulting form to bring up the Quantus Pegasus Parasitic Extraction Run form. Set up various tabs and fields in this form as per your extraction specifications.

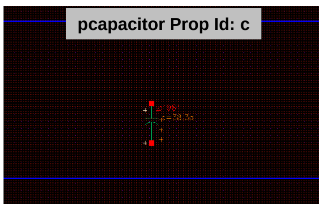
In case you have an existing state file from the previous one, use the load file option to load it, and make any corrections, if need be. This simplifies the Quantus GUI and reduces user mistakes. Click **OK** to start the parasitic extraction run. Track the progress in the Quantus progress form. You may track the incrementally updating log file with the watch log file option. This command does a UNIX tailing to display the file updates. A pop-up window will notify the end of the extraction run. You may opt for the appropriate extraction output format to analyze the results.

What Is Quantus Extracted View?

In the Extracted View, graphical symbols of parasitic elements are overlaid onto the layout shapes seen by Quantus; that helps cross-probing between the schematic and layout.

Parasitic Resistor in Extracted View



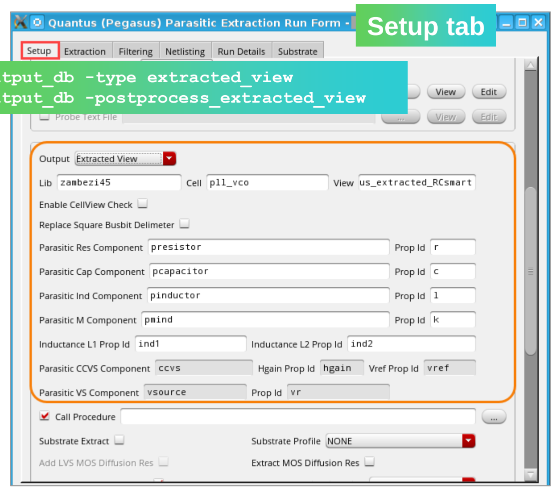


Parasitic Capacitor in Extracted View

Setup tab

CCL

```
output_db -type extracted_view
output_db -postprocess_extracted_view
```



Let's take a closer look at one of the important output formats in Quantus, which is the Cadence DFII Extracted View.

Extracted View outputs the extraction result graphically. The graphical symbols of parasitic elements are overlaid onto the layout shapes seen by Quantus. The extracted view is of importance to layout designers, as well, due to this graphical viewing feature.

As seen in an Extracted View, we have a sample resistor and capacitor on the left-hand side – the symbols with their corresponding RC values overlaid at the relative layout positions. The Quantus gui setup for the Extracted View generation is on the right-hand side. Select Extracted View as the output in the Setup tab, and the default output name is "av_extracted". Use Parasitic Capacitance, Resistance, Inductance, and Mutual Inductance Component fields, to specify the parasitic components from your DFII library that you want to place in the extracted view.

These properties are stored with each component. The Inductance Prop ID properties are used for mutual inductors. The CCL Command for Extracted View generation is to set the "output_db" type as Extracted View. You can generate a DFII Extracted View, without the parasitic elements, known as LVS Extracted View, by selecting the none option from the Extracted Type list under the Extraction tab.

The optional Call Procedure field lets you specify a SKILL file procedure to modify the extracted view output. It allows you to customize or post-process the Quantus extracted view output to meet the requirements of your design flow and environment.

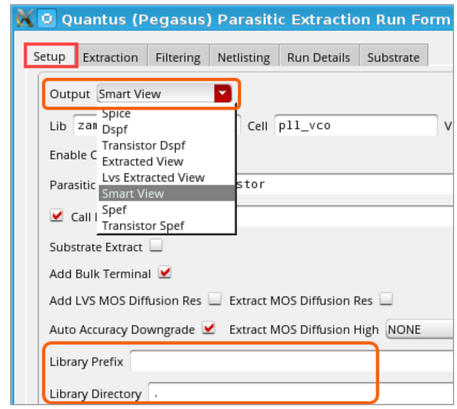
Select the Call Procedure checkbox to write the Call Procedure option to the CCL file to be run.

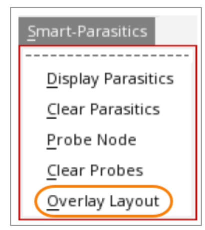
What Is Quantus Smart View?

Smart View (SV) is the enhanced version of Extracted View output that leverages massively parallel technology to improve the Quantus runtime, memory usage, and netlist size; supported by Pegasus, Assura, and Calibre-LVS flows.

- Tightly integrated with the Virtuoso ADE platforms (supported by PAD flow).
- SV provides key post-layout simulation/verification functionalities in the Virtuoso ADE:
  - Out-of-context (OOC) and In-context cross-probing, parasitic reporting, and parasitic back annotation.
  - SV visualization with Smart-Parasitics menu (Virtuoso Layout Suite GXL).
  - SV has no layout shapes but can Overlay Layout on a need basis → high efficiency.

```
input_db -type [caliber | pegasus | assura]
output_db -type smart_view
output_setup -net_name_space schematic | layout
```





What is Quantus Smart View?

Smart View is an enhanced next-edition version of the Quantus Extracted View that leverages massively parallel technology to significantly improve the Quantus runtime, memory usage, and netlist size. Quantus Smart View is supported for Pegasus, Assura, and Calibre LVS flows. Corresponding gui output selection in Smart View and the CCL option is to set the “output_db” type as Smart View. Smart View is tightly integrated with the Virtuoso ADE platforms, supported by the PAD flow, which stands for Parasitic Aware Design flow. Smart View provides some of the key post-layout simulation verification functionalities, in the Virtuoso ADE environment, like Out-of-context and In-context cross-probing, parasitic reporting, and parasitic back-annotation. Out-of-context probing is offered on Schematic. That is the ability to automatically map information from the schematic namespace to the post-layout Smart View namespace. In-context cross-probing is probing the Smart View itself. Parasitic reporting is a summary of the Sum Capacitance and Effective Resistance for all nets, and you can back-annotate this parasitic summary info to the Schematic as well.

Smart View has no layout shapes but can be visualized with the Smart-Parasitics menu, which is displayed by default from the Smart View window when working with Virtuoso ADE XL or GXL version. The Overlay Layout option here can be applied on a need basis.

Net namespace – layout and schematic options are supported for Smart View output generation.

The Smart View flow does not support some of the key Quantus flows, like Multi-Corner Extraction, Substrate Extraction, and EMIR flow with the Voltus™-Fi XL. The support will be provided in future depending on ADE readiness.

Inductance extraction and Hierarchical Extraction are supported for the Smart View in GUI mode.

For Hierarchical Extraction, Library Prefix and Library Directory fields on the Setup tab of the Quantus Run Form are enabled for Smart View output.

Submodule 7.2

Creating Design Configurations with the Hierarchy Editor

This page does not contain notes.

Submodule Objectives

In this submodule, you

- Use the Hierarchy Editor(HED) to create interchangeable design configurations
- Modify cell bindings in a design

This page does not contain notes.

What Is the Virtuoso Hierarchy Editor?

The **Hierarchy Editor (HED)** tool in the Virtuoso® Studio allows you to create a design configuration or “**config**” cellview in which you can interchange the *view to use* for your simulation without having to change your original design.

- When you work with the Virtuoso use model, you work mostly with design schematics in the library:cell:view hierarchy structure.
  - The hierarchical schematics can be at a transistor level or can contain behavioral code, especially if you are at the verification phase. You can also use extracted views or pure digital text views in your design.
- Depending on which stage of the design you are at, it is necessary to seamlessly integrate and simulate such designs in a GUI environment using the available cellviews.
- The config views that are created using the HED are associated with a top-level schematic view and contain certain cell bindings, depending on the type of cell views available and the design hierarchy.

You can use the Virtuoso Hierarchy Editor to specify the cellviews that you want to use in your design.

A configuration (config) is a set of binding rules that defines which cellviews are part of the design for a given purpose (such as netlisting and simulation).

Using the Virtuoso Hierarchy Editor, you can view the hierarchy of these cellviews and apply view switching to mix and match abstraction levels depending on which phase of the design cycle you are performing.

You can start the Hierarchy Editor in standalone mode from the command line by entering **hierEditor**.

What Is the Purpose of Design Configurations?

A design configuration lets you **bind** different cellviews to different hierarchical blocks in your design, as your design evolves from concept to finish.

- Configurations are a record of the views created and made available for your simulation strategy.
  - In essence, they provide expansion information for mixed-signal partitioning.
- Config views are visible in your Library Manager. Use these views to open the HED, associated schematic view or both.
- Using a *config* view makes things easy:
  - When you are working on a post-layout simulation that needs extracted parasitic views.
  - When you are working with a mixed-signal/mixed-language design.
  - When there is more than one view of a cell because the elaborator needs to be instructed which view to choose.

310

Config views bind defined views to specific cells in your design library, and serve the same function as choosing the view to descend into from a schematic.

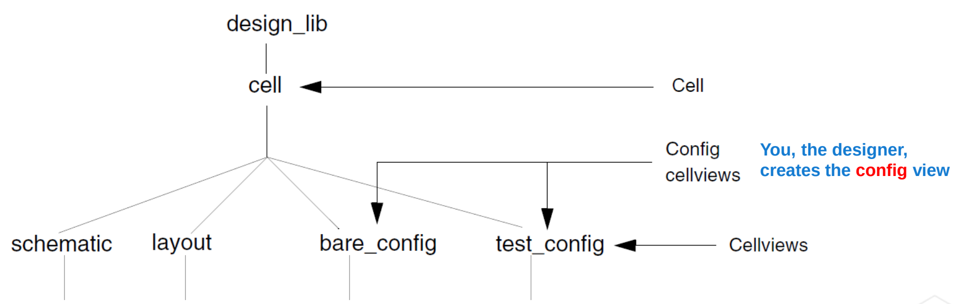
For example, you might begin the design process using high-level behavioral models of your design components; later, you might insert modules into test fixtures; you might replace behavioral descriptions with schematics and finally, add post-layout views, as the design process approaches implementation.

Example: Design Configuration for the Mixed-Signal Designs

- Configurations created using the Hierarchy Editor permit you to control the views used for simulation.
  - These config views can be schematics, Extracted Views, Verilog/A/AMS modules, primitive symbols, etc.

Important: To simulate your design with Spectre® in the ADE Explorer/Assembler, you must specify a top-level config cellview for your design hierarchy.

- Config views are displayed graphically (in a table or tree structure showing cell bindings) in the HED.



The top-level config cellview can be a HDL text module or a schematic. Note that your design can contain other config cellviews at lower levels in the hierarchy.

You can open the HED in standalone mode from the command line without invoking Virtuoso, by using **`cdsHierEditor &`**.

You can create configuration rules that define what views to include in the hierarchy at three different levels:

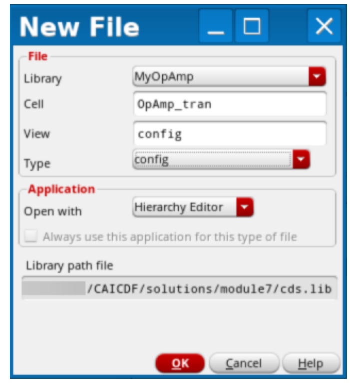
- Global level – Using a global view list and stop list.
- Cell level – Using cell-based view lists — which affect the cell as well as the structures below the cell in the hierarchy — and cell bindings.
- Instance level – Using instance-based view lists — which affect the instance as well as structures lower in the hierarchy — and instance bindings.

How to Create the Initial Configuration (New config Cellview)

When working within the Virtuoso environment, you mostly *invoke the HED from within Virtuoso* rather than invoking it standalone.

1. To create a new configuration using the Hierarchy Editor, choose **File – New –Cellview** from either one of the following:
   - Command Interpreter Window (CIW)
   - The Library Manager
   - The Schematic Editor
2. In the **New File** form:
   - Choose config as the *Type*. The View name is arbitrary. The default is simply *config*.
   - Click **OK** to bring up the **New Configuration** form.

2



When you choose the Type as config, the Application to Open with is automatically set to Hierarchy Editor

Continued on next slide

cādence

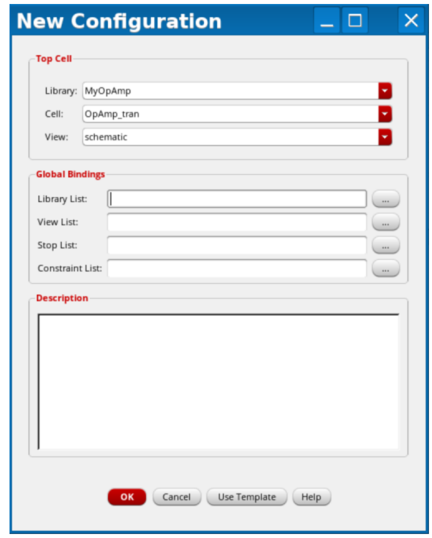
- There are multiple ways to create a new config cellview using the Hierarchy Editor. These are discussed here.
- When the New Configuration form comes up, the HED opens in the background but you cannot access it.

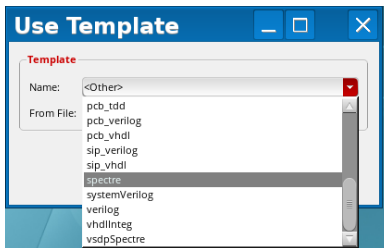
How to Create the Initial Configuration (New config Cellview)

(continued)

3. In the **New Configuration** form:
   - Set the top-level view (usually *schematic*) in the New Configuration form by selecting it.
   - Fill in the Global Bindings for the Library, View and Stop lists by either:
     - Clicking **Use Template** (recommended) and selecting a template (**Spectre**) to fill in the view list and stop list in the Hierarchy Editor.
     - Entering your own library, view and stop lists.

3





Continued on next slide

cadence

This page does not contain notes.

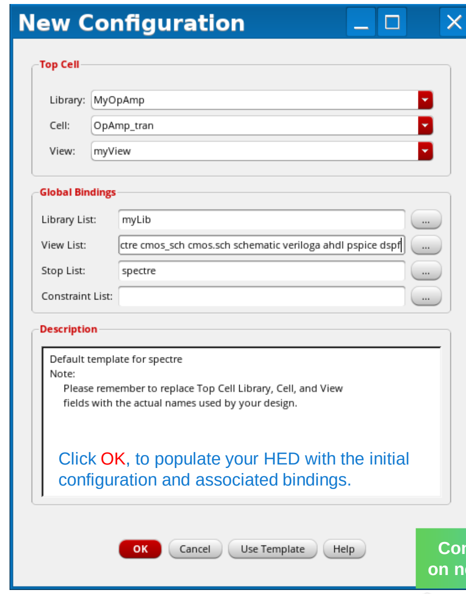
Choosing the Default Spectre Template

4. Click **OK** in the **Use Template** form after selecting **Spectre** to populate the bindings.

The Global Bindings provide priority search lists to resolve cell bindings to libraries (Library List) and view bindings to cells (View List).

- The Library List is a list of libraries that the netlister searches in for cells in the order of preference.
- The netlister will search for views in the order they are listed in the View List.
- The Stop List indicates that on finding which view the hierarchy expansion should stop.
  - This cell should be considered a leaf node in the hierarchy.
  - When using the Spectre template, it chooses Spectre as the default stopping view.

4



Continued on next slide

cadence

- The lists can be customized for any design, for special view names or library search ordering.
- *Library List* is a list of libraries, specified in order of preference, that are searched for cells. Each cell is obtained from the first library in the list that contains a cell of that name.
- *View List* is a list of views, specified in order of preference, for cells. For each cell, the first view in the list that is found is used.
  - For example, if the view list is **`systemVerilog`**, **`schematic`**, **`veriloga`**, then the netlister first looks for a **`systemVerilog`** view. If it is not found, it looks for a **`schematic`** view. If that is not found either, it looks for a **`veriloga`** view.
- *Stop List* is the list of views used to determine when the hierarchy expansion will stop and when a cell should be considered a leaf node in the hierarchy.
- *Constraint List* is a list of constraint views, listed in order of preference. The first view that is found is used. If none of the constraint views in the list is found, the schematic editor creates an empty view named after the first view in the constraint list.
- Selecting the browse (**...**) button for a specific Global Bindings list option will display a view list building form that will allow you to build up a view list without the need to manually type in the libraries, views, and so on.

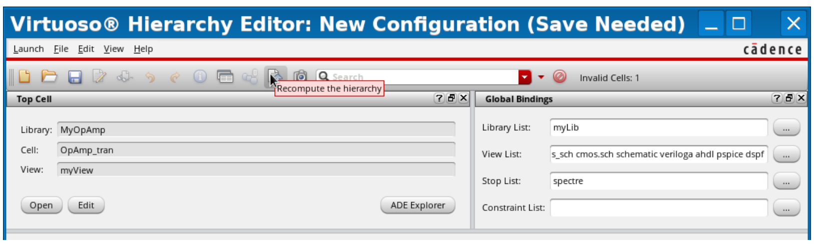
HED Populated with the Library, View and Stop Lists

Clicking **OK** after filling out the form opens the Hierarchy Editor, which uses the global bindings lists to create the list of design cells and view bindings.

- The library, view, and stop lists provide ordered search lists for the design parser, which walks through the entire design hierarchy, searching for views to use for the configuration.
- Unless overrides are provided in the Hierarchy Editor, the parser stops at the first valid view for a cell that it finds and uses that view for the netlist.

<u>Important:</u> Save the configuration in the Hierarchy Editor (**File – Save**) after the initial creation, and update and save it (**Recompute the Hierarchy** button) after any changes.

Recompute the Hierarchy button



A config view, when created, needs to be saved to be made available in the Library Manager.

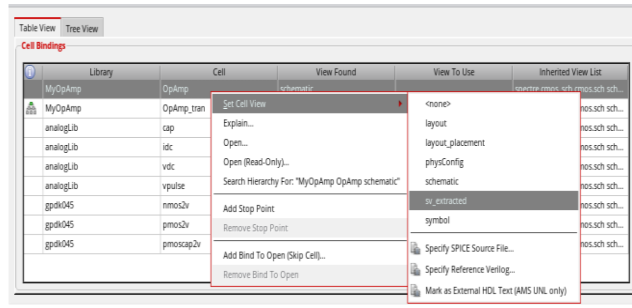
- The top portion of the HED is visible in this snapshot. You cannot see the configured design here.
- Views added to HED Stop List disable visibility/access to the cellview's hierarchy for netlisting but *do not* disable visibility/access for the xrun binding (BIND2) engine; therefore, to ensure accurate and consistent binding it is imperative that no text view with hierarchy should be used as a stopping view. Stopping view must be leaf-level text. More on this is discussed in subsequent slides.

How to Bind Cells to Cellviews in the Table View Tab

The HED lists the views to be used (bound) for each cell in the **View to Use** column. If nothing is listed the **View Found** entry is used.

1. Place the mouse pointer at a cell bindings line item.
2. **Right**-click to open the context-sensitive menu.
3. Select the **Set Cell View** option to show you a list of views available
4. Choose the view you want to use. This will replace the one in the binding list.

Right-click to change one view to another, for example, *schematic to sv_extracted* in the figure below.



- The *View to Use* column lists the view that is currently bound to the cell under consideration. The *View Found* entry is determined by the order of the view list. Different views can exist for each cell. The *View to Use* column controls overrides to the view list order.
- To change the views used for all cells at the same time, change the view list order and update the configuration.
- Using the right-click context-sensitive menu, schematics and modules can be opened in read-only mode or editing mode by choosing **Open** or **Open (Read-Only)** on the resulting menu. Once changed and saved, the modified cell can be re-compiled by pulling down to **Compile Netlist**.

Submodule 7.3

Post-Layout Simulation

This page does not contain notes.

Submodule Objectives

In this submodule, you

- Set up the designs and simulation files for post-layout simulation in the ADE Assembler
- Run post-layout simulations
- Analyze the post-layout simulation results

cadence

This page does not contain notes.

Recap of Pre-Layout Simulation Setup in the ADE Assembler

The environment setup for post-layout simulation will be very similar to that for pre-layout simulation, but a few more steps are included.

- In the pre-layout simulation stage, we:
  - Invoke ADE Assembler environment
  - Define Model Libraries and design variables
  - Setup multiple tests(AC, DC and transient)
  - Create output expressions using the Calculator
  - Define the outputs and parameters
  - Run the simulations
  - Analyze the simulation results
- Before running the post-layout simulation, additionally we will:
  - Create copies of each test that we had in the pre-layout setup
    - Rename the copied test. For example, tran_dspf, ac_sv
  - Change the design(from schematic to Config) for the copied tests

    (or)
  - Update the newly created tests with respective simulation files(DSPF/SPEF)

Note: The same pre-layout simulation setup can be used for post-layout simulations.

This page does not contain notes.

Post-Layout Simulation

Post-layout simulation is essential to reduce the risk that, a design passes verification before tape-out but fails after fabrication due to the parasitic effects. Post-layout netlists are created by layout extraction tools in formats such as Spectre netlist format, SPICE netlist format, or DSPF netlist format.

Post-layout simulation supports three types of files.

1. **Backannotation of parasitic files**
   - DSPF/SPEF/DPF files containing parasitic information. This methodology combines the parasitic information with the pre-layout netlist through stitching (backannotation).
   - It enables Spectre to automatically plug in the parasitic elements during the simulation.
2. **Hierarchical RC netlist file**
   - A netlist file that contains a hierarchy of extracted subcircuits.
3. **Flat RC netlist file**
   - Contains a large number of elements and devices.
     - **Example 1:** Directly include the DSPF file
     - **Example 2:** Netlist from the Quantus extracted view(av_extracted Smart_View)

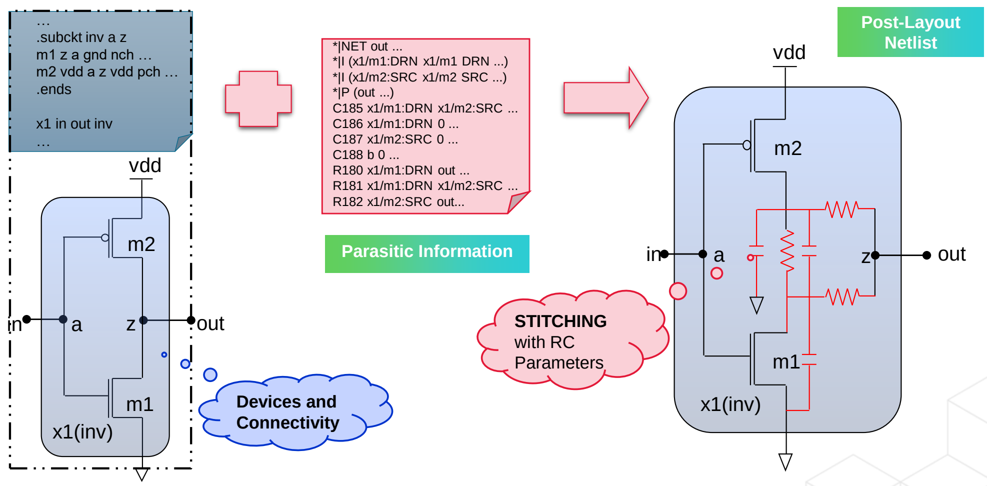
Note: Compared to the flat and hierarchical RC netlist methodologies, the stitching methodology has the benefits listed above.

What Is Backannotation?

Backannotation enables Spectre APS to plug in the parasitic elements while simulation is running. The simulation database is built upon the pre-layout netlist. The parasitics from the parasitic files are backannotated into the database.

Pre-Layout Netlist

- The hierarchy and the net names from the pre-layout netlist are retained, and the device models are regenerated according to the device parasitic information (DPF/Instance Section from the DSPF).
- The nodes are backannotated with R and C from the DSPF Net section.



- Pre-layout: device and connectivity
- Post-layout: parasitic information
- 1 node in pre-layout became 1 net in post-layout (in red). Stitching enables APS to stitch back the parasitic elements on the fly during simulation.

Stitching

- DSPF (Detailed Standard Parasitic Format) – Active Device + RC Parasitic Info
- DPF (Device Parameter Format) – Only extracted DEVICE Info
- SPEF (Standard Parasitic Exchange Format) – Only RC Parasitic Info, NO ACTIVE DEVICE

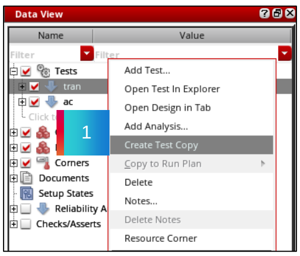
RC stitching, which is also known as RC backannotation, expands a node in the schematics to an RC net as defined in a DSPF or SPEF netlist.

The stitching process is case insensitive.

Creating Copy of Tests in the Data View Assistant

Tests can be copied/duplicated in the ADE Assembler Editing window.

1. **To copy/duplicate tests in the Data View assistant**:
   - **Right**-click the test name and select the *Create Test Copy* option.
2. **To change the name of the copied/duplicated test**:
   - **Double**-click the test name to invoke an inline editor.



Tests can also be renamed in the ADE Explorer Editing window.

How to Set Up Parasitic Files: Use Model

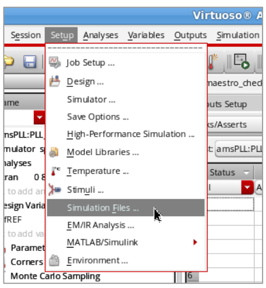
Include paths, definition files and stimulus files can be set up using the **Paths/Files** tab. Digital vector files can be set up using the **Vector Files** tab. DSPF files contain the post-layout data of the design, including fingered devices and net parasitics.

GUI support in ADE:

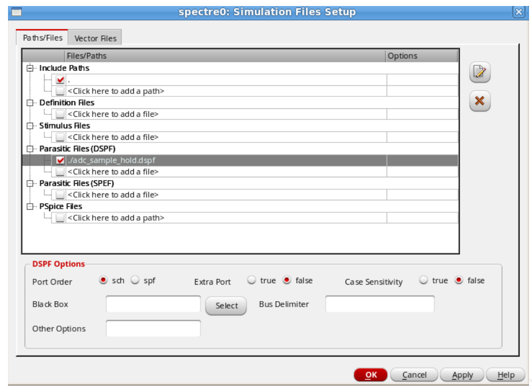
1. Choose **Setup – Simulation Files** in the ADE.
2. Select **Parasitic Files (DSPF)** and load **adc_sample_hold.dspf** in the Simulation Files Setup window.

`dspf_include "adc_sample_hold.dspf"`

1



2



cādence

- Include Paths specify the directories that contain the files you want to include when simulating your design.
- Files that contain function and parameter definitions that are not displayed in the Design Variables section of the simulation window can be added to the Definition Files section.
- The Stimulus Files section can contain input and power supply stimuli, initialize nodes, and include estimated parasitics in the netlist.
- The DSPF file containing post-layout data of the design, fingered devices and net parasitics, are included in the Parasitic Files (DSPF) section. With DSPF selected, notice the DSPF Options at the bottom of the form to provide **Port Order**, **Extra Port**, **Case Sensitivity**, **Black Box**, **Bus Delimiter** and **Other Options**.
  - *Port Order:* Specifies whether to take the port order of the subcircuit definition from the pre-layout schematic netlist or from the DSPF file subcircuit definition.
    - sch: port order is taken from schematic subcircuit definition.
    - Spf: port order is taken from the DSPF subcircuit definition.
  - *Extra Port*: Specifies how to handle extra ports in the DSPF or schematic subcircuit definition.
  - *Case Sensitivity*: Specifies case sensitivity of DSPF file.
  - *Black Box*: Specifies the instances to be considered as blackboxes.
  - *Bus Delimiter*: Enables you to map the bus delimiter between the schematic and DSPF or SPEF files.
  - *Other Options*: Any dspf_include option other than those available on this form.
- The SPEF files are included in the Parasitic Files (SPEF) section. Unlike the DSPF flow, the SPEF approach requires a combination of a schematic netlist with the active devices and the SPEF netlist with the parasitic element information.
- The PSpice® netlist format targeted to include PCB components is included in the PSpice Files section.

Pre- and Post-Layout Simulation Results Analysis

We will compare and analyze the pre-layout and post-layout simulation results.

tran	Vout_tran				
tran	Vin_tran				
tran	Vout_dc	1.004			
tran	Vin_dc	1			
tran	Slew Rate	51.02M	> 50M		pass
tran	Settling Time	22.35n	< 50n		pass
tran	Pdiss	184.9u	< 200u		pass
tran	Offset	3.85m	range -10m 10m		pass
ac	Av_ac				
ac	UGB	121.1M	> 50M		pass
ac	Av0	60.36	> 60		pass

Pre-Layout Simulation Results

tran_DSPF	Vout_tran				
tran_DSPF	Vin_tran				
tran_DSPF	Vout_dc	1.004			
tran_DSPF	Vin_dc	1			
tran_DSPF	Slew Rate	50.5M	> 50M		pass
tran_DSPF	Settling Time	26.48n	< 50n		pass
tran_DSPF	Pdiss	184.7u	< 200u		pass
tran_DSPF	Offset	4.364m	range -10m 10m		pass
ac_DSPF	Av_ac				
ac_DSPF	UGB	119.5M	> 50M		pass
ac_DSPF	Av0	60.54	> 60		pass

Post-Layout Simulation Results Using DSPF File

This page does not contain notes.

Submodule 7.4

Generating GDSII File

This page does not contain notes.

Submodule Objectives

In this submodule, you will:

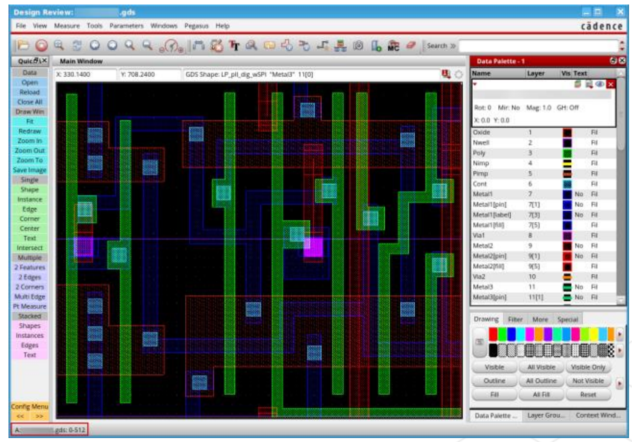
- Outline the components and the purpose of a GDSII file
- Export a layout into a GDSII file
- View the GDSII file

This page does not contain notes.

What Is GDSII file?

GDSII (Graphic data System II) is an industry-standard binary database file format used for exchanging IC layout data between an EDA and a foundry for Integrated Circuit (IC) design.

- GDSII file contains:
  - **Geometric Shapes**: Rectangles, polygons, etc.
  - **Layers**: Define different material types for fabrication.
  - **Text labels**: For annotations and identification.
- The final GDSII file is sent to the foundry, which creates the photomasks that define the patterns for etching the circuit on the silicon wafer.
- After layout design, physical verification, and post-layout simulation stages, we export the final layout to generate the GDSII file.
- This file is widely adopted by all EDA tools and foundries.



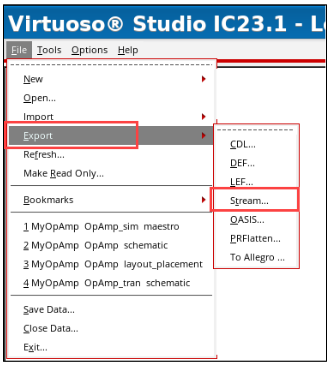
cādence

This page does not contain notes.

Generating GDSII File Using Xstream Translator

In the CIW, go to the **File → Export → Stream** to access the translation options.

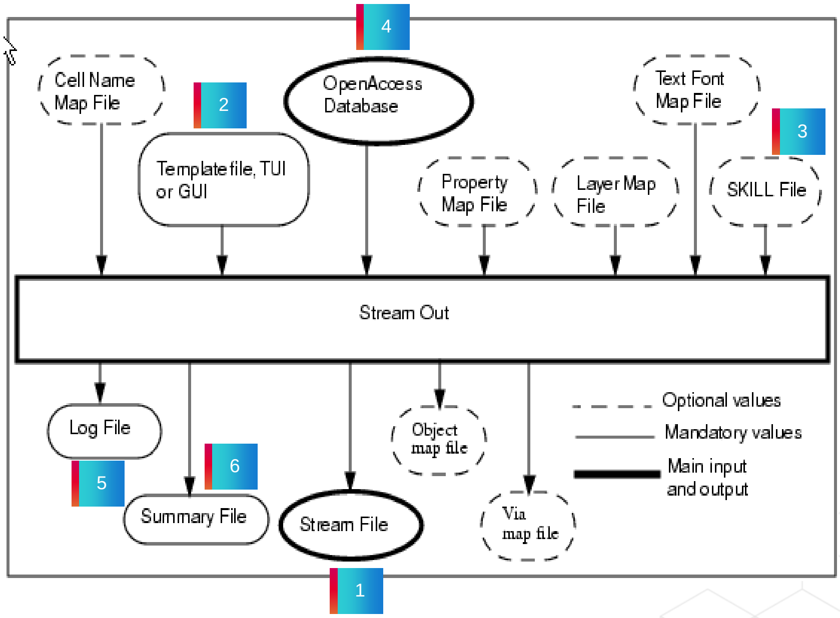
- At some point in the design process, you may want to export the design data for various reasons.
  - To Export the layout to GDS format, we use the **Xstream OUT** form.



This page does not contain notes.

XStream Out: Use Model

1. Stream File: Output Stream file.
2. Template File / GUI: A file that contains parameter settings you want to use to run XStream Out. Load the parameter settings in the template file into the Virtuoso® XStream Out form.
3. SKILL File: A cell used to map cell names, font and layer names.
4. Open Access Database: The database you want to translate to a stream file.
5. Log file: A file used for recording the translation process. It also keeps a log of all the messages.
6. Summary File: A file that has a summary of the translated design.



Templates

Prior to running the translator, you can save the run information in a template file. If there is a need to change an option and rerun the translator, you can load the template instead of specifying all the options, again.

Map Files

You use map files to create a correspondence among Cadence and Stream layers, cell names, and text height. An example is a layer map, which specifies the Stream layers to be translated and their corresponding Cadence layer name. By default, all layers are translated and assigned layer names that match the layer numbers.

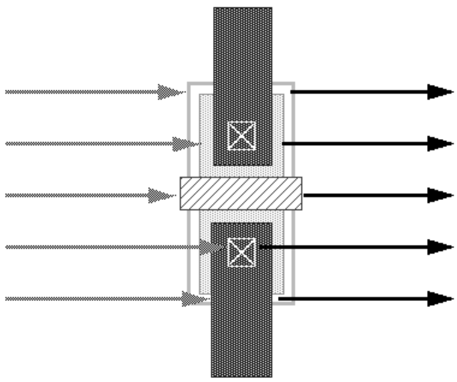
Technology File

An alternative to using a layer map file is to create a technology file with the techfile-generation option. You can edit the technology file to give the layers specific names, as well as customizing colors and adding technology information.

Layer Map File

Technology Layers

Layer	Purpose
nwell	drawing
pdiff	drawing
poly	drawing
contact	drawing
metal1	drawing
metal1	power
metal1	ground



XStream Layers

Layer #	Datatype #
1	0
2	0
3	0
5	0
6	0
6	1
6	2

The default for XStream In to a new library is to create a Cadence layer for every Stream layer. The layer names are defined as L#, where # is the Stream layer number. For example, data on Stream layer 5 is on Cadence layer L5.

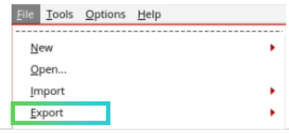
The default for XStream In to an existing library is to attempt to match the internal Cadence layer number to the Stream layer number. For example, if a Cadence library contained a layer named poly and its internal number is 5, then Stream data on Stream layer 5 is placed on the layer poly.

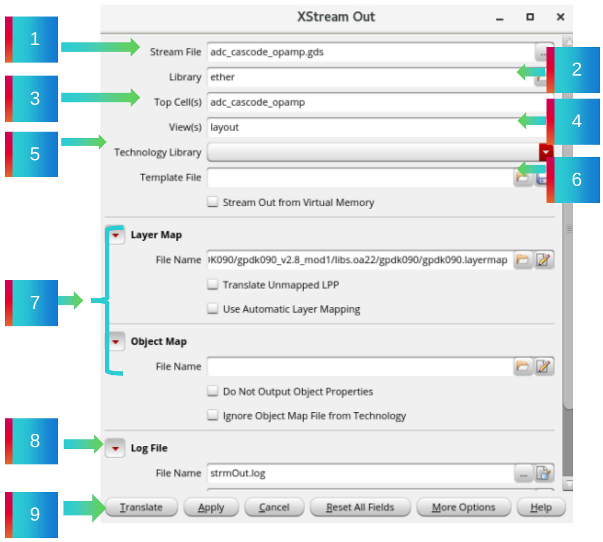
XStream Out translation uses the layer properties to determine what Stream layer to put data on. You can access and change these properties with the Technology File pull-down menu.

You can use a layer-mapping table for both XStream In and XStream Out. If you specify a layer map table, XStream In translates only the layers that are listed.

Running XStream Out Translator

XStream Out translates designs from the OpenAccess database to the Stream format.





1. Specify a stream output filename (**Required)**
2. Specify the Library (**Required)**
3. Specify top-level cellview
4. Specify the view (Layout)
5. Choose technology Library**(Required)**
6. Save and load template
7. Select and choose options
8. Specify the name in Log Field
9. Translate

Important – Select a library *before* streaming out the data.

Stream File – Specifies the output Stream file. (Required)

Library – Specifies the library from which the design is to be translated to the Stream format. (Required)

Top Cell(s) – Specifies the name of the cell at the top of the hierarchy that you want to translate to the Stream format.

Technology Library – Collects information about layers and purposes used in the design. By default, the technology file of the library is used.

Library – Specifies a library from which a design is to be translated to the Stream format. The library name should be a logical name supported in the *cds.lib* file.

Top Level Cell(s) – Specifies the name of the cell at the top of the hierarchy that you want to translate to the Stream file.

View(s) – Specifies a view name for the top cell. If you do not specify a view name, the layout view is the default view. If you specify any other name, the view name and all the hierarchy below is translated.

Template File – When this file is loaded, the XStream Out form is updated with the option settings specified in it.

Layer Map Field – Specifies an existing layer map file. Click on the Browser button to select the file

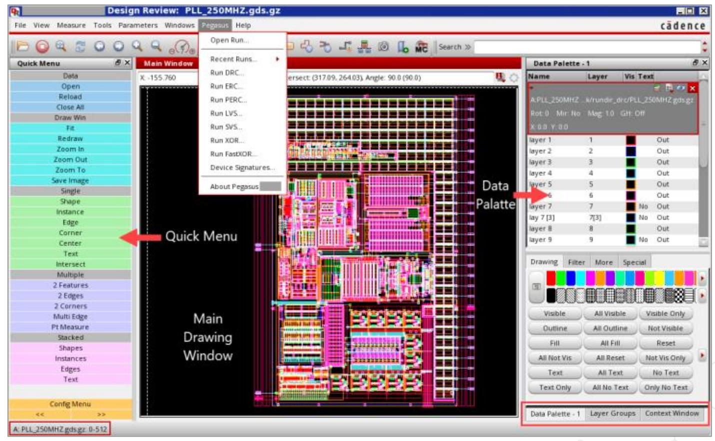
Object Map Field – Specifies an existing object map file. Click on the Browser button to select the file.

Log File Field – Specifies the name of the log file that will be used to record the translation process steps and the messages generated by XStream.

What Is Design Review?

Design Review (DR) is the industry-leading design data and results viewer. It is the central cockpit for review and signoff for Pegasus/PVS flows (DRC, LVS) and activities – A standalone, faster, easy-to-use, high-performance environment.

- Tightly integrated with the Cadence® platform.
- Central job submission, high-capacity/multi-format viewing, analysis, and debugging.
- Net connectivity tracing, visualization, overlay, and editing.
- Loads large layouts in seconds.
- Draws, examines, compares, and validates IC data in various formats.
- Extracts connectivity nets and cross sections from GDSII & OASIS.
- Assigns drawing colors and fill patterns.
- Point-to-point path analysis.



Design Review, and is the Cadence, standalone data, and results viewer. Design Review offers design and manufacturing teams a standalone, faster, easy-to-use, high-performance, and extensible environment for efficient tape-out and chip finishing.

Design Review is tightly integrated with the Cadence verification platform, and it acts as a central cockpit for job submission, high-capacity-multi-format viewing, analysis, debugging, and signoff for verification flows like DRC, XOR, LVS, and ERC. The debugging features of Design Review include net connectivity tracing, visualization, overlay, editing, interactive error browsing, and cross-probing. Design Review can load large layouts in seconds. It can draw, examine, compare, and validate IC data in various layout and pattern data formats.

Design Review can import and apply Virtuoso technology files to load data and perform point-to-point path analysis through identified connected geometries.

Design Review uses proprietary, high-capacity, high-performance algorithms to draw data directly from the layout and pattern data files. Design Review allows you to view patterns, job decks, and layout libraries individually and in combination. Flexibility in assigning, drawing colors, and fill patterns allows you to obtain the appearance and contrasts you desire. Design Review provides flexibility in customizing the display environment and user interface.

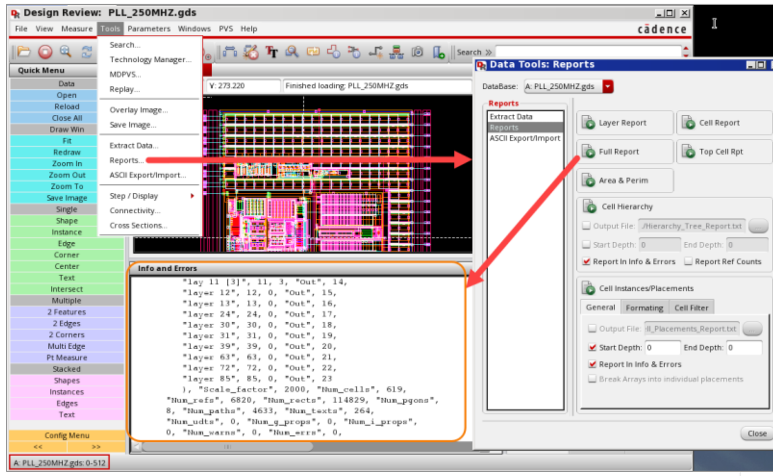
Here is a snapshot of the Design Review Canvas, with a sample GDS File Loaded. The “main drawing window” that appears automatically when you invoke Design Review is the principal Design Review window. The Data Palette on the right-hand side lists the patterns, job decks, and layout database files, which you have opened for viewing. It serves as the cockpit for controlling individual database attributes like rotation, mirror, magnification, and offset, along with individual layer attributes like visibility, color, and stipples.

Starting Design Review and Viewing Reports

```
Pegasus: % <install_dir>/bin/pegasusDesignReview

PVS: % <install_dir>/bin/k2_viewer
```

- Tools → Reports form to view Design Review reports:
  - Layer Report
  - Cell Report
  - Full Report
  - Top Cell Report
  - Area & Perimeter Report
  - Cell Hierarchy Report
- Interactive error browsing and cross-probing. RV is the default error browser.
- Customize display environment and UI.
- Imports and applies Virtuoso Tech files to loaded data.



Watch QuickView videos (predecessor to Design Review) at the Cadence Learning and Support portal.

The install directory must be set before you can invoke the Design Review. Once set, invoke Design Review by entering Pegasus Design Review from the Pegasus environment or k2_viewer from the PVS environment.

All forms and access to PVS or Pegasus rule decks are available in the Virtuoso environment.

Use the reports tool invoked from the Design Review tools drop-down menu to view various reports like Layer Report, Cell Report, Full Report, Top Cell Report, Area and Perimeter Report, Cell Hierarchy Report, etc. The Results Viewer is the default error browser for reviewing Design Review DRC and LVS results.

A series of videos on various QuickView features, the predecessor to Design Review, is available on the Cadence Learning and Support PVS QuickView channel.

References

Parasitic Extraction, Post-Layout Simulation course reference

- Quantus Transistor-Level T1: Overview and Technology Setup
- Quantus Transistor-Level T2: Parasitic Extraction
- Quantus Transistor-Level T3: Extracted View Flows and Advanced Features
- High-Performance Spectre Simulation

Parasitic Extraction and Post-Layout Simulation Training Bytes

- PVS Quantus QRC Overview (Video)
- Post-Layout Simulation Methodologies
- Setting Up Parasitic Files – Use Model

Parasitic Extraction and Post-Layout Simulation User Manual

- Quantus Extraction Users Manual

These links are accessible from the Reference section of the course interface.

This page does not contain notes.

Module Summary

In this module, you learned how to

- Set up Quantus Extraction Solution for extracting parasitics from the layout
- Extract RC parasitics using Pegasus-Quantus flow
- Create design configuration using Hierarchy Editor
- Perform post-layout simulations using ADE Assembler
- Generate the GDSII file for tapeout

This page does not contain notes.

Labs

This page does not contain notes.

Module 8

Next Steps

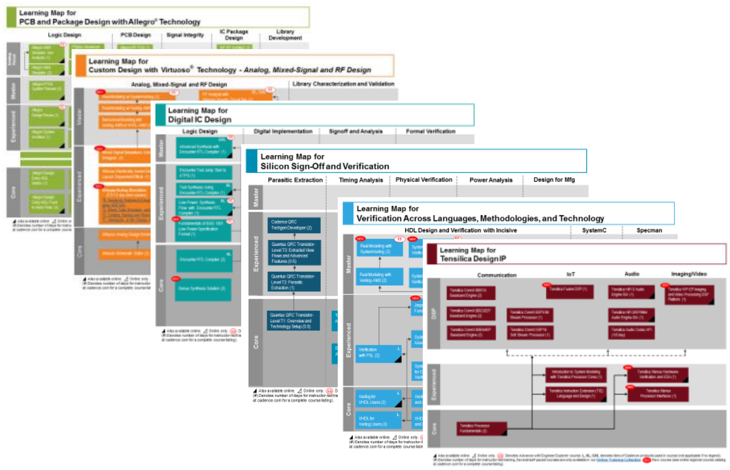
cadence

This page does not contain notes.

Learning Maps

Cadence® Training Services learning maps provide a comprehensive visual overview of the learning opportunities for Cadence customers.

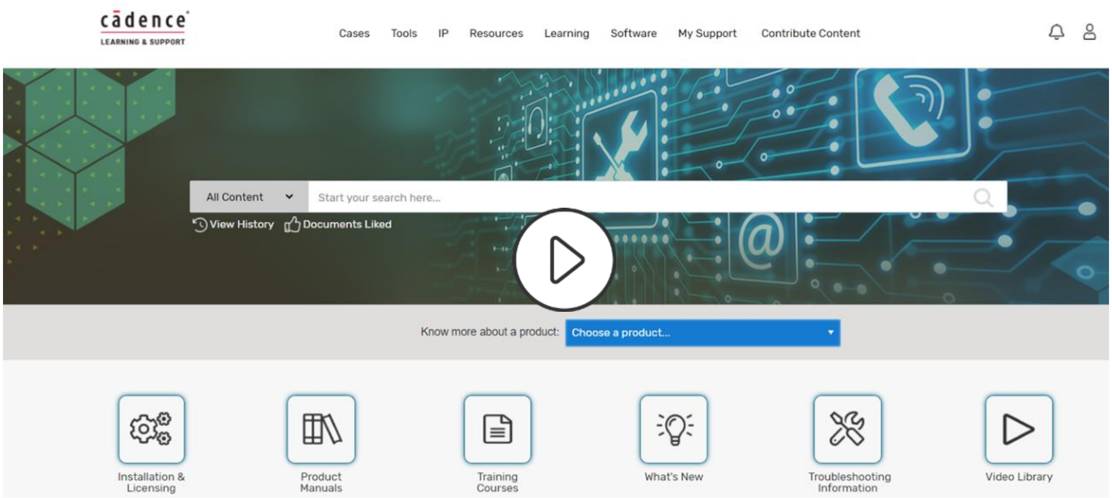
Click here to see all our courses in each technology area and the recommended order in which to take them.



Go here to view the learning maps:

http://www.cadence.com/Training/Pages/learning_maps.aspx

Cadence Learning and Support



Cadence Support now includes over 2000 product/language/methodology videos (“Training Bytes”)!

Click here to view the demo of Cadence Learning and Support.

Wrap Up

- Complete Post Assessment, if provided
- Complete the Course Evaluation
- Get a Certificate of Course Completion

Thank you!

This page does not contain notes.

This page does not contain notes.